

TCX Three Candle EA

Website Control Technical Manual

Full project architecture, Supabase flow, MT5 EA logic, web controls, performance tuning, deployment steps, troubleshooting, and line-by-line EA source appendix.

Generated: 2026-05-15 20:00:58

Repository: /Users/vedkanani/Downloads/tcx-supabase-react-web

Note: the .mq5 source must be compiled in MetaEditor before MT5 runs the updated binary.

Executive Summary

This manual documents the TCX Three Candle EA web-control project end to end. It covers the MT5 Expert Advisor, the React website, Supabase tables and Edge Functions, Telegram notifications, command synchronization, and the latest performance and command fixes.

- 1ST BE web button now uses SET_PARTIALS with firstBreakEven=true, avoiding Invalid command on older deployed functions.
- Command buttons use optimistic UI and independent pending state so 2nd Entry and 1st Trail do not flip each other.
- EA command polling is tuned to 200 ms with a 500 ms failure backoff for faster pickup.
- Supabase notification and audit side work is moved to background tasks so trading commands return faster.
- The EA contains break-even, first-trade break-even exit, second-entry, first-trade trailing, partial closes, dashboard UI, and Supabase web control.

High Level Architecture



Telegram notifications listen beside the command and state flow. They should never block order control.

Command Sequence

Step	Actor	What happens
1	User clicks a website button	React immediately updates optimistic state for mode buttons.
2	create-command Edge Function	Validates user, checks EA ownership, inserts one pending command row.
3	EA polls ea-next-command	EA authenticates with ea_id and token, fetches oldest pending command.
4	EA applies command	Runs SetArmMode, MoveFirstTradeToBreakEven, CloseFirstTradeAtBreakEvenExit, or state toggle.
5	EA acknowledges command	Posts done or failed to ea-ack-command.
6	EA posts state	ea-post-state updates website data and Telegram state monitoring in background.

Project Inventory

Every important source file is listed with line count and SHA-256 prefix so the document can be matched to the exact local code state.

File	Lines	SHA-256 prefix
Three_Candle_EA_SUPABASE_WEB_FULL.mq5	5155	ec39deab15ed97cd
src/App.jsx	2345	506cda22fc7ea071
src/styles.css	2873	6e45b57e39a984fc
src/supabaseClient.js	12	82d3043e6e570790
supabase/schema.sql	161	df317d623bd2549d
supabase/functions/create-command/index.ts	66	c417fc6e9d90c14e
supabase/functions/ea-next-command/index.ts	52	2789baf12d3f399d
supabase/functions/ea-ack-command/index.ts	49	f25ae2f69bf974e5
supabase/functions/ea-post-state/index.ts	60	63d56b0e5edf88b2
supabase/functions/telegram-settings/index.ts	135	03b3d6dae3a1ee7a
supabase/functions/telegram-chat-id/index.ts	65	3752af5ded454ac8
supabase/functions/telegram-offline-monitor/index.ts	56	1043989de3fd1a75
supabase/functions/_shared/telegram.ts	449	0293e5b42db71de7
README_SETUP.md	146	fb8373c67d4f4dd

Runtime Components

Component	Purpose
React website	Displays EA status, risk controls, trade controls, rules dashboard, trade history, and Telegram settings.
Supabase tables	Persist EA instances, latest EA state, pending/done commands, audit log rows, and Telegram settings.
Supabase Edge Functions	Bridge browser commands to database rows and EA polling to authenticated command pickup.
MT5 EA	Detects three-candle models, executes trades, manages protection, and syncs state/commands with Supabase.
Telegram integration	Sends command, ack, online/offline, and state-change notifications according to user preferences.

MT5 EA Technical Map

The EA is a single MQL5 source file. Its modules are organized by inputs, global state, trading logic, management logic, panel drawing, JSON/state sync, and MT5 events.

Line	Function	Purpose
140	HasOurPosition	EA helper used by trade, panel, or state logic.
232	LogMsg	EA helper used by trade, panel, or state logic.
237	SetLastMessage	EA helper used by trade, panel, or state logic.
245	TradeRetcodeOK	Executes, protects, or closes managed positions.
253	ClipText	EA helper used by trade, panel, or state logic.
262	ArmStatusText	EA helper used by trade, panel, or state logic.
275	DrawEmergencyStatus	EA helper used by trade, panel, or state logic.
296	ChartHeartbeat	EA helper used by trade, panel, or state logic.
302	BrokerMinLot	Calculates risk, lot size, margin, or panel risk display.
310	BrokerMaxLot	Calculates risk, lot size, margin, or panel risk display.
318	BrokerStepLot	Calculates risk, lot size, margin, or panel risk display.
326	VolumeDigits	Calculates risk, lot size, margin, or panel risk display.
337	NormalizeVolumeDown	Calculates risk, lot size, margin, or panel risk display.
355	NormalizeVolumeForOrder	Calculates risk, lot size, margin, or panel risk display.
372	NormalizeVolumeForPanel	Draws or updates the MT5 chart dashboard.
397	CleanNumber	EA helper used by trade, panel, or state logic.
421	NormalizeRiskMoney	Calculates risk, lot size, margin, or panel risk display.
430	CleanMoney	EA helper used by trade, panel, or state logic.
435	RiskPercentOfBalance	Calculates risk, lot size, margin, or panel risk display.
443	CleanPercent	EA helper used by trade, panel, or state logic.
448	PriceText	EA helper used by trade, panel, or state logic.
455	EffectiveRR	EA helper used by trade, panel, or state logic.
464	ClampPercent	EA helper used by trade, panel, or state logic.
473	CurrentSpreadPoints	EA helper used by trade, panel, or state logic.
478	SpreadProtectionPoints	EA helper used by trade, panel, or state logic.
497	ApplySpreadSpaceToSL	EA helper used by trade, panel, or state logic.
515	ApplySpreadSpaceToTP	EA helper used by trade, panel, or state logic.
536	SpreadBlocksTrades	Executes, protects, or closes managed positions.
541	RequiredStopPoints	EA helper used by trade, panel, or state logic.
549	HasEnoughBars	EA helper used by trade, panel, or state logic.
557	SelectOurPosition	EA helper used by trade, panel, or state logic.
575	CountOurPositions	EA helper used by trade, panel, or state logic.
594	IsDashboardMagic	EA helper used by trade, panel, or state logic.
599	IsManagedMagic	EA helper used by trade, panel, or state logic.
604	DashboardSourceText	EA helper used by trade, panel, or state logic.
613	CountDashboardPositions	EA helper used by trade, panel, or state logic.
631	MaxEntriesPerCycle	EA helper used by trade, panel, or state logic.
636	CanStartModelEntry	EA helper used by trade, panel, or state logic.
664	SelectOldestOurPosition	EA helper used by trade, panel, or state logic.
696	SelectNewestOurPosition	EA helper used by trade, panel, or state logic.
728	IsSecondEntryPositionComment	Manages second-entry state, execution, or display.
734	SelectFirstEntryPosition	EA helper used by trade, panel, or state logic.
798	HasOurPosition	EA helper used by trade, panel, or state logic.
803	SetArmMode	EA helper used by trade, panel, or state logic.
822	SetAutoArm	EA helper used by trade, panel, or state logic.
840	IsHedgingAccount	EA helper used by trade, panel, or state logic.
846	StateKey	EA helper used by trade, panel, or state logic.
852	SecondEntryCycleKey	Manages second-entry state, execution, or display.
858	LoadStateValue	EA helper used by trade, panel, or state logic.
868	LoadSecondEntryCycleValue	Manages second-entry state, execution, or display.
878	ClearSecondEntryCycleState	Manages second-entry state, execution, or display.

884	SavePositionState	EA helper used by trade, panel, or state logic.
897	ResetLocalTradeState	Executes, protects, or closes managed positions.
908	SaveSecondEntryState	Manages second-entry state, execution, or display.
920	ResetSecondEntryState	Manages second-entry state, execution, or display.
929	SaveFirstTrailState	Manages first-trade trailing or display state.
938	ResetFirstTrailState	Manages first-trade trailing or display state.
945	SyncFirstTrailState	Manages first-trade trailing or display state.
973	SyncSecondEntryState	Manages second-entry state, execution, or display.
1026	SyncPositionState	EA helper used by trade, panel, or state logic.
1089	ResetSignal	Detects or draws the three-candle pattern and sweep structure.
1111	DetectPattern	Detects or draws the three-candle pattern and sweep structure.
1214	IsJustClosedThirdCandleSignal	Detects or draws the three-candle pattern and sweep structure.
1233	DrawSignalMarker	Detects or draws the three-candle pattern and sweep structure.
1266	DeleteTradeLevelObjects	Executes, protects, or closes managed positions.
1277	DrawTradeLevels	Executes, protects, or closes managed positions.
1287	DrawHistoricalSignals	Detects or draws the three-candle pattern and sweep structure.
1307	DrawLatestClosedSignal	Detects or draws the three-candle pattern and sweep structure.
1322	IsTradingAllowedForDirection	EA helper used by trade, panel, or state logic.
1369	SpreadAllowed	EA helper used by trade, panel, or state logic.
1383	ValidateStops	EA helper used by trade, panel, or state logic.
1448	MoneyRiskForLot	Calculates risk, lot size, margin, or panel risk display.
1472	FitVolumeToFreeMargin	Calculates risk, lot size, margin, or panel risk display.
1508	CalculateRiskVolume	Calculates risk, lot size, margin, or panel risk display.
1545	GetRiskReference	Calculates risk, lot size, margin, or panel risk display.
1598	RiskPreviewText	Calculates risk, lot size, margin, or panel risk display.
1635	EnforceActualProtection	EA helper used by trade, panel, or state logic.
1676	CloseUnprotectedPosition	Executes, protects, or closes managed positions.
1690	ExecuteSignal	Detects or draws the three-candle pattern and sweep structure.
1839	PositionRRFromValues	EA helper used by trade, panel, or state logic.
1853	EnforceSecondEntryProtection	Manages second-entry state, execution, or display.
1916	ExecuteSecondEntryFromBase	Manages second-entry state, execution, or display.
2037	MonitorSecondEntry	Manages second-entry state, execution, or display.
2105	SecondEntryStatusText	Manages second-entry state, execution, or display.
2120	FirstTrailStatusText	Manages first-trade trailing or display state.
2133	MonitorFirstTradeTrailing	Manages first-trade trailing or display state.
2226	TryArmedExecution	EA helper used by trade, panel, or state logic.
2293	DoPartialCloseLevel	Handles partial close percentages and execution levels.
2374	MonitorPartialClose	Handles partial close percentages and execution levels.
2435	EntryCommissionForPosition	EA helper used by trade, panel, or state logic.
2462	EstimatedCloseCommissionFromEntry	Executes, protects, or closes managed positions.
2469	BreakEvenCostTargetMoney	Moves stops to break even or handles break-even exit.
2480	CommissionAdjustedBreakEvenPrice	Moves stops to break even or handles break-even exit.
2502	ExpectedCloseNetForTicket	Executes, protects, or closes managed positions.
2537	MoveTicketToBreakEven	Moves stops to break even or handles break-even exit.
2584	MoveToBreakEven	Moves stops to break even or handles break-even exit.
2630	MoveFirstTradeToBreakEven	Moves stops to break even or handles break-even exit.
2652	CloseFirstTradeAtBreakEvenExit	Moves stops to break even or handles break-even exit.
2694	CloseAllPositions	Executes, protects, or closes managed positions.
2722	CloseCurrentPercent	Executes, protects, or closes managed positions.
2770	CalcPanelGeometry	Draws or updates the MT5 chart dashboard.
2813	Rect	Draws or updates the MT5 chart dashboard.
2831	Border	EA helper used by trade, panel, or state logic.
2839	Txt	Draws or updates the MT5 chart dashboard.
2863	DotLabel	EA helper used by trade, panel, or state logic.
2880	SetDotColor	EA helper used by trade, panel, or state logic.
2889	SetCandleDotColor	EA helper used by trade, panel, or state logic.
2904	Btn	Draws or updates the MT5 chart dashboard.

2932	EditBox	EA helper used by trade, panel, or state logic.
2961	SetTxt	EA helper used by trade, panel, or state logic.
2969	SetRect	EA helper used by trade, panel, or state logic.
2977	SetBtn	Draws or updates the MT5 chart dashboard.
2988	SetEditText	EA helper used by trade, panel, or state logic.
2995	SectionTitle	EA helper used by trade, panel, or state logic.
3001	DayStart	EA helper used by trade, panel, or state logic.
3011	MoneyText	EA helper used by trade, panel, or state logic.
3019	SignedMoneyText	EA helper used by trade, panel, or state logic.
3029	CalcDealStats	EA helper used by trade, panel, or state logic.
3077	OpenProfit	EA helper used by trade, panel, or state logic.
3093	CurrentPositionRR	EA helper used by trade, panel, or state logic.
3112	SessionName	EA helper used by trade, panel, or state logic.
3126	SessionTimeLeft	EA helper used by trade, panel, or state logic.
3146	CandleCloseTimerText	Executes, protects, or closes managed positions.
3164	QualityScore	EA helper used by trade, panel, or state logic.
3191	QualityText	EA helper used by trade, panel, or state logic.
3200	QualityColor	EA helper used by trade, panel, or state logic.
3209	CardBox	EA helper used by trade, panel, or state logic.
3219	StatBox	EA helper used by trade, panel, or state logic.
3227	MiniCard	EA helper used by trade, panel, or state logic.
3248	MiniStat	EA helper used by trade, panel, or state logic.
3265	MetricRow	EA helper used by trade, panel, or state logic.
3281	H4BlockNumber	EA helper used by trade, panel, or state logic.
3288	LatestH4ShiftByParity	EA helper used by trade, panel, or state logic.
3314	PriceAboveOpen	EA helper used by trade, panel, or state logic.
3333	StandardSweep	Detects or draws the three-candle pattern and sweep structure.
3354	H4ParitySweep	Detects or draws the three-candle pattern and sweep structure.
3379	BuildCandleStructureRow	EA helper used by trade, panel, or state logic.
3387	BuildSweepStructureRow	Detects or draws the three-candle pattern and sweep structure.
3402	BuildStructureSections	EA helper used by trade, panel, or state logic.
3445	UpdateStructureSections	EA helper used by trade, panel, or state logic.
3496	BuildPanel	Draws or updates the MT5 chart dashboard.
3671	UpdatePanel	Draws or updates the MT5 chart dashboard.
3902	RefreshLotEdit	Calculates risk, lot size, margin, or panel risk display.
3903	RefreshRiskEdit	Calculates risk, lot size, margin, or panel risk display.
3904	TCX_SupabasePostState	Posts EA state to Supabase.
3906	TCX_TrimRightSlash	TCX web/state helper.
3916	TCX_JsonEscape	Builds or parses JSON for the website bridge.
3926	TCX_BoolText	TCX web/state helper.
3931	TCX_CommandStatusText	TCX web/state helper.
3945	TCX_ArmText	TCX web/state helper.
3953	TCX_PositionJson	Builds or parses JSON for the website bridge.
4051	TCX_CandleRowJson	Builds or parses JSON for the website bridge.
4058	TCX_SweepRowJson	Builds or parses JSON for the website bridge.
4063	TCX_StructureJson	Builds or parses JSON for the website bridge.
4089	TCX_DealReasonText	TCX web/state helper.
4104	TCX_DealSideText	TCX web/state helper.
4117	TCX_JsonPrice	Builds or parses JSON for the website bridge.
4145	TCX_TradeHistoryJson	Builds or parses JSON for the website bridge.
4220	TCX_BuildStateJson	Builds or parses JSON for the website bridge.
4314	TCX_HttpPostFunction	Posts EA state to Supabase.
4345	TCX_JsonString	Builds or parses JSON for the website bridge.
4370	TCX_JsonNumber	Builds or parses JSON for the website bridge.
4395	TCX_JsonHasKey	Builds or parses JSON for the website bridge.
4401	TCX_JsonBool	Builds or parses JSON for the website bridge.
4426	TCX_AckCommand	Acknowledges command status back to Supabase.

4434	TCX_ApplyWebCommand	TCX web/state helper.
4668	TCX_WebReady	TCX web/state helper.
4681	TCX_SupabasePollCommands	Polls Supabase for commands.
4710	TCX_SupabasePostState	Posts EA state to Supabase.
4727	EnsurePanel	Draws or updates the MT5 chart dashboard.
4736	RuntimeLoop	EA helper used by trade, panel, or state logic.
4774	RefreshLotEdit	Calculates risk, lot size, margin, or panel risk display.
4780	RefreshRiskEdit	Calculates risk, lot size, margin, or panel risk display.
4786	HandleLotEdit	Calculates risk, lot size, margin, or panel risk display.
4802	HandleRiskEdit	Calculates risk, lot size, margin, or panel risk display.
4811	HandlePartialEdit	Handles partial close percentages and execution levels.
4838	OnInit	MetaTrader event handler.
4881	OnDeinit	MetaTrader event handler.
4889	OnTick	MetaTrader event handler.
4894	OnTimer	MetaTrader event handler.
4899	OnChartEvent	MetaTrader event handler.

EA Inputs and Controls

Line	Input			Explanation
23	input group "TRADE"			Starts a user input group shown in MT5 EA settings.
24	input double	InpLot	= 0.01;	Defines MT5 input InpLot, configurable before attaching the EA.
25	input double	InpRiskReward	= 3.0;	Defines MT5 input InpRiskReward, configurable before attaching the EA.
26	input double	InpRiskMoney	= 100.0;	Defines MT5 input InpRiskMoney, configurable before attaching the EA.
27	input double	InpRiskMoneyButtonStep	= 10.0;	Defines MT5 input InpRiskMoneyButtonStep, configurable before attaching the EA.
28	input int	InpMagic	= 55555;	Defines MT5 input InpMagic, configurable before attaching the EA.
29	input int	InpDeviationPoints	= 30;	Defines MT5 input InpDeviationPoints, configurable before attaching the EA.
30	input int	InpStopBufferPoints	= 0;	Defines MT5 input InpStopBufferPoints, configurable before attaching the EA.
31	input ENUM_TC_STOP_MODE	InpStopMode	= STOP_FROM_PATTERN_EXTREME;	Defines MT5 input InpStopMode, configurable before attaching the EA.
32	input bool	InpFallbackNoInitialSLTP	= true;	Defines MT5 input InpFallbackNoInitialSLTP, configurable before attaching the EA.
33	input bool	InpCloseIfProtectionFails	= true;	Defines MT5 input InpCloseIfProtectionFails, configurable before attaching the EA.
34	input int	InpMaxEntriesPerCycle	= 2;	Defines MT5 input InpMaxEntriesPerCycle, configurable before attaching the EA.
35	input double	InpDailyLimitR	= 2.0;	Defines MT5 input InpDailyLimitR, configurable before attaching the EA.
36	input int	InpDailyLossLimitCount	= 2;	Defines MT5 input InpDailyLossLimitCount, configurable before attaching the EA.
38	input group "PATTERN"			Starts a user input group shown in MT5 EA settings.
39	input bool	InpAllowSweep	= true;	Defines MT5 input InpAllowSweep, configurable before attaching the EA.
40	input bool	InpAllowSwingPattern	= true;	Defines MT5 input InpAllowSwingPattern, configurable before attaching the EA.
41	input bool	InpShowSignalArrows	= true;	Defines MT5 input InpShowSignalArrows, configurable before attaching the EA.
42	input int	InpSignalLookbackBars	= 150;	Defines MT5 input InpSignalLookbackBars, configurable before attaching the EA.
44	input group "DASHBOARD"			Starts a user input group shown in MT5 EA settings.
45	input bool	InpShowManualTradesOnDashboard	= true;	Defines MT5 input InpShowManualTradesOnDashboard, configurable before attaching the EA.
47	input group "FILTERS"			Starts a user input group shown in MT5 EA settings.
48	input bool	InpUseSpreadFilter	= false;	Defines MT5 input InpUseSpreadFilter, configurable before attaching the EA.
49	input int	InpMaxSpreadPoints	= 5000;	Defines MT5 input InpMaxSpreadPoints, configurable before attaching the EA.
50	input bool	InpBlockTradeOnHighSpread	= false;	Defines MT5 input InpBlockTradeOnHighSpread, configurable before attaching the EA.
52	input group "SPREAD SL/TP SPACE"			Starts a user input group shown in MT5 EA settings.
53	input bool	InpUseSpreadProtection	= true;	Defines MT5 input InpUseSpreadProtection, configurable before attaching the EA.
54	input double	InpSpreadProtectionMultiplier	= 1.0;	Defines MT5 input InpSpreadProtectionMultiplier, configurable before attaching the EA.
55	input int	InpExtraProtectionPoints	= 0;	Defines MT5 input InpExtraProtectionPoints, configurable before attaching the EA.
57	input group "SECOND ENTRY"			Starts a user input group shown in MT5 EA settings.
58	input bool	InpSecondEntryEnabled	= false;	Defines MT5 input InpSecondEntryEnabled, configurable before attaching the EA.
59	input double	InpSecondEntryTriggerLossR	= 0.5;	Defines MT5 input InpSecondEntryTriggerLossR, configurable before attaching the EA.
60	input double	InpSecondEntryCancelAtR	= 2.0;	Defines MT5 input InpSecondEntryCancelAtR, configurable before attaching the EA.
62	input group "FIRST TRADE TRAILING"			Starts a user input group shown in MT5 EA settings.
63	input bool	InpFirstTrailEnabled	= false;	Defines MT5 input InpFirstTrailEnabled, configurable before attaching the EA.
65	input group "LOT PANEL"			Starts a user input group shown in MT5 EA settings.

66	input double	InpLotButtonStep	= 0.01;	Defines MT5 input InpLotButtonStep, configurable before attaching the EA.
67	input double	InpPanelLotMax	= 100.0;	Defines MT5 input InpPanelLotMax, configurable before attaching the EA.
68	input bool	InpClampLotToBrokerMax	= true;	Defines MT5 input InpClampLotToBrokerMax, configurable before attaching the EA.
70	input group "PARTIAL CLOSE"			Starts a user input group shown in MT5 EA settings.
71	input bool	InpPartialCloseEnabled	= false;	Defines MT5 input InpPartialCloseEnabled, configurable before attaching the EA.
72	input double	InpPC1_RR	= 1.0;	Defines MT5 input InpPC1_RR, configurable before attaching the EA.
73	input double	InpPC1_Percent	= 30.0;	Defines MT5 input InpPC1_Percent, configurable before attaching the EA.
74	input double	InpPC2_RR	= 2.0;	Defines MT5 input InpPC2_RR, configurable before attaching the EA.
75	input double	InpPC2_Percent	= 30.0;	Defines MT5 input InpPC2_Percent, configurable before attaching the EA.
76	input double	InpPC3_RR	= 3.0;	Defines MT5 input InpPC3_RR, configurable before attaching the EA.
77	input double	InpPC3_Percent	= 40.0;	Defines MT5 input InpPC3_Percent, configurable before attaching the EA.
79	input group "BREAK EVEN"			Starts a user input group shown in MT5 EA settings.
80	input int	InpBreakEvenPlusPoints	= 1;	Defines MT5 input InpBreakEvenPlusPoints, configurable before attaching the EA.
81	input double	InpBreakEvenExitExtraMoney	= 0.0;	Defines MT5 input InpBreakEvenExitExtraMoney, configurable before attaching the EA.
83	input group "PANEL"			Starts a user input group shown in MT5 EA settings.
84	input int	InpPanelX	= 10;	Defines MT5 input InpPanelX, configurable before attaching the EA.
85	input int	InpPanelY	= 10;	Defines MT5 input InpPanelY, configurable before attaching the EA.
86	input int	InpPanelW	= 950;	Defines MT5 input InpPanelW, configurable before attaching the EA.
87	input bool	InpAutoFitPanelWidth	= true;	Defines MT5 input InpAutoFitPanelWidth, configurable before attaching the EA.
90	input group "SUPABASE WEB CONTROL"			Starts a user input group shown in MT5 EA settings.
91	input bool	InpWebControlEnabled	= true;	Defines MT5 input InpWebControlEnabled, configurable before attaching the EA.
92	input string	InpSupabaseFunctionsUrl	= "https://YOUR_PROJECT_REF.functions.supabase.co";	Defines MT5 input InpSupabaseFunctionsUrl, configurable before attaching the EA.
93	input string	InpSupabaseEald	= "PASTE_EA_ID_FROM_WEBSITE";	Defines MT5 input InpSupabaseEald, configurable before attaching the EA.
94	input string	InpSupabaseEaToken	= "PASTE_EA_SECRET_TOKEN_FROM_WEBSITE";	Defines MT5 input InpSupabaseEaToken, configurable before attaching the EA.
95	input int	InpWebPollMilliseconds	= 200;	Defines MT5 input InpWebPollMilliseconds, configurable before attaching the EA.
96	input int	InpWebTimeoutMs	= 1500;	Defines MT5 input InpWebTimeoutMs, configurable before attaching the EA.
97	input bool	InpWebPostLiveState	= true;	Defines MT5 input InpWebPostLiveState, configurable before attaching the EA.
98	input bool	InpWebAllowCloseCommands	= true;	Defines MT5 input InpWebAllowCloseCommands, configurable before attaching the EA.

Website Technical Map

The website is a Vite React application. It connects to Supabase using the public anon key, then calls Edge Functions with the signed-in user's JWT.

Line	Function or component	Purpose
267	n	Utility, UI component, or calculation helper in the React website.
273	money	Utility, UI component, or calculation helper in the React website.
280	moneyPlain	Utility, UI component, or calculation helper in the React website.
286	pct	Utility, UI component, or calculation helper in the React website.
290	clsPL	Utility, UI component, or calculation helper in the React website.
294	statusClass	Utility, UI component, or calculation helper in the React website.
303	formatClock	Utility, UI component, or calculation helper in the React website.
310	formatHistoryTime	Utility, UI component, or calculation helper in the React website.
318	nearNumber	Utility, UI component, or calculation helper in the React website.
324	clampNumber	Utility, UI component, or calculation helper in the React website.
330	moneyValue	Utility, UI component, or calculation helper in the React website.
336	rulePct	Utility, UI component, or calculation helper in the React website.
343	numberValue	Utility, UI component, or calculation helper in the React website.
348	avg	Utility, UI component, or calculation helper in the React website.
354	pad2	Utility, UI component, or calculation helper in the React website.
358	dateKeyFromSeconds	Utility, UI component, or calculation helper in the React website.
364	displayDateKey	Utility, UI component, or calculation helper in the React website.
371	historySeconds	Utility, UI component, or calculation helper in the React website.
381	durationParts	Utility, UI component, or calculation helper in the React website.
394	tradeR	Utility, UI component, or calculation helper in the React website.
406	ringStatus	Utility, UI component, or calculation helper in the React website.
415	buildRuleMetrics	Utility, UI component, or calculation helper in the React website.
566	sha256Hex	Utility, UI component, or calculation helper in the React website.
571	token	Utility, UI component, or calculation helper in the React website.
580	uid	Utility, UI component, or calculation helper in the React website.
584	Login	Magic-link sign-in screen.
591	submit	Utility, UI component, or calculation helper in the React website.
637	EASetup	Creates an EA instance and secret token for MT5.
643	createEA	Utility, UI component, or calculation helper in the React website.
696	CopyRow	Utility, UI component, or calculation helper in the React website.
708	MissingConfig	Utility, UI component, or calculation helper in the React website.
738	CollapsibleSection	Utility, UI component, or calculation helper in the React website.
751	toggleOpen	Utility, UI component, or calculation helper in the React website.
782	Topbar	EA selector, refresh, edit, and navigation.
805	handleSave	Utility, UI component, or calculation helper in the React website.
815	toggleMenuOpen	Utility, UI component, or calculation helper in the React website.
890	StatusHero	Main live status summary.
936	StatCards	Utility, UI component, or calculation helper in the React website.
963	StepperInput	Utility, UI component, or calculation helper in the React website.
977	optimisticControlValue	Utility, UI component, or calculation helper in the React website.
983	pruneOptimisticControls	Utility, UI component, or calculation helper in the React website.
996	CommandPanel	Trade command and risk-control buttons.
1037	cmd	Utility, UI component, or calculation helper in the React website.
1059	updateRiskField	Utility, UI component, or calculation helper in the React website.
1064	updatePcField	Utility, UI component, or calculation helper in the React website.
1069	stepRiskField	Utility, UI component, or calculation helper in the React website.
1078	stepPcField	Utility, UI component, or calculation helper in the React website.
1083	sendRiskSettings	Utility, UI component, or calculation helper in the React website.
1094	sendPartialSettings	Utility, UI component, or calculation helper in the React website.
1105	setBooleanControl	Utility, UI component, or calculation helper in the React website.
1125	togglePartials	Utility, UI component, or calculation helper in the React website.

1129	toggleSecondEntry	Utility, UI component, or calculation helper in the React website.
1133	toggleFirstTrail	Utility, UI component, or calculation helper in the React website.
1137	firstBreakEven	Utility, UI component, or calculation helper in the React website.
1141	firstBreakEvenExit	Utility, UI component, or calculation helper in the React website.
1220	PositionPanel	Open-position details.
1224	sourceText	Utility, UI component, or calculation helper in the React website.
1261	StructurePanel	Candle/sweep structure display.
1297	StructureRow	Utility, UI component, or calculation helper in the React website.
1306	CommandsLog	Recent command status table.
1329	TelegramSettingsPage	Telegram bot, chat id, and notification preferences.
1377	setPref	Utility, UI component, or calculation helper in the React website.
1381	saveSettings	Utility, UI component, or calculation helper in the React website.
1412	getChatId	Utility, UI component, or calculation helper in the React website.
1453	disconnect	Utility, UI component, or calculation helper in the React website.
1575	RulesPage	Funding rules and account objective dashboard.
1937	ObjectiveCard	Utility, UI component, or calculation helper in the React website.
1955	AccountInfoRow	Utility, UI component, or calculation helper in the React website.
1965	StatValue	Utility, UI component, or calculation helper in the React website.
1974	AccountCurveChart	Utility, UI component, or calculation helper in the React website.
2018	DailySummaryTable	Utility, UI component, or calculation helper in the React website.
2040	RuleBar	Utility, UI component, or calculation helper in the React website.
2053	RulesTable	Utility, UI component, or calculation helper in the React website.
2077	TradeHistoryPage	Closed trade history table.
2152	Dashboard	Loads EA instances, state, commands, realtime channels, and polling.
2232	pollLiveData	Utility, UI component, or calculation helper in the React website.
2330	App	Top-level auth and app routing.

Important Website State

Name	Purpose
ACTIONS	Central list of command strings sent to Supabase.
LIVE_POLL_MS	Website live-state polling interval, currently 500 ms, with realtime subscription also active.
CONTROL_OPTIMISTIC_MS	How long a button keeps optimistic state while waiting for EA confirmation.
CONTROL_KEYS	The three independent control toggles: partialsOn, secondEntryOn, firstTrailOn.
controlStateRef	Holds latest optimistic button values so rapid clicks invert the visible current value, not stale server state.

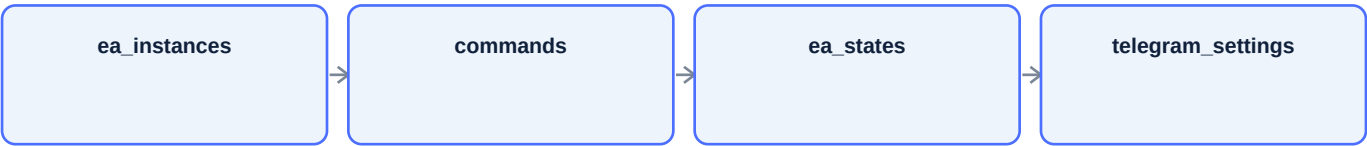
Button Command Reference

This table maps the website and MT5 buttons to command payloads and EA effects.

Button	Command or payload	EA handler
ARM BUY	ARM BUY	SetArmMode(ARM BUY), then TryArmedExecution waits for a fresh buy model.
ARM SELL	ARM_SELL	SetArmMode(ARM_SELL), then TryArmedExecution waits for a fresh sell model.
AUTO ARM	AUTO_ARM	SetAutoArm(true), so the next valid model can trade automatically.
CANCEL	CANCEL	Clears armed mode and auto-arm.
CLOSE 50%	CLOSE 50	CloseCurrentPercent(50.0), guarded by close-command input setting.
BREAK EVEN	BREAK_EVEN	MoveToBreakEven for all managed positions.
1ST BE	SET_PARTIALS { firstBreakEven: true }	MoveFirstTradeToBreakEven. This compatibility path avoids Invalid command.
1ST BE EXIT	SET_PARTIALS { firstBeExit: true }	CloseFirstTradeAtBreakEvenExit only when second trade exists and estimated net is not loss.
CLOSE ALL	CLOSE_ALL	CloseAllPositions for matching managed EA positions.
Partials	SET_PARTIALS { partialsOn }	Updates g_PC_On only.
2nd	SET_PARTIALS { secondEntryOn }	Updates g_SecondEntryOn only and syncs second-entry state.
1st Trail	SET_PARTIALS { firstTrailOn }	Updates g_FirstTrailOn only and syncs first-trail state.
Safe/Advanced	SET_MODE { mode }	Switches safe or advanced mode and partial behavior.
Send Risk Settings	SET_RISK { lot, risk, rr }	Updates lot, money risk, and RR target.
Send Partial Settings	SET_PARTIALS { pc1, pc2, pc3, partialsOn }	Updates partial close percentages and partials enabled flag.

Supabase and Database

Database Model



Additional tables include audit_logs, notification_events, telegram_ea_status, and profiles.

Table line	Table	Role
6	profiles	User profile rows tied to auth users.
13	ea_instances	One row per MT5 EA connection, including token hash and enabled state.
25	ea_states	Latest live state JSON posted by each EA.
32	commands	Command queue between website and EA with pending, sent, done, failed, or expired state.
47	audit_logs	Audit trail of important user/system actions.
56	telegram_settings	Encrypted Telegram bot token, chat id, enabled flag, and preferences.
82	notification_events	Notification delivery records for dedupe and troubleshooting.
97	telegram_ea_status	Online/offline status tracking for each EA.

Edge Function Roles

Function	Role
create-command	Browser entry point for control commands. Inserts command rows quickly and performs audit/Telegram work in background.
ea-next-command	EA polling endpoint. Authenticates EA token, expires stale commands, returns oldest pending command.
ea-ack-command	EA command result endpoint. Updates command done/failed and sends Telegram ack in background.
ea-post-state	EA state endpoint. Upserts latest dashboard state and sends online/state notifications in background.
telegram-settings	Stores encrypted bot token, chat id, enabled flag, and notification preferences.
telegram-chat-id	Helps detect the chat id from bot updates.
telegram-offline-monitor	Scheduled monitor that marks EAs offline after stale last_seen_at.

Performance and Reliability

- Website control toggles call cmd with lock=false, so another command does not freeze 2nd Entry, 1st Trail, or Partials.
- Optimistic controls are stored per key. Updating secondEntryOn no longer replaces firstTrailOn pending state.
- The 1ST BE button uses SET_PARTIALS payload compatibility, preventing Invalid command on deployed functions that do not yet include FIRST_BREAK_EVEN.
- EA polling minimum is 200 ms. This reduces pickup delay while still avoiding an uncontrolled request loop.
- EA network backoff after a failed web call is 500 ms, not 1500 ms, reducing recovery time after short connection failures.
- Command create, command ack, and state post endpoints return after their required database write. Telegram/audit work runs in EdgeRuntime.waitUntil.
- EA command handling marks state dirty and lets the normal loop post state after ack, so acknowledgement is not held by a forced state post.

Latency Budget

Segment	Expected behavior
Web click	Immediate visual response for toggles due optimistic state.
create-command	One authenticated insert plus ownership check; no Telegram wait.
EA pickup	Up to about 200 ms plus network time under healthy conditions.
EA apply	Local MQL logic runs immediately once the command arrives.
Ack and state	Ack first, then state post in the loop, both with notification work in background.

Deployment and Operations

- Run the SQL in supabase/schema.sql before first deployment.
- Deploy all Supabase functions after code changes, especially create-command, ea-next-command, ea-ack-command, ea-post-state, and telegram functions.
- Build the website with npm run build and deploy the dist output to the chosen host.
- Compile Three_Candle_EA_SUPABASE_WEB_FULL.mq5 in MetaEditor. The existing .ex5 in this folder is old until MetaEditor recompiles it.
- Attach the compiled EA to the MT5 chart and configure InpSupabaseFunctionsUrl, InpSupabaseEaId, and InpSupabaseEaToken.
- For fastest control, keep InpWebControlEnabled=true, InpWebPollMilliseconds=200, and confirm MT5 WebRequest allows the Supabase functions URL.

Troubleshooting

Symptom	Likely cause	Fix
Invalid command	Website sends action unsupported by deployed create-command.	Deploy functions or use compatibility SET_PARTIALS payloads as done for 1ST BE.
Website button flips back	EA state has not confirmed yet or old optimistic state was overwritten.	Current code stores pending controls separately and prunes only when live state matches.
MT5 cannot operate from web	Bad EA token/id, WebRequest URL missing, function not deployed, or EA disabled.	Check setup values, Supabase logs, ea_instances.enabled, and MT5 Experts tab.
Web cannot operate from MT5	State post blocked or wrong Supabase URL/token.	Check ea-post-state logs, last seen at, and MT5 WebRequest permissions.
Too slow	Function waits, network timeout, or EA polling interval too high.	Use this version: background notifications, 200 ms poll, 500 ms backoff.

Important Source Snippets

Website 1ST BE compatibility path

```

1137: function firstBreakEven() {
1138:     return cmd(ACTIONS.SET_PARTIALS, { firstBreakEven: true }, { lock: false })
1139: }
1140:
1141: function firstBreakEvenExit() {
1142:     return cmd(ACTIONS.SET_PARTIALS, { firstBeExit: true })
1143: }

```

Optimistic control state isolation

```

1105: async function setBooleanControl(key, value) {
1106:     const requestId = uid()
1107:     const now = Date.now()
1108:     controlStateRef.current = { ...controlStateRef.current, [key]: value }
1109:     setOptimisticControls(prev => ({
1110:         ...pruneOptimisticControls(prev, s, now),
1111:         [key]: { value, at: now, requestId }
1112:     }))
1113:     const ok = await cmd(ACTIONS.SET_PARTIALS, { [key]: value, request_id: requestId }, { lock: false })
1114:     if (!ok) {
1115:         setOptimisticControls(prev => {
1116:             if (prev[key]?.requestId !== requestId) return prev
1117:             const copy = { ...prev }
1118:             delete copy[key]
1119:             return copy
1120:         })
1121:     }
1122:     return ok

```

Fast create-command background work

```

0044:     client_id,
0045:     action,
0046:     payload: safePayload,
0047:     status: 'pending',
0048:     expires_at: new Date(Date.now() + 45_000).toISOString()
0049: }).select().single()
0050: if (error) throw error
0051:
0052: const backgroundTasks = Promise.allSettled([
0053:     admin.from('audit_logs').insert({ user_id:userData.user.id, ea_id, action:'create_command',
details:{ command_id:data.id, action } })),
0054:     notifyCommandCreated(admin, data, ea)
0055: ]).then((results) => {
0056:     const labels = ['audit log', 'telegram command notification']
0057:     results.forEach((result, index) => {
0058:         if (result.status === 'rejected') console.error(`${labels[index]} failed`, result.reason)
0059:     })
0060: })
0061: ;(globalThis as any).EdgeRuntime?.waitUntil?.(backgroundTasks)
0062: return json({ ok:true, command:data })

```

EA command poll timing

```

4681: void TCX_SupabasePollCommands()
4682: {
4683:     if(!TCX_WebReady())
4684:         return;
4685:
4686:     uint nowTick=GetTickCount();
4687:     int ms=MathMax(200,InpWebPollMilliseconds);
4688:     if(g_WebLastPollTick>0 && (uint)(nowTick-g_WebLastPollTick)<(uint)ms)
4689:         return;
4690:     g_WebLastPollTick=nowTick;

```

EA command ack and dirty state

```

4658:     SetLastMessage(msg,clrTomato);
4659:     return false;
4660: }
4661:
4662: g_WebLastCommandId=id;
4663: UpdatePanel();
4664: g_WebStateDirty=true;
4665: return true;

```


4666: }

Appendix A - EA Line-by-Line Source Explanation

This appendix covers every source line in Three_Candle_EA_SUPABASE_WEB_FULLL.mq5. It includes line number, exact source text, and a concise purpose statement generated from the code context.

Total lines: 5155.

Line	EA source	Explanation
1	//+-----+	Banner comment marking a major EA section.
2	/// Three Candle Execution Controller	Comment explaining the nearby EA code.
3	/// Indicator-matched model detection with armed BUY/SELL execution	Comment explaining the nearby EA code.
4	//+-----+	Banner comment marking a major EA section.
5	#property copyright "Three Candle Execution Controller"	EA compiler metadata or strict compilation setting.
6	#property version "1.298"	EA compiler metadata or strict compilation setting.
7	#property description "Attach to a chart as an Expert Advisor. BUY/SELL buttons arm the next matching 3-candle model."	EA compiler metadata or strict compilation setting.
8	#property strict	EA compiler metadata or strict compilation setting.
9		Blank separator for readability.
10	#include <Trade\Trade.mqh>	Imports a MetaTrader include library.
11		Blank separator for readability.
12	CTrade Trade;	Executes one statement.
13		Blank separator for readability.
14	enum ENUM_TC_STOP_MODE	Starts an enumeration of named constants.
15	{	Opens or closes a code block.
16	STOP_FROM_CANDLE2_LINE = 0, // Matches indicator C2 low/high line	Part of the EA logic.
17	STOP_FROM_PATTERN_EXTREME = 1 // Uses lowest/highest of all 3 model candles	Part of the EA logic.
18	};	Executes one statement.
19		Blank separator for readability.
20	//=====	Comment explaining the nearby EA code.
21	// INPUTS	Comment explaining the nearby EA code.
22	//=====	Comment explaining the nearby EA code.
23	input group "TRADE"	Starts a user input group shown in MT5 EA settings.
24	input double InpLot = 0.01;	Defines MT5 input InpLot, configurable before attaching the EA.
25	input double InpRiskReward = 3.0;	Defines MT5 input InpRiskReward, configurable before attaching the EA.
26	input double InpRiskMoney = 100.0;	Defines MT5 input InpRiskMoney, configurable before attaching the EA.
27	input double InpRiskMoneyButtonStep = 10.0;	Defines MT5 input InpRiskMoneyButtonStep, configurable before attaching the EA.
28	input int InpMagic = 55555;	Defines MT5 input InpMagic, configurable before attaching the EA.
29	input int InpDeviationPoints = 30;	Defines MT5 input InpDeviationPoints, configurable before attaching the EA.
30	input int InpStopBufferPoints = 0;	Defines MT5 input InpStopBufferPoints, configurable before attaching the EA.
31	input ENUM_TC_STOP_MODE InpStopMode = STOP_FROM_PATTERN_EXTREME;	Defines MT5 input InpStopMode, configurable before attaching the EA.
32	input bool InpFallbackNoInitialSLTP = true;	Defines MT5 input InpFallbackNoInitialSLTP, configurable before attaching the EA.
33	input bool InpCloseIfProtectionFails = true;	Defines MT5 input InpCloseIfProtectionFails, configurable before attaching the EA.
34	input int InpMaxEntriesPerCycle = 2;	Defines MT5 input InpMaxEntriesPerCycle, configurable before attaching the EA.
35	input double InpDailyLimitR = 2.0;	Defines MT5 input InpDailyLimitR, configurable before attaching the EA.
36	input int InpDailyLossLimitCount = 2;	Defines MT5 input InpDailyLossLimitCount, configurable before attaching the EA.
37		Blank separator for readability.
38	input group "PATTERN"	Starts a user input group shown in MT5 EA settings.
39	input bool InpAllowSweep = true;	Defines MT5 input InpAllowSweep, configurable before attaching the EA.
40	input bool InpAllowSwingPattern = true;	Defines MT5 input InpAllowSwingPattern, configurable before attaching the EA.
41	input bool InpShowSignalArrows = true;	Defines MT5 input InpShowSignalArrows, configurable before attaching the EA.
42	input int InpSignalLookbackBars = 150;	Defines MT5 input InpSignalLookbackBars, configurable before attaching the EA.
43		Blank separator for readability.
44	input group "DASHBOARD"	Starts a user input group shown in MT5 EA settings.
45	input bool InpShowManualTradesOnDashboard = true;	Defines MT5 input InpShowManualTradesOnDashboard, configurable before attaching the EA.
46		Blank separator for readability.
47	input group "FILTERS"	Starts a user input group shown in MT5 EA settings.
48	input bool InpUseSpreadFilter = false;	Defines MT5 input InpUseSpreadFilter, configurable before attaching the EA.
49	input int InpMaxSpreadPoints = 5000;	Defines MT5 input InpMaxSpreadPoints, configurable before attaching the EA.
50	input bool InpBlockTradeOnHighSpread = false;	Defines MT5 input InpBlockTradeOnHighSpread, configurable before attaching the EA.
51		Blank separator for readability.
52	input group "SPREAD SL/TP SPACE"	Starts a user input group shown in MT5 EA settings.
53	input bool InpUseSpreadProtection = true;	Defines MT5 input InpUseSpreadProtection, configurable before attaching the EA.
54	input double InpSpreadProtectionMultiplier = 1.0;	Defines MT5 input InpSpreadProtectionMultiplier, configurable before attaching the EA.
55	input int InpExtraProtectionPoints = 0;	Defines MT5 input InpExtraProtectionPoints, configurable before attaching the EA.
56		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
57	input group "SECOND ENTRY"	Starts a user input group shown in MT5 EA settings.
58	input bool InpSecondEntryEnabled = false;	Defines MT5 input InpSecondEntryEnabled, configurable before attaching the EA.
59	input double InpSecondEntryTriggerLossR = 0.5;	Defines MT5 input InpSecondEntryTriggerLossR, configurable before attaching the EA.
60	input double InpSecondEntryCancelAtR = 2.0;	Defines MT5 input InpSecondEntryCancelAtR, configurable before attaching the EA.
61		Blank separator for readability.
62	input group "FIRST TRADE TRAILING"	Starts a user input group shown in MT5 EA settings.
63	input bool InpFirstTrailEnabled = false;	Defines MT5 input InpFirstTrailEnabled, configurable before attaching the EA.
64		Blank separator for readability.
65	input group "LOT PANEL"	Starts a user input group shown in MT5 EA settings.
66	input double InpLotButtonStep = 0.01;	Defines MT5 input InpLotButtonStep, configurable before attaching the EA.
67	input double InpPanelLotMax = 100.0;	Defines MT5 input InpPanelLotMax, configurable before attaching the EA.
68	input bool InpClampLotToBrokerMax = true;	Defines MT5 input InpClampLotToBrokerMax, configurable before attaching the EA.
69		Blank separator for readability.
70	input group "PARTIAL CLOSE"	Starts a user input group shown in MT5 EA settings.
71	input bool InpPartialCloseEnabled = false;	Defines MT5 input InpPartialCloseEnabled, configurable before attaching the EA.
72	input double InpPC1_RR = 1.0;	Defines MT5 input InpPC1_RR, configurable before attaching the EA.
73	input double InpPC1_Percent = 30.0;	Defines MT5 input InpPC1_Percent, configurable before attaching the EA.
74	input double InpPC2_RR = 2.0;	Defines MT5 input InpPC2_RR, configurable before attaching the EA.
75	input double InpPC2_Percent = 30.0;	Defines MT5 input InpPC2_Percent, configurable before attaching the EA.
76	input double InpPC3_RR = 3.0;	Defines MT5 input InpPC3_RR, configurable before attaching the EA.
77	input double InpPC3_Percent = 40.0;	Defines MT5 input InpPC3_Percent, configurable before attaching the EA.
78		Blank separator for readability.
79	input group "BREAK EVEN"	Starts a user input group shown in MT5 EA settings.
80	input int InpBreakEvenPlusPoints = 1;	Defines MT5 input InpBreakEvenPlusPoints, configurable before attaching the EA.
81	input double InpBreakEvenExitExtraMoney = 0.0;	Defines MT5 input InpBreakEvenExitExtraMoney, configurable before attaching the EA.
82		Blank separator for readability.
83	input group "PANEL"	Starts a user input group shown in MT5 EA settings.
84	input int InpPanelX = 10;	Defines MT5 input InpPanelX, configurable before attaching the EA.
85	input int InpPanelY = 10;	Defines MT5 input InpPanelY, configurable before attaching the EA.
86	input int InpPanelW = 950;	Defines MT5 input InpPanelW, configurable before attaching the EA.
87	input bool InpAutoFitPanelWidth = true;	Defines MT5 input InpAutoFitPanelWidth, configurable before attaching the EA.
88		Blank separator for readability.
89		Blank separator for readability.
90	input group "SUPABASE WEB CONTROL"	Starts a user input group shown in MT5 EA settings.
91	input bool InpWebControlEnabled = true;	Defines MT5 input InpWebControlEnabled, configurable before attaching the EA.
92	input string InpSupabaseFunctionsUrl = "https://YOUR_PROJECT_REF.functions.supabase.co";	Defines MT5 input InpSupabaseFunctionsUrl, configurable before attaching the EA.
93	input string InpSupabaseEaId = "PASTE_EA_ID_FROM_WEBSITE";	Defines MT5 input InpSupabaseEaId, configurable before attaching the EA.
94	input string InpSupabaseEaToken = "PASTE_EA_SECRET_TOKEN_FROM_WEBSITE";	Defines MT5 input InpSupabaseEaToken, configurable before attaching the EA.
95	input int InpWebPollMilliseconds = 200;	Defines MT5 input InpWebPollMilliseconds, configurable before attaching the EA.
96	input int InpWebTimeoutMs = 1500;	Defines MT5 input InpWebTimeoutMs, configurable before attaching the EA.
97	input bool InpWebPostLiveState = true;	Defines MT5 input InpWebPostLiveState, configurable before attaching the EA.
98	input bool InpWebAllowCloseCommands = true;	Defines MT5 input InpWebAllowCloseCommands, configurable before attaching the EA.
99		Blank separator for readability.
100	//=====	Comment explaining the nearby EA code.
101	// CONSTANTS / OBJECT NAMES	Comment explaining the nearby EA code.
102	//=====	Comment explaining the nearby EA code.
103	#define ARM_NONE 0	Defines constant ARM_NONE used later by the EA.
104	#define ARM_BUY 1	Defines constant ARM_BUY used later by the EA.
105	#define ARM_SELL 2	Defines constant ARM_SELL used later by the EA.
106		Blank separator for readability.
107	string PX = "TCX_";	Updates or initializes string PX.
108	string UI = "TCX_UI_";	Updates or initializes string UI.
109		Blank separator for readability.
110	#define B_BUY "TCX_UI_B_BUY"	Defines constant B_BUY used later by the EA.
111	#define B_SELL "TCX_UI_B_SELL"	Defines constant B_SELL used later by the EA.
112	#define B_AUTO "TCX_UI_B_AUTO"	Defines constant B_AUTO used later by the EA.
113	#define B_CANCEL "TCX_UI_B_CANCEL"	Defines constant B_CANCEL used later by the EA.
114	#define B_CLOSE "TCX_UI_B_CLOSE"	Defines constant B_CLOSE used later by the EA.
115	#define B_CLOSE_ALL "TCX_UI_B_CLOSE_ALL"	Defines constant B_CLOSE_ALL used later by the EA.
116	#define B_CLOSE50 "TCX_UI_B_CLOSE50"	Defines constant B_CLOSE50 used later by the EA.
117	#define B_BE "TCX_UI_B_BE"	Defines constant B_BE used later by the EA.
118	#define B_PC "TCX_UI_B_PC"	Defines constant B_PC used later by the EA.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
119	#define B_LOT_UP "TCX_UI_B_LOT_UP"	Defines constant B_LOT_UP used later by the EA.
120	#define B_LOT_DN "TCX_UI_B_LOT_DN"	Defines constant B_LOT_DN used later by the EA.
121	#define B_RISK_UP "TCX_UI_B_RISK_UP"	Defines constant B_RISK_UP used later by the EA.
122	#define B_RISK_DN "TCX_UI_B_RISK_DN"	Defines constant B_RISK_DN used later by the EA.
123	#define B_RR_UP "TCX_UI_B_RR_UP"	Defines constant B_RR_UP used later by the EA.
124	#define B_RR_DN "TCX_UI_B_RR_DN"	Defines constant B_RR_DN used later by the EA.
125	#define B_MODE_SAFE "TCX_UI_B_MODE_SAFE"	Defines constant B_MODE_SAFE used later by the EA.
126	#define B_MODE_ADV "TCX_UI_B_MODE_ADV"	Defines constant B_MODE_ADV used later by the EA.
127	#define B_SECOND "TCX_UI_B_SECOND"	Defines constant B_SECOND used later by the EA.
128	#define B_FIRST_TRAIL "TCX_UI_B_FIRST_TRAIL"	Defines constant B_FIRST_TRAIL used later by the EA.
129	#define B_FIRST_BE "TCX_UI_B_FIRST_BE"	Defines constant B_FIRST_BE used later by the EA.
130	#define B_FIRST_BE_EXIT "TCX_UI_B_FIRST_BE_EXIT"	Defines constant B_FIRST_BE_EXIT used later by the EA.
131		Blank separator for readability.
132	#define E_LOT "TCX_UI_E_LOT"	Defines constant E_LOT used later by the EA.
133	#define E_RISK "TCX_UI_E_RISK"	Defines constant E_RISK used later by the EA.
134	#define E_PC1 "TCX_UI_E_PC1"	Defines constant E_PC1 used later by the EA.
135	#define E_PC2 "TCX_UI_E_PC2"	Defines constant E_PC2 used later by the EA.
136	#define E_PC3 "TCX_UI_E_PC3"	Defines constant E_PC3 used later by the EA.
137		Blank separator for readability.
138	#define EMERGENCY_LABEL "TCX_EMERGENCY_STATUS"	Defines constant EMERGENCY_LABEL used later by the EA.
139		Blank separator for readability.
140	bool HasOurPosition();	Declares or starts function HasOurPosition.
141		Blank separator for readability.
142	//=====	Comment explaining the nearby EA code.
143	// GLOBAL STATE	Comment explaining the nearby EA code.
144	//=====	Comment explaining the nearby EA code.
145	int g_ArmedMode = ARM_NONE;	Updates or initializes int g_ArmedMode in HasOurPosition.
146	datetime g_LastBarTime = 0;	Updates or initializes datetime g_LastBarTime in HasOurPosition.
147	bool g_PanelBuilt = false;	Updates or initializes bool g_PanelBuilt in HasOurPosition.
148	datetime g_InitTime = 0;	Updates or initializes datetime g_InitTime in HasOurPosition.
149	datetime g_ArmMinSignalTime = 0;	Updates or initializes datetime g_ArmMinSignalTime in HasOurPosition.
150	datetime g_LastTradeSignalTime = 0;	Updates or initializes datetime g_LastTradeSignalTime in HasOurPosition.
151	datetime g_LastTradeScanBarTime = 0;	Updates or initializes datetime g_LastTradeScanBarTime in HasOurPosition.
152		Blank separator for readability.
153	double g_Lot = 0.01;	Updates or initializes double g_Lot in HasOurPosition.
154	double g_RRTarget = 3.0;	Updates or initializes double g_RRTarget in HasOurPosition.
155	double g_RiskMoney = 100.0;	Updates or initializes double g_RiskMoney in HasOurPosition.
156	bool g_PC_On = false;	Updates or initializes bool g_PC_On in HasOurPosition.
157	bool g_AutoArm = false;	Updates or initializes bool g_AutoArm in HasOurPosition.
158	bool g_AdvancedMode = false;	Updates or initializes bool g_AdvancedMode in HasOurPosition.
159	bool g_SecondEntryOn = false;	Updates or initializes bool g_SecondEntryOn in HasOurPosition.
160	bool g_FirstTrailOn = false;	Updates or initializes bool g_FirstTrailOn in HasOurPosition.
161	double g_PC1_Pct = 30.0;	Updates or initializes double g_PC1_Pct in HasOurPosition.
162	double g_PC2_Pct = 30.0;	Updates or initializes double g_PC2_Pct in HasOurPosition.
163	double g_PC3_Pct = 40.0;	Updates or initializes double g_PC3_Pct in HasOurPosition.
164		Blank separator for readability.
165	ulong g_StateTicket = 0;	Updates or initializes ulong g_StateTicket in HasOurPosition.
166	double g_InitialVolume = 0.0;	Updates or initializes double g_InitialVolume in HasOurPosition.
167	double g_RiskDistance = 0.0;	Updates or initializes double g_RiskDistance in HasOurPosition.
168	bool g_PC1_Done = false;	Updates or initializes bool g_PC1_Done in HasOurPosition.
169	bool g_PC2_Done = false;	Updates or initializes bool g_PC2_Done in HasOurPosition.
170	bool g_PC3_Done = false;	Updates or initializes bool g_PC3_Done in HasOurPosition.
171	bool g_BE_Done = false;	Updates or initializes bool g_BE_Done in HasOurPosition.
172		Blank separator for readability.
173	ulong g_SecondBaseTicket = 0;	Updates or initializes ulong g_SecondBaseTicket in HasOurPosition.
174	bool g_SecondEntryDone = false;	Updates or initializes bool g_SecondEntryDone in HasOurPosition.
175	bool g_SecondEntryBlocked = false;	Updates or initializes bool g_SecondEntryBlocked in HasOurPosition.
176	datetime g_SecondEntryLastAttempt = 0;	Updates or initializes datetime g_SecondEntryLastAttempt in HasOurPosition.
177		Blank separator for readability.
178	ulong g_FirstTrailTicket = 0;	Updates or initializes ulong g_FirstTrailTicket in HasOurPosition.
179	bool g_FirstTrailActive = false;	Updates or initializes bool g_FirstTrailActive in HasOurPosition.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
180	datetime g_FirstTrailLastM5BarTime = 0;	Updates or initializes datetime g_FirstTrailLastM5BarTime in HasOurPosition.
181		Blank separator for readability.
182	string g_LastMessage = "Ready";	Updates or initializes string g_LastMessage in HasOurPosition.
183	color g_LastMsgColor = clrSilver;	Updates or initializes color g_LastMsgColor in HasOurPosition.
184		Blank separator for readability.
185	int g_X = 10;	Updates or initializes int g_X in HasOurPosition.
186	int g_Y = 20;	Updates or initializes int g_Y in HasOurPosition.
187	int g_W = 1080;	Updates or initializes int g_W in HasOurPosition.
188	int g_H = 1360;	Updates or initializes int g_H in HasOurPosition.
189		Blank separator for readability.
190		Blank separator for readability.
191	//=====	Comment explaining the nearby EA code.
192	// SUPABASE WEB STATE	Comment explaining the nearby EA code.
193	//=====	Comment explaining the nearby EA code.
194	datetime g_WebLastPollSecond = 0;	Updates or initializes datetime g_WebLastPollSecond in HasOurPosition.
195	datetime g_WebLastPostSecond = 0;	Updates or initializes datetime g_WebLastPostSecond in HasOurPosition.
196	uint g_WebLastPollTick = 0;	Updates or initializes uint g_WebLastPollTick in HasOurPosition.
197	uint g_WebLastPostTick = 0;	Updates or initializes uint g_WebLastPostTick in HasOurPosition.
198	uint g_WebBackoffUntilTick = 0;	Updates or initializes uint g_WebBackoffUntilTick in HasOurPosition.
199	bool g_WebConnected = false;	Updates or initializes bool g_WebConnected in HasOurPosition.
200	string g_WebLastError = "";	Updates or initializes string g_WebLastError in HasOurPosition.
201	string g_WebLastCommandId = "";	Updates or initializes string g_WebLastCommandId in HasOurPosition.
202	bool g_WebStateDirty = false;	Updates or initializes bool g_WebStateDirty in HasOurPosition.
203		Blank separator for readability.
204	//=====	Comment explaining the nearby EA code.
205	// STRUCTS	Comment explaining the nearby EA code.
206	//=====	Comment explaining the nearby EA code.
207	struct PatternSignal	Starts a data structure used to group related values.
208	{	Opens or closes a code block in HasOurPosition.
209	bool ok;	Executes one statement in HasOurPosition.
210	bool buy;	Executes one statement in HasOurPosition.
211	bool swing;	Executes one statement in HasOurPosition.
212	datetime time;	Executes one statement in HasOurPosition.
213	double c1o;	Executes one statement in HasOurPosition.
214	double c1c;	Executes one statement in HasOurPosition.
215	double c1h;	Executes one statement in HasOurPosition.
216	double c1l;	Executes one statement in HasOurPosition.
217	double c2o;	Executes one statement in HasOurPosition.
218	double c2c;	Executes one statement in HasOurPosition.
219	double c2h;	Executes one statement in HasOurPosition.
220	double c2l;	Executes one statement in HasOurPosition.
221	double c3o;	Executes one statement in HasOurPosition.
222	double c3c;	Executes one statement in HasOurPosition.
223	double c3h;	Executes one statement in HasOurPosition.
224	double c3l;	Executes one statement in HasOurPosition.
225	double s1;	Executes one statement in HasOurPosition.
226	double breakLevel;	Executes one statement in HasOurPosition.
227	};	Executes one statement in HasOurPosition.
228		Blank separator for readability.
229	//=====	Comment explaining the nearby EA code.
230	// BASIC HELPERS	Comment explaining the nearby EA code.
231	//=====	Comment explaining the nearby EA code.
232	void LogMsg(string tag,string text)	Declares or starts function LogMsg.
233	{	Opens or closes a code block in LogMsg.
234	Print("[TCX]","",tag," ",text);	Calls helper function Print in LogMsg.
235	}	Opens or closes a code block in LogMsg.
236		Blank separator for readability.
237	void SetLastMessage(string text,color clr)	Declares or starts function SetLastMessage.
238	{	Opens or closes a code block in SetLastMessage.
239	g_LastMessage=text;	Updates or initializes g_LastMessage in SetLastMessage.
240	g_LastMsgColor=clr;	Updates or initializes g_LastMsgColor in SetLastMessage.
241	g_WebStateDirty=true;	Updates or initializes g_WebStateDirty in SetLastMessage.
242	LogMsg("STATUS",text);	Calls helper function LogMsg in SetLastMessage.
243	}	Opens or closes a code block in SetLastMessage.
244		Blank separator for readability.
245	bool TradeRetcodeOK()	Declares or starts function TradeRetcodeOK.
246	{	Opens or closes a code block in TradeRetcodeOK.
247	uint rc=Trade.ResultRetcode();	Executes or modifies a trade through the MetaTrader trade API.
248	return (rc==TRADE_RETCODE_DONE	Returns a value or exits the current function in TradeRetcodeOK.
249	rc==TRADE_RETCODE_DONE_PARTIAL	Part of the EA logic in TradeRetcodeOK.
250	rc==TRADE_RETCODE_PLACED);	Updates or initializes rc in TradeRetcodeOK.
251	}	Opens or closes a code block in TradeRetcodeOK.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
252		Blank separator for readability.
253	string ClipText(string text,int maxlen)	Declares or starts function ClipText.
254	{	Opens or closes a code block in ClipText.
255	if(StringLen(text)<=maxLen)	Checks a condition and runs the guarded branch in ClipText.
256	return text;	Returns a value or exits the current function in ClipText.
257	if(maxLen<=3)	Checks a condition and runs the guarded branch in ClipText.
258	return StringSubstr(text,0,maxLen);	Returns a value or exits the current function in ClipText.
259	return StringSubstr(text,0,maxLen-3)+"...";	Returns a value or exits the current function in ClipText.
260	}	Opens or closes a code block in ClipText.
261		Blank separator for readability.
262	string ArmStatusText()	Declares or starts function ArmStatusText.
263	{	Opens or closes a code block in ArmStatusText.
264	if(g_AutoArm)	Checks a condition and runs the guarded branch in ArmStatusText.
265	return "AUTO ARM";	Returns a value or exits the current function in ArmStatusText.
266	if(g_ArmedMode==ARM_BUY)	Checks a condition and runs the guarded branch in ArmStatusText.
267	return "WAITING BUY MODEL";	Returns a value or exits the current function in ArmStatusText.
268	if(g_ArmedMode==ARM_SELL)	Checks a condition and runs the guarded branch in ArmStatusText.
269	return "WAITING SELL MODEL";	Returns a value or exits the current function in ArmStatusText.
270	if(HasOurPosition())	Checks a condition and runs the guarded branch in ArmStatusText.
271	return "POSITION OPEN";	Returns a value or exits the current function in ArmStatusText.
272	return "IDLE";	Returns a value or exits the current function in ArmStatusText.
273	}	Opens or closes a code block in ArmStatusText.
274		Blank separator for readability.
275	void DrawEmergencyStatus()	Declares or starts function DrawEmergencyStatus.
276	{	Opens or closes a code block in DrawEmergencyStatus.
277	if(ObjectFind(0,EMERGENCY_LABEL)<0)	Checks a condition and runs the guarded branch in DrawEmergencyStatus.
278	{	Opens or closes a code block in DrawEmergencyStatus.
279	ObjectCreate(0,EMERGENCY_LABEL,OBJ_LABEL,0,0,0);	Creates, updates, or removes an MT5 chart panel object.
280	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_CORNER,CORNER_LEFT_UPPER);	Creates, updates, or removes an MT5 chart panel object.
281	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_XDISTANCE,12);	Creates, updates, or removes an MT5 chart panel object.
282	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_YDISTANCE,12);	Creates, updates, or removes an MT5 chart panel object.
283	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_FONTSIZE,11);	Creates, updates, or removes an MT5 chart panel object.
284	ObjectSetString(0,EMERGENCY_LABEL,OBJPROP_FONT,"Arial Bold");	Creates, updates, or removes an MT5 chart panel object.
285	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_SELECTABLE,false);	Creates, updates, or removes an MT5 chart panel object.
286	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_HIDDEN,true);	Creates, updates, or removes an MT5 chart panel object.
287	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_BACK,false);	Creates, updates, or removes an MT5 chart panel object.
288	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_ZORDER,1000);	Creates, updates, or removes an MT5 chart panel object.
289	}	Opens or closes a code block in DrawEmergencyStatus.
290		Blank separator for readability.
291	string text="TCX EA RUNNING "+_Symbol+" "+EnumToString(_Period)+" "+ArmStatusText();	Updates or initializes string text in DrawEmergencyStatus.
292	ObjectSetString(0,EMERGENCY_LABEL,OBJPROP_TEXT,text);	Creates, updates, or removes an MT5 chart panel object.
293	ObjectSetInteger(0,EMERGENCY_LABEL,OBJPROP_COLOR,clrLime);	Creates, updates, or removes an MT5 chart panel object.
294	}	Opens or closes a code block in DrawEmergencyStatus.
295		Blank separator for readability.
296	void ChartHeartbeat()	Declares or starts function ChartHeartbeat.
297	{	Opens or closes a code block in ChartHeartbeat.
298	Comment("");	Calls helper function Comment in ChartHeartbeat.
299	ObjectDelete(0,EMERGENCY_LABEL);	Creates, updates, or removes an MT5 chart panel object.
300	}	Opens or closes a code block in ChartHeartbeat.
301		Blank separator for readability.
302	double BrokerMinLot()	Declares or starts function BrokerMinLot.
303	{	Opens or closes a code block in BrokerMinLot.
304	double v=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MIN);	Updates or initializes double v in BrokerMinLot.
305	if(v<=0.0)	Checks a condition and runs the guarded branch in BrokerMinLot.
306	v=0.01;	Updates or initializes v in BrokerMinLot.
307	return v;	Returns a value or exits the current function in BrokerMinLot.
308	}	Opens or closes a code block in BrokerMinLot.
309		Blank separator for readability.
310	double BrokerMaxLot()	Declares or starts function BrokerMaxLot.
311	{	Opens or closes a code block in BrokerMaxLot.
312	double v=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MAX);	Updates or initializes double v in BrokerMaxLot.
313	if(v<=0.0)	Checks a condition and runs the guarded branch in BrokerMaxLot.
314	v=100.0;	Updates or initializes v in BrokerMaxLot.
315	return v;	Returns a value or exits the current function in BrokerMaxLot.
316	}	Opens or closes a code block in BrokerMaxLot.
317		Blank separator for readability.
318	double BrokerStepLot()	Declares or starts function BrokerStepLot.
319	{	Opens or closes a code block in BrokerStepLot.
320	double v=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_STEP);	Updates or initializes double v in BrokerStepLot.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
321	if(v<=0.0)	Checks a condition and runs the guarded branch in BrokerStepLot.
322	v=0.01;	Updates or initializes v in BrokerStepLot.
323	return v;	Returns a value or exits the current function in BrokerStepLot.
324	}	Opens or closes a code block in BrokerStepLot.
325		Blank separator for readability.
326	int VolumeDigits()	Declares or starts function VolumeDigits.
327	{	Opens or closes a code block in VolumeDigits.
328	double step=BrokerStepLot();	Updates or initializes double step in VolumeDigits.
329	int digits=0;	Updates or initializes int digits in VolumeDigits.
330		Blank separator for readability.
331	while(digits<8 && MathAbs(step-NormalizeDouble(step,digits))>0.000000001)	Repeats while the condition remains true in VolumeDigits.
332	digits++;	Executes one statement in VolumeDigits.
333		Blank separator for readability.
334	return digits;	Returns a value or exits the current function in VolumeDigits.
335	}	Opens or closes a code block in VolumeDigits.
336		Blank separator for readability.
337	double NormalizeVolumeDown(double volume)	Declares or starts function NormalizeVolumeDown.
338	{	Opens or closes a code block in NormalizeVolumeDown.
339	double step=BrokerStepLot();	Updates or initializes double step in NormalizeVolumeDown.
340	if(step<=0.0)	Checks a condition and runs the guarded branch in NormalizeVolumeDown.
341	step=0.01;	Updates or initializes step in NormalizeVolumeDown.
342		Blank separator for readability.
343	volume=MathFloor((volume/step)+0.000000001)*step;	Updates or initializes volume in NormalizeVolumeDown.
344	volume=NormalizeDouble(volume,VolumeDigits());	Updates or initializes volume in NormalizeVolumeDown.
345		Blank separator for readability.
346	if(volume+0.000000001<BrokerMinLot())	Checks a condition and runs the guarded branch in NormalizeVolumeDown.
347	return 0.0;	Returns a value or exits the current function in NormalizeVolumeDown.
348		Blank separator for readability.
349	if(volume>BrokerMaxLot())	Checks a condition and runs the guarded branch in NormalizeVolumeDown.
350	volume=BrokerMaxLot();	Updates or initializes volume in NormalizeVolumeDown.
351		Blank separator for readability.
352	return NormalizeDouble(volume,VolumeDigits());	Returns a value or exits the current function in NormalizeVolumeDown.
353	}	Opens or closes a code block in NormalizeVolumeDown.
354		Blank separator for readability.
355	double NormalizeVolumeForOrder(double volume)	Declares or starts function NormalizeVolumeForOrder.
356	{	Opens or closes a code block in NormalizeVolumeForOrder.
357	double minLot=BrokerMinLot();	Updates or initializes double minLot in NormalizeVolumeForOrder.
358	double maxLot=BrokerMaxLot();	Updates or initializes double maxLot in NormalizeVolumeForOrder.
359		Blank separator for readability.
360	if(volume<minLot)	Checks a condition and runs the guarded branch in NormalizeVolumeForOrder.
361	volume=minLot;	Updates or initializes volume in NormalizeVolumeForOrder.
362	if(volume>maxLot)	Checks a condition and runs the guarded branch in NormalizeVolumeForOrder.
363	volume=maxLot;	Updates or initializes volume in NormalizeVolumeForOrder.
364		Blank separator for readability.
365	double out=NormalizeVolumeDown(volume);	Updates or initializes double out in NormalizeVolumeForOrder.
366	if(out<=0.0)	Checks a condition and runs the guarded branch in NormalizeVolumeForOrder.
367	out=minLot;	Updates or initializes out in NormalizeVolumeForOrder.
368		Blank separator for readability.
369	return NormalizeDouble(out,VolumeDigits());	Returns a value or exits the current function in NormalizeVolumeForOrder.
370	}	Opens or closes a code block in NormalizeVolumeForOrder.
371		Blank separator for readability.
372	double NormalizeVolumeForPanel(double volume)	Declares or starts function NormalizeVolumeForPanel.
373	{	Opens or closes a code block in NormalizeVolumeForPanel.
374	double minLot=BrokerMinLot();	Updates or initializes double minLot in NormalizeVolumeForPanel.
375	double maxLot=InpPanelLotMax;	Updates or initializes double maxLot in NormalizeVolumeForPanel.
376		Blank separator for readability.
377	if(maxLot<=0.0)	Checks a condition and runs the guarded branch in NormalizeVolumeForPanel.
378	maxLot=100.0;	Updates or initializes maxLot in NormalizeVolumeForPanel.
379	if(InpClampLotToBrokerMax)	Checks a condition and runs the guarded branch in NormalizeVolumeForPanel.
380	maxLot=MathMin(maxLot,BrokerMaxLot());	Updates or initializes maxLot in NormalizeVolumeForPanel.
381		Blank separator for readability.
382	if(volume<minLot)	Checks a condition and runs the guarded branch in NormalizeVolumeForPanel.
383	volume=minLot;	Updates or initializes volume in NormalizeVolumeForPanel.
384	if(volume>maxLot)	Checks a condition and runs the guarded branch in NormalizeVolumeForPanel.
385	volume=maxLot;	Updates or initializes volume in NormalizeVolumeForPanel.
386		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
387	double out=NormalizeVolumeDown(volume);	Updates or initializes double out in NormalizeVolumeForPanel.
388	if(out<=0.0)	Checks a condition and runs the guarded branch in NormalizeVolumeForPanel.
389	out=minLot;	Updates or initializes out in NormalizeVolumeForPanel.
390		Blank separator for readability.
391	if(out>maxLot)	Checks a condition and runs the guarded branch in NormalizeVolumeForPanel.
392	out=maxLot;	Updates or initializes out in NormalizeVolumeForPanel.
393		Blank separator for readability.
394	return NormalizeDouble(out,VolumeDigits());	Returns a value or exits the current function in NormalizeVolumeForPanel.
395	}	Opens or closes a code block in NormalizeVolumeForPanel.
396		Blank separator for readability.
397	double CleanNumber(string text,double oldValue,double minValue,double maxValue)	Declares or starts function CleanNumber.
398	{	Opens or closes a code block in CleanNumber.
399	string s=text;	Updates or initializes string s in CleanNumber.
400	StringReplace(s,"%","");	Calls helper function StringReplace in CleanNumber.
401	StringReplace(s,"\$","");	Calls helper function StringReplace in CleanNumber.
402	String cur=AccountInfoString(ACCOUNT_CURRENCY);	Updates or initializes string cur in CleanNumber.
403	if(StringLen(cur)>0)	Checks a condition and runs the guarded branch in CleanNumber.
404	StringReplace(s,cur,"");	Calls helper function StringReplace in CleanNumber.
405	StringReplace(s,"",".");	Calls helper function StringReplace in CleanNumber.
406	StringTrimLeft(s);	Calls helper function StringTrimLeft in CleanNumber.
407	StringTrimRight(s);	Calls helper function StringTrimRight in CleanNumber.
408		Blank separator for readability.
409	if(StringLen(s)==0)	Checks a condition and runs the guarded branch in CleanNumber.
410	return oldValue;	Returns a value or exits the current function in CleanNumber.
411		Blank separator for readability.
412	double v=StringToDouble(s);	Updates or initializes double v in CleanNumber.
413	if(v<minValue)	Checks a condition and runs the guarded branch in CleanNumber.
414	v=minValue;	Updates or initializes v in CleanNumber.
415	if(v>maxValue)	Checks a condition and runs the guarded branch in CleanNumber.
416	v=maxValue;	Updates or initializes v in CleanNumber.
417		Blank separator for readability.
418	return v;	Returns a value or exits the current function in CleanNumber.
419	}	Opens or closes a code block in CleanNumber.
420		Blank separator for readability.
421	double NormalizeRiskMoney(double value)	Declares or starts function NormalizeRiskMoney.
422	{	Opens or closes a code block in NormalizeRiskMoney.
423	if(value<0.01)	Checks a condition and runs the guarded branch in NormalizeRiskMoney.
424	value=0.01;	Updates or initializes value in NormalizeRiskMoney.
425	if(value>100000000.0)	Checks a condition and runs the guarded branch in NormalizeRiskMoney.
426	value=100000000.0;	Updates or initializes value in NormalizeRiskMoney.
427	return NormalizeDouble(value,2);	Returns a value or exits the current function in NormalizeRiskMoney.
428	}	Opens or closes a code block in NormalizeRiskMoney.
429		Blank separator for readability.
430	double CleanMoney(string text,double oldValue)	Declares or starts function CleanMoney.
431	{	Opens or closes a code block in CleanMoney.
432	return NormalizeRiskMoney(CleanNumber(text,oldValue,0.01,100000000.0));	Returns a value or exits the current function in CleanMoney.
433	}	Opens or closes a code block in CleanMoney.
434		Blank separator for readability.
435	double RiskPercentOfBalance(double riskMoney)	Declares or starts function RiskPercentOfBalance.
436	{	Opens or closes a code block in RiskPercentOfBalance.
437	double balance=AccountInfoDouble(ACCOUNT_BALANCE);	Updates or initializes double balance in RiskPercentOfBalance.
438	if(balance<=0.0)	Checks a condition and runs the guarded branch in RiskPercentOfBalance.
439	return 0.0;	Returns a value or exits the current function in RiskPercentOfBalance.
440	return riskMoney*100.0/balance;	Returns a value or exits the current function in RiskPercentOfBalance.
441	}	Opens or closes a code block in RiskPercentOfBalance.
442		Blank separator for readability.
443	double CleanPercent(string text,double oldValue)	Declares or starts function CleanPercent.
444	{	Opens or closes a code block in CleanPercent.
445	return NormalizeDouble(CleanNumber(text,oldValue,0.0,100.0),1);	Returns a value or exits the current function in CleanPercent.
446	}	Opens or closes a code block in CleanPercent.
447		Blank separator for readability.
448	string PriceText(double price)	Declares or starts function PriceText.
449	{	Opens or closes a code block in PriceText.
450	if(price<=0.0)	Checks a condition and runs the guarded branch in PriceText.
451	return "---";	Returns a value or exits the current function in PriceText.
452	return DoubleToString(price,_Digits);	Returns a value or exits the current function in PriceText.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
453	}	Opens or closes a code block in PriceText.
454		Blank separator for readability.
455	double EffectiveRR()	Declares or starts function EffectiveRR.
456	{	Opens or closes a code block in EffectiveRR.
457	if(g_RRTarget>0.0)	Checks a condition and runs the guarded branch in EffectiveRR.
458	return g_RRTarget;	Returns a value or exits the current function in EffectiveRR.
459	if(InpRiskReward<=0.0)	Checks a condition and runs the guarded branch in EffectiveRR.
460	return 3.0;	Returns a value or exits the current function in EffectiveRR.
461	return InpRiskReward;	Returns a value or exits the current function in EffectiveRR.
462	}	Opens or closes a code block in EffectiveRR.
463		Blank separator for readability.
464	double ClampPercent(double value)	Declares or starts function ClampPercent.
465	{	Opens or closes a code block in ClampPercent.
466	if(value<0.0)	Checks a condition and runs the guarded branch in ClampPercent.
467	value=0.0;	Updates or initializes value in ClampPercent.
468	if(value>100.0)	Checks a condition and runs the guarded branch in ClampPercent.
469	value=100.0;	Updates or initializes value in ClampPercent.
470	return NormalizeDouble(value,1);	Returns a value or exits the current function in ClampPercent.
471	}	Opens or closes a code block in ClampPercent.
472		Blank separator for readability.
473	int CurrentSpreadPoints()	Declares or starts function CurrentSpreadPoints.
474	{	Opens or closes a code block in CurrentSpreadPoints.
475	return (int)SymbolInfoInteger(_Symbol,SYMBOL_SPREAD);	Returns a value or exits the current function in CurrentSpreadPoints.
476	}	Opens or closes a code block in CurrentSpreadPoints.
477		Blank separator for readability.
478	int SpreadProtectionPoints()	Declares or starts function SpreadProtectionPoints.
479	{	Opens or closes a code block in SpreadProtectionPoints.
480	if(!InpUseSpreadProtection)	Checks a condition and runs the guarded branch in SpreadProtectionPoints.
481	return 0;	Returns a value or exits the current function in SpreadProtectionPoints.
482		Blank separator for readability.
483	double multiplier=InpSpreadProtectionMultiplier;	Updates or initializes double multiplier in SpreadProtectionPoints.
484	if(multiplier<0.0)	Checks a condition and runs the guarded branch in SpreadProtectionPoints.
485	multiplier=0.0;	Updates or initializes multiplier in SpreadProtectionPoints.
486		Blank separator for readability.
487	int extra=InpExtraProtectionPoints;	Updates or initializes int extra in SpreadProtectionPoints.
488	if(extra<0)	Checks a condition and runs the guarded branch in SpreadProtectionPoints.
489	extra=0;	Updates or initializes extra in SpreadProtectionPoints.
490		Blank separator for readability.
491	int pts=(int)MathCeil((double)CurrentSpreadPoints()*multiplier)+(int)extra;	Updates or initializes int pts in SpreadProtectionPoints.
492	if(pts<0)	Checks a condition and runs the guarded branch in SpreadProtectionPoints.
493	pts=0;	Updates or initializes pts in SpreadProtectionPoints.
494	return pts;	Returns a value or exits the current function in SpreadProtectionPoints.
495	}	Opens or closes a code block in SpreadProtectionPoints.
496		Blank separator for readability.
497	void ApplySpreadSpaceToSL(bool buy,double &sl)	Declares or starts function ApplySpreadSpaceToSL.
498	{	Opens or closes a code block in ApplySpreadSpaceToSL.
499	int pts=SpreadProtectionPoints();	Updates or initializes int pts in ApplySpreadSpaceToSL.
500	if(pts<=0)	Checks a condition and runs the guarded branch in ApplySpreadSpaceToSL.
501	{	Opens or closes a code block in ApplySpreadSpaceToSL.
502	sl=NormalizeDouble(sl,_Digits);	Updates or initializes sl in ApplySpreadSpaceToSL.
503	return;	Returns a value or exits the current function in ApplySpreadSpaceToSL.
504	}	Opens or closes a code block in ApplySpreadSpaceToSL.
505		Blank separator for readability.
506	double space=(double)pts*_Point;	Updates or initializes double space in ApplySpreadSpaceToSL.
507	if(buy)	Checks a condition and runs the guarded branch in ApplySpreadSpaceToSL.
508	sl-=space;	Updates or initializes sl- in ApplySpreadSpaceToSL.
509	else	Fallback branch when prior conditions did not match in ApplySpreadSpaceToSL.
510	sl+=space;	Updates or initializes sl+ in ApplySpreadSpaceToSL.
511		Blank separator for readability.
512	sl=NormalizeDouble(sl,_Digits);	Updates or initializes sl in ApplySpreadSpaceToSL.
513	}	Opens or closes a code block in ApplySpreadSpaceToSL.
514		Blank separator for readability.
515	void ApplySpreadSpaceToTP(bool buy,double &tp)	Declares or starts function ApplySpreadSpaceToTP.
516	{	Opens or closes a code block in ApplySpreadSpaceToTP.
517	if(tp<=0.0)	Checks a condition and runs the guarded branch in ApplySpreadSpaceToTP.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
518	return;	Returns a value or exits the current function in ApplySpreadSpaceToTP.
519		Blank separator for readability.
520	int pts=SpreadProtectionPoints();	Updates or initializes int pts in ApplySpreadSpaceToTP.
521	if(pts<=0)	Checks a condition and runs the guarded branch in ApplySpreadSpaceToTP.
522	{	Opens or closes a code block in ApplySpreadSpaceToTP.
523	tp=NormalizeDouble(tp,_Digits);	Updates or initializes tp in ApplySpreadSpaceToTP.
524	return;	Returns a value or exits the current function in ApplySpreadSpaceToTP.
525	}	Opens or closes a code block in ApplySpreadSpaceToTP.
526		Blank separator for readability.
527	double space=(double)pts*_Point;	Updates or initializes double space in ApplySpreadSpaceToTP.
528	if(buy)	Checks a condition and runs the guarded branch in ApplySpreadSpaceToTP.
529	tp+=space;	Updates or initializes tp+ in ApplySpreadSpaceToTP.
530	else if(tp>space+_Point)	Checks an alternate condition when the previous branch did not run in ApplySpreadSpaceToTP.
531	tp-=space;	Updates or initializes tp- in ApplySpreadSpaceToTP.
532		Blank separator for readability.
533	tp=NormalizeDouble(tp,_Digits);	Updates or initializes tp in ApplySpreadSpaceToTP.
534	}	Opens or closes a code block in ApplySpreadSpaceToTP.
535		Blank separator for readability.
536	bool SpreadBlocksTrades()	Declares or starts function SpreadBlocksTrades.
537	{	Opens or closes a code block in SpreadBlocksTrades.
538	return (InpBlockTradeOnHighSpread && InpUseSpreadFilter);	Returns a value or exits the current function in SpreadBlocksTrades.
539	}	Opens or closes a code block in SpreadBlocksTrades.
540		Blank separator for readability.
541	int RequiredStopPoints()	Declares or starts function RequiredStopPoints.
542	{	Opens or closes a code block in RequiredStopPoints.
543	int stops=(int)SymbolInfoInteger(_Symbol,SYMBOL_TRADE_STOPS_LEVEL);	Updates or initializes int stops in RequiredStopPoints.
544	int freeze=(int)SymbolInfoInteger(_Symbol,SYMBOL_TRADE_FREEZE_LEVEL);	Updates or initializes int freeze in RequiredStopPoints.
545	int needed=(int)MathMax((double)stops,(double)freeze);	Updates or initializes int needed in RequiredStopPoints.
546	return needed+1;	Returns a value or exits the current function in RequiredStopPoints.
547	}	Opens or closes a code block in RequiredStopPoints.
548		Blank separator for readability.
549	bool HasEnoughBars()	Declares or starts function HasEnoughBars.
550	{	Opens or closes a code block in HasEnoughBars.
551	return Bars(_Symbol,_Period)>=10;	Returns a value or exits the current function in HasEnoughBars.
552	}	Opens or closes a code block in HasEnoughBars.
553		Blank separator for readability.
554	//=====	Comment explaining the nearby EA code.
555	// POSITION HELPERS	Comment explaining the nearby EA code.
556	//=====	Comment explaining the nearby EA code.
557	bool SelectOurPosition()	Declares or starts function SelectOurPosition.
558	{	Opens or closes a code block in SelectOurPosition.
559	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in SelectOurPosition.
560	{	Opens or closes a code block in SelectOurPosition.
561	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in SelectOurPosition.
562	if(ticket==0)	Checks a condition and runs the guarded branch in SelectOurPosition.
563	continue;	Executes one statement in SelectOurPosition.
564		Blank separator for readability.
565	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in SelectOurPosition.
566	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in SelectOurPosition.
567		Blank separator for readability.
568	if(sym==_Symbol && magic==InpMagic)	Checks a condition and runs the guarded branch in SelectOurPosition.
569	return true;	Returns a value or exits the current function in SelectOurPosition.
570	}	Opens or closes a code block in SelectOurPosition.
571		Blank separator for readability.
572	return false;	Returns a value or exits the current function in SelectOurPosition.
573	}	Opens or closes a code block in SelectOurPosition.
574		Blank separator for readability.
575	int CountOurPositions()	Declares or starts function CountOurPositions.
576	{	Opens or closes a code block in CountOurPositions.
577	int count=0;	Updates or initializes int count in CountOurPositions.
578		Blank separator for readability.
579	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in CountOurPositions.
580	{	Opens or closes a code block in CountOurPositions.
581	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in CountOurPositions.
582	if(ticket==0)	Checks a condition and runs the guarded branch in CountOurPositions.
583	continue;	Executes one statement in CountOurPositions.
584		Blank separator for readability.
585	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in CountOurPositions.
586	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in CountOurPositions.
587	if(sym==_Symbol && magic==InpMagic)	Checks a condition and runs the guarded branch in CountOurPositions.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
588	count++;	Executes one statement in CountOurPositions.
589	}	Opens or closes a code block in CountOurPositions.
590		Blank separator for readability.
591	return count;	Returns a value or exits the current function in CountOurPositions.
592	}	Opens or closes a code block in CountOurPositions.
593		Blank separator for readability.
594	bool IsDashboardMagic(long magic)	Declares or starts function IsDashboardMagic.
595	{	Opens or closes a code block in IsDashboardMagic.
596	return (magic==InpMagic (InpShowManualTradesOnDashboard && magic==0));	Returns a value or exits the current function in IsDashboardMagic.
597	}	Opens or closes a code block in IsDashboardMagic.
598		Blank separator for readability.
599	bool IsManagedMagic(long magic)	Declares or starts function IsManagedMagic.
600	{	Opens or closes a code block in IsManagedMagic.
601	return (magic==InpMagic);	Returns a value or exits the current function in IsManagedMagic.
602	}	Opens or closes a code block in IsManagedMagic.
603		Blank separator for readability.
604	string DashboardSourceText(long magic)	Declares or starts function DashboardSourceText.
605	{	Opens or closes a code block in DashboardSourceText.
606	if(IsManagedMagic(magic))	Checks a condition and runs the guarded branch in DashboardSourceText.
607	return "EA";	Returns a value or exits the current function in DashboardSourceText.
608	if(magic==0)	Checks a condition and runs the guarded branch in DashboardSourceText.
609	return "Manual";	Returns a value or exits the current function in DashboardSourceText.
610	return "Other";	Returns a value or exits the current function in DashboardSourceText.
611	}	Opens or closes a code block in DashboardSourceText.
612		Blank separator for readability.
613	int CountDashboardPositions()	Declares or starts function CountDashboardPositions.
614	{	Opens or closes a code block in CountDashboardPositions.
615	int count=0;	Updates or initializes int count in CountDashboardPositions.
616		Blank separator for readability.
617	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in CountDashboardPositions.
618	{	Opens or closes a code block in CountDashboardPositions.
619	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in CountDashboardPositions.
620	if(ticket==0)	Checks a condition and runs the guarded branch in CountDashboardPositions.
621	continue;	Executes one statement in CountDashboardPositions.
622		Blank separator for readability.
623	if(PositionGetString(POSITION_SYMBOL)==_Symbol &&	Checks a condition and runs the guarded branch in CountDashboardPositions.
624	IsDashboardMagic((long)PositionGetInteger(POSITION_MAGIC)))	Part of the EA logic in CountDashboardPositions.
625	count++;	Executes one statement in CountDashboardPositions.
626	}	Opens or closes a code block in CountDashboardPositions.
627		Blank separator for readability.
628	return count;	Returns a value or exits the current function in CountDashboardPositions.
629	}	Opens or closes a code block in CountDashboardPositions.
630		Blank separator for readability.
631	int MaxEntriesPerCycle()	Declares or starts function MaxEntriesPerCycle.
632	{	Opens or closes a code block in MaxEntriesPerCycle.
633	return MathMax(1,InpMaxEntriesPerCycle);	Returns a value or exits the current function in MaxEntriesPerCycle.
634	}	Opens or closes a code block in MaxEntriesPerCycle.
635		Blank separator for readability.
636	bool CanStartModelEntry(string &reason)	Declares or starts function CanStartModelEntry.
637	{	Opens or closes a code block in CanStartModelEntry.
638	reason="";	Updates or initializes reason in CanStartModelEntry.
639	int openCount=CountOurPositions();	Updates or initializes int openCount in CanStartModelEntry.
640	if(openCount<=0)	Checks a condition and runs the guarded branch in CanStartModelEntry.
641	return true;	Returns a value or exits the current function in CanStartModelEntry.
642		Blank separator for readability.
643	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in CanStartModelEntry.
644		Blank separator for readability.
645	if(g_SecondEntryDone)	Checks a condition and runs the guarded branch in CanStartModelEntry.
646	{	Opens or closes a code block in CanStartModelEntry.
647	reason="Second entry already used for this trade";	Updates or initializes reason in CanStartModelEntry.
648	return false;	Returns a value or exits the current function in CanStartModelEntry.
649	}	Opens or closes a code block in CanStartModelEntry.
650		Blank separator for readability.
651	if(g_SecondEntryBlocked)	Checks a condition and runs the guarded branch in CanStartModelEntry.
652	{	Opens or closes a code block in CanStartModelEntry.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
653	reason="Second entry blocked after first trade reached	Updates or initializes reason in CanStartModelEntry.
654	"+"DoubleToString(MathMax(0.1, InpSecondEntryCancelAtR), 1)+"R";	
654	return false;	Returns a value or exits the current function in CanStartModelEntry.
655	}	Opens or closes a code block in CanStartModelEntry.
656		Blank separator for readability.
657	if(g_SecondEntryOn)	Checks a condition and runs the guarded branch in CanStartModelEntry.
658	reason="Trade already running - 2nd entry only triggers at	Updates or initializes reason in CanStartModelEntry.
658	"+"DoubleToString(MathMax(0.1, InpSecondEntryTriggerLossR), 1)+"R";	
659	else	Fallback branch when prior conditions did not match in CanStartModelEntry.
660	reason="Trade already running - one entry mode";	Updates or initializes reason in CanStartModelEntry.
661	return false;	Returns a value or exits the current function in CanStartModelEntry.
662	}	Opens or closes a code block in CanStartModelEntry.
663		Blank separator for readability.
664	bool SelectOldestOurPosition()	Declares or starts function SelectOldestOurPosition.
665	{	Opens or closes a code block in SelectOldestOurPosition.
666	bool found=false;	Updates or initializes bool found in SelectOldestOurPosition.
667	ulong selectedTicket=0;	Updates or initializes ulong selectedTicket in SelectOldestOurPosition.
668	datetime selectedTime=0;	Updates or initializes datetime selectedTime in SelectOldestOurPosition.
669		Blank separator for readability.
670	for(int i=PositionsTotal()-1; i>=0; i--)	Loops over a fixed range or collection in SelectOldestOurPosition.
671	{	Opens or closes a code block in SelectOldestOurPosition.
672	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in SelectOldestOurPosition.
673	if(ticket==0)	Checks a condition and runs the guarded branch in SelectOldestOurPosition.
674	continue;	Executes one statement in SelectOldestOurPosition.
675		Blank separator for readability.
676	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in SelectOldestOurPosition.
677	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in SelectOldestOurPosition.
678	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in SelectOldestOurPosition.
679	continue;	Executes one statement in SelectOldestOurPosition.
680		Blank separator for readability.
681	datetime openTime=(datetime)PositionGetInteger(POSITION_TIME);	Updates or initializes datetime openTime in SelectOldestOurPosition.
682	if(!found openTime<selectedTime (openTime==selectedTime && ticket<selectedTicket))	Checks a condition and runs the guarded branch in SelectOldestOurPosition.
683	{	Opens or closes a code block in SelectOldestOurPosition.
684	found=true;	Updates or initializes found in SelectOldestOurPosition.
685	selectedTicket=ticket;	Updates or initializes selectedTicket in SelectOldestOurPosition.
686	selectedTime=openTime;	Updates or initializes selectedTime in SelectOldestOurPosition.
687	}	Opens or closes a code block in SelectOldestOurPosition.
688	}	Opens or closes a code block in SelectOldestOurPosition.
689		Blank separator for readability.
690	if(!found selectedTicket==0)	Checks a condition and runs the guarded branch in SelectOldestOurPosition.
691	return false;	Returns a value or exits the current function in SelectOldestOurPosition.
692		Blank separator for readability.
693	return PositionSelectByTicket(selectedTicket);	Returns a value or exits the current function in SelectOldestOurPosition.
694	}	Opens or closes a code block in SelectOldestOurPosition.
695		Blank separator for readability.
696	bool SelectNewestOurPosition()	Declares or starts function SelectNewestOurPosition.
697	{	Opens or closes a code block in SelectNewestOurPosition.
698	bool found=false;	Updates or initializes bool found in SelectNewestOurPosition.
699	ulong selectedTicket=0;	Updates or initializes ulong selectedTicket in SelectNewestOurPosition.
700	datetime selectedTime=0;	Updates or initializes datetime selectedTime in SelectNewestOurPosition.
701		Blank separator for readability.
702	for(int i=PositionsTotal()-1; i>=0; i--)	Loops over a fixed range or collection in SelectNewestOurPosition.
703	{	Opens or closes a code block in SelectNewestOurPosition.
704	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in SelectNewestOurPosition.
705	if(ticket==0)	Checks a condition and runs the guarded branch in SelectNewestOurPosition.
706	continue;	Executes one statement in SelectNewestOurPosition.
707		Blank separator for readability.
708	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in SelectNewestOurPosition.
709	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in SelectNewestOurPosition.
710	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in SelectNewestOurPosition.
711	continue;	Executes one statement in SelectNewestOurPosition.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
712		Blank separator for readability.
713	datetime openTime=(datetime)PositionGetInteger(POSITION_TIME);	Updates or initializes datetime openTime in SelectNewestOurPosition.
714	if(!found openTime>selectedTime (openTime==selectedTime && ticket>selectedTicket))	Checks a condition and runs the guarded branch in SelectNewestOurPosition.
715	{	Opens or closes a code block in SelectNewestOurPosition.
716	found=true;	Updates or initializes found in SelectNewestOurPosition.
717	selectedTicket=ticket;	Updates or initializes selectedTicket in SelectNewestOurPosition.
718	selectedTime=openTime;	Updates or initializes selectedTime in SelectNewestOurPosition.
719	}	Opens or closes a code block in SelectNewestOurPosition.
720	}	Opens or closes a code block in SelectNewestOurPosition.
721		Blank separator for readability.
722	if(!found selectedTicket==0)	Checks a condition and runs the guarded branch in SelectNewestOurPosition.
723	return false;	Returns a value or exits the current function in SelectNewestOurPosition.
724		Blank separator for readability.
725	return PositionSelectByTicket(selectedTicket);	Returns a value or exits the current function in SelectNewestOurPosition.
726	}	Opens or closes a code block in SelectNewestOurPosition.
727		Blank separator for readability.
728	bool IsSecondEntryPositionComment()	Declares or starts function IsSecondEntryPositionComment.
729	{	Opens or closes a code block in IsSecondEntryPositionComment.
730	string comment=PositionGetString(POSITION_COMMENT);	Updates or initializes string comment in IsSecondEntryPositionComment.
731	return (StringFind(comment,"TCX Second")>=0);	Returns a value or exits the current function in IsSecondEntryPositionComment.
732	}	Opens or closes a code block in IsSecondEntryPositionComment.
733		Blank separator for readability.
734	bool SelectFirstEntryPosition()	Declares or starts function SelectFirstEntryPosition.
735	{	Opens or closes a code block in SelectFirstEntryPosition.
736	bool found=false;	Updates or initializes bool found in SelectFirstEntryPosition.
737	ulong selectedTicket=0;	Updates or initializes ulong selectedTicket in SelectFirstEntryPosition.
738	datetime selectedTime=0;	Updates or initializes datetime selectedTime in SelectFirstEntryPosition.
739		Blank separator for readability.
740	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in SelectFirstEntryPosition.
741	{	Opens or closes a code block in SelectFirstEntryPosition.
742	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in SelectFirstEntryPosition.
743	if(ticket==0)	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
744	continue;	Executes one statement in SelectFirstEntryPosition.
745		Blank separator for readability.
746	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in SelectFirstEntryPosition.
747	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in SelectFirstEntryPosition.
748	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
749	continue;	Executes one statement in SelectFirstEntryPosition.
750		Blank separator for readability.
751	if(IsSecondEntryPositionComment())	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
752	continue;	Executes one statement in SelectFirstEntryPosition.
753		Blank separator for readability.
754	datetime openTime=(datetime)PositionGetInteger(POSITION_TIME);	Updates or initializes datetime openTime in SelectFirstEntryPosition.
755	if(!found openTime<selectedTime (openTime==selectedTime && ticket<selectedTicket))	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
756	{	Opens or closes a code block in SelectFirstEntryPosition.
757	found=true;	Updates or initializes found in SelectFirstEntryPosition.
758	selectedTicket=ticket;	Updates or initializes selectedTicket in SelectFirstEntryPosition.
759	selectedTime=openTime;	Updates or initializes selectedTime in SelectFirstEntryPosition.
760	}	Opens or closes a code block in SelectFirstEntryPosition.
761	}	Opens or closes a code block in SelectFirstEntryPosition.
762		Blank separator for readability.
763	if(!found selectedTicket==0)	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
764	return false;	Returns a value or exits the current function in SelectFirstEntryPosition.
765		Blank separator for readability.
766	return PositionSelectByTicket(selectedTicket);	Returns a value or exits the current function in SelectFirstEntryPosition.
767	}	Opens or closes a code block in SelectFirstEntryPosition.
768		Blank separator for readability.
769	ulong FindNewestOurPositionTicket(ulong exceptTicket)	Part of the EA logic in SelectFirstEntryPosition.
770	{	Opens or closes a code block in SelectFirstEntryPosition.
771	bool found=false;	Updates or initializes bool found in SelectFirstEntryPosition.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
772	ulong selectedTicket=0;	Updates or initializes ulong selectedTicket in SelectFirstEntryPosition.
773	datetime selectedTime=0;	Updates or initializes datetime selectedTime in SelectFirstEntryPosition.
774		Blank separator for readability.
775	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in SelectFirstEntryPosition.
776	{	Opens or closes a code block in SelectFirstEntryPosition.
777	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in SelectFirstEntryPosition.
778	if(ticket==0 (exceptTicket>0 && ticket==exceptTicket))	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
779	continue;	Executes one statement in SelectFirstEntryPosition.
780		Blank separator for readability.
781	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in SelectFirstEntryPosition.
782	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in SelectFirstEntryPosition.
783	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
784	continue;	Executes one statement in SelectFirstEntryPosition.
785		Blank separator for readability.
786	datetime openTime=(datetime)PositionGetInteger(POSITION_TIME);	Updates or initializes datetime openTime in SelectFirstEntryPosition.
787	if(!found openTime>selectedTime (openTime==selectedTime && ticket>selectedTicket))	Checks a condition and runs the guarded branch in SelectFirstEntryPosition.
788	{	Opens or closes a code block in SelectFirstEntryPosition.
789	found=true;	Updates or initializes found in SelectFirstEntryPosition.
790	selectedTicket=ticket;	Updates or initializes selectedTicket in SelectFirstEntryPosition.
791	selectedTime=openTime;	Updates or initializes selectedTime in SelectFirstEntryPosition.
792	}	Opens or closes a code block in SelectFirstEntryPosition.
793	}	Opens or closes a code block in SelectFirstEntryPosition.
794		Blank separator for readability.
795	return found ? selectedTicket : 0;	Returns a value or exits the current function in SelectFirstEntryPosition.
796	}	Opens or closes a code block in SelectFirstEntryPosition.
797		Blank separator for readability.
798	bool HasOurPosition()	Declares or starts function HasOurPosition.
799	{	Opens or closes a code block in HasOurPosition.
800	return SelectOurPosition();	Returns a value or exits the current function in HasOurPosition.
801	}	Opens or closes a code block in HasOurPosition.
802		Blank separator for readability.
803	void SetArmMode(int mode)	Declares or starts function SetArmMode.
804	{	Opens or closes a code block in SetArmMode.
805	g_ArmedMode=mode;	Updates or initializes g_ArmedMode in SetArmMode.
806	if(mode!=ARM_NONE)	Checks a condition and runs the guarded branch in SetArmMode.
807	g_AutoArm=false;	Updates or initializes g_AutoArm in SetArmMode.
808		Blank separator for readability.
809	if(mode==ARM_NONE)	Checks a condition and runs the guarded branch in SetArmMode.
810	{	Opens or closes a code block in SetArmMode.
811	g_ArmMinSignalTime=0;	Updates or initializes g_ArmMinSignalTime in SetArmMode.
812	return;	Returns a value or exits the current function in SetArmMode.
813	}	Opens or closes a code block in SetArmMode.
814		Blank separator for readability.
815	datetime currentBar=iTime(_Symbol,_Period,0);	Updates or initializes datetime currentBar in SetArmMode.
816	if(currentBar<=0)	Checks a condition and runs the guarded branch in SetArmMode.
817	currentBar=TimeCurrent();	Updates or initializes currentBar in SetArmMode.
818		Blank separator for readability.
819	g_ArmMinSignalTime=currentBar;	Updates or initializes g_ArmMinSignalTime in SetArmMode.
820	}	Opens or closes a code block in SetArmMode.
821		Blank separator for readability.
822	void SetAutoArm(bool enabled)	Declares or starts function SetAutoArm.
823	{	Opens or closes a code block in SetAutoArm.
824	g_AutoArm=enabled;	Updates or initializes g_AutoArm in SetAutoArm.
825		Blank separator for readability.
826	if(enabled)	Checks a condition and runs the guarded branch in SetAutoArm.
827	{	Opens or closes a code block in SetAutoArm.
828	g_ArmedMode=ARM_NONE;	Updates or initializes g_ArmedMode in SetAutoArm.
829	datetime currentBar=iTime(_Symbol,_Period,0);	Updates or initializes datetime currentBar in SetAutoArm.
830	if(currentBar<=0)	Checks a condition and runs the guarded branch in SetAutoArm.
831	currentBar=TimeCurrent();	Updates or initializes currentBar in SetAutoArm.
832	g_ArmMinSignalTime=currentBar;	Updates or initializes g_ArmMinSignalTime in SetAutoArm.
833	}	Opens or closes a code block in SetAutoArm.
834	else if(g_ArmedMode==ARM_NONE)	Checks an alternate condition when the previous branch did not run in SetAutoArm.
835	{	Opens or closes a code block in SetAutoArm.
836	g_ArmMinSignalTime=0;	Updates or initializes g_ArmMinSignalTime in SetAutoArm.
837	}	Opens or closes a code block in SetAutoArm.
838	}	Opens or closes a code block in SetAutoArm.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
839		Blank separator for readability.
840	bool IsHedgingAccount()	Declares or starts function IsHedgingAccount.
841	{	Opens or closes a code block in IsHedgingAccount.
842	long mode=AccountInfoInteger(ACCOUNT_MARGIN_MODE);	Updates or initializes long mode in IsHedgingAccount.
843	return (mode==ACCOUNT_MARGIN_MODE_RETAIL_HEDGING);	Returns a value or exits the current function in IsHedgingAccount.
844	}	Opens or closes a code block in IsHedgingAccount.
845		Blank separator for readability.
846	string StateKey(ulong ticket,string field)	Declares or starts function StateKey.
847	{	Opens or closes a code block in StateKey.
848	long login=(long)AccountInfoInteger(ACCOUNT_LOGIN);	Updates or initializes long login in StateKey.
849	return "TCX."+IntegerToString(login)+"."+_Symbol+"."+IntegerToString(InpMagic)+"."+IntegerToString((long)ticket)+"."+field;	Returns a value or exits the current function in StateKey.
850	}	Opens or closes a code block in StateKey.
851		Blank separator for readability.
852	string SecondEntryCycleKey(string field)	Declares or starts function SecondEntryCycleKey.
853	{	Opens or closes a code block in SecondEntryCycleKey.
854	long login=(long)AccountInfoInteger(ACCOUNT_LOGIN);	Updates or initializes long login in SecondEntryCycleKey.
855	return "TCX."+IntegerToString(login)+"."+_Symbol+"."+IntegerToString(InpMagic)+"."+cond_cycle."+field;	Returns a value or exits the current function in SecondEntryCycleKey.
856	}	Opens or closes a code block in SecondEntryCycleKey.
857		Blank separator for readability.
858	bool LoadStateValue(ulong ticket,string field,double &value)	Declares or starts function LoadStateValue.
859	{	Opens or closes a code block in LoadStateValue.
860	string key=StateKey(ticket,field);	Updates or initializes string key in LoadStateValue.
861	if(!GlobalVariableCheck(key))	Checks a condition and runs the guarded branch in LoadStateValue.
862	return false;	Returns a value or exits the current function in LoadStateValue.
863		Blank separator for readability.
864	value=GlobalVariableGet(key);	Updates or initializes value in LoadStateValue.
865	return true;	Returns a value or exits the current function in LoadStateValue.
866	}	Opens or closes a code block in LoadStateValue.
867		Blank separator for readability.
868	bool LoadSecondEntryCycleValue(string field,double &value)	Declares or starts function LoadSecondEntryCycleValue.
869	{	Opens or closes a code block in LoadSecondEntryCycleValue.
870	string key=SecondEntryCycleKey(field);	Updates or initializes string key in LoadSecondEntryCycleValue.
871	if(!GlobalVariableCheck(key))	Checks a condition and runs the guarded branch in LoadSecondEntryCycleValue.
872	return false;	Returns a value or exits the current function in LoadSecondEntryCycleValue.
873		Blank separator for readability.
874	value=GlobalVariableGet(key);	Updates or initializes value in LoadSecondEntryCycleValue.
875	return true;	Returns a value or exits the current function in LoadSecondEntryCycleValue.
876	}	Opens or closes a code block in LoadSecondEntryCycleValue.
877		Blank separator for readability.
878	void ClearSecondEntryCycleState()	Declares or starts function ClearSecondEntryCycleState.
879	{	Opens or closes a code block in ClearSecondEntryCycleState.
880	GlobalVariableDel(SecondEntryCycleKey("second_entry_done"));	Calls helper function GlobalVariableDel in ClearSecondEntryCycleState.
881	GlobalVariableDel(SecondEntryCycleKey("second_entry_blocked"));	Calls helper function GlobalVariableDel in ClearSecondEntryCycleState.
882	}	Opens or closes a code block in ClearSecondEntryCycleState.
883		Blank separator for readability.
884	void SavePositionState()	Declares or starts function SavePositionState.
885	{	Opens or closes a code block in SavePositionState.
886	if(g_StateTicket==0)	Checks a condition and runs the guarded branch in SavePositionState.
887	return;	Returns a value or exits the current function in SavePositionState.
888		Blank separator for readability.
889	GlobalVariableSet(StateKey(g_StateTicket,"initial_volume"),g_InitialVolume);	Calls helper function GlobalVariableSet in SavePositionState.
890	GlobalVariableSet(StateKey(g_StateTicket,"risk_distance"),g_RiskDistance);	Calls helper function GlobalVariableSet in SavePositionState.
891	GlobalVariableSet(StateKey(g_StateTicket,"pc1_done"),g_PC1_Done ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SavePositionState.
892	GlobalVariableSet(StateKey(g_StateTicket,"pc2_done"),g_PC2_Done ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SavePositionState.
893	GlobalVariableSet(StateKey(g_StateTicket,"pc3_done"),g_PC3_Done ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SavePositionState.
894	GlobalVariableSet(StateKey(g_StateTicket,"be_done"),g_BE_Done ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SavePositionState.
895	}	Opens or closes a code block in SavePositionState.
896		Blank separator for readability.
897	void ResetLocalTradeState()	Declares or starts function ResetLocalTradeState.
898	{	Opens or closes a code block in ResetLocalTradeState.
899	g_StateTicket=0;	Updates or initializes g_StateTicket in ResetLocalTradeState.
900	g_InitialVolume=0.0;	Updates or initializes g_InitialVolume in ResetLocalTradeState.
901	g_RiskDistance=0.0;	Updates or initializes g_RiskDistance in ResetLocalTradeState.
902	g_PC1_Done=false;	Updates or initializes g_PC1_Done in ResetLocalTradeState.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
903	g_PC2_Done=false;	Updates or initializes g_PC2_Done in ResetLocalTradeState.
904	g_PC3_Done=false;	Updates or initializes g_PC3_Done in ResetLocalTradeState.
905	g_BE_Done=false;	Updates or initializes g_BE_Done in ResetLocalTradeState.
906	}	Opens or closes a code block in ResetLocalTradeState.
907		Blank separator for readability.
908	void SaveSecondEntryState()	Declares or starts function SaveSecondEntryState.
909	{	Opens or closes a code block in SaveSecondEntryState.
910	GlobalVariableSet(SecondEntryCycleKey("second_entry_done"),g_SecondEntryDone ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SaveSecondEntryState.
911	GlobalVariableSet(SecondEntryCycleKey("second_entry_blocked"),g_SecondEntryBlocked ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SaveSecondEntryState.
912		Blank separator for readability.
913	if(g_SecondBaseTicket>0)	Checks a condition and runs the guarded branch in SaveSecondEntryState.
914	{	Opens or closes a code block in SaveSecondEntryState.
915	GlobalVariableSet(StateKey(g_SecondBaseTicket,"second_entry_done"),g_SecondEntryDone ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SaveSecondEntryState.
916	GlobalVariableSet(StateKey(g_SecondBaseTicket,"second_entry_blocked"),g_SecondEntryBlocked ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SaveSecondEntryState.
917	}	Opens or closes a code block in SaveSecondEntryState.
918	}	Opens or closes a code block in SaveSecondEntryState.
919		Blank separator for readability.
920	void ResetSecondEntryState()	Declares or starts function ResetSecondEntryState.
921	{	Opens or closes a code block in ResetSecondEntryState.
922	g_SecondBaseTicket=0;	Updates or initializes g_SecondBaseTicket in ResetSecondEntryState.
923	g_SecondEntryDone=false;	Updates or initializes g_SecondEntryDone in ResetSecondEntryState.
924	g_SecondEntryBlocked=false;	Updates or initializes g_SecondEntryBlocked in ResetSecondEntryState.
925	g_SecondEntryLastAttempt=0;	Updates or initializes g_SecondEntryLastAttempt in ResetSecondEntryState.
926	ClearSecondEntryCycleState();	Calls helper function ClearSecondEntryCycleState in ResetSecondEntryState.
927	}	Opens or closes a code block in ResetSecondEntryState.
928		Blank separator for readability.
929	void SaveFirstTrailState()	Declares or starts function SaveFirstTrailState.
930	{	Opens or closes a code block in SaveFirstTrailState.
931	if(g_FirstTrailTicket==0)	Checks a condition and runs the guarded branch in SaveFirstTrailState.
932	return;	Returns a value or exits the current function in SaveFirstTrailState.
933		Blank separator for readability.
934	GlobalVariableSet(StateKey(g_FirstTrailTicket,"first_trail_active"),g_FirstTrailActive ? 1.0 : 0.0);	Calls helper function GlobalVariableSet in SaveFirstTrailState.
935	GlobalVariableSet(StateKey(g_FirstTrailTicket,"first_trail_last_m5"),(double)g_FirstTrailLastM5BarTime);	Calls helper function GlobalVariableSet in SaveFirstTrailState.
936	}	Opens or closes a code block in SaveFirstTrailState.
937		Blank separator for readability.
938	void ResetFirstTrailState()	Declares or starts function ResetFirstTrailState.
939	{	Opens or closes a code block in ResetFirstTrailState.
940	g_FirstTrailTicket=0;	Updates or initializes g_FirstTrailTicket in ResetFirstTrailState.
941	g_FirstTrailActive=false;	Updates or initializes g_FirstTrailActive in ResetFirstTrailState.
942	g_FirstTrailLastM5BarTime=0;	Updates or initializes g_FirstTrailLastM5BarTime in ResetFirstTrailState.
943	}	Opens or closes a code block in ResetFirstTrailState.
944		Blank separator for readability.
945	void SyncFirstTrailState()	Declares or starts function SyncFirstTrailState.
946	{	Opens or closes a code block in SyncFirstTrailState.
947	if(!SelectFirstEntryPosition())	Checks a condition and runs the guarded branch in SyncFirstTrailState.
948	{	Opens or closes a code block in SyncFirstTrailState.
949	ResetFirstTrailState();	Calls helper function ResetFirstTrailState in SyncFirstTrailState.
950	return;	Returns a value or exits the current function in SyncFirstTrailState.
951	}	Opens or closes a code block in SyncFirstTrailState.
952		Blank separator for readability.
953	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in SyncFirstTrailState.
954	if(ticket!=g_FirstTrailTicket)	Checks a condition and runs the guarded branch in SyncFirstTrailState.
955	{	Opens or closes a code block in SyncFirstTrailState.
956	g_FirstTrailTicket=ticket;	Updates or initializes g_FirstTrailTicket in SyncFirstTrailState.
957		Blank separator for readability.
958	double stored=0.0;	Updates or initializes double stored in SyncFirstTrailState.
959	if(LoadStateValue(ticket,"first_trail_active",stored))	Checks a condition and runs the guarded branch in SyncFirstTrailState.
960	g_FirstTrailActive=(stored>=0.5);	Updates or initializes g_FirstTrailActive in SyncFirstTrailState.
961	else	Fallback branch when prior conditions did not match in SyncFirstTrailState.
962	g_FirstTrailActive=false;	Updates or initializes g_FirstTrailActive in SyncFirstTrailState.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
963		Blank separator for readability.
964	if(LoadStateValue(ticket,"first_trail_last_m5",stored) && stored>0.0)	Checks a condition and runs the guarded branch in SyncFirstTrailState.
965	g_FirstTrailLastM5BarTime=(datetime)stored;	Updates or initializes g_FirstTrailLastM5BarTime in SyncFirstTrailState.
966	else	Fallback branch when prior conditions did not match in SyncFirstTrailState.
967	g_FirstTrailLastM5BarTime=0;	Updates or initializes g_FirstTrailLastM5BarTime in SyncFirstTrailState.
968		Blank separator for readability.
969	SaveFirstTrailState();	Calls helper function SaveFirstTrailState in SyncFirstTrailState.
970	}	Opens or closes a code block in SyncFirstTrailState.
971	}	Opens or closes a code block in SyncFirstTrailState.
972		Blank separator for readability.
973	void SyncSecondEntryState()	Declares or starts function SyncSecondEntryState.
974	{	Opens or closes a code block in SyncSecondEntryState.
975	if(!SelectOldestOurPosition())	Checks a condition and runs the guarded branch in SyncSecondEntryState.
976	{	Opens or closes a code block in SyncSecondEntryState.
977	if(g_SecondBaseTicket>0 && g_SecondEntryOn)	Checks a condition and runs the guarded branch in SyncSecondEntryState.
978	g_SecondEntryOn=false;	Updates or initializes g_SecondEntryOn in SyncSecondEntryState.
979	ResetSecondEntryState();	Calls helper function ResetSecondEntryState in SyncSecondEntryState.
980	return;	Returns a value or exits the current function in SyncSecondEntryState.
981	}	Opens or closes a code block in SyncSecondEntryState.
982		Blank separator for readability.
983	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in SyncSecondEntryState.
984	double cycleStored=0.0;	Updates or initializes double cycleStored in SyncSecondEntryState.
985	bool cycleDone=(LoadSecondEntryCycleValue("second_entry_done",cycleStored) && cycleStored>=0.5);	Updates or initializes bool cycleDone in SyncSecondEntryState.
986	bool cycleBlocked=(LoadSecondEntryCycleValue("second_entry_blocked",cycleStored) && cycleStored>=0.5);	Updates or initializes bool cycleBlocked in SyncSecondEntryState.
987		Blank separator for readability.
988	if(ticket!=g_SecondBaseTicket)	Checks a condition and runs the guarded branch in SyncSecondEntryState.
989	{	Opens or closes a code block in SyncSecondEntryState.
990	g_SecondBaseTicket=ticket;	Updates or initializes g_SecondBaseTicket in SyncSecondEntryState.
991	g_SecondEntryLastAttempt=0;	Updates or initializes g_SecondEntryLastAttempt in SyncSecondEntryState.
992		Blank separator for readability.
993	double stored=0.0;	Updates or initializes double stored in SyncSecondEntryState.
994	if(LoadStateValue(ticket,"second_entry_done",stored))	Checks a condition and runs the guarded branch in SyncSecondEntryState.
995	g_SecondEntryDone=(stored>=0.5);	Updates or initializes g_SecondEntryDone in SyncSecondEntryState.
996	else	Fallback branch when prior conditions did not match in SyncSecondEntryState.
997	g_SecondEntryDone=false;	Updates or initializes g_SecondEntryDone in SyncSecondEntryState.
998		Blank separator for readability.
999	if(LoadStateValue(ticket,"second_entry_blocked",stored))	Checks a condition and runs the guarded branch in SyncSecondEntryState.
1000	g_SecondEntryBlocked=(stored>=0.5);	Updates or initializes g_SecondEntryBlocked in SyncSecondEntryState.
1001	else	Fallback branch when prior conditions did not match in SyncSecondEntryState.
1002	g_SecondEntryBlocked=false;	Updates or initializes g_SecondEntryBlocked in SyncSecondEntryState.
1003		Blank separator for readability.
1004	if(cycleDone)	Checks a condition and runs the guarded branch in SyncSecondEntryState.
1005	g_SecondEntryDone=true;	Updates or initializes g_SecondEntryDone in SyncSecondEntryState.
1006	if(cycleBlocked)	Checks a condition and runs the guarded branch in SyncSecondEntryState.
1007	g_SecondEntryBlocked=true;	Updates or initializes g_SecondEntryBlocked in SyncSecondEntryState.
1008		Blank separator for readability.
1009	SaveSecondEntryState();	Calls helper function SaveSecondEntryState in SyncSecondEntryState.
1010	}	Opens or closes a code block in SyncSecondEntryState.
1011	else	Fallback branch when prior conditions did not match in SyncSecondEntryState.
1012	{	Opens or closes a code block in SyncSecondEntryState.
1013	if(cycleDone && !g_SecondEntryDone)	Checks a condition and runs the guarded branch in SyncSecondEntryState.
1014	{	Opens or closes a code block in SyncSecondEntryState.
1015	g_SecondEntryDone=true;	Updates or initializes g_SecondEntryDone in SyncSecondEntryState.
1016	SaveSecondEntryState();	Calls helper function SaveSecondEntryState in SyncSecondEntryState.
1017	}	Opens or closes a code block in SyncSecondEntryState.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1018	if(cycleBlocked && !g_SecondEntryBlocked)	Checks a condition and runs the guarded branch in SyncSecondEntryState.
1019	{	Opens or closes a code block in SyncSecondEntryState.
1020	g_SecondEntryBlocked=true;	Updates or initializes g_SecondEntryBlocked in SyncSecondEntryState.
1021	SaveSecondEntryState();	Calls helper function SaveSecondEntryState in SyncSecondEntryState.
1022	}	Opens or closes a code block in SyncSecondEntryState.
1023	}	Opens or closes a code block in SyncSecondEntryState.
1024	}	Opens or closes a code block in SyncSecondEntryState.
1025		Blank separator for readability.
1026	void SyncPositionState()	Declares or starts function SyncPositionState.
1027	{	Opens or closes a code block in SyncPositionState.
1028	if(!SelectOurPosition())	Checks a condition and runs the guarded branch in SyncPositionState.
1029	{	Opens or closes a code block in SyncPositionState.
1030	ResetLocalTradeState();	Calls helper function ResetLocalTradeState in SyncPositionState.
1031	return;	Returns a value or exits the current function in SyncPositionState.
1032	}	Opens or closes a code block in SyncPositionState.
1033		Blank separator for readability.
1034	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in SyncPositionState.
1035	double volume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double volume in SyncPositionState.
1036	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double entry in SyncPositionState.
1037	double sl=PositionGetDouble(POSITION_SL);	Updates or initializes double sl in SyncPositionState.
1038	double risk=MathAbs(entry-sl);	Updates or initializes double risk in SyncPositionState.
1039		Blank separator for readability.
1040	if(ticket!=g_StateTicket)	Checks a condition and runs the guarded branch in SyncPositionState.
1041	{	Opens or closes a code block in SyncPositionState.
1042	g_StateTicket=ticket;	Updates or initializes g_StateTicket in SyncPositionState.
1043		Blank separator for readability.
1044	double stored=0.0;	Updates or initializes double stored in SyncPositionState.
1045	if(LoadStateValue(ticket,"initial_volume",stored) && stored>0.0)	Checks a condition and runs the guarded branch in SyncPositionState.
1046	g_InitialVolume=stored;	Updates or initializes g_InitialVolume in SyncPositionState.
1047	else	Fallback branch when prior conditions did not match in SyncPositionState.
1048	g_InitialVolume=volume;	Updates or initializes g_InitialVolume in SyncPositionState.
1049		Blank separator for readability.
1050	if(LoadStateValue(ticket,"risk_distance",stored) && stored>0.0)	Checks a condition and runs the guarded branch in SyncPositionState.
1051	g_RiskDistance=stored;	Updates or initializes g_RiskDistance in SyncPositionState.
1052	else	Fallback branch when prior conditions did not match in SyncPositionState.
1053	g_RiskDistance=risk;	Updates or initializes g_RiskDistance in SyncPositionState.
1054		Blank separator for readability.
1055	if(LoadStateValue(ticket,"pc1_done",stored))	Checks a condition and runs the guarded branch in SyncPositionState.
1056	g_PC1_Done=(stored>=0.5);	Updates or initializes g_PC1_Done in SyncPositionState.
1057	else	Fallback branch when prior conditions did not match in SyncPositionState.
1058	g_PC1_Done=false;	Updates or initializes g_PC1_Done in SyncPositionState.
1059		Blank separator for readability.
1060	if(LoadStateValue(ticket,"pc2_done",stored))	Checks a condition and runs the guarded branch in SyncPositionState.
1061	g_PC2_Done=(stored>=0.5);	Updates or initializes g_PC2_Done in SyncPositionState.
1062	else	Fallback branch when prior conditions did not match in SyncPositionState.
1063	g_PC2_Done=false;	Updates or initializes g_PC2_Done in SyncPositionState.
1064		Blank separator for readability.
1065	if(LoadStateValue(ticket,"pc3_done",stored))	Checks a condition and runs the guarded branch in SyncPositionState.
1066	g_PC3_Done=(stored>=0.5);	Updates or initializes g_PC3_Done in SyncPositionState.
1067	else	Fallback branch when prior conditions did not match in SyncPositionState.
1068	g_PC3_Done=false;	Updates or initializes g_PC3_Done in SyncPositionState.
1069		Blank separator for readability.
1070	if(LoadStateValue(ticket,"be_done",stored))	Checks a condition and runs the guarded branch in SyncPositionState.
1071	g_BE_Done=(stored>=0.5);	Updates or initializes g_BE_Done in SyncPositionState.
1072	else	Fallback branch when prior conditions did not match in SyncPositionState.
1073	g_BE_Done=false;	Updates or initializes g_BE_Done in SyncPositionState.
1074		Blank separator for readability.
1075	SavePositionState();	Calls helper function SavePositionState in SyncPositionState.
1076	}	Opens or closes a code block in SyncPositionState.
1077	else	Fallback branch when prior conditions did not match in SyncPositionState.
1078	{	Opens or closes a code block in SyncPositionState.
1079	if(g_InitialVolume<=0.0)	Checks a condition and runs the guarded branch in SyncPositionState.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1080	g_InitialVolume=volume;	Updates or initializes g_InitialVolume in SyncPositionState.
1081	if(g_RiskDistance<=0.0 && risk>0.0)	Checks a condition and runs the guarded branch in SyncPositionState.
1082	g_RiskDistance=risk;	Updates or initializes g_RiskDistance in SyncPositionState.
1083	}	Opens or closes a code block in SyncPositionState.
1084	}	Opens or closes a code block in SyncPositionState.
1085		Blank separator for readability.
1086	//=====	Comment explaining the nearby EA code.
1087	// PATTERN DETECTION - MATCHES PROVIDED INDICATOR RULES	Comment explaining the nearby EA code.
1088	//=====	Comment explaining the nearby EA code.
1089	void ResetSignal(PatternSignal &sig)	Declares or starts function ResetSignal.
1090	{	Opens or closes a code block in ResetSignal.
1091	sig.ok=false;	Updates or initializes sig.ok in ResetSignal.
1092	sig.buy=false;	Updates or initializes sig.buy in ResetSignal.
1093	sig.swing=false;	Updates or initializes sig.swing in ResetSignal.
1094	sig.time=0;	Updates or initializes sig.time in ResetSignal.
1095	sig.c1o=0.0;	Updates or initializes sig.c1o in ResetSignal.
1096	sig.c1c=0.0;	Updates or initializes sig.c1c in ResetSignal.
1097	sig.c1h=0.0;	Updates or initializes sig.c1h in ResetSignal.
1098	sig.c1l=0.0;	Updates or initializes sig.c1l in ResetSignal.
1099	sig.c2o=0.0;	Updates or initializes sig.c2o in ResetSignal.
1100	sig.c2c=0.0;	Updates or initializes sig.c2c in ResetSignal.
1101	sig.c2h=0.0;	Updates or initializes sig.c2h in ResetSignal.
1102	sig.c2l=0.0;	Updates or initializes sig.c2l in ResetSignal.
1103	sig.c3o=0.0;	Updates or initializes sig.c3o in ResetSignal.
1104	sig.c3c=0.0;	Updates or initializes sig.c3c in ResetSignal.
1105	sig.c3h=0.0;	Updates or initializes sig.c3h in ResetSignal.
1106	sig.c3l=0.0;	Updates or initializes sig.c3l in ResetSignal.
1107	sig.sl=0.0;	Updates or initializes sig.sl in ResetSignal.
1108	sig.breakLevel=0.0;	Updates or initializes sig.breakLevel in ResetSignal.
1109	}	Opens or closes a code block in ResetSignal.
1110		Blank separator for readability.
1111	bool DetectPattern(int shift,bool wantBuy,PatternSignal &sig)	Declares or starts function DetectPattern.
1112	{	Opens or closes a code block in DetectPattern.
1113	ResetSignal(sig);	Calls helper function ResetSignal in DetectPattern.
1114		Blank separator for readability.
1115	if(!HasEnoughBars())	Checks a condition and runs the guarded branch in DetectPattern.
1116	return false;	Returns a value or exits the current function in DetectPattern.
1117	if(shift<1)	Checks a condition and runs the guarded branch in DetectPattern.
1118	shift=1;	Updates or initializes shift in DetectPattern.
1119	if(Bars(_Symbol,_Period)<shift+4)	Checks a condition and runs the guarded branch in DetectPattern.
1120	return false;	Returns a value or exits the current function in DetectPattern.
1121		Blank separator for readability.
1122	double c1o=iOpen(_Symbol,_Period,shift+2);	Updates or initializes double c1o in DetectPattern.
1123	double c1c=iClose(_Symbol,_Period,shift+2);	Updates or initializes double c1c in DetectPattern.
1124	double c1h=iHigh(_Symbol,_Period,shift+2);	Updates or initializes double c1h in DetectPattern.
1125	double c1l=iLow(_Symbol,_Period,shift+2);	Updates or initializes double c1l in DetectPattern.
1126		Blank separator for readability.
1127	double c2o=iOpen(_Symbol,_Period,shift+1);	Updates or initializes double c2o in DetectPattern.
1128	double c2c=iClose(_Symbol,_Period,shift+1);	Updates or initializes double c2c in DetectPattern.
1129	double c2h=iHigh(_Symbol,_Period,shift+1);	Updates or initializes double c2h in DetectPattern.
1130	double c2l=iLow(_Symbol,_Period,shift+1);	Updates or initializes double c2l in DetectPattern.
1131		Blank separator for readability.
1132	double c3o=iOpen(_Symbol,_Period,shift);	Updates or initializes double c3o in DetectPattern.
1133	double c3c=iClose(_Symbol,_Period,shift);	Updates or initializes double c3c in DetectPattern.
1134	double c3h=iHigh(_Symbol,_Period,shift);	Updates or initializes double c3h in DetectPattern.
1135	double c3l=iLow(_Symbol,_Period,shift);	Updates or initializes double c3l in DetectPattern.
1136		Blank separator for readability.
1137	if(c1o<=0.0 c2o<=0.0 c3o<=0.0)	Checks a condition and runs the guarded branch in DetectPattern.
1138	return false;	Returns a value or exits the current function in DetectPattern.
1139		Blank separator for readability.
1140	bool b1=c1c>c1o;	Updates or initializes bool b1 in DetectPattern.
1141	bool b2=c2c<c2o;	Updates or initializes bool b2 in DetectPattern.
1142	bool b3=c3c>c3o;	Updates or initializes bool b3 in DetectPattern.
1143	bool candle1IsLowest=(c1l<c2l && c1l<c3l);	Updates or initializes bool candle1IsLowest in DetectPattern.
1144	bool buySweep=InpAllowSweep ? (c2l<=c1l c3l<=c1l) : true;	Updates or initializes bool buySweep in DetectPattern.
1145	bool buyCloseBreak=c3c>c2o;	Updates or initializes bool buyCloseBreak in DetectPattern.
1146	bool buyOriginal=(!candle1IsLowest && buySweep && buyCloseBreak);	Updates or initializes bool buyOriginal in DetectPattern.
1147	bool buySwingStrong=(c3l<c2l && c3c>c2h);	Updates or initializes bool buySwingStrong in DetectPattern.
1148	bool buySwingWeak=(c3l<c2l && c3c>c2o);	Updates or initializes bool buySwingWeak in DetectPattern.
1149	bool buySwing=(InpAllowSwingPattern && (buySwingStrong buySwingWeak));	Updates or initializes bool buySwing in DetectPattern.
1150	bool buyOK=b1 && b2 && b3 && (buyOriginal buySwing);	Updates or initializes bool buyOK in DetectPattern.
1151		Blank separator for readability.
1152	bool s1=c1c<c1o;	Updates or initializes bool s1 in DetectPattern.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1153	bool s2=c2c>c2o;	Updates or initializes bool s2 in DetectPattern.
1154	bool s3=c3c<c3o;	Updates or initializes bool s3 in DetectPattern.
1155	bool candle1IsHighest=(c1h>c2h && c1h>c3h);	Updates or initializes bool candle1IsHighest in DetectPattern.
1156	bool sellSweep=InpAllowSweep ? (c2h>=c1h c3h>=c1h) : true;	Updates or initializes bool sellSweep in DetectPattern.
1157	bool sellCloseBreak=c3c<c2o;	Updates or initializes bool sellCloseBreak in DetectPattern.
1158	bool sellOriginal=(!candle1IsHighest && sellSweep && sellCloseBreak);	Updates or initializes bool sellOriginal in DetectPattern.
1159	bool sellSwingStrong=(c3h>c2h && c3c<c2l);	Updates or initializes bool sellSwingStrong in DetectPattern.
1160	bool sellSwingWeak=(c3h>c2h && c3c<c2o);	Updates or initializes bool sellSwingWeak in DetectPattern.
1161	bool sellSwing=(InpAllowSwingPattern && (sellSwingStrong sellSwingWeak));	Updates or initializes bool sellSwing in DetectPattern.
1162	bool sellOK=s1 && s2 && s3 && (sellOriginal sellSwing);	Updates or initializes bool sellOK in DetectPattern.
1163		Blank separator for readability.
1164	if(wantBuy && !buyOK)	Checks a condition and runs the guarded branch in DetectPattern.
1165	return false;	Returns a value or exits the current function in DetectPattern.
1166	if(!wantBuy && !sellOK)	Checks a condition and runs the guarded branch in DetectPattern.
1167	return false;	Returns a value or exits the current function in DetectPattern.
1168		Blank separator for readability.
1169	sig.ok=true;	Updates or initializes sig.ok in DetectPattern.
1170	sig.buy=wantBuy;	Updates or initializes sig.buy in DetectPattern.
1171	sig.swing=wantBuy ? buySwing : sellSwing;	Updates or initializes sig.swing in DetectPattern.
1172	sig.time=iTime(_Symbol,_Period,shift);	Updates or initializes sig.time in DetectPattern.
1173	sig.c1o=c1o;	Updates or initializes sig.c1o in DetectPattern.
1174	sig.c1c=c1c;	Updates or initializes sig.c1c in DetectPattern.
1175	sig.c1h=c1h;	Updates or initializes sig.c1h in DetectPattern.
1176	sig.c1l=c1l;	Updates or initializes sig.c1l in DetectPattern.
1177	sig.c2o=c2o;	Updates or initializes sig.c2o in DetectPattern.
1178	sig.c2c=c2c;	Updates or initializes sig.c2c in DetectPattern.
1179	sig.c2h=c2h;	Updates or initializes sig.c2h in DetectPattern.
1180	sig.c2l=c2l;	Updates or initializes sig.c2l in DetectPattern.
1181	sig.c3o=c3o;	Updates or initializes sig.c3o in DetectPattern.
1182	sig.c3c=c3c;	Updates or initializes sig.c3c in DetectPattern.
1183	sig.c3h=c3h;	Updates or initializes sig.c3h in DetectPattern.
1184	sig.c3l=c3l;	Updates or initializes sig.c3l in DetectPattern.
1185		Blank separator for readability.
1186	double buffer=MathMax(0.0,(double)InpStopBufferPoints)*_Point;	Updates or initializes double buffer in DetectPattern.
1187		Blank separator for readability.
1188	bool isSwingStop=wantBuy ? buySwing : sellSwing;	Updates or initializes bool isSwingStop in DetectPattern.
1189		Blank separator for readability.
1190	if(wantBuy)	Checks a condition and runs the guarded branch in DetectPattern.
1191	{	Opens or closes a code block in DetectPattern.
1192	double baseSL=0.0;	Updates or initializes double baseSL in DetectPattern.
1193	if(isSwingStop)	Checks a condition and runs the guarded branch in DetectPattern.
1194	baseSL=c3l;	Updates or initializes baseSL in DetectPattern.
1195	else	Fallback branch when prior conditions did not match in DetectPattern.
1196	baseSL=(InpStopMode==STOP_FROM_CANDLE2_LINE ? c2l : MathMin(c1l,MathMin(c2l,c3l)));	Updates or initializes baseSL in DetectPattern.
1197	sig.sl=NormalizeDouble(baseSL-buffer,_Digits);	Updates or initializes sig.sl in DetectPattern.
1198	sig.breakLevel=sig.swing ? (buySwingStrong ? c2h : c2o) : c2o;	Updates or initializes sig.breakLevel in DetectPattern.
1199	}	Opens or closes a code block in DetectPattern.
1200	else	Fallback branch when prior conditions did not match in DetectPattern.
1201	{	Opens or closes a code block in DetectPattern.
1202	double baseSL=0.0;	Updates or initializes double baseSL in DetectPattern.
1203	if(isSwingStop)	Checks a condition and runs the guarded branch in DetectPattern.
1204	baseSL=c3h;	Updates or initializes baseSL in DetectPattern.
1205	else	Fallback branch when prior conditions did not match in DetectPattern.
1206	baseSL=(InpStopMode==STOP_FROM_CANDLE2_LINE ? c2h : MathMax(c1h,MathMax(c2h,c3h)));	Updates or initializes baseSL in DetectPattern.
1207	sig.sl=NormalizeDouble(baseSL+buffer,_Digits);	Updates or initializes sig.sl in DetectPattern.
1208	sig.breakLevel=sig.swing ? (sellSwingStrong ? c2l : c2o) : c2o;	Updates or initializes sig.breakLevel in DetectPattern.
1209	}	Opens or closes a code block in DetectPattern.
1210		Blank separator for readability.
1211	return true;	Returns a value or exits the current function in DetectPattern.
1212	}	Opens or closes a code block in DetectPattern.
1213		Blank separator for readability.
1214	bool IsJustClosedThirdCandleSignal(PatternSignal &sig)	Declares or starts function IsJustClosedThirdCandleSignal.
1215	{	Opens or closes a code block in IsJustClosedThirdCandleSignal.
1216	if(!sig.ok sig.time<=0)	Checks a condition and runs the guarded branch in IsJustClosedThirdCandleSignal.
1217	return false;	Returns a value or exits the current function in IsJustClosedThirdCandleSignal.
1218		Blank separator for readability.
1219	datetime currentBar=iTime(_Symbol,_Period,0);	Updates or initializes datetime currentBar in IsJustClosedThirdCandleSignal.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1220	<code>datetime justClosedBar=iTime(_Symbol,_Period,1);</code>	Updates or initializes datetime justClosedBar in IsJustClosedThirdCandleSignal.
1221	<code>if(currentBar<=0 justClosedBar<=0)</code>	Checks a condition and runs the guarded branch in IsJustClosedThirdCandleSignal.
1222	<code>return false;</code>	Returns a value or exits the current function in IsJustClosedThirdCandleSignal.
1223		Blank separator for readability.
1224	<code>if(currentBar<=justClosedBar)</code>	Checks a condition and runs the guarded branch in IsJustClosedThirdCandleSignal.
1225	<code>return false;</code>	Returns a value or exits the current function in IsJustClosedThirdCandleSignal.
1226		Blank separator for readability.
1227	<code>return (sig.time==justClosedBar);</code>	Returns a value or exits the current function in IsJustClosedThirdCandleSignal.
1228	<code>}</code>	Opens or closes a code block in IsJustClosedThirdCandleSignal.
1229		Blank separator for readability.
1230	<code>//=====</code>	Comment explaining the nearby EA code.
1231	<code>// CHART SIGNAL MARKERS</code>	Comment explaining the nearby EA code.
1232	<code>//=====</code>	Comment explaining the nearby EA code.
1233	<code>void DrawSignalMarker(PatternSignal &sig)</code>	Declares or starts function DrawSignalMarker.
1234	<code>{</code>	Opens or closes a code block in DrawSignalMarker.
1235	<code>if(!InpShowSignalArrows !sig.ok)</code>	Checks a condition and runs the guarded branch in DrawSignalMarker.
1236	<code>return;</code>	Returns a value or exits the current function in DrawSignalMarker.
1237		Blank separator for readability.
1238	<code>string side=sig.buy ? "BUY" : "SELL";</code>	Updates or initializes string side in DrawSignalMarker.
1239	<code>string name=PX+"SIG_"+side+"_"+IntegerToString((long)sig.time);</code>	Updates or initializes string name in DrawSignalMarker.
1240	<code>if(ObjectFind(0,name)>=0)</code>	Checks a condition and runs the guarded branch in DrawSignalMarker.
1241	<code>return;</code>	Returns a value or exits the current function in DrawSignalMarker.
1242		Blank separator for readability.
1243	<code>double candleRange=MathMax(_Point,MathAbs(sig.c3h-sig.c3l));</code>	Updates or initializes double candleRange in DrawSignalMarker.
1244	<code>double arrowGap=MathMax(60*_Point,candleRange*0.35);</code>	Updates or initializes double arrowGap in DrawSignalMarker.
1245	<code>double textGap=MathMax(95.0*_Point,candleRange*0.55);</code>	Updates or initializes double textGap in DrawSignalMarker.
1246	<code>double y=sig.buy ? sig.c3l-arrowGap : sig.c3h+arrowGap;</code>	Updates or initializes double y in DrawSignalMarker.
1247	<code>double textY=sig.buy ? sig.c3l-textGap : sig.c3h+textGap;</code>	Updates or initializes double textY in DrawSignalMarker.
1248		Blank separator for readability.
1249	<code>ObjectCreate(0,name,OBJ_ARROW,0,sig.time,y);</code>	Creates, updates, or removes an MT5 chart panel object.
1250	<code>ObjectSetInteger(0,name,OBJPROP_ARROWCODE,sig.buy ? 233 : 234);</code>	Creates, updates, or removes an MT5 chart panel object.
1251	<code>ObjectSetInteger(0,name,OBJPROP_COLOR,sig.buy ? clrLime : clrTomato);</code>	Creates, updates, or removes an MT5 chart panel object.
1252	<code>ObjectSetInteger(0,name,OBJPROP_WIDTH,1);</code>	Creates, updates, or removes an MT5 chart panel object.
1253	<code>ObjectSetInteger(0,name,OBJPROP_SELECTABLE,false);</code>	Creates, updates, or removes an MT5 chart panel object.
1254	<code>ObjectSetInteger(0,name,OBJPROP_HIDDEN,true);</code>	Creates, updates, or removes an MT5 chart panel object.
1255		Blank separator for readability.
1256	<code>string textName=name+"_TXT";</code>	Updates or initializes string textName in DrawSignalMarker.
1257	<code>ObjectCreate(0,textName,OBJ_TEXT,0,sig.time,textY);</code>	Creates, updates, or removes an MT5 chart panel object.
1258	<code>ObjectSetString(0,textName,OBJPROP_TEXT,sig.swing ? side+" SWING" : side);</code>	Creates, updates, or removes an MT5 chart panel object.
1259	<code>ObjectSetInteger(0,textName,OBJPROP_COLOR,sig.buy ? clrLime : clrTomato);</code>	Creates, updates, or removes an MT5 chart panel object.
1260	<code>ObjectSetInteger(0,textName,OBJPROP_FONTSIZE,7);</code>	Creates, updates, or removes an MT5 chart panel object.
1261	<code>ObjectSetString(0,textName,OBJPROP_FONT,"Arial Bold");</code>	Creates, updates, or removes an MT5 chart panel object.
1262	<code>ObjectSetInteger(0,textName,OBJPROP_SELECTABLE,false);</code>	Creates, updates, or removes an MT5 chart panel object.
1263	<code>ObjectSetInteger(0,textName,OBJPROP_HIDDEN,true);</code>	Creates, updates, or removes an MT5 chart panel object.
1264	<code>}</code>	Opens or closes a code block in DrawSignalMarker.
1265		Blank separator for readability.
1266	<code>void DeleteTradeLevelObjects()</code>	Declares or starts function DeleteTradeLevelObjects.
1267	<code>{</code>	Opens or closes a code block in DeleteTradeLevelObjects.
1268	<code>string tradePrefix=PX+"TRADE_";</code>	Updates or initializes string tradePrefix in DeleteTradeLevelObjects.
1269	<code>for(int i=ObjectsTotal(0)-1;i>=0;i--)</code>	Loops over a fixed range or collection in DeleteTradeLevelObjects.
1270	<code>{</code>	Opens or closes a code block in DeleteTradeLevelObjects.
1271	<code>string name=ObjectName(0,i);</code>	Updates or initializes string name in DeleteTradeLevelObjects.
1272	<code>if(StringFind(name,tradePrefix)==0)</code>	Checks a condition and runs the guarded branch in DeleteTradeLevelObjects.
1273	<code>ObjectDelete(0,name);</code>	Creates, updates, or removes an MT5 chart panel object.
1274	<code>}</code>	Opens or closes a code block in DeleteTradeLevelObjects.
1275	<code>}</code>	Opens or closes a code block in DeleteTradeLevelObjects.
1276		Blank separator for readability.
1277	<code>void DrawTradeLevels(ulong ticket,bool buy,double sl,double tp)</code>	Declares or starts function DrawTradeLevels.
1278	<code>{</code>	Opens or closes a code block in DrawTradeLevels.
1279	<code>// UI request: do not draw SL/TP lines on chart.</code>	Comment explaining the nearby EA code.
1280	<code>// Broker SL/TP still works because trade execution logic is unchanged.</code>	Comment explaining the nearby EA code.
1281	<code>string base=PX+"TRADE_"+IntegerToString((long)ticket)+"_";</code>	Updates or initializes string base in DrawTradeLevels.
1282	<code>ObjectDelete(0,base+"SL");</code>	Creates, updates, or removes an MT5 chart panel object.
1283	<code>ObjectDelete(0,base+"TP");</code>	Creates, updates, or removes an MT5 chart panel object.
1284	<code>}</code>	Opens or closes a code block in DrawTradeLevels.
1285		Blank separator for readability.
1286		Blank separator for readability.
1287	<code>void DrawHistoricalSignals()</code>	Declares or starts function DrawHistoricalSignals.
1288	<code>{</code>	Opens or closes a code block in DrawHistoricalSignals.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1289	if(!InpShowSignalArrows)	Checks a condition and runs the guarded branch in DrawHistoricalSignals.
1290	return;	Returns a value or exits the current function in DrawHistoricalSignals.
1291		Blank separator for readability.
1292	int bars=Bars(_Symbol,_Period);	Updates or initializes int bars in DrawHistoricalSignals.
1293	int maxShift=MathMin(InpSignalLookbackBars,bars-4);	Updates or initializes int maxShift in DrawHistoricalSignals.
1294	if(maxShift<1)	Checks a condition and runs the guarded branch in DrawHistoricalSignals.
1295	return;	Returns a value or exits the current function in DrawHistoricalSignals.
1296		Blank separator for readability.
1297	for(int shift=maxShift;shift>=1;shift--)	Loops over a fixed range or collection in DrawHistoricalSignals.
1298	{	Opens or closes a code block in DrawHistoricalSignals.
1299	PatternSignal sig;	Executes one statement in DrawHistoricalSignals.
1300	if(DetectPattern(shift,true,sig))	Checks a condition and runs the guarded branch in DrawHistoricalSignals.
1301	DrawSignalMarker(sig);	Calls helper function DrawSignalMarker in DrawHistoricalSignals.
1302	if(DetectPattern(shift,false,sig))	Checks a condition and runs the guarded branch in DrawHistoricalSignals.
1303	DrawSignalMarker(sig);	Calls helper function DrawSignalMarker in DrawHistoricalSignals.
1304	}	Opens or closes a code block in DrawHistoricalSignals.
1305	}	Opens or closes a code block in DrawHistoricalSignals.
1306		Blank separator for readability.
1307	void DrawLatestClosedSignal()	Declares or starts function DrawLatestClosedSignal.
1308	{	Opens or closes a code block in DrawLatestClosedSignal.
1309	if(!InpShowSignalArrows)	Checks a condition and runs the guarded branch in DrawLatestClosedSignal.
1310	return;	Returns a value or exits the current function in DrawLatestClosedSignal.
1311		Blank separator for readability.
1312	PatternSignal sig;	Executes one statement in DrawLatestClosedSignal.
1313	if(DetectPattern(1,true,sig))	Checks a condition and runs the guarded branch in DrawLatestClosedSignal.
1314	DrawSignalMarker(sig);	Calls helper function DrawSignalMarker in DrawLatestClosedSignal.
1315	if(DetectPattern(1,false,sig))	Checks a condition and runs the guarded branch in DrawLatestClosedSignal.
1316	DrawSignalMarker(sig);	Calls helper function DrawSignalMarker in DrawLatestClosedSignal.
1317	}	Opens or closes a code block in DrawLatestClosedSignal.
1318		Blank separator for readability.
1319	//=====	Comment explaining the nearby EA code.
1320	// TRADE VALIDATION / EXECUTION	Comment explaining the nearby EA code.
1321	//=====	Comment explaining the nearby EA code.
1322	bool IsTradingAllowedForDirection(bool buy,string &reason)	Declares or starts function IsTradingAllowedForDirection.
1323	{	Opens or closes a code block in IsTradingAllowedForDirection.
1324	reason="";	Updates or initializes reason in IsTradingAllowedForDirection.
1325		Blank separator for readability.
1326	if(!TerminalInfoInteger(TERMINAL_TRADE_ALLOWED))	Checks a condition and runs the guarded branch in IsTradingAllowedForDirection.
1327	{	Opens or closes a code block in IsTradingAllowedForDirection.
1328	reason="Terminal Algo Trading is disabled";	Updates or initializes reason in IsTradingAllowedForDirection.
1329	return false;	Returns a value or exits the current function in IsTradingAllowedForDirection.
1330	}	Opens or closes a code block in IsTradingAllowedForDirection.
1331		Blank separator for readability.
1332	if(!MQLInfoInteger(MQL_TRADE_ALLOWED))	Checks a condition and runs the guarded branch in IsTradingAllowedForDirection.
1333	{	Opens or closes a code block in IsTradingAllowedForDirection.
1334	reason="EA trading permission is disabled";	Updates or initializes reason in IsTradingAllowedForDirection.
1335	return false;	Returns a value or exits the current function in IsTradingAllowedForDirection.
1336	}	Opens or closes a code block in IsTradingAllowedForDirection.
1337		Blank separator for readability.
1338	if(!AccountInfoInteger(ACCOUNT_TRADE_ALLOWED))	Checks a condition and runs the guarded branch in IsTradingAllowedForDirection.
1339	{	Opens or closes a code block in IsTradingAllowedForDirection.
1340	reason="Account trading is disabled";	Updates or initializes reason in IsTradingAllowedForDirection.
1341	return false;	Returns a value or exits the current function in IsTradingAllowedForDirection.
1342	}	Opens or closes a code block in IsTradingAllowedForDirection.
1343		Blank separator for readability.
1344	long mode=SymbolInfoInteger(_Symbol,SYMBOL_TRADE_MODE);	Updates or initializes long mode in IsTradingAllowedForDirection.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1345	<code>if(mode==SYMBOL_TRADE_MODE_DISABLED)</code>	Checks a condition and runs the guarded branch in <code>IsTradingAllowedForDirection</code> .
1346	<code>{</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1347	<code>reason="Symbol trading is disabled";</code>	Updates or initializes reason in <code>IsTradingAllowedForDirection</code> .
1348	<code>return false;</code>	Returns a value or exits the current function in <code>IsTradingAllowedForDirection</code> .
1349	<code>}</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1350	<code>if(mode==SYMBOL_TRADE_MODE_CLOSEONLY)</code>	Checks a condition and runs the guarded branch in <code>IsTradingAllowedForDirection</code> .
1351	<code>{</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1352	<code>reason="Symbol is close-only";</code>	Updates or initializes reason in <code>IsTradingAllowedForDirection</code> .
1353	<code>return false;</code>	Returns a value or exits the current function in <code>IsTradingAllowedForDirection</code> .
1354	<code>}</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1355	<code>if(buy && mode==SYMBOL_TRADE_MODE_SHORTONLY)</code>	Checks a condition and runs the guarded branch in <code>IsTradingAllowedForDirection</code> .
1356	<code>{</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1357	<code>reason="Symbol allows sell only";</code>	Updates or initializes reason in <code>IsTradingAllowedForDirection</code> .
1358	<code>return false;</code>	Returns a value or exits the current function in <code>IsTradingAllowedForDirection</code> .
1359	<code>}</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1360	<code>if(!buy && mode==SYMBOL_TRADE_MODE_LONGONLY)</code>	Checks a condition and runs the guarded branch in <code>IsTradingAllowedForDirection</code> .
1361	<code>{</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1362	<code>reason="Symbol allows buy only";</code>	Updates or initializes reason in <code>IsTradingAllowedForDirection</code> .
1363	<code>return false;</code>	Returns a value or exits the current function in <code>IsTradingAllowedForDirection</code> .
1364	<code>}</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1365		Blank separator for readability.
1366	<code>return true;</code>	Returns a value or exits the current function in <code>IsTradingAllowedForDirection</code> .
1367	<code>}</code>	Opens or closes a code block in <code>IsTradingAllowedForDirection</code> .
1368		Blank separator for readability.
1369	<code>bool SpreadAllowed(string &reason)</code>	Declares or starts function <code>SpreadAllowed</code> .
1370	<code>{</code>	Opens or closes a code block in <code>SpreadAllowed</code> .
1371	<code>reason="";</code>	Updates or initializes reason in <code>SpreadAllowed</code> .
1372	<code>int spread=CurrentSpreadPoints();</code>	Updates or initializes int <code>spread</code> in <code>SpreadAllowed</code> .
1373		Blank separator for readability.
1374	<code>if(SpreadBlocksTrades() && spread>InpMaxSpreadPoints)</code>	Checks a condition and runs the guarded branch in <code>SpreadAllowed</code> .
1375	<code>{</code>	Opens or closes a code block in <code>SpreadAllowed</code> .
1376	<code>reason="Spread too high: "+IntegerToString(spread)+" pts";</code>	Updates or initializes reason in <code>SpreadAllowed</code> .
1377	<code>return false;</code>	Returns a value or exits the current function in <code>SpreadAllowed</code> .
1378	<code>}</code>	Opens or closes a code block in <code>SpreadAllowed</code> .
1379		Blank separator for readability.
1380	<code>return true;</code>	Returns a value or exits the current function in <code>SpreadAllowed</code> .
1381	<code>}</code>	Opens or closes a code block in <code>SpreadAllowed</code> .
1382		Blank separator for readability.
1383	<code>bool ValidateStops(bool buy,double sl,double tp,string &reason)</code>	Declares or starts function <code>ValidateStops</code> .
1384	<code>{</code>	Opens or closes a code block in <code>ValidateStops</code> .
1385	<code>reason="";</code>	Updates or initializes reason in <code>ValidateStops</code> .
1386		Blank separator for readability.
1387	<code>double ask=SymbolInfoDouble(_Symbol,SYMBOL_ASK);</code>	Updates or initializes double <code>ask</code> in <code>ValidateStops</code> .
1388	<code>double bid=SymbolInfoDouble(_Symbol,SYMBOL_BID);</code>	Updates or initializes double <code>bid</code> in <code>ValidateStops</code> .
1389	<code>int minPts=RequiredStopPoints();</code>	Updates or initializes int <code>minPts</code> in <code>ValidateStops</code> .
1390	<code>double minDist=minPts*_Point;</code>	Updates or initializes double <code>minDist</code> in <code>ValidateStops</code> .
1391		Blank separator for readability.
1392	<code>if(ask<=0.0 bid<=0.0)</code>	Checks a condition and runs the guarded branch in <code>ValidateStops</code> .
1393	<code>{</code>	Opens or closes a code block in <code>ValidateStops</code> .
1394	<code>reason="No valid bid/ask price";</code>	Updates or initializes reason in <code>ValidateStops</code> .
1395	<code>return false;</code>	Returns a value or exits the current function in <code>ValidateStops</code> .
1396	<code>}</code>	Opens or closes a code block in <code>ValidateStops</code> .
1397		Blank separator for readability.
1398	<code>if(buy)</code>	Checks a condition and runs the guarded branch in <code>ValidateStops</code> .
1399	<code>{</code>	Opens or closes a code block in <code>ValidateStops</code> .
1400	<code>if(sl>=ask)</code>	Checks a condition and runs the guarded branch in <code>ValidateStops</code> .
1401	<code>{</code>	Opens or closes a code block in <code>ValidateStops</code> .
1402	<code>reason="BUY SL is not below entry";</code>	Updates or initializes reason in <code>ValidateStops</code> .
1403	<code>return false;</code>	Returns a value or exits the current function in <code>ValidateStops</code> .
1404	<code>}</code>	Opens or closes a code block in <code>ValidateStops</code> .

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1405	if(tp>0.0 && tp<=ask)	Checks a condition and runs the guarded branch in ValidateStops.
1406	{	Opens or closes a code block in ValidateStops.
1407	reason="BUY TP is not above entry";	Updates or initializes reason in ValidateStops.
1408	return false;	Returns a value or exits the current function in ValidateStops.
1409	}	Opens or closes a code block in ValidateStops.
1410	if(bid-sl<minDist)	Checks a condition and runs the guarded branch in ValidateStops.
1411	{	Opens or closes a code block in ValidateStops.
1412	reason="BUY SL too close for broker stop level";	Updates or initializes reason in ValidateStops.
1413	return false;	Returns a value or exits the current function in ValidateStops.
1414	}	Opens or closes a code block in ValidateStops.
1415	if(tp>0.0 && tp-bid<minDist)	Checks a condition and runs the guarded branch in ValidateStops.
1416	{	Opens or closes a code block in ValidateStops.
1417	reason="BUY TP too close for broker stop level";	Updates or initializes reason in ValidateStops.
1418	return false;	Returns a value or exits the current function in ValidateStops.
1419	}	Opens or closes a code block in ValidateStops.
1420	}	Opens or closes a code block in ValidateStops.
1421	else	Fallback branch when prior conditions did not match in ValidateStops.
1422	{	Opens or closes a code block in ValidateStops.
1423	if(sl<=bid)	Checks a condition and runs the guarded branch in ValidateStops.
1424	{	Opens or closes a code block in ValidateStops.
1425	reason="SELL SL is not above entry";	Updates or initializes reason in ValidateStops.
1426	return false;	Returns a value or exits the current function in ValidateStops.
1427	}	Opens or closes a code block in ValidateStops.
1428	if(tp>0.0 && tp>=bid)	Checks a condition and runs the guarded branch in ValidateStops.
1429	{	Opens or closes a code block in ValidateStops.
1430	reason="SELL TP is not below entry";	Updates or initializes reason in ValidateStops.
1431	return false;	Returns a value or exits the current function in ValidateStops.
1432	}	Opens or closes a code block in ValidateStops.
1433	if(sl-ask<minDist)	Checks a condition and runs the guarded branch in ValidateStops.
1434	{	Opens or closes a code block in ValidateStops.
1435	reason="SELL SL too close for broker stop level";	Updates or initializes reason in ValidateStops.
1436	return false;	Returns a value or exits the current function in ValidateStops.
1437	}	Opens or closes a code block in ValidateStops.
1438	if(tp>0.0 && ask-tp<minDist)	Checks a condition and runs the guarded branch in ValidateStops.
1439	{	Opens or closes a code block in ValidateStops.
1440	reason="SELL TP too close for broker stop level";	Updates or initializes reason in ValidateStops.
1441	return false;	Returns a value or exits the current function in ValidateStops.
1442	}	Opens or closes a code block in ValidateStops.
1443	}	Opens or closes a code block in ValidateStops.
1444		Blank separator for readability.
1445	return true;	Returns a value or exits the current function in ValidateStops.
1446	}	Opens or closes a code block in ValidateStops.
1447		Blank separator for readability.
1448	double MoneyRiskForLot(bool buy,double entry,double sl,double lot)	Declares or starts function MoneyRiskForLot.
1449	{	Opens or closes a code block in MoneyRiskForLot.
1450	if(lot<=0.0 entry<=0.0 sl<=0.0)	Checks a condition and runs the guarded branch in MoneyRiskForLot.
1451	return 0.0;	Returns a value or exits the current function in MoneyRiskForLot.
1452		Blank separator for readability.
1453	double priceRisk=buy ? (entry-sl) : (sl-entry);	Updates or initializes double priceRisk in MoneyRiskForLot.
1454	if(priceRisk<=_Point*0.1)	Checks a condition and runs the guarded branch in MoneyRiskForLot.
1455	return 0.0;	Returns a value or exits the current function in MoneyRiskForLot.
1456		Blank separator for readability.
1457	ENUM_ORDER_TYPE orderType=ORDER_TYPE_BUY;	Updates or initializes ENUM_ORDER_TYPE orderType in MoneyRiskForLot.
1458	if(!buy)	Checks a condition and runs the guarded branch in MoneyRiskForLot.
1459	orderType=ORDER_TYPE_SELL;	Updates or initializes orderType in MoneyRiskForLot.
1460	double profit=0.0;	Updates or initializes double profit in MoneyRiskForLot.
1461	if(OrderCalcProfit(orderType,_Symbol,lot,entry,sl,profit))	Checks a condition and runs the guarded branch in MoneyRiskForLot.
1462	return MathAbs(profit);	Returns a value or exits the current function in MoneyRiskForLot.
1463		Blank separator for readability.
1464	double tickSize=SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE);	Updates or initializes double tickSize in MoneyRiskForLot.
1465	double tickValue=SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_VALUE);	Updates or initializes double tickValue in MoneyRiskForLot.
1466	if(tickSize<=0.0 tickValue<=0.0)	Checks a condition and runs the guarded branch in MoneyRiskForLot.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1467	return 0.0;	Returns a value or exits the current function in MoneyRiskForLot.
1468		Blank separator for readability.
1469	return MathAbs(entry-sl)/tickSize*tickValue*lot;	Returns a value or exits the current function in MoneyRiskForLot.
1470	}	Opens or closes a code block in MoneyRiskForLot.
1471		Blank separator for readability.
1472	double FitVolumeToFreeMargin(bool buy,double volume,double entry,string &reason)	Declares or starts function FitVolumeToFreeMargin.
1473	{	Opens or closes a code block in FitVolumeToFreeMargin.
1474	reason="";	Updates or initializes reason in FitVolumeToFreeMargin.
1475		Blank separator for readability.
1476	double minLot=BrokerMinLot();	Updates or initializes double minLot in FitVolumeToFreeMargin.
1477	double step=BrokerStepLot();	Updates or initializes double step in FitVolumeToFreeMargin.
1478	double lot=NormalizeVolumeDown(volume);	Updates or initializes double lot in FitVolumeToFreeMargin.
1479	if(lot<=0.0)	Checks a condition and runs the guarded branch in FitVolumeToFreeMargin.
1480	lot=minLot;	Updates or initializes lot in FitVolumeToFreeMargin.
1481		Blank separator for readability.
1482	double freeMargin=AccountInfoDouble(ACCOUNT_MARGIN_FREE);	Updates or initializes double freeMargin in FitVolumeToFreeMargin.
1483	if(freeMargin<=0.0)	Checks a condition and runs the guarded branch in FitVolumeToFreeMargin.
1484	return lot;	Returns a value or exits the current function in FitVolumeToFreeMargin.
1485		Blank separator for readability.
1486	ENUM_ORDER_TYPE orderType=ORDER_TYPE_BUY;	Updates or initializes ENUM_ORDER_TYPE orderType in FitVolumeToFreeMargin.
1487	if(!buy)	Checks a condition and runs the guarded branch in FitVolumeToFreeMargin.
1488	orderType=ORDER_TYPE_SELL;	Updates or initializes orderType in FitVolumeToFreeMargin.
1489	while(lot+0.000000001>=minLot)	Repeats while the condition remains true in FitVolumeToFreeMargin.
1490	{	Opens or closes a code block in FitVolumeToFreeMargin.
1491	double margin=0.0;	Updates or initializes double margin in FitVolumeToFreeMargin.
1492	if(!OrderCalcMargin(orderType,_Symbol,lot,entry,margin))	Checks a condition and runs the guarded branch in FitVolumeToFreeMargin.
1493	return lot;	Returns a value or exits the current function in FitVolumeToFreeMargin.
1494		Blank separator for readability.
1495	if(margin<=freeMargin)	Checks a condition and runs the guarded branch in FitVolumeToFreeMargin.
1496	return NormalizeDouble(lot,VolumeDigits());	Returns a value or exits the current function in FitVolumeToFreeMargin.
1497		Blank separator for readability.
1498	double nextLot=NormalizeVolumeDown(lot-step);	Updates or initializes double nextLot in FitVolumeToFreeMargin.
1499	if(nextLot<=0.0 nextLot>=lot)	Checks a condition and runs the guarded branch in FitVolumeToFreeMargin.
1500	break;	Executes one statement in FitVolumeToFreeMargin.
1501	lot=nextLot;	Updates or initializes lot in FitVolumeToFreeMargin.
1502	}	Opens or closes a code block in FitVolumeToFreeMargin.
1503		Blank separator for readability.
1504	reason="Not enough free margin for min lot";	Updates or initializes reason in FitVolumeToFreeMargin.
1505	return 0.0;	Returns a value or exits the current function in FitVolumeToFreeMargin.
1506	}	Opens or closes a code block in FitVolumeToFreeMargin.
1507		Blank separator for readability.
1508	bool CalculateRiskVolume(bool buy,double entry,double sl,double &lot,double &targetRisk,double &actualRisk,string &reason)	Declares or starts function CalculateRiskVolume.
1509	{	Opens or closes a code block in CalculateRiskVolume.
1510	lot=0.0;	Updates or initializes lot in CalculateRiskVolume.
1511	actualRisk=0.0;	Updates or initializes actualRisk in CalculateRiskVolume.
1512	reason="";	Updates or initializes reason in CalculateRiskVolume.
1513		Blank separator for readability.
1514	targetRisk=NormalizeRiskMoney(g_RiskMoney);	Updates or initializes targetRisk in CalculateRiskVolume.
1515		Blank separator for readability.
1516	if(targetRisk<=0.0)	Checks a condition and runs the guarded branch in CalculateRiskVolume.
1517	{	Opens or closes a code block in CalculateRiskVolume.
1518	reason="Risk amount is zero";	Updates or initializes reason in CalculateRiskVolume.
1519	return false;	Returns a value or exits the current function in CalculateRiskVolume.
1520	}	Opens or closes a code block in CalculateRiskVolume.
1521		Blank separator for readability.
1522	double riskPerLot=MoneyRiskForLot(buy,entry,sl,1.0);	Updates or initializes double riskPerLot in CalculateRiskVolume.
1523	if(riskPerLot<=0.0)	Checks a condition and runs the guarded branch in CalculateRiskVolume.
1524	{	Opens or closes a code block in CalculateRiskVolume.
1525	reason="Cannot calculate risk value for this symbol";	Updates or initializes reason in CalculateRiskVolume.
1526	return false;	Returns a value or exits the current function in CalculateRiskVolume.
1527	}	Opens or closes a code block in CalculateRiskVolume.
1528		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1529	double rawLot=targetRisk/riskPerLot;	Updates or initializes double rawLot in CalculateRiskVolume.
1530	if(rawLot>BrokerMaxLot())	Checks a condition and runs the guarded branch in CalculateRiskVolume.
1531	rawLot=BrokerMaxLot();	Updates or initializes rawLot in CalculateRiskVolume.
1532		Blank separator for readability.
1533	lot=NormalizeVolumeDown(rawLot);	Updates or initializes lot in CalculateRiskVolume.
1534	if(lot<=0.0)	Checks a condition and runs the guarded branch in CalculateRiskVolume.
1535	lot=BrokerMinLot();	Updates or initializes lot in CalculateRiskVolume.
1536		Blank separator for readability.
1537	lot=FitVolumeToFreeMargin(buy, lot, entry, reason);	Updates or initializes lot in CalculateRiskVolume.
1538	if(lot<=0.0)	Checks a condition and runs the guarded branch in CalculateRiskVolume.
1539	return false;	Returns a value or exits the current function in CalculateRiskVolume.
1540		Blank separator for readability.
1541	actualRisk=MoneyRiskForLot(buy, entry, sl, lot);	Updates or initializes actualRisk in CalculateRiskVolume.
1542	return true;	Returns a value or exits the current function in CalculateRiskVolume.
1543	}	Opens or closes a code block in CalculateRiskVolume.
1544		Blank separator for readability.
1545	bool GetRiskReference(bool &buy,double &entry,double &sl)	Declares or starts function GetRiskReference.
1546	{	Opens or closes a code block in GetRiskReference.
1547	buy=true;	Updates or initializes buy in GetRiskReference.
1548	entry=0.0;	Updates or initializes entry in GetRiskReference.
1549	sl=0.0;	Updates or initializes sl in GetRiskReference.
1550		Blank separator for readability.
1551	if(SelectOurPosition())	Checks a condition and runs the guarded branch in GetRiskReference.
1552	{	Opens or closes a code block in GetRiskReference.
1553	buy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Updates or initializes buy in GetRiskReference.
1554	entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes entry in GetRiskReference.
1555	sl=PositionGetDouble(POSITION_SL);	Updates or initializes sl in GetRiskReference.
1556	if(sl<=0.0 && g_RiskDistance>0.0)	Checks a condition and runs the guarded branch in GetRiskReference.
1557	sl=buy ? entry-g_RiskDistance : entry+g_RiskDistance;	Updates or initializes sl in GetRiskReference.
1558	return (entry>0.0 && sl>0.0);	Returns a value or exits the current function in GetRiskReference.
1559	}	Opens or closes a code block in GetRiskReference.
1560		Blank separator for readability.
1561	PatternSignal sig;	Executes one statement in GetRiskReference.
1562	if(g_ArmedMode==ARM_BUY && DetectPattern(1,true,sig))	Checks a condition and runs the guarded branch in GetRiskReference.
1563	{	Opens or closes a code block in GetRiskReference.
1564	buy=true;	Updates or initializes buy in GetRiskReference.
1565	entry=SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes entry in GetRiskReference.
1566	sl=sig.sl;	Updates or initializes sl in GetRiskReference.
1567	ApplySpreadSpaceToSL(buy,sl);	Calls helper function ApplySpreadSpaceToSL in GetRiskReference.
1568	return (entry>0.0 && sl>0.0);	Returns a value or exits the current function in GetRiskReference.
1569	}	Opens or closes a code block in GetRiskReference.
1570	if(g_ArmedMode==ARM_SELL && DetectPattern(1,false,sig))	Checks a condition and runs the guarded branch in GetRiskReference.
1571	{	Opens or closes a code block in GetRiskReference.
1572	buy=false;	Updates or initializes buy in GetRiskReference.
1573	entry=SymbolInfoDouble(_Symbol,SYMBOL_BID);	Updates or initializes entry in GetRiskReference.
1574	sl=sig.sl;	Updates or initializes sl in GetRiskReference.
1575	ApplySpreadSpaceToSL(buy,sl);	Calls helper function ApplySpreadSpaceToSL in GetRiskReference.
1576	return (entry>0.0 && sl>0.0);	Returns a value or exits the current function in GetRiskReference.
1577	}	Opens or closes a code block in GetRiskReference.
1578	if(DetectPattern(1,true,sig))	Checks a condition and runs the guarded branch in GetRiskReference.
1579	{	Opens or closes a code block in GetRiskReference.
1580	buy=true;	Updates or initializes buy in GetRiskReference.
1581	entry=SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes entry in GetRiskReference.
1582	sl=sig.sl;	Updates or initializes sl in GetRiskReference.
1583	ApplySpreadSpaceToSL(buy,sl);	Calls helper function ApplySpreadSpaceToSL in GetRiskReference.
1584	return (entry>0.0 && sl>0.0);	Returns a value or exits the current function in GetRiskReference.
1585	}	Opens or closes a code block in GetRiskReference.
1586	if(DetectPattern(1,false,sig))	Checks a condition and runs the guarded branch in GetRiskReference.
1587	{	Opens or closes a code block in GetRiskReference.
1588	buy=false;	Updates or initializes buy in GetRiskReference.
1589	entry=SymbolInfoDouble(_Symbol,SYMBOL_BID);	Updates or initializes entry in GetRiskReference.
1590	sl=sig.sl;	Updates or initializes sl in GetRiskReference.
1591	ApplySpreadSpaceToSL(buy,sl);	Calls helper function ApplySpreadSpaceToSL in GetRiskReference.
1592	return (entry>0.0 && sl>0.0);	Returns a value or exits the current function in GetRiskReference.
1593	}	Opens or closes a code block in GetRiskReference.
1594		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1595	return false;	Returns a value or exits the current function in GetRiskReference.
1596	}	Opens or closes a code block in GetRiskReference.
1597		Blank separator for readability.
1598	string RiskPreviewText()	Declares or starts function RiskPreviewText.
1599	{	Opens or closes a code block in RiskPreviewText.
1600	double targetRisk=NormalizeRiskMoney(g_RiskMoney);	Updates or initializes double targetRisk in RiskPreviewText.
1601		Blank separator for readability.
1602	if(SelectOurPosition())	Checks a condition and runs the guarded branch in RiskPreviewText.
1603	{	Opens or closes a code block in RiskPreviewText.
1604	bool posBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Updates or initializes bool posBuy in RiskPreviewText.
1605	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double entry in RiskPreviewText.
1606	double sl=PositionGetDouble(POSITION_SL);	Updates or initializes double sl in RiskPreviewText.
1607	double volume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double volume in RiskPreviewText.
1608	if(sl<=0.0 && g_RiskDistance>0.0)	Checks a condition and runs the guarded branch in RiskPreviewText.
1609	sl=posBuy ? entry-g_RiskDistance : entry+g_RiskDistance;	Updates or initializes sl in RiskPreviewText.
1610		Blank separator for readability.
1611	double openRisk=MoneyRiskForLot(posBuy,entry,sl,volume);	Updates or initializes double openRisk in RiskPreviewText.
1612	double openPct=RiskPercentOfBalance(openRisk);	Updates or initializes double openPct in RiskPreviewText.
1613	return "Open risk "+MoneyText(openRisk)+	Returns a value or exits the current function in RiskPreviewText.
1614	" (" +DoubleToString(openPct,2)+"%) Target "+MoneyText(targetRisk);	Executes one statement in RiskPreviewText.
1615	}	Opens or closes a code block in RiskPreviewText.
1616		Blank separator for readability.
1617	bool buy=true;	Updates or initializes bool buy in RiskPreviewText.
1618	double entry=0.0;	Updates or initializes double entry in RiskPreviewText.
1619	double sl=0.0;	Updates or initializes double sl in RiskPreviewText.
1620	if(!GetRiskReference(buy,entry,sl))	Checks a condition and runs the guarded branch in RiskPreviewText.
1621	return "Risk "+MoneyText(targetRisk)+" wait model SL";	Returns a value or exits the current function in RiskPreviewText.
1622		Blank separator for readability.
1623	double lot=0.0;	Updates or initializes double lot in RiskPreviewText.
1624	double actualRisk=0.0;	Updates or initializes double actualRisk in RiskPreviewText.
1625	string reason="";	Updates or initializes string reason in RiskPreviewText.
1626	if(CalculateRiskVolume(buy,entry,sl,lot,targetRisk,actualRisk,reason))	Checks a condition and runs the guarded branch in RiskPreviewText.
1627	{	Opens or closes a code block in RiskPreviewText.
1628	return "Risk "+MoneyText(targetRisk)+	Returns a value or exits the current function in RiskPreviewText.
1629	" Auto lot "+DoubleToString(lot,VolumeDigits());	Executes one statement in RiskPreviewText.
1630	}	Opens or closes a code block in RiskPreviewText.
1631		Blank separator for readability.
1632	return "Risk "+MoneyText(targetRisk)+" "+reason;	Returns a value or exits the current function in RiskPreviewText.
1633	}	Opens or closes a code block in RiskPreviewText.
1634		Blank separator for readability.
1635	bool EnforceActualProtection(ulong ticket,bool buy,double sl,double &tp)	Declares or starts function EnforceActualProtection.
1636	{	Opens or closes a code block in EnforceActualProtection.
1637	if(!PositionSelectByTicket(ticket))	Checks a condition and runs the guarded branch in EnforceActualProtection.
1638	return false;	Returns a value or exits the current function in EnforceActualProtection.
1639		Blank separator for readability.
1640	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double entry in EnforceActualProtection.
1641	double risk=buy ? (entry-sl) : (sl-entry);	Updates or initializes double risk in EnforceActualProtection.
1642	if(risk<=_Point)	Checks a condition and runs the guarded branch in EnforceActualProtection.
1643	{	Opens or closes a code block in EnforceActualProtection.
1644	SetLastMessage("Trade opened but SL risk is invalid",clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1645	return false;	Returns a value or exits the current function in EnforceActualProtection.
1646	}	Opens or closes a code block in EnforceActualProtection.
1647		Blank separator for readability.
1648	tp=buy ? entry+risk*EffectiveRR() : entry-risk*EffectiveRR();	Updates or initializes tp in EnforceActualProtection.
1649	sl=NormalizeDouble(sl,_Digits);	Updates or initializes sl in EnforceActualProtection.
1650	ApplySpreadSpaceToTP(buy,tp);	Calls helper function ApplySpreadSpaceToTP in EnforceActualProtection.
1651	tp=NormalizeDouble(tp,_Digits);	Updates or initializes tp in EnforceActualProtection.
1652		Blank separator for readability.
1653	string reason="";	Updates or initializes string reason in EnforceActualProtection.
1654	if(!ValidateStops(buy,sl,tp,reason))	Checks a condition and runs the guarded branch in EnforceActualProtection.
1655	{	Opens or closes a code block in EnforceActualProtection.
1656	SetLastMessage("Protection check failed: "+reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1657	return false;	Returns a value or exits the current function in EnforceActualProtection.
1658	}	Opens or closes a code block in EnforceActualProtection.
1659		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1660	double oldSL=PositionGetDouble(POSITION_SL);	Updates or initializes double oldSL in EnforceActualProtection.
1661	double oldTP=PositionGetDouble(POSITION_TP);	Updates or initializes double oldTP in EnforceActualProtection.
1662	bool needModify=(MathAbs(oldSL-sl)>_Point*0.5 MathAbs(oldTP-tp)>_Point*0.5);	Updates or initializes bool needModify in EnforceActualProtection.
1663		Blank separator for readability.
1664	if(needModify)	Checks a condition and runs the guarded branch in EnforceActualProtection.
1665	{	Opens or closes a code block in EnforceActualProtection.
1666	if(!Trade.PositionModify(ticket,sl,tp) !TradeRetcodeOK())	Checks a condition and runs the guarded branch in EnforceActualProtection.
1667	{	Opens or closes a code block in EnforceActualProtection.
1668	SetLastMessage("SL/TP modify failed: "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
1669	return false;	Returns a value or exits the current function in EnforceActualProtection.
1670	}	Opens or closes a code block in EnforceActualProtection.
1671	}	Opens or closes a code block in EnforceActualProtection.
1672		Blank separator for readability.
1673	return true;	Returns a value or exits the current function in EnforceActualProtection.
1674	}	Opens or closes a code block in EnforceActualProtection.
1675		Blank separator for readability.
1676	void CloseUnprotectedPosition(ulong ticket,string why)	Declares or starts function CloseUnprotectedPosition.
1677	{	Opens or closes a code block in CloseUnprotectedPosition.
1678	if(ticket==0)	Checks a condition and runs the guarded branch in CloseUnprotectedPosition.
1679	return;	Returns a value or exits the current function in CloseUnprotectedPosition.
1680		Blank separator for readability.
1681	if(PositionSelectByTicket(ticket))	Checks a condition and runs the guarded branch in CloseUnprotectedPosition.
1682	{	Opens or closes a code block in CloseUnprotectedPosition.
1683	Trade.PositionClose(ticket);	Executes or modifies a trade through the MetaTrader trade API.
1684	ResetLocalTradeState();	Calls helper function ResetLocalTradeState in CloseUnprotectedPosition.
1685	}	Opens or closes a code block in CloseUnprotectedPosition.
1686		Blank separator for readability.
1687	SetLastMessage(why,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1688	}	Opens or closes a code block in CloseUnprotectedPosition.
1689		Blank separator for readability.
1690	bool ExecuteSignal(PatternSignal &sig)	Declares or starts function ExecuteSignal.
1691	{	Opens or closes a code block in ExecuteSignal.
1692	if(!sig.ok)	Checks a condition and runs the guarded branch in ExecuteSignal.
1693	return false;	Returns a value or exits the current function in ExecuteSignal.
1694		Blank separator for readability.
1695	if(!IsJustClosedThirdCandleSignal(sig))	Checks a condition and runs the guarded branch in ExecuteSignal.
1696	{	Opens or closes a code block in ExecuteSignal.
1697	SetLastMessage("Signal skipped - waiting for 3rd candle close",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
1698	return false;	Returns a value or exits the current function in ExecuteSignal.
1699	}	Opens or closes a code block in ExecuteSignal.
1700		Blank separator for readability.
1701	string reason="";	Updates or initializes string reason in ExecuteSignal.
1702		Blank separator for readability.
1703	if(!CanStartModelEntry(reason))	Checks a condition and runs the guarded branch in ExecuteSignal.
1704	{	Opens or closes a code block in ExecuteSignal.
1705	SetLastMessage(reason,clrGold);	Updates the EA status message, panel display, and web state dirty flag.
1706	return false;	Returns a value or exits the current function in ExecuteSignal.
1707	}	Opens or closes a code block in ExecuteSignal.
1708		Blank separator for readability.
1709	if(!IsTradingAllowedForDirection(sig.buy,reason))	Checks a condition and runs the guarded branch in ExecuteSignal.
1710	{	Opens or closes a code block in ExecuteSignal.
1711	SetLastMessage(reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1712	return false;	Returns a value or exits the current function in ExecuteSignal.
1713	}	Opens or closes a code block in ExecuteSignal.
1714		Blank separator for readability.
1715	if(!SpreadAllowed(reason))	Checks a condition and runs the guarded branch in ExecuteSignal.
1716	{	Opens or closes a code block in ExecuteSignal.
1717	SetLastMessage(reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1718	return false;	Returns a value or exits the current function in ExecuteSignal.
1719	}	Opens or closes a code block in ExecuteSignal.
1720		Blank separator for readability.
1721	double ask=SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes double ask in ExecuteSignal.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1722	double bid=SymbolInfoDouble(_Symbol,SYMBOL_BID);	Updates or initializes double bid in ExecuteSignal.
1723	double entry=sig.buy ? ask : bid;	Updates or initializes double entry in ExecuteSignal.
1724	double sl=NormalizeDouble(sig.sl,_Digits);	Updates or initializes double sl in ExecuteSignal.
1725	ApplySpreadSpaceToSL(sig.buy,sl);	Calls helper function ApplySpreadSpaceToSL in ExecuteSignal.
1726	double risk=sig.buy ? (entry-sl) : (sl-entry);	Updates or initializes double risk in ExecuteSignal.
1727		Blank separator for readability.
1728	if(risk<=_Point)	Checks a condition and runs the guarded branch in ExecuteSignal.
1729	{	Opens or closes a code block in ExecuteSignal.
1730	SetLastMessage("Signal risk invalid - SL is too close",clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1731	return false;	Returns a value or exits the current function in ExecuteSignal.
1732	}	Opens or closes a code block in ExecuteSignal.
1733		Blank separator for readability.
1734	double tp=sig.buy ? entry+risk*EffectiveRR() : entry-risk*EffectiveRR();	Updates or initializes double tp in ExecuteSignal.
1735	ApplySpreadSpaceToTP(sig.buy,tp);	Calls helper function ApplySpreadSpaceToTP in ExecuteSignal.
1736	tp=NormalizeDouble(tp,_Digits);	Updates or initializes tp in ExecuteSignal.
1737		Blank separator for readability.
1738	if(!ValidateStops(sig.buy,sl,tp,reason))	Checks a condition and runs the guarded branch in ExecuteSignal.
1739	{	Opens or closes a code block in ExecuteSignal.
1740	SetLastMessage(reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1741	return false;	Returns a value or exits the current function in ExecuteSignal.
1742	}	Opens or closes a code block in ExecuteSignal.
1743		Blank separator for readability.
1744	double lot=0.0;	Updates or initializes double lot in ExecuteSignal.
1745	double targetRisk=0.0;	Updates or initializes double targetRisk in ExecuteSignal.
1746	double actualRisk=0.0;	Updates or initializes double actualRisk in ExecuteSignal.
1747	if(!CalculateRiskVolume(sig.buy,entry,sl,lot,targetRisk,actualRisk,reason))	Checks a condition and runs the guarded branch in ExecuteSignal.
1748	{	Opens or closes a code block in ExecuteSignal.
1749	SetLastMessage("Risk lot failed: "+reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1750	return false;	Returns a value or exits the current function in ExecuteSignal.
1751	}	Opens or closes a code block in ExecuteSignal.
1752		Blank separator for readability.
1753	g_Lot=NormalizeDouble(lot,VolumeDigits());	Updates or initializes g_Lot in ExecuteSignal.
1754	RefreshLotEdit();	Calls helper function RefreshLotEdit in ExecuteSignal.
1755		Blank separator for readability.
1756	bool ok=false;	Updates or initializes bool ok in ExecuteSignal.
1757	string comment=sig.swing ? "TCX Swing" : "TCX Model";	Updates or initializes string comment in ExecuteSignal.
1758		Blank separator for readability.
1759	if(sig.buy)	Checks a condition and runs the guarded branch in ExecuteSignal.
1760	ok=Trade.Buy(lot,_Symbol,0.0,sl,tp,comment+" BUY");	Executes or modifies a trade through the MetaTrader trade API.
1761	else	Fallback branch when prior conditions did not match in ExecuteSignal.
1762	ok=Trade.Sell(lot,_Symbol,0.0,sl,tp,comment+" SELL");	Executes or modifies a trade through the MetaTrader trade API.
1763		Blank separator for readability.
1764	if(!ok !TradeRetcodeOK())	Checks a condition and runs the guarded branch in ExecuteSignal.
1765	{	Opens or closes a code block in ExecuteSignal.
1766	string firstError=Trade.ResultRetcodeDescription();	Executes or modifies a trade through the MetaTrader trade API.
1767		Blank separator for readability.
1768	if(!InpFallbackNoInitialSLTP)	Checks a condition and runs the guarded branch in ExecuteSignal.
1769	{	Opens or closes a code block in ExecuteSignal.
1770	SetLastMessage("Order failed: "+firstError,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1771	return false;	Returns a value or exits the current function in ExecuteSignal.
1772	}	Opens or closes a code block in ExecuteSignal.
1773		Blank separator for readability.
1774	if(sig.buy)	Checks a condition and runs the guarded branch in ExecuteSignal.
1775	ok=Trade.Buy(lot,_Symbol,0.0,0.0,0.0,comment+" BUY protect");	Executes or modifies a trade through the MetaTrader trade API.
1776	else	Fallback branch when prior conditions did not match in ExecuteSignal.
1777	ok=Trade.Sell(lot,_Symbol,0.0,0.0,0.0,comment+" SELL protect");	Executes or modifies a trade through the MetaTrader trade API.
1778		Blank separator for readability.
1779	if(!ok !TradeRetcodeOK())	Checks a condition and runs the guarded branch in ExecuteSignal.
1780	{	Opens or closes a code block in ExecuteSignal.
1781	SetLastMessage("Order failed: "+firstError+" retry "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
1782	return false;	Returns a value or exits the current function in ExecuteSignal.
1783	}	Opens or closes a code block in ExecuteSignal.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1784	}	Opens or closes a code block in ExecuteSignal.
1785		Blank separator for readability.
1786	Sleep(150);	Calls helper function Sleep in ExecuteSignal.
1787		Blank separator for readability.
1788	if(!SelectNewestOurPosition())	Checks a condition and runs the guarded branch in ExecuteSignal.
1789	{	Opens or closes a code block in ExecuteSignal.
1790	SetLastMessage("Order sent, position not found yet",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
1791	return true;	Returns a value or exits the current function in ExecuteSignal.
1792	}	Opens or closes a code block in ExecuteSignal.
1793		Blank separator for readability.
1794	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in ExecuteSignal.
1795	bool resultBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE))==POSITION_TYPE_BUY;	Updates or initializes bool resultBuy in ExecuteSignal.
1796	double finalTP=tp;	Updates or initializes double finalTP in ExecuteSignal.
1797	bool protectionOK=true;	Updates or initializes bool protectionOK in ExecuteSignal.
1798		Blank separator for readability.
1799	if(resultBuy==sig.buy)	Checks a condition and runs the guarded branch in ExecuteSignal.
1800	protectionOK=EnforceActualProtection(ticket,sig.buy,sl,finalTP);	Updates or initializes protectionOK in ExecuteSignal.
1801	else	Fallback branch when prior conditions did not match in ExecuteSignal.
1802	{	Opens or closes a code block in ExecuteSignal.
1803	SyncPositionState();	Calls helper function SyncPositionState in ExecuteSignal.
1804	SetLastMessage("Order sent - existing net position adjusted",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
1805	return true;	Returns a value or exits the current function in ExecuteSignal.
1806	}	Opens or closes a code block in ExecuteSignal.
1807		Blank separator for readability.
1808	if(!protectionOK && InpCloseIfProtectionFails)	Checks a condition and runs the guarded branch in ExecuteSignal.
1809	{	Opens or closes a code block in ExecuteSignal.
1810	CloseUnprotectedPosition(ticket,(sig.buy ? "BUY" : "SELL")+" closed - SL/TP protection failed");	Calls helper function CloseUnprotectedPosition in ExecuteSignal.
1811	return false;	Returns a value or exits the current function in ExecuteSignal.
1812	}	Opens or closes a code block in ExecuteSignal.
1813		Blank separator for readability.
1814	g_StateTicket=ticket;	Updates or initializes g_StateTicket in ExecuteSignal.
1815	g_InitialVolume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes g_InitialVolume in ExecuteSignal.
1816	g_RiskDistance=MathAbs(PositionGetDouble(POSITION_PRICE_OPEN)-sl);	Updates or initializes g_RiskDistance in ExecuteSignal.
1817	g_PC1_Done=false;	Updates or initializes g_PC1_Done in ExecuteSignal.
1818	g_PC2_Done=false;	Updates or initializes g_PC2_Done in ExecuteSignal.
1819	g_PC3_Done=false;	Updates or initializes g_PC3_Done in ExecuteSignal.
1820	g_BE_Done=false;	Updates or initializes g_BE_Done in ExecuteSignal.
1821	SavePositionState();	Calls helper function SavePositionState in ExecuteSignal.
1822		Blank separator for readability.
1823	DrawTradeLevels(ticket,sig.buy,sl,finalTP);	Calls helper function DrawTradeLevels in ExecuteSignal.
1824		Blank separator for readability.
1825	if(protectionOK)	Checks a condition and runs the guarded branch in ExecuteSignal.
1826	{	Opens or closes a code block in ExecuteSignal.
1827	SetLastMessage((sig.buy ? "BUY" : "SELL")+" placed Lot "+DoubleToString(lot,VolumeDigits())+" Risk "+MoneyText(actualRisk)+" SL "+PriceText(sl)+" TP "+PriceText(finalTP),clrLime);	Updates the EA status message, panel display, and web state dirty flag.
1828		Part of the EA logic in ExecuteSignal.
1829		Executes one statement in ExecuteSignal.
1830	}	Opens or closes a code block in ExecuteSignal.
1831	else	Fallback branch when prior conditions did not match in ExecuteSignal.
1832	{	Opens or closes a code block in ExecuteSignal.
1833	SetLastMessage((sig.buy ? "BUY" : "SELL")+" placed - check SL/TP message in Experts tab",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
1834	}	Opens or closes a code block in ExecuteSignal.
1835		Blank separator for readability.
1836	return true;	Returns a value or exits the current function in ExecuteSignal.
1837	}	Opens or closes a code block in ExecuteSignal.
1838		Blank separator for readability.
1839	double PositionRRFromValues(bool buy,double entry,double sl)	Declares or starts function PositionRRFromValues.
1840	{	Opens or closes a code block in PositionRRFromValues.
1841	double risk=buy ? (entry-sl) : (sl-entry);	Updates or initializes double risk in PositionRRFromValues.
1842	if(risk<=_Point)	Checks a condition and runs the guarded branch in PositionRRFromValues.
1843	return 0.0;	Returns a value or exits the current function in PositionRRFromValues.
1844		Blank separator for readability.
1845	double price=buy ? SymbolInfoDouble(_Symbol,SYMBOL_BID) : SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes double price in PositionRRFromValues.
1846	if(price<=0.0)	Checks a condition and runs the guarded branch in PositionRRFromValues.
1847	return 0.0;	Returns a value or exits the current function in PositionRRFromValues.
1848		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1849	double move=buy ? price-entry : entry-price;	Updates or initializes double move in PositionRRFromValues.
1850	return move/risk;	Returns a value or exits the current function in PositionRRFromValues.
1851	}	Opens or closes a code block in PositionRRFromValues.
1852		Blank separator for readability.
1853	bool EnforceSecondEntryProtection(bool buy,double sl,double tp,string &reason)	Declares or starts function EnforceSecondEntryProtection.
1854	{	Opens or closes a code block in EnforceSecondEntryProtection.
1855	reason="";	Updates or initializes reason in EnforceSecondEntryProtection.
1856	sl=NormalizeDouble(sl,_Digits);	Updates or initializes sl in EnforceSecondEntryProtection.
1857	tp=NormalizeDouble(tp,_Digits);	Updates or initializes tp in EnforceSecondEntryProtection.
1858		Blank separator for readability.
1859	int matched=0;	Updates or initializes int matched in EnforceSecondEntryProtection.
1860	int failed=0;	Updates or initializes int failed in EnforceSecondEntryProtection.
1861	bool stopsChecked=false;	Updates or initializes bool stopsChecked in EnforceSecondEntryProtection.
1862	string lastFail="";	Updates or initializes string lastFail in EnforceSecondEntryProtection.
1863		Blank separator for readability.
1864	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in EnforceSecondEntryProtection.
1865	{	Opens or closes a code block in EnforceSecondEntryProtection.
1866	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in EnforceSecondEntryProtection.
1867	if(ticket==0)	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1868	continue;	Executes one statement in EnforceSecondEntryProtection.
1869		Blank separator for readability.
1870	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in EnforceSecondEntryProtection.
1871	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in EnforceSecondEntryProtection.
1872	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1873	continue;	Executes one statement in EnforceSecondEntryProtection.
1874		Blank separator for readability.
1875	bool	Updates or initializes bool posBuy in EnforceSecondEntryProtection.
1876	posBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
	if(posBuy!=buy)	
1877	continue;	Executes one statement in EnforceSecondEntryProtection.
1878		Blank separator for readability.
1879	matched++;	Executes one statement in EnforceSecondEntryProtection.
1880		Blank separator for readability.
1881	double oldSL=PositionGetDouble(POSITION_SL);	Updates or initializes double oldSL in EnforceSecondEntryProtection.
1882	double oldTP=PositionGetDouble(POSITION_TP);	Updates or initializes double oldTP in EnforceSecondEntryProtection.
1883	bool needModify=(MathAbs(oldSL-sl)>_Point*0.5 MathAbs(oldTP-tp)>_Point*0.5);	Updates or initializes bool needModify in EnforceSecondEntryProtection.
1884	if(!needModify)	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1885	continue;	Executes one statement in EnforceSecondEntryProtection.
1886		Blank separator for readability.
1887	if(!stopsChecked)	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1888	{	Opens or closes a code block in EnforceSecondEntryProtection.
1889	if(!ValidateStops(buy,sl,tp,reason))	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1890	return false;	Returns a value or exits the current function in EnforceSecondEntryProtection.
1891	stopsChecked=true;	Updates or initializes stopsChecked in EnforceSecondEntryProtection.
1892	}	Opens or closes a code block in EnforceSecondEntryProtection.
1893		Blank separator for readability.
1894	if(!Trade.PositionModify(ticket,sl,tp) !TradeRetcodeOK())	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1895	{	Opens or closes a code block in EnforceSecondEntryProtection.
1896	failed++;	Executes one statement in EnforceSecondEntryProtection.
1897	lastFail=Trade.ResultRetcodeDescription();	Executes or modifies a trade through the MetaTrader trade API.
1898	}	Opens or closes a code block in EnforceSecondEntryProtection.
1899	}	Opens or closes a code block in EnforceSecondEntryProtection.
1900		Blank separator for readability.
1901	if(matched<=0)	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1902	{	Opens or closes a code block in EnforceSecondEntryProtection.
1903	reason="position not found";	Updates or initializes reason in EnforceSecondEntryProtection.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1904	return false;	Returns a value or exits the current function in EnforceSecondEntryProtection.
1905	}	Opens or closes a code block in EnforceSecondEntryProtection.
1906		Blank separator for readability.
1907	if(failed>0)	Checks a condition and runs the guarded branch in EnforceSecondEntryProtection.
1908	{	Opens or closes a code block in EnforceSecondEntryProtection.
1909	reason=lastFail;	Updates or initializes reason in EnforceSecondEntryProtection.
1910	return false;	Returns a value or exits the current function in EnforceSecondEntryProtection.
1911	}	Opens or closes a code block in EnforceSecondEntryProtection.
1912		Blank separator for readability.
1913	return true;	Returns a value or exits the current function in EnforceSecondEntryProtection.
1914	}	Opens or closes a code block in EnforceSecondEntryProtection.
1915		Blank separator for readability.
1916	bool ExecuteSecondEntryFromBase(ulong baseTicket)	Declares or starts function ExecuteSecondEntryFromBase.
1917	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1918	if(baseTicket==0 !PositionSelectByTicket(baseTicket))	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1919	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1920		Blank separator for readability.
1921	bool	Updates or initializes bool buy in ExecuteSecondEntryFromBase.
1922	buy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Updates or initializes double baseSL in ExecuteSecondEntryFromBase.
1923	double baseSL=NormalizeDouble(PositionGetDouble(POSITION_SL),_Digits);	Updates or initializes double baseTP in ExecuteSecondEntryFromBase.
1924	double baseTP=NormalizeDouble(PositionGetDouble(POSITION_TP),_Digits);	Blank separator for readability.
1925		Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1926	if(baseSL<=0.0 baseTP<=0.0)	Opens or closes a code block in ExecuteSecondEntryFromBase.
1927	{	Updates the EA status message, panel display, and web state dirty flag.
1928	SetLastMessage("Second entry skipped - first trade has no SL/TP",clrGold);	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1929	return false;	Opens or closes a code block in ExecuteSecondEntryFromBase.
1930	}	Blank separator for readability.
1931	string reason="";	Updates or initializes string reason in ExecuteSecondEntryFromBase.
1932	int maxEntries=MaxEntriesPerCycle();	Updates or initializes int maxEntries in ExecuteSecondEntryFromBase.
1933	if(CountOurPositions())>=maxEntries)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1934	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1935	SetLastMessage("Second entry skipped - maximum "+IntegerToString(maxEntries)+" entries already open",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
1936	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1937	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1938		Blank separator for readability.
1939	if(g_SecondEntryDone g_SecondEntryBlocked)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1940	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1941		Blank separator for readability.
1942	if(!IsTradingAllowedForDirection(buy,reason))	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1943	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1944	SetLastMessage(reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1945	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1946	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1947		Blank separator for readability.
1948	if(!SpreadAllowed(reason))	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1949	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1950	SetLastMessage(reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1951	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1952	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1953		Blank separator for readability.
1954	double ask=SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes double ask in ExecuteSecondEntryFromBase.
1955	double bid=SymbolInfoDouble(_Symbol,SYMBOL_BID);	Updates or initializes double bid in ExecuteSecondEntryFromBase.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
1956	double entry=buy ? ask : bid;	Updates or initializes double entry in ExecuteSecondEntryFromBase.
1957	if(entry<=0.0)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1958	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1959	SetLastMessage("Second entry skipped - no valid price",clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1960	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1961	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1962		Blank separator for readability.
1963	if(!ValidateStops(buy,baseSL,baseTP,reason))	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1964	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1965	SetLastMessage("Second entry skipped: "+reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1966	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1967	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1968		Blank separator for readability.
1969	double lot=0.0;	Updates or initializes double lot in ExecuteSecondEntryFromBase.
1970	double targetRisk=0.0;	Updates or initializes double targetRisk in ExecuteSecondEntryFromBase.
1971	double actualRisk=0.0;	Updates or initializes double actualRisk in ExecuteSecondEntryFromBase.
1972	if(!CalculateRiskVolume(buy,entry,baseSL,lot,targetRisk,actualRisk,reason))	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1973	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1974	SetLastMessage("Second entry risk failed: "+reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1975	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1976	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1977		Blank separator for readability.
1978	bool ok=false;	Updates or initializes bool ok in ExecuteSecondEntryFromBase.
1979	string comment="TCX Second";	Updates or initializes string comment in ExecuteSecondEntryFromBase.
1980	if(buy)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1981	ok=Trade.Buy(lot,_Symbol,0.0,baseSL,baseTP,comment+" BUY");	Executes or modifies a trade through the MetaTrader trade API.
1982	else	Fallback branch when prior conditions did not match in ExecuteSecondEntryFromBase.
1983	ok=Trade.Sell(lot,_Symbol,0.0,baseSL,baseTP,comment+" SELL");	Executes or modifies a trade through the MetaTrader trade API.
1984		Blank separator for readability.
1985	if(!ok !TradeRetcodeOK())	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1986	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1987	string firstError=Trade.ResultRetcodeDescription();	Executes or modifies a trade through the MetaTrader trade API.
1988		Blank separator for readability.
1989	if(!InpFallbackNoInitialSLTP)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1990	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
1991	SetLastMessage("Second entry failed: "+firstError,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
1992	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
1993	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
1994		Blank separator for readability.
1995	if(buy)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
1996	ok=Trade.Buy(lot,_Symbol,0.0,0.0,0.0,comment+" BUY protect");	Executes or modifies a trade through the MetaTrader trade API.
1997	else	Fallback branch when prior conditions did not match in ExecuteSecondEntryFromBase.
1998	ok=Trade.Sell(lot,_Symbol,0.0,0.0,0.0,comment+" SELL protect");	Executes or modifies a trade through the MetaTrader trade API.
1999		Blank separator for readability.
2000	if(!ok !TradeRetcodeOK())	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
2001	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
2002	SetLastMessage("Second entry failed: "+firstError+" retry "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
2003	return false;	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
2004	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
2005	}	Opens or closes a code block in ExecuteSecondEntryFromBase.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2006		Blank separator for readability.
2007	g_SecondEntryDone=true;	Updates or initializes g_SecondEntryDone in ExecuteSecondEntryFromBase.
2008	g_SecondEntryOn=false;	Updates or initializes g_SecondEntryOn in ExecuteSecondEntryFromBase.
2009	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in ExecuteSecondEntryFromBase.
2010	g_AutoArm=false;	Updates or initializes g_AutoArm in ExecuteSecondEntryFromBase.
2011	SaveSecondEntryState();	Calls helper function SaveSecondEntryState in ExecuteSecondEntryFromBase.
2012		Blank separator for readability.
2013	Sleep(150);	Calls helper function Sleep in ExecuteSecondEntryFromBase.
2014		Blank separator for readability.
2015	bool protectionOK=true;	Updates or initializes bool protectionOK in ExecuteSecondEntryFromBase.
2016	if(!EnforceSecondEntryProtection(buy, baseSL, baseTP, reason))	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
2017	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
2018	protectionOK=false;	Updates or initializes protectionOK in ExecuteSecondEntryFromBase.
2019	if(reason=="")	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
2020	reason=Trade.ResultRetcodeDescription();	Executes or modifies a trade through the MetaTrader trade API.
2021	}	Opens or closes a code block in ExecuteSecondEntryFromBase.
2022		Blank separator for readability.
2023	if(!protectionOK && InpCloseIfProtectionFails)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
2024	{	Opens or closes a code block in ExecuteSecondEntryFromBase.
2025	ulong protectTicket=FindNewestOurPositionTicket(baseTicket);	Updates or initializes ulong protectTicket in ExecuteSecondEntryFromBase.
2026	if(protectTicket==0)	Checks a condition and runs the guarded branch in ExecuteSecondEntryFromBase.
2027	protectTicket=FindNewestOurPositionTicket(0);	Updates or initializes protectTicket in ExecuteSecondEntryFromBase.
2028	CloseUnprotectedPosition(protectTicket,(buy ? "BUY" : "SELL")+ " second entry	Calls helper function CloseUnprotectedPosition in ExecuteSecondEntryFromBase.
2029	closed - SL/TP protection failed");	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
2030	return false;	Opens or closes a code block in ExecuteSecondEntryFromBase.
2031	}	Blank separator for readability.
2032	SetLastMessage((buy ? "BUY" : "SELL")+ " second entry placed Lot	Updates the EA status message, panel display, and web state dirty flag.
2033	"+DoubleToString(lot,VolumeDigits())+	Executes one statement in ExecuteSecondEntryFromBase.
2034	" SL "+PriceText(baseSL)+" TP "+PriceText(baseTP),clrLime);	Returns a value or exits the current function in ExecuteSecondEntryFromBase.
2035	return true;	Opens or closes a code block in ExecuteSecondEntryFromBase.
2036	}	Blank separator for readability.
2037	void MonitorSecondEntry()	Declares or starts function MonitorSecondEntry.
2038	{	Opens or closes a code block in MonitorSecondEntry.
2039	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in MonitorSecondEntry.
2040		Blank separator for readability.
2041	if(g_SecondBaseTicket==0 !PositionSelectByTicket(g_SecondBaseTicket))	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2042	return;	Returns a value or exits the current function in MonitorSecondEntry.
2043		Blank separator for readability.
2044	int openCount=CountOurPositions();	Updates or initializes int openCount in MonitorSecondEntry.
2045	if(openCount<=0)	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2046	{	Opens or closes a code block in MonitorSecondEntry.
2047	ResetSecondEntryState();	Calls helper function ResetSecondEntryState in MonitorSecondEntry.
2048	return;	Returns a value or exits the current function in MonitorSecondEntry.
2049	}	Opens or closes a code block in MonitorSecondEntry.
2050		Blank separator for readability.
2051	if(openCount>1 && !g_SecondEntryDone)	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2052	{	Opens or closes a code block in MonitorSecondEntry.
2053	g_SecondEntryDone=true;	Updates or initializes g_SecondEntryDone in MonitorSecondEntry.
2054	g_SecondEntryOn=false;	Updates or initializes g_SecondEntryOn in MonitorSecondEntry.
2055	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in MonitorSecondEntry.
2056	g_AutoArm=false;	Updates or initializes g_AutoArm in MonitorSecondEntry.
2057	SaveSecondEntryState();	Calls helper function SaveSecondEntryState in MonitorSecondEntry.
2058	return;	Returns a value or exits the current function in MonitorSecondEntry.
2059	}	Opens or closes a code block in MonitorSecondEntry.
2060		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2061	<code>if(!PositionSelectByTicket(g_SecondBaseTicket))</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2062	<code>return;</code>	Returns a value or exits the current function in MonitorSecondEntry.
2063		Blank separator for readability.
2064	<code>bool buy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);</code>	Updates or initializes bool buy in MonitorSecondEntry.
2065	<code>double entry=PositionGetDouble(POSITION_PRICE_OPEN);</code>	Updates or initializes double entry in MonitorSecondEntry.
2066	<code>double sl=PositionGetDouble(POSITION_SL);</code>	Updates or initializes double sl in MonitorSecondEntry.
2067	<code>if(entry<=0.0 sl<=0.0)</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2068	<code>return;</code>	Returns a value or exits the current function in MonitorSecondEntry.
2069		Blank separator for readability.
2070	<code>double rr=PositionRRFromValues(buy,entry,sl);</code>	Updates or initializes double rr in MonitorSecondEntry.
2071	<code>double cancelAt=MathMax(0.1,InpSecondEntryCancelAtR);</code>	Updates or initializes double cancelAt in MonitorSecondEntry.
2072	<code>if(rr>=cancelAt && !g_SecondEntryBlocked)</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2073	<code>{</code>	Opens or closes a code block in MonitorSecondEntry.
2074	<code>string beReason="";</code>	Updates or initializes string beReason in MonitorSecondEntry.
2075	<code>bool beOK=MoveTicketToBreakEven(g_SecondBaseTicket,beReason);</code>	Updates or initializes bool beOK in MonitorSecondEntry.
2076	<code>g_SecondEntryBlocked=true;</code>	Updates or initializes g_SecondEntryBlocked in MonitorSecondEntry.
2077	<code>g_SecondEntryOn=false;</code>	Updates or initializes g_SecondEntryOn in MonitorSecondEntry.
2078	<code>SetArmMode(ARM_NONE);</code>	Calls helper function SetArmMode in MonitorSecondEntry.
2079	<code>g_AutoArm=false;</code>	Updates or initializes g_AutoArm in MonitorSecondEntry.
2080	<code>SaveSecondEntryState();</code>	Calls helper function SaveSecondEntryState in MonitorSecondEntry.
2081	<code>string msg="Second entry blocked - first trade reached</code> <code>" +DoubleToString(cancelAt,1)+"R";</code>	Updates or initializes string msg in MonitorSecondEntry.
2082	<code>if(beOK)</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2083	<code>msg+=(beReason=="already protected" ? " SL already BE" : " SL moved BE");</code>	Updates or initializes msg+ in MonitorSecondEntry.
2084	<code>else if(beReason!="")</code>	Checks an alternate condition when the previous branch did not run in MonitorSecondEntry.
2085	<code>msg+=" BE failed: "+beReason;</code>	Updates or initializes msg+ in MonitorSecondEntry.
2086	<code>SetLastMessage(msg,beOK ? clrLime : clrGold);</code>	Updates the EA status message, panel display, and web state dirty flag.
2087	<code>return;</code>	Returns a value or exits the current function in MonitorSecondEntry.
2088	<code>}</code>	Opens or closes a code block in MonitorSecondEntry.
2089		Blank separator for readability.
2090	<code>if(!g_SecondEntryOn g_SecondEntryDone g_SecondEntryBlocked)</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2091	<code>return;</code>	Returns a value or exits the current function in MonitorSecondEntry.
2092		Blank separator for readability.
2093	<code>double trigger=MathMax(0.1,InpSecondEntryTriggerLossR);</code>	Updates or initializes double trigger in MonitorSecondEntry.
2094	<code>if(rr>-trigger)</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2095	<code>return;</code>	Returns a value or exits the current function in MonitorSecondEntry.
2096		Blank separator for readability.
2097	<code>datetime now=TimeCurrent();</code>	Updates or initializes datetime now in MonitorSecondEntry.
2098	<code>if(g_SecondEntryLastAttempt>0 && now-g_SecondEntryLastAttempt<10)</code>	Checks a condition and runs the guarded branch in MonitorSecondEntry.
2099	<code>return;</code>	Returns a value or exits the current function in MonitorSecondEntry.
2100	<code>g_SecondEntryLastAttempt=now;</code>	Updates or initializes g_SecondEntryLastAttempt in MonitorSecondEntry.
2101		Blank separator for readability.
2102	<code>ExecuteSecondEntryFromBase(g_SecondBaseTicket);</code>	Calls helper function ExecuteSecondEntryFromBase in MonitorSecondEntry.
2103	<code>}</code>	Opens or closes a code block in MonitorSecondEntry.
2104		Blank separator for readability.
2105	<code>string SecondEntryStatusText()</code>	Declares or starts function SecondEntryStatusText.
2106	<code>{</code>	Opens or closes a code block in SecondEntryStatusText.
2107	<code>if(!g_SecondEntryOn)</code>	Checks a condition and runs the guarded branch in SecondEntryStatusText.
2108	<code>return "2nd entry OFF";</code>	Returns a value or exits the current function in SecondEntryStatusText.
2109	<code>if(!HasOurPosition())</code>	Checks a condition and runs the guarded branch in SecondEntryStatusText.
2110	<code>return "2nd entry ready";</code>	Returns a value or exits the current function in SecondEntryStatusText.
2111	<code>if(CountOurPositions())>=MaxEntriesPerCycle())</code>	Checks a condition and runs the guarded branch in SecondEntryStatusText.
2112	<code>return "Max "+IntegerToString(MaxEntriesPerCycle())+" entries";</code>	Returns a value or exits the current function in SecondEntryStatusText.
2113	<code>if(g_SecondEntryBlocked)</code>	Checks a condition and runs the guarded branch in SecondEntryStatusText.
2114	<code>return "2nd blocked after 2R";</code>	Returns a value or exits the current function in SecondEntryStatusText.
2115	<code>if(g_SecondEntryDone CountOurPositions())>1)</code>	Checks a condition and runs the guarded branch in SecondEntryStatusText.
2116	<code>return "2nd entry done";</code>	Returns a value or exits the current function in SecondEntryStatusText.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2117	return "2nd waits -"+DoubleToString(MathMax(0.1, InpSecondEntryTriggerLossR),1)+"R";	Returns a value or exits the current function in SecondEntryStatusText.
2118	}	Opens or closes a code block in SecondEntryStatusText.
2119		Blank separator for readability.
2120	string FirstTrailStatusText()	Declares or starts function FirstTrailStatusText.
2121	{	Opens or closes a code block in FirstTrailStatusText.
2122	if(!g_FirstTrailOn)	Checks a condition and runs the guarded branch in FirstTrailStatusText.
2123	return "1st trail OFF";	Returns a value or exits the current function in FirstTrailStatusText.
2124	if(!g_SecondEntryDone)	Checks a condition and runs the guarded branch in FirstTrailStatusText.
2125	return "1st trail waits 2nd";	Returns a value or exits the current function in FirstTrailStatusText.
2126	if(!SelectFirstEntryPosition())	Checks a condition and runs the guarded branch in FirstTrailStatusText.
2127	return "1st trade closed";	Returns a value or exits the current function in FirstTrailStatusText.
2128	if(g_FirstTrailActive)	Checks a condition and runs the guarded branch in FirstTrailStatusText.
2129	return "1st trail active";	Returns a value or exits the current function in FirstTrailStatusText.
2130	return "1st trail waits M5";	Returns a value or exits the current function in FirstTrailStatusText.
2131	}	Opens or closes a code block in FirstTrailStatusText.
2132		Blank separator for readability.
2133	void MonitorFirstTradeTrailing()	Declares or starts function MonitorFirstTradeTrailing.
2134	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2135	if(!g_FirstTrailOn)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2136	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2137		Blank separator for readability.
2138	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in MonitorFirstTradeTrailing.
2139	SyncFirstTrailState();	Calls helper function SyncFirstTrailState in MonitorFirstTradeTrailing.
2140		Blank separator for readability.
2141	if(!g_SecondEntryDone g_FirstTrailTicket==0)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2142	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2143		Blank separator for readability.
2144	if(!PositionSelectByTicket(g_FirstTrailTicket))	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2145	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2146	ResetFirstTrailState();	Calls helper function ResetFirstTrailState in MonitorFirstTradeTrailing.
2147	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2148	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2149		Blank separator for readability.
2150	if(IsSecondEntryPositionComment())	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2151	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2152		Blank separator for readability.
2153	if(iBars(_Symbol,PERIOD_M5)<4)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2154	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2155		Blank separator for readability.
2156	datetime m5Time=iTime(_Symbol,PERIOD_M5,1);	Updates or initializes datetime m5Time in MonitorFirstTradeTrailing.
2157	if(m5Time<=0 m5Time==g_FirstTrailLastM5BarTime)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2158	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2159		Blank separator for readability.
2160	bool buy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE))==POSITION_TYPE_BUY;	Updates or initializes bool buy in MonitorFirstTradeTrailing.
2161	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double entry in MonitorFirstTradeTrailing.
2162	double oldSL=PositionGetDouble(POSITION_SL);	Updates or initializes double oldSL in MonitorFirstTradeTrailing.
2163	double tp=PositionGetDouble(POSITION_TP);	Updates or initializes double tp in MonitorFirstTradeTrailing.
2164		Blank separator for readability.
2165	double close1=iClose(_Symbol,PERIOD_M5,1);	Updates or initializes double close1 in MonitorFirstTradeTrailing.
2166	double low1=iLow(_Symbol,PERIOD_M5,1);	Updates or initializes double low1 in MonitorFirstTradeTrailing.
2167	double high1=iHigh(_Symbol,PERIOD_M5,1);	Updates or initializes double high1 in MonitorFirstTradeTrailing.
2168	double low2=iLow(_Symbol,PERIOD_M5,2);	Updates or initializes double low2 in MonitorFirstTradeTrailing.
2169	double high2=iHigh(_Symbol,PERIOD_M5,2);	Updates or initializes double high2 in MonitorFirstTradeTrailing.
2170		Blank separator for readability.
2171	if(entry<=0.0 close1<=0.0 low1<=0.0 high1<=0.0 low2<=0.0 high2<=0.0)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2172	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2173		Blank separator for readability.
2174	bool trigger=false;	Updates or initializes bool trigger in MonitorFirstTradeTrailing.
2175	double newSL=0.0;	Updates or initializes double newSL in MonitorFirstTradeTrailing.
2176		Blank separator for readability.
2177	if(buy)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2178	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2179	trigger=(!g_FirstTrailActive ? close1>entry+_Point*0.1 : close1>high2+_Point*0.1);	Updates or initializes trigger in MonitorFirstTradeTrailing.
2180	newSL=NormalizeDouble(low1,_Digits);	Updates or initializes newSL in MonitorFirstTradeTrailing.
2181	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2182	else	Fallback branch when prior conditions did not match in MonitorFirstTradeTrailing.
2183	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2184	trigger=(!g_FirstTrailActive ? close1<entry-_Point*0.1 : close1<low2-_Point*0.1);	Updates or initializes trigger in MonitorFirstTradeTrailing.
2185	newSL=NormalizeDouble(high1,_Digits);	Updates or initializes newSL in MonitorFirstTradeTrailing.
2186	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2187		Blank separator for readability.
2188	g_FirstTrailLastM5BarTime=m5Time;	Updates or initializes g_FirstTrailLastM5BarTime in MonitorFirstTradeTrailing.
2189		Blank separator for readability.
2190	if(!trigger)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2191	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2192	SaveFirstTrailState();	Calls helper function SaveFirstTrailState in MonitorFirstTradeTrailing.
2193	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2194	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2195		Blank separator for readability.
2196	if(!g_FirstTrailActive)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2197	g_FirstTrailActive=true;	Updates or initializes g_FirstTrailActive in MonitorFirstTradeTrailing.
2198		Blank separator for readability.
2199	bool improves=(oldSL<=0.0 (buy ? newSL>oldSL+_Point*0.5 : newSL<oldSL-_Point*0.5));	Updates or initializes bool improves in MonitorFirstTradeTrailing.
2200	if(!improves)	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2201	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2202	SaveFirstTrailState();	Calls helper function SaveFirstTrailState in MonitorFirstTradeTrailing.
2203	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2204	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2205		Blank separator for readability.
2206	string reason="";	Updates or initializes string reason in MonitorFirstTradeTrailing.
2207	if(!ValidateStops(buy,newSL,tp,reason))	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2208	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2209	SaveFirstTrailState();	Calls helper function SaveFirstTrailState in MonitorFirstTradeTrailing.
2210	SetLastMessage("1st trail skipped: "+reason,clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2211	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2212	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2213		Blank separator for readability.
2214	ulong ticket=g_FirstTrailTicket;	Updates or initializes ulong ticket in MonitorFirstTradeTrailing.
2215	if(Trade.PositionModify(ticket,newSL,tp) && TradeRetcodeOK())	Checks a condition and runs the guarded branch in MonitorFirstTradeTrailing.
2216	{	Opens or closes a code block in MonitorFirstTradeTrailing.
2217	SaveFirstTrailState();	Calls helper function SaveFirstTrailState in MonitorFirstTradeTrailing.
2218	SetLastMessage((buy ? "BUY" : "SELL")+ " 1st trail SL -> "+PriceText(newSL)+" on M5 close",clrLime);	Updates the EA status message, panel display, and web state dirty flag.
2219	return;	Returns a value or exits the current function in MonitorFirstTradeTrailing.
2220	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2221		Blank separator for readability.
2222	SaveFirstTrailState();	Calls helper function SaveFirstTrailState in MonitorFirstTradeTrailing.
2223	SetLastMessage("1st trail failed: "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
2224	}	Opens or closes a code block in MonitorFirstTradeTrailing.
2225		Blank separator for readability.
2226	void TryArmedExecution()	Declares or starts function TryArmedExecution.
2227	{	Opens or closes a code block in TryArmedExecution.
2228	if(g_ArmedMode==ARM_NONE && !g_AutoArm)	Checks a condition and runs the guarded branch in TryArmedExecution.
2229	return;	Returns a value or exits the current function in TryArmedExecution.
2230		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2231	<code>datetime currentBar=iTime(_Symbol,_Period,0);</code>	Updates or initializes datetime currentBar in TryArmedExecution.
2232	<code>datetime justClosedBar=iTime(_Symbol,_Period,1);</code>	Updates or initializes datetime justClosedBar in TryArmedExecution.
2233	<code>if(currentBar<=0 justClosedBar<=0 currentBar<=justClosedBar)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2234	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2235		Blank separator for readability.
2236	<code>if(g_LastTradeScanBarTime==currentBar)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2237	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2238	<code>g_LastTradeScanBarTime=currentBar;</code>	Updates or initializes g_LastTradeScanBarTime in TryArmedExecution.
2239		Blank separator for readability.
2240	<code>string gateReason="";</code>	Updates or initializes string gateReason in TryArmedExecution.
2241	<code>if(!CanStartModelEntry(gateReason))</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2242	<code>{</code>	Opens or closes a code block in TryArmedExecution.
2243	<code>SetLastMessage(gateReason,clrGold);</code>	Updates the EA status message, panel display, and web state dirty flag.
2244	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2245	<code>}</code>	Opens or closes a code block in TryArmedExecution.
2246		Blank separator for readability.
2247	<code>PatternSignal sig;</code>	Executes one statement in TryArmedExecution.
2248	<code>bool wantBuy=(g_ArmedMode==ARM_BUY);</code>	Updates or initializes bool wantBuy in TryArmedExecution.
2249		Blank separator for readability.
2250	<code>if(g_AutoArm)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2251	<code>{</code>	Opens or closes a code block in TryArmedExecution.
2252	<code>if(DetectPattern(1,true,sig))</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2253	<code>wantBuy=true;</code>	Updates or initializes wantBuy in TryArmedExecution.
2254	<code>else if(DetectPattern(1,false,sig))</code>	Checks an alternate condition when the previous branch did not run in TryArmedExecution.
2255	<code>wantBuy=false;</code>	Updates or initializes wantBuy in TryArmedExecution.
2256	<code>else</code>	Fallback branch when prior conditions did not match in TryArmedExecution.
2257	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2258	<code>}</code>	Opens or closes a code block in TryArmedExecution.
2259	<code>else</code>	Fallback branch when prior conditions did not match in TryArmedExecution.
2260	<code>{</code>	Opens or closes a code block in TryArmedExecution.
2261	<code>if(!DetectPattern(1,wantBuy,sig))</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2262	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2263	<code>}</code>	Opens or closes a code block in TryArmedExecution.
2264		Blank separator for readability.
2265	<code>if(g_ArmMinSignalTime>0 && sig.time<g_ArmMinSignalTime)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2266	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2267		Blank separator for readability.
2268	<code>if(sig.time!=justClosedBar)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2269	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2270		Blank separator for readability.
2271	<code>if(g_LastTradeSignalTime>0 && sig.time==g_LastTradeSignalTime)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2272	<code>return;</code>	Returns a value or exits the current function in TryArmedExecution.
2273		Blank separator for readability.
2274	<code>DrawSignalMarker(sig);</code>	Calls helper function DrawSignalMarker in TryArmedExecution.
2275		Blank separator for readability.
2276	<code>string signalText=(wantBuy ? "BUY" : "SELL");</code>	Updates or initializes string signalText in TryArmedExecution.
2277	<code>if(sig.swing)</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2278	<code>signalText+=" swing";</code>	Updates or initializes signalText+ in TryArmedExecution.
2279		Blank separator for readability.
2280	<code>SetLastMessage(signalText+" model found - sending order",clrGold);</code>	Updates the EA status message, panel display, and web state dirty flag.
2281		Blank separator for readability.
2282	<code>if(ExecuteSignal(sig))</code>	Checks a condition and runs the guarded branch in TryArmedExecution.
2283	<code>{</code>	Opens or closes a code block in TryArmedExecution.
2284	<code>g_LastTradeSignalTime=sig.time;</code>	Updates or initializes g_LastTradeSignalTime in TryArmedExecution.
2285	<code>SetArmMode(ARM_NONE);</code>	Calls helper function SetArmMode in TryArmedExecution.
2286	<code>g_AutoArm=false;</code>	Updates or initializes g_AutoArm in TryArmedExecution.
2287	<code>}</code>	Opens or closes a code block in TryArmedExecution.
2288	<code>}</code>	Opens or closes a code block in TryArmedExecution.
2289		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2290	//=====	Comment explaining the nearby EA code.
2291	// PARTIAL CLOSE / BREAK EVEN / CLOSE ALL	Comment explaining the nearby EA code.
2292	//=====	Comment explaining the nearby EA code.
2293	bool DoPartialCloseLevel(int level,double pct,double rrLevel,bool &markDone)	Declares or starts function DoPartialCloseLevel.
2294	{	Opens or closes a code block in DoPartialCloseLevel.
2295	markDone=false;	Updates or initializes markDone in DoPartialCloseLevel.
2296		Blank separator for readability.
2297	if(!SelectOurPosition())	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2298	return false;	Returns a value or exits the current function in DoPartialCloseLevel.
2299		Blank separator for readability.
2300	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in DoPartialCloseLevel.
2301	double volume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double volume in DoPartialCloseLevel.
2302		Blank separator for readability.
2303	if(g_InitialVolume<=0.0)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2304	g_InitialVolume=volume;	Updates or initializes g_InitialVolume in DoPartialCloseLevel.
2305		Blank separator for readability.
2306	double rawLot=g_InitialVolume*pct/100.0;	Updates or initializes double rawLot in DoPartialCloseLevel.
2307	double closeLot=NormalizeVolumeDown(rawLot);	Updates or initializes double closeLot in DoPartialCloseLevel.
2308	double minLot=BrokerMinLot();	Updates or initializes double minLot in DoPartialCloseLevel.
2309	double step=BrokerStepLot();	Updates or initializes double step in DoPartialCloseLevel.
2310		Blank separator for readability.
2311	if(closeLot<=0.0)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2312	{	Opens or closes a code block in DoPartialCloseLevel.
2313	SetLastMessage("PC"+IntegerToString(level)+" skipped - lot too small",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2314	markDone=true;	Updates or initializes markDone in DoPartialCloseLevel.
2315	return false;	Returns a value or exits the current function in DoPartialCloseLevel.
2316	}	Opens or closes a code block in DoPartialCloseLevel.
2317		Blank separator for readability.
2318	if(closeLot>volume)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2319	closeLot=NormalizeVolumeDown(volume);	Updates or initializes closeLot in DoPartialCloseLevel.
2320		Blank separator for readability.
2321	if(closeLot<=0.0)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2322	{	Opens or closes a code block in DoPartialCloseLevel.
2323	SetLastMessage("PC"+IntegerToString(level)+" skipped - no closable volume",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2324	markDone=true;	Updates or initializes markDone in DoPartialCloseLevel.
2325	return false;	Returns a value or exits the current function in DoPartialCloseLevel.
2326	}	Opens or closes a code block in DoPartialCloseLevel.
2327		Blank separator for readability.
2328	bool closeFull=false;	Updates or initializes bool closeFull in DoPartialCloseLevel.
2329	if(closeLot>=volume-step*0.1)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2330	closeFull=true;	Updates or initializes closeFull in DoPartialCloseLevel.
2331	else if(volume-closeLot<minLot)	Checks an alternate condition when the previous branch did not run in DoPartialCloseLevel.
2332	{	Opens or closes a code block in DoPartialCloseLevel.
2333	double adjusted=NormalizeVolumeDown(volume-minLot);	Updates or initializes double adjusted in DoPartialCloseLevel.
2334	if(adjusted<=0.0)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2335	{	Opens or closes a code block in DoPartialCloseLevel.
2336	SetLastMessage("PC"+IntegerToString(level)+" skipped - would leave less than min lot",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2337	markDone=true;	Updates or initializes markDone in DoPartialCloseLevel.
2338	return false;	Returns a value or exits the current function in DoPartialCloseLevel.
2339	}	Opens or closes a code block in DoPartialCloseLevel.
2340	closeLot=adjusted;	Updates or initializes closeLot in DoPartialCloseLevel.
2341	}	Opens or closes a code block in DoPartialCloseLevel.
2342		Blank separator for readability.
2343	bool ok=false;	Updates or initializes bool ok in DoPartialCloseLevel.
2344	bool isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE))==POSITION_TYPE_BUY;	Updates or initializes bool isBuy in DoPartialCloseLevel.
2345		Blank separator for readability.
2346	if(closeFull)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2347	{	Opens or closes a code block in DoPartialCloseLevel.
2348	ok=Trade.PositionClose(ticket);	Executes or modifies a trade through the MetaTrader trade API.
2349	}	Opens or closes a code block in DoPartialCloseLevel.
2350	else if(IsHedgingAccount())	Checks an alternate condition when the previous branch did not run in DoPartialCloseLevel.
2351	{	Opens or closes a code block in DoPartialCloseLevel.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2352	ok=Trade.PositionClosePartial(ticket,closeLot);	Executes or modifies a trade through the MetaTrader trade API.
2353	}	Opens or closes a code block in DoPartialCloseLevel.
2354	else	Fallback branch when prior conditions did not match in DoPartialCloseLevel.
2355	{	Opens or closes a code block in DoPartialCloseLevel.
2356	if(isBuy)	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2357	ok=Trade.Sell(closeLot,_Symbol,0.0,0.0,0.0,"TCX partial reduce");	Executes or modifies a trade through the MetaTrader trade API.
2358	else	Fallback branch when prior conditions did not match in DoPartialCloseLevel.
2359	ok=Trade.Buy(closeLot,_Symbol,0.0,0.0,0.0,"TCX partial reduce");	Executes or modifies a trade through the MetaTrader trade API.
2360	}	Opens or closes a code block in DoPartialCloseLevel.
2361		Blank separator for readability.
2362	if(ok && TradeRetcodeOK())	Checks a condition and runs the guarded branch in DoPartialCloseLevel.
2363	{	Opens or closes a code block in DoPartialCloseLevel.
2364	SetLastMessage("PC"+IntegerToString(level)+" booked "+DoubleToString(closeLot,VolumeDigits())+ " lot at "+DoubleToString(rrLevel,1)+"R",clrLime);	Updates the EA status message, panel display, and web state dirty flag.
2365		Executes one statement in DoPartialCloseLevel.
2366	markDone=true;	Updates or initializes markDone in DoPartialCloseLevel.
2367	return true;	Returns a value or exits the current function in DoPartialCloseLevel.
2368	}	Opens or closes a code block in DoPartialCloseLevel.
2369		Blank separator for readability.
2370	SetLastMessage("PC"+IntegerToString(level)+" failed: "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
2371	return false;	Returns a value or exits the current function in DoPartialCloseLevel.
2372	}	Opens or closes a code block in DoPartialCloseLevel.
2373		Blank separator for readability.
2374	void MonitorPartialClose()	Declares or starts function MonitorPartialClose.
2375	{	Opens or closes a code block in MonitorPartialClose.
2376	if(!g_PC_On)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2377	return;	Returns a value or exits the current function in MonitorPartialClose.
2378	if(!SelectOurPosition())	Checks a condition and runs the guarded branch in MonitorPartialClose.
2379	return;	Returns a value or exits the current function in MonitorPartialClose.
2380		Blank separator for readability.
2381	SyncPositionState();	Calls helper function SyncPositionState in MonitorPartialClose.
2382		Blank separator for readability.
2383	bool	Updates or initializes bool isBuy in MonitorPartialClose.
2384	isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY); double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double entry in MonitorPartialClose.
2385	double sl=PositionGetDouble(POSITION_SL);	Updates or initializes double sl in MonitorPartialClose.
2386	double price=isBuy ? SymbolInfoDouble(_Symbol,SYMBOL_BID) : SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes double price in MonitorPartialClose.
2387		Blank separator for readability.
2388	if(g_RiskDistance<=0.0)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2389	g_RiskDistance=MathAbs(entry-sl);	Updates or initializes g_RiskDistance in MonitorPartialClose.
2390	if(g_RiskDistance<=0.0)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2391	return;	Returns a value or exits the current function in MonitorPartialClose.
2392		Blank separator for readability.
2393	double profitDistance=isBuy ? price-entry : entry-price;	Updates or initializes double profitDistance in MonitorPartialClose.
2394	double rr=profitDistance/g_RiskDistance;	Updates or initializes double rr in MonitorPartialClose.
2395		Blank separator for readability.
2396	if(rr<0.0)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2397	return;	Returns a value or exits the current function in MonitorPartialClose.
2398		Blank separator for readability.
2399	bool mark=false;	Updates or initializes bool mark in MonitorPartialClose.
2400		Blank separator for readability.
2401	if(!g_PC1_Done && g_PC1_Pct>0.0 && rr>=InpPC1_RR)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2402	{	Opens or closes a code block in MonitorPartialClose.
2403	DoPartialCloseLevel(1,g_PC1_Pct,InpPC1_RR,mark);	Calls helper function DoPartialCloseLevel in MonitorPartialClose.
2404	if(mark)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2405	{	Opens or closes a code block in MonitorPartialClose.
2406	g_PC1_Done=true;	Updates or initializes g_PC1_Done in MonitorPartialClose.
2407	SavePositionState();	Calls helper function SavePositionState in MonitorPartialClose.
2408	}	Opens or closes a code block in MonitorPartialClose.
2409	return;	Returns a value or exits the current function in MonitorPartialClose.
2410	}	Opens or closes a code block in MonitorPartialClose.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2411		Blank separator for readability.
2412	if(!g_PC2_Done && g_PC2_Pct>0.0 && rr>=InpPC2_RR)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2413	{	Opens or closes a code block in MonitorPartialClose.
2414	DoPartialCloseLevel(2,g_PC2_Pct,InpPC2_RR,mark);	Calls helper function DoPartialCloseLevel in MonitorPartialClose.
2415	if(mark)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2416	{	Opens or closes a code block in MonitorPartialClose.
2417	g_PC2_Done=true;	Updates or initializes g_PC2_Done in MonitorPartialClose.
2418	SavePositionState();	Calls helper function SavePositionState in MonitorPartialClose.
2419	}	Opens or closes a code block in MonitorPartialClose.
2420	return;	Returns a value or exits the current function in MonitorPartialClose.
2421	}	Opens or closes a code block in MonitorPartialClose.
2422		Blank separator for readability.
2423	if(!g_PC3_Done && g_PC3_Pct>0.0 && rr>=InpPC3_RR)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2424	{	Opens or closes a code block in MonitorPartialClose.
2425	DoPartialCloseLevel(3,g_PC3_Pct,InpPC3_RR,mark);	Calls helper function DoPartialCloseLevel in MonitorPartialClose.
2426	if(mark)	Checks a condition and runs the guarded branch in MonitorPartialClose.
2427	{	Opens or closes a code block in MonitorPartialClose.
2428	g_PC3_Done=true;	Updates or initializes g_PC3_Done in MonitorPartialClose.
2429	SavePositionState();	Calls helper function SavePositionState in MonitorPartialClose.
2430	}	Opens or closes a code block in MonitorPartialClose.
2431	return;	Returns a value or exits the current function in MonitorPartialClose.
2432	}	Opens or closes a code block in MonitorPartialClose.
2433	}	Opens or closes a code block in MonitorPartialClose.
2434		Blank separator for readability.
2435	double EntryCommissionForPosition(long positionId)	Declares or starts function EntryCommissionForPosition.
2436	{	Opens or closes a code block in EntryCommissionForPosition.
2437	if(positionId<=0)	Checks a condition and runs the guarded branch in EntryCommissionForPosition.
2438	return 0.0;	Returns a value or exits the current function in EntryCommissionForPosition.
2439	if(!HistorySelect((datetime)0,TimeCurrent()))	Checks a condition and runs the guarded branch in EntryCommissionForPosition.
2440	return 0.0;	Returns a value or exits the current function in EntryCommissionForPosition.
2441		Blank separator for readability.
2442	double commission=0.0;	Updates or initializes double commission in EntryCommissionForPosition.
2443	int total=HistoryDealsTotal();	Updates or initializes int total in EntryCommissionForPosition.
2444	for(int i=0;i<total;i++)	Loops over a fixed range or collection in EntryCommissionForPosition.
2445	{	Opens or closes a code block in EntryCommissionForPosition.
2446	ulong deal=HistoryDealGetTicket(i);	Updates or initializes ulong deal in EntryCommissionForPosition.
2447	if(deal==0)	Checks a condition and runs the guarded branch in EntryCommissionForPosition.
2448	continue;	Executes one statement in EntryCommissionForPosition.
2449	if(HistoryDealGetString(deal,DEAL_SYMBOL)!=_Symbol)	Checks a condition and runs the guarded branch in EntryCommissionForPosition.
2450	continue;	Executes one statement in EntryCommissionForPosition.
2451	if((long)HistoryDealGetInteger(deal,DEAL_POSITION_ID)!=positionId)	Checks a condition and runs the guarded branch in EntryCommissionForPosition.
2452	continue;	Executes one statement in EntryCommissionForPosition.
2453		Blank separator for readability.
2454	long entry=HistoryDealGetInteger(deal,DEAL_ENTRY);	Updates or initializes long entry in EntryCommissionForPosition.
2455	if(entry==DEAL_ENTRY_IN entry==DEAL_ENTRY_INOUT)	Checks a condition and runs the guarded branch in EntryCommissionForPosition.
2456	commission+=HistoryDealGetDouble(deal,DEAL_COMMISSION);	Updates or initializes commission+ in EntryCommissionForPosition.
2457	}	Opens or closes a code block in EntryCommissionForPosition.
2458		Blank separator for readability.
2459	return commission;	Returns a value or exits the current function in EntryCommissionForPosition.
2460	}	Opens or closes a code block in EntryCommissionForPosition.
2461		Blank separator for readability.
2462	double EstimatedCloseCommissionFromEntry(double entryCommission)	Declares or starts function EstimatedCloseCommissionFromEntry.
2463	{	Opens or closes a code block in EstimatedCloseCommissionFromEntry.
2464	if(entryCommission<0.0)	Checks a condition and runs the guarded branch in EstimatedCloseCommissionFromEntry.
2465	return -MathAbs(entryCommission);	Returns a value or exits the current function in EstimatedCloseCommissionFromEntry.
2466	return 0.0;	Returns a value or exits the current function in EstimatedCloseCommissionFromEntry.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2467	}	Opens or closes a code block in EstimatedCloseCommissionFromEntry.
2468		Blank separator for readability.
2469	double BreakEvenCostTargetMoney()	Declares or starts function BreakEvenCostTargetMoney.
2470	{	Opens or closes a code block in BreakEvenCostTargetMoney.
2471	long positionId=(long)PositionGetInteger(POSITION_IDENTIFIER);	Updates or initializes long positionId in BreakEvenCostTargetMoney.
2472	double entryCommission=EntryCommissionForPosition(positionId);	Updates or initializes double entryCommission in BreakEvenCostTargetMoney.
2473	double estimatedCloseCommission=EstimatedCloseCommissionFromEntry(entryCommission);	Updates or initializes double estimatedCloseCommission in BreakEvenCostTargetMoney.
2474	double swap=PositionGetDouble(POSITION_SWAP);	Updates or initializes double swap in BreakEvenCostTargetMoney.
2475	double extra=MathMax(0.0,InpBreakEvenExitExtraMoney);	Updates or initializes double extra in BreakEvenCostTargetMoney.
2476	double costs=swap+entryCommission+estimatedCloseCommission-extra;	Updates or initializes double costs in BreakEvenCostTargetMoney.
2477	return MathMax(0.0,-costs);	Returns a value or exits the current function in BreakEvenCostTargetMoney.
2478	}	Opens or closes a code block in BreakEvenCostTargetMoney.
2479		Blank separator for readability.
2480	double CommissionAdjustedBreakEvenPrice(bool isBuy,double entry,double volume)	Declares or starts function CommissionAdjustedBreakEvenPrice.
2481	{	Opens or closes a code block in CommissionAdjustedBreakEvenPrice.
2482	double targetMoney=BreakEvenCostTargetMoney();	Updates or initializes double targetMoney in CommissionAdjustedBreakEvenPrice.
2483	int plusPoints=InpBreakEvenPlusPoints;	Updates or initializes int plusPoints in CommissionAdjustedBreakEvenPrice.
2484	if(plusPoints<0)	Checks a condition and runs the guarded branch in CommissionAdjustedBreakEvenPrice.
2485	plusPoints=0;	Updates or initializes plusPoints in CommissionAdjustedBreakEvenPrice.
2486	ENUM_ORDER_TYPE orderType=isBuy ? ORDER_TYPE_BUY : ORDER_TYPE_SELL;	Updates or initializes ENUM_ORDER_TYPE orderType in CommissionAdjustedBreakEvenPrice.
2487	double onePointPrice=isBuy ? entry+_Point : entry-_Point;	Updates or initializes double onePointPrice in CommissionAdjustedBreakEvenPrice.
2488	double onePointProfit=0.0;	Updates or initializes double onePointProfit in CommissionAdjustedBreakEvenPrice.
2489		Blank separator for readability.
2490	if(OrderCalcProfit(orderType,_Symbol,volume,entry,onePointPrice,onePointProfit) && onePointProfit>0.0)	Checks a condition and runs the guarded branch in CommissionAdjustedBreakEvenPrice.
2491	{	Opens or closes a code block in CommissionAdjustedBreakEvenPrice.
2492	int costPoints=(int)MathCeil(targetMoney/onePointProfit);	Updates or initializes int costPoints in CommissionAdjustedBreakEvenPrice.
2493	double price=isBuy ? entry+(costPoints+plusPoints)*_Point	Part of the EA logic in CommissionAdjustedBreakEvenPrice.
2494	: entry-(costPoints+plusPoints)*_Point;	
2495	return NormalizeDouble(price,_Digits);	Executes one statement in CommissionAdjustedBreakEvenPrice.
2496	}	Returns a value or exits the current function in CommissionAdjustedBreakEvenPrice.
2497		Opens or closes a code block in CommissionAdjustedBreakEvenPrice.
2498	double fallback=isBuy ? entry+plusPoints*_Point : entry-plusPoints*_Point;	Blank separator for readability.
2499	return NormalizeDouble(fallback,_Digits);	Updates or initializes double fallback in CommissionAdjustedBreakEvenPrice.
2500	}	Returns a value or exits the current function in CommissionAdjustedBreakEvenPrice.
2501		Opens or closes a code block in CommissionAdjustedBreakEvenPrice.
2502	bool ExpectedCloseNetForTicket(ulong ticket,double &net,string &reason)	Blank separator for readability.
2503	{	Declares or starts function ExpectedCloseNetForTicket.
2504	net=0.0;	Opens or closes a code block in ExpectedCloseNetForTicket.
2505	reason="";	Updates or initializes net in ExpectedCloseNetForTicket.
2506	if(ticket==0 !PositionSelectByTicket(ticket))	Updates or initializes reason in ExpectedCloseNetForTicket.
2507	{	Checks a condition and runs the guarded branch in ExpectedCloseNetForTicket.
2508	reason="position not found";	Opens or closes a code block in ExpectedCloseNetForTicket.
2509	return false;	Updates or initializes reason in ExpectedCloseNetForTicket.
2510	}	Returns a value or exits the current function in ExpectedCloseNetForTicket.
2511		Opens or closes a code block in ExpectedCloseNetForTicket.
2512	bool	Updates or closes a code block in ExpectedCloseNetForTicket.
2513	isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Blank separator for readability.
2514	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes bool isBuy in ExpectedCloseNetForTicket.
2515	double volume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double entry in ExpectedCloseNetForTicket.
2516	double closePrice=isBuy ? SymbolInfoDouble(_Symbol,SYMBOL_BID) : SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes double volume in ExpectedCloseNetForTicket.
2517	if(entry<=0.0 volume<=0.0 closePrice<=0.0)	Updates or initializes double closePrice in ExpectedCloseNetForTicket.
2518	{	Checks a condition and runs the guarded branch in ExpectedCloseNetForTicket.
2519	reason="price not ready";	Opens or closes a code block in ExpectedCloseNetForTicket.
2520	return false;	Updates or initializes reason in ExpectedCloseNetForTicket.
2521	}	Returns a value or exits the current function in ExpectedCloseNetForTicket.
		Opens or closes a code block in ExpectedCloseNetForTicket.
		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2522	ENUM_ORDER_TYPE orderType=isBuy ? ORDER_TYPE_BUY : ORDER_TYPE_SELL;	Updates or initializes ENUM_ORDER_TYPE orderType in ExpectedCloseNetForTicket.
2523	double gross=0.0;	Updates or initializes double gross in ExpectedCloseNetForTicket.
2524	if(!OrderCalcProfit(orderType,_Symbol,volume,entry,closePrice,gross))	Checks a condition and runs the guarded branch in ExpectedCloseNetForTicket.
2525	gross=PositionGetDouble(POSITION_PROFIT);	Updates or initializes gross in ExpectedCloseNetForTicket.
2526		Blank separator for readability.
2527	long positionId=(long)PositionGetInteger(POSITION_IDENTIFIER);	Updates or initializes long positionId in ExpectedCloseNetForTicket.
2528	double entryCommission=EntryCommissionForPosition(positionId);	Updates or initializes double entryCommission in ExpectedCloseNetForTicket.
2529	double estimatedCloseCommission=EstimatedCloseCommissionFromEntry(entryCommission);	Updates or initializes double estimatedCloseCommission in ExpectedCloseNetForTicket.
2530	double swap=PositionGetDouble(POSITION_SWAP);	Updates or initializes double swap in ExpectedCloseNetForTicket.
2531	double extra=MathMax(0.0,InpBreakEvenExitExtraMoney);	Updates or initializes double extra in ExpectedCloseNetForTicket.
2532		Blank separator for readability.
2533	net=gross+swap+entryCommission+estimatedCloseCommission-extra;	Updates or initializes net in ExpectedCloseNetForTicket.
2534	return true;	Returns a value or exits the current function in ExpectedCloseNetForTicket.
2535	}	Opens or closes a code block in ExpectedCloseNetForTicket.
2536		Blank separator for readability.
2537	bool MoveTicketToBreakEven(ulong ticket,string &reason)	Declares or starts function MoveTicketToBreakEven.
2538	{	Opens or closes a code block in MoveTicketToBreakEven.
2539	reason="";	Updates or initializes reason in MoveTicketToBreakEven.
2540	if(ticket==0 !PositionSelectByTicket(ticket))	Checks a condition and runs the guarded branch in MoveTicketToBreakEven.
2541	{	Opens or closes a code block in MoveTicketToBreakEven.
2542	reason="position not found";	Updates or initializes reason in MoveTicketToBreakEven.
2543	return false;	Returns a value or exits the current function in MoveTicketToBreakEven.
2544	}	Opens or closes a code block in MoveTicketToBreakEven.
2545		Blank separator for readability.
2546	bool	Updates or initializes bool isBuy in MoveTicketToBreakEven.
2547	isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Updates or initializes double entry in MoveTicketToBreakEven.
2548	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double volume in MoveTicketToBreakEven.
2549	double volume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double oldSL in MoveTicketToBreakEven.
2550	double oldSL=PositionGetDouble(POSITION_SL);	Updates or initializes double tp in MoveTicketToBreakEven.
2551	double tp=PositionGetDouble(POSITION_TP);	Updates or initializes double newSL in MoveTicketToBreakEven.
2552	double newSL=CommissionAdjustedBreakEvenPrice(isBuy,entry,volume);	Blank separator for readability.
2553		Updates or initializes newSL in MoveTicketToBreakEven.
2554	newSL=NormalizeDouble(newSL,_Digits);	Blank separator for readability.
2555	if(isBuy && oldSL>=newSL && oldSL>0.0)	Checks a condition and runs the guarded branch in MoveTicketToBreakEven.
2556	{	Opens or closes a code block in MoveTicketToBreakEven.
2557	reason="already protected";	Updates or initializes reason in MoveTicketToBreakEven.
2558	return true;	Returns a value or exits the current function in MoveTicketToBreakEven.
2559	}	Opens or closes a code block in MoveTicketToBreakEven.
2560	if(!isBuy && oldSL<=newSL && oldSL>0.0)	Checks a condition and runs the guarded branch in MoveTicketToBreakEven.
2561	{	Opens or closes a code block in MoveTicketToBreakEven.
2562	reason="already protected";	Updates or initializes reason in MoveTicketToBreakEven.
2563	return true;	Returns a value or exits the current function in MoveTicketToBreakEven.
2564	}	Opens or closes a code block in MoveTicketToBreakEven.
2565		Blank separator for readability.
2566	if(!ValidateStops(isBuy,newSL,tp,reason))	Checks a condition and runs the guarded branch in MoveTicketToBreakEven.
2567	return false;	Returns a value or exits the current function in MoveTicketToBreakEven.
2568		Blank separator for readability.
2569	if(Trade.PositionModify(ticket,newSL,tp) && TradeRetcodeOK())	Checks a condition and runs the guarded branch in MoveTicketToBreakEven.
2570	{	Opens or closes a code block in MoveTicketToBreakEven.
2571	if(ticket==g_StateTicket)	Checks a condition and runs the guarded branch in MoveTicketToBreakEven.
2572	{	Opens or closes a code block in MoveTicketToBreakEven.
2573	g_BE_Done=true;	Updates or initializes g_BE_Done in MoveTicketToBreakEven.
2574	SavePositionState();	Calls helper function SavePositionState in MoveTicketToBreakEven.
2575	}	Opens or closes a code block in MoveTicketToBreakEven.
2576	reason="moved";	Updates or initializes reason in MoveTicketToBreakEven.
2577	return true;	Returns a value or exits the current function in MoveTicketToBreakEven.
2578	}	Opens or closes a code block in MoveTicketToBreakEven.
2579		Blank separator for readability.
2580	reason=Trade.ResultRetcodeDescription();	Executes or modifies a trade through the MetaTrader trade API.
2581	return false;	Returns a value or exits the current function in MoveTicketToBreakEven.
2582	}	Opens or closes a code block in MoveTicketToBreakEven.
2583		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2584	void MoveToBreakEven()	Declares or starts function MoveToBreakEven.
2585	{	Opens or closes a code block in MoveToBreakEven.
2586	int found=0;	Updates or initializes int found in MoveToBreakEven.
2587	int moved=0;	Updates or initializes int moved in MoveToBreakEven.
2588	int already=0;	Updates or initializes int already in MoveToBreakEven.
2589	int failed=0;	Updates or initializes int failed in MoveToBreakEven.
2590	string lastFail="";	Updates or initializes string lastFail in MoveToBreakEven.
2591		Blank separator for readability.
2592	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in MoveToBreakEven.
2593	{	Opens or closes a code block in MoveToBreakEven.
2594	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in MoveToBreakEven.
2595	if(ticket==0)	Checks a condition and runs the guarded branch in MoveToBreakEven.
2596	continue;	Executes one statement in MoveToBreakEven.
2597		Blank separator for readability.
2598	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in MoveToBreakEven.
2599	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in MoveToBreakEven.
2600	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in MoveToBreakEven.
2601	continue;	Executes one statement in MoveToBreakEven.
2602		Blank separator for readability.
2603	found++;	Executes one statement in MoveToBreakEven.
2604		Blank separator for readability.
2605	string reason="";	Updates or initializes string reason in MoveToBreakEven.
2606	if(MoveTicketToBreakEven(ticket, reason))	Checks a condition and runs the guarded branch in MoveToBreakEven.
2607	{	Opens or closes a code block in MoveToBreakEven.
2608	if(reason=="already protected")	Checks a condition and runs the guarded branch in MoveToBreakEven.
2609	already++;	Executes one statement in MoveToBreakEven.
2610	else	Fallback branch when prior conditions did not match in MoveToBreakEven.
2611	moved++;	Executes one statement in MoveToBreakEven.
2612	}	Opens or closes a code block in MoveToBreakEven.
2613	else	Fallback branch when prior conditions did not match in MoveToBreakEven.
2614	{	Opens or closes a code block in MoveToBreakEven.
2615	failed++;	Executes one statement in MoveToBreakEven.
2616	lastFail=reason;	Updates or initializes lastFail in MoveToBreakEven.
2617	}	Opens or closes a code block in MoveToBreakEven.
2618	}	Opens or closes a code block in MoveToBreakEven.
2619		Blank separator for readability.
2620	if(found==0)	Checks a condition and runs the guarded branch in MoveToBreakEven.
2621	SetLastMessage("No open position for break even",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2622	else if(moved>0)	Checks an alternate condition when the previous branch did not run in MoveToBreakEven.
2623	SetLastMessage("Stop moved to break even on "+IntegerToString(moved)+"	Updates the EA status message, panel display, and web state dirty flag.
2624	position(s)",clrLime);	
2624	else if(failed>0)	Checks an alternate condition when the previous branch did not run in MoveToBreakEven.
2625	SetLastMessage("BE failed: "+lastFail,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
2626	else if(already>0)	Checks an alternate condition when the previous branch did not run in MoveToBreakEven.
2627	SetLastMessage("Break even already protected",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2628	}	Opens or closes a code block in MoveToBreakEven.
2629		Blank separator for readability.
2630	void MoveFirstTradeToBreakEven()	Declares or starts function MoveFirstTradeToBreakEven.
2631	{	Opens or closes a code block in MoveFirstTradeToBreakEven.
2632	if(!SelectFirstEntryPosition())	Checks a condition and runs the guarded branch in MoveFirstTradeToBreakEven.
2633	{	Opens or closes a code block in MoveFirstTradeToBreakEven.
2634	SetLastMessage("No first trade for break even",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2635	return;	Returns a value or exits the current function in MoveFirstTradeToBreakEven.
2636	}	Opens or closes a code block in MoveFirstTradeToBreakEven.
2637		Blank separator for readability.
2638	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in MoveFirstTradeToBreakEven.
2639	string reason="";	Updates or initializes string reason in MoveFirstTradeToBreakEven.
2640	if(MoveTicketToBreakEven(ticket, reason))	Checks a condition and runs the guarded branch in MoveFirstTradeToBreakEven.
2641	{	Opens or closes a code block in MoveFirstTradeToBreakEven.
2642	if(reason=="already protected")	Checks a condition and runs the guarded branch in MoveFirstTradeToBreakEven.
2643	SetLastMessage("First trade already protected",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2644	else	Fallback branch when prior conditions did not match in MoveFirstTradeToBreakEven.
2645	SetLastMessage("First trade SL moved to break even",clrLime);	Updates the EA status message, panel display, and web state dirty flag.
2646	return;	Returns a value or exits the current function in MoveFirstTradeToBreakEven.
2647	}	Opens or closes a code block in MoveFirstTradeToBreakEven.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2648		Blank separator for readability.
2649	SetLastMessage("First BE failed: "+reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
2650	}	Opens or closes a code block in MoveFirstTradeToBreakEven.
2651		Blank separator for readability.
2652	void CloseFirstTradeAtBreakEvenExit()	Declares or starts function CloseFirstTradeAtBreakEvenExit.
2653	{	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2654	if(CountOurPositions())<2)	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2655	{	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2656	SetLastMessage("1st BE exit waits until the second trade is open",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2657	return;	Returns a value or exits the current function in CloseFirstTradeAtBreakEvenExit.
2658	}	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2659		Blank separator for readability.
2660	if(!SelectFirstEntryPosition())	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2661	{	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2662	SetLastMessage("No first trade for BE exit",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2663	return;	Returns a value or exits the current function in CloseFirstTradeAtBreakEvenExit.
2664	}	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2665		Blank separator for readability.
2666	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in CloseFirstTradeAtBreakEvenExit.
2667	double expectedNet=0.0;	Updates or initializes double expectedNet in CloseFirstTradeAtBreakEvenExit.
2668	string reason="";	Updates or initializes string reason in CloseFirstTradeAtBreakEvenExit.
2669	if(!ExpectedCloseNetForTicket(ticket,expectedNet,reason))	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2670	{	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2671	SetLastMessage("1st BE exit failed: "+reason,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
2672	return;	Returns a value or exits the current function in CloseFirstTradeAtBreakEvenExit.
2673	}	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2674		Blank separator for readability.
2675	if(expectedNet<-0.01)	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2676	{	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2677	SetLastMessage("1st BE exit blocked - net would be "+SignedMoneyText(expectedNet)+" after spread/commission",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2678	return;	Returns a value or exits the current function in CloseFirstTradeAtBreakEvenExit.
2679	}	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2680		Blank separator for readability.
2681	if(Trade.PositionClose(ticket) && TradeRetcodeOK())	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2682	{	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2683	if(ticket==g_StateTicket)	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2684	ResetLocalTradeState();	Calls helper function ResetLocalTradeState in CloseFirstTradeAtBreakEvenExit.
2685	if(ticket==g_FirstTrailTicket)	Checks a condition and runs the guarded branch in CloseFirstTradeAtBreakEvenExit.
2686	ResetFirstTrailState();	Calls helper function ResetFirstTrailState in CloseFirstTradeAtBreakEvenExit.
2687	SetLastMessage("First trade BE exit closed est net "+SignedMoneyText(expectedNet),clrLime);	Updates the EA status message, panel display, and web state dirty flag.
2688	return;	Returns a value or exits the current function in CloseFirstTradeAtBreakEvenExit.
2689	}	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2690		Blank separator for readability.
2691	SetLastMessage("1st BE exit failed: "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
2692	}	Opens or closes a code block in CloseFirstTradeAtBreakEvenExit.
2693		Blank separator for readability.
2694	void CloseAllPositions()	Declares or starts function CloseAllPositions.
2695	{	Opens or closes a code block in CloseAllPositions.
2696	bool any=false;	Updates or initializes bool any in CloseAllPositions.
2697		Blank separator for readability.
2698	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in CloseAllPositions.
2699	{	Opens or closes a code block in CloseAllPositions.
2700	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in CloseAllPositions.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2701	if(ticket==0)	Checks a condition and runs the guarded branch in CloseAllPositions.
2702	continue;	Executes one statement in CloseAllPositions.
2703		Blank separator for readability.
2704	string sym=PositionGetString(POSITION_SYMBOL);	Updates or initializes string sym in CloseAllPositions.
2705	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in CloseAllPositions.
2706	if(sym!=_Symbol magic!=InpMagic)	Checks a condition and runs the guarded branch in CloseAllPositions.
2707	continue;	Executes one statement in CloseAllPositions.
2708		Blank separator for readability.
2709	any=true;	Updates or initializes any in CloseAllPositions.
2710	Trade.PositionClose(ticket);	Executes or modifies a trade through the MetaTrader trade API.
2711	}	Opens or closes a code block in CloseAllPositions.
2712		Blank separator for readability.
2713	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in CloseAllPositions.
2714	ResetLocalTradeState();	Calls helper function ResetLocalTradeState in CloseAllPositions.
2715		Blank separator for readability.
2716	if(any)	Checks a condition and runs the guarded branch in CloseAllPositions.
2717	SetLastMessage("Close command sent",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2718	else	Fallback branch when prior conditions did not match in CloseAllPositions.
2719	SetLastMessage("No matching position to close",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2720	}	Opens or closes a code block in CloseAllPositions.
2721		Blank separator for readability.
2722	void CloseCurrentPercent(double pct)	Declares or starts function CloseCurrentPercent.
2723	{	Opens or closes a code block in CloseCurrentPercent.
2724	if(pct<=0.0)	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2725	return;	Returns a value or exits the current function in CloseCurrentPercent.
2726		Blank separator for readability.
2727	if(!SelectOurPosition())	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2728	{	Opens or closes a code block in CloseCurrentPercent.
2729	SetLastMessage("No matching position to close",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2730	return;	Returns a value or exits the current function in CloseCurrentPercent.
2731	}	Opens or closes a code block in CloseCurrentPercent.
2732		Blank separator for readability.
2733	ulong ticket=(ulong)PositionGetInteger(POSITION_TICKET);	Updates or initializes ulong ticket in CloseCurrentPercent.
2734	double volume=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double volume in CloseCurrentPercent.
2735	bool isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE))==POSITION_TYPE_BUY;	Updates or initializes bool isBuy in CloseCurrentPercent.
2736	double minLot=BrokerMinLot();	Updates or initializes double minLot in CloseCurrentPercent.
2737	double closeLot=NormalizeVolumeDown(volume*pct/100.0);	Updates or initializes double closeLot in CloseCurrentPercent.
2738		Blank separator for readability.
2739	if(closeLot<=0.0)	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2740	{	Opens or closes a code block in CloseCurrentPercent.
2741	SetLastMessage("Close "+DoubleToString(pct,0)+"% skipped - lot too small",clrGold);	Updates the EA status message, panel display, and web state dirty flag.
2742	return;	Returns a value or exits the current function in CloseCurrentPercent.
2743	}	Opens or closes a code block in CloseCurrentPercent.
2744		Blank separator for readability.
2745	if(volume-closeLot<minLot)	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2746	closeLot=volume;	Updates or initializes closeLot in CloseCurrentPercent.
2747		Blank separator for readability.
2748	bool ok=false;	Updates or initializes bool ok in CloseCurrentPercent.
2749	if(closeLot>=volume)	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2750	ok=Trade.PositionClose(ticket);	Executes or modifies a trade through the MetaTrader trade API.
2751	else if(IsHedgingAccount())	Checks an alternate condition when the previous branch did not run in CloseCurrentPercent.
2752	ok=Trade.PositionClosePartial(ticket,closeLot);	Executes or modifies a trade through the MetaTrader trade API.
2753	else	Fallback branch when prior conditions did not match in CloseCurrentPercent.
2754	{	Opens or closes a code block in CloseCurrentPercent.
2755	if(isBuy)	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2756	ok=Trade.Sell(closeLot,_Symbol,0.0,0.0,0.0,"TCX manual close");	Executes or modifies a trade through the MetaTrader trade API.
2757	else	Fallback branch when prior conditions did not match in CloseCurrentPercent.
2758	ok=Trade.Buy(closeLot,_Symbol,0.0,0.0,0.0,"TCX manual close");	Executes or modifies a trade through the MetaTrader trade API.
2759	}	Opens or closes a code block in CloseCurrentPercent.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2760		Blank separator for readability.
2761	if(ok && TradeRetcodeOK())	Checks a condition and runs the guarded branch in CloseCurrentPercent.
2762	SetLastMessage("Closed "+DoubleToString(closeLot,VolumeDigits())+"lot",clrTime);	Updates the EA status message, panel display, and web state dirty flag.
2763	else	Fallback branch when prior conditions did not match in CloseCurrentPercent.
2764	SetLastMessage("Close failed: "+Trade.ResultRetcodeDescription(),clrTomato);	Executes or modifies a trade through the MetaTrader trade API.
2765	}	Opens or closes a code block in CloseCurrentPercent.
2766		Blank separator for readability.
2767	//=====	Comment explaining the nearby EA code.
2768	// UI HELPERS	Comment explaining the nearby EA code.
2769	//=====	Comment explaining the nearby EA code.
2770	void CalcPanelGeometry()	Declares or starts function CalcPanelGeometry.
2771	{	Opens or closes a code block in CalcPanelGeometry.
2772	int chartW=(int)ChartGetInteger(0,CHART_WIDTH_IN_PIXELS);	Updates or initializes int chartW in CalcPanelGeometry.
2773	int chartH=(int)ChartGetInteger(0,CHART_HEIGHT_IN_PIXELS);	Updates or initializes int chartH in CalcPanelGeometry.
2774		Blank separator for readability.
2775	g_X=InpPanelX;	Updates or initializes g_X in CalcPanelGeometry.
2776	g_Y=InpPanelY;	Updates or initializes g_Y in CalcPanelGeometry.
2777		Blank separator for readability.
2778	if(g_X<0)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2779	g_X=0;	Updates or initializes g_X in CalcPanelGeometry.
2780	if(g_Y<0)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2781	g_Y=0;	Updates or initializes g_Y in CalcPanelGeometry.
2782		Blank separator for readability.
2783	int wantedW=InpPanelW;	Updates or initializes int wantedW in CalcPanelGeometry.
2784		Blank separator for readability.
2785	// Clean side-controller width. Keep it wide enough for all blocks,	Comment explaining the nearby EA code.
2786	// but never force horizontal overflow on smaller charts.	Comment explaining the nearby EA code.
2787	if(wantedW<930)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2788	wantedW=930;	Updates or initializes wantedW in CalcPanelGeometry.
2789	if(wantedW>1120)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2790	wantedW=1120;	Updates or initializes wantedW in CalcPanelGeometry.
2791		Blank separator for readability.
2792	g_W=wantedW;	Updates or initializes g_W in CalcPanelGeometry.
2793		Blank separator for readability.
2794	if(InpAutoFitPanelWidth && chartW>100)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2795	{	Opens or closes a code block in CalcPanelGeometry.
2796	int fitW=chartW-g_X-12;	Updates or initializes int fitW in CalcPanelGeometry.
2797	if(fitW>=900)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2798	g_W=MathMin(wantedW,fitW);	Updates or initializes g_W in CalcPanelGeometry.
2799	else	Fallback branch when prior conditions did not match in CalcPanelGeometry.
2800	g_W=MathMax(860,fitW);	Updates or initializes g_W in CalcPanelGeometry.
2801	}	Opens or closes a code block in CalcPanelGeometry.
2802		Blank separator for readability.
2803	if(g_W<860)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2804	g_W=860;	Updates or initializes g_W in CalcPanelGeometry.
2805	if(g_W>1120)	Checks a condition and runs the guarded branch in CalcPanelGeometry.
2806	g_W=1120;	Updates or initializes g_W in CalcPanelGeometry.
2807		Blank separator for readability.
2808	// Content-fit height: removes the bad empty full-height area while keeping	Comment explaining the nearby EA code.
2809	// every block readable and aligned.	Comment explaining the nearby EA code.
2810	g_H=732;	Updates or initializes g_H in CalcPanelGeometry.
2811	}	Opens or closes a code block in CalcPanelGeometry.
2812		Blank separator for readability.
2813	void Rect(string name,int x,int y,int w,int h,color bg,int z=1)	Declares or starts function Rect.
2814	{	Opens or closes a code block in Rect.
2815	ObjectDelete(0,name);	Creates, updates, or removes an MT5 chart panel object.
2816	ObjectCreate(0,name,OBJ_RECTANGLE_LABEL,0,0,0);	Creates, updates, or removes an MT5 chart panel object.
2817	ObjectSetInteger(0,name,OBJPROP_XDISTANCE,x);	Creates, updates, or removes an MT5 chart panel object.
2818	ObjectSetInteger(0,name,OBJPROP_YDISTANCE,y);	Creates, updates, or removes an MT5 chart panel object.
2819	ObjectSetInteger(0,name,OBJPROP_XSIZE,w);	Creates, updates, or removes an MT5 chart panel object.
2820	ObjectSetInteger(0,name,OBJPROP_YSIZE,h);	Creates, updates, or removes an MT5 chart panel object.
2821	ObjectSetInteger(0,name,OBJPROP_BGCOLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2822	ObjectSetInteger(0,name,OBJPROP_COLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2823	ObjectSetInteger(0,name,OBJPROP_BORDER_TYPE,BORDER_FLAT);	Creates, updates, or removes an MT5 chart panel object.
2824	ObjectSetInteger(0,name,OBJPROP_CORNER,CORNER_LEFT_UPPER);	Creates, updates, or removes an MT5 chart panel object.
2825	ObjectSetInteger(0,name,OBJPROP_BACK,false);	Creates, updates, or removes an MT5 chart panel object.
2826	ObjectSetInteger(0,name,OBJPROP_SELECTABLE,false);	Creates, updates, or removes an MT5 chart panel object.
2827	ObjectSetInteger(0,name,OBJPROP_HIDDEN,true);	Creates, updates, or removes an MT5 chart panel object.
2828	ObjectSetInteger(0,name,OBJPROP_ZORDER,z);	Creates, updates, or removes an MT5 chart panel object.
2829	}	Opens or closes a code block in Rect.
2830		Blank separator for readability.
2831	void Border(string name,int x,int y,int w,int h,color clr)	Declares or starts function Border.
2832	{	Opens or closes a code block in Border.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2833	<code>Rect(name+"_T",x,y,w,2,clr,2);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
2834	<code>Rect(name+"_B",x,y+h-2,w,2,clr,2);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
2835	<code>Rect(name+"_L",x,y,2,h,clr,2);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
2836	<code>Rect(name+"_R",x+w-2,y,2,h,clr,2);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
2837	<code>}</code>	Opens or closes a code block in Border.
2838		Blank separator for readability.
2839	<code>void Txt(string name,string text,int x,int y,color clr,int size,bool bold=false,int z=20)</code>	Declares or starts function Txt.
2840	<code>{</code>	Opens or closes a code block in Txt.
2841	<code>ObjectDelete(0,name);</code>	Creates, updates, or removes an MT5 chart panel object.
2842	<code>ObjectCreate(0,name,OBJ_LABEL,0,0,0);</code>	Creates, updates, or removes an MT5 chart panel object.
2843	<code>ObjectSetInteger(0,name,OBJPROP_XDISTANCE,x);</code>	Creates, updates, or removes an MT5 chart panel object.
2844	<code>ObjectSetInteger(0,name,OBJPROP_YDISTANCE,y);</code>	Creates, updates, or removes an MT5 chart panel object.
2845	<code>ObjectSetString(0,name,OBJPROP_TEXT,text);</code>	Creates, updates, or removes an MT5 chart panel object.
2846	<code>ObjectSetInteger(0,name,OBJPROP_COLOR,clr);</code>	Creates, updates, or removes an MT5 chart panel object.
2847	<code>int uiSize=size;</code>	Updates or initializes int uiSize in Txt.
2848	<code>if(uiSize>=22) uiSize-=6;</code>	Checks a condition and runs the guarded branch in Txt.
2849	<code>else if(uiSize>=18) uiSize-=5;</code>	Checks an alternate condition when the previous branch did not run in Txt.
2850	<code>else if(uiSize>=14) uiSize-=3;</code>	Checks an alternate condition when the previous branch did not run in Txt.
2851	<code>else if(uiSize>=10) uiSize-=1;</code>	Checks an alternate condition when the previous branch did not run in Txt.
2852	<code>if(uiSize<6) uiSize=6;</code>	Checks a condition and runs the guarded branch in Txt.
2853	<code>ObjectSetInteger(0,name,OBJPROP_FONTSIZE,uiSize);</code>	Creates, updates, or removes an MT5 chart panel object.
2854	<code>ObjectSetString(0,name,OBJPROP_FONT,bold ? "Arial Bold" : "Arial");</code>	Creates, updates, or removes an MT5 chart panel object.
2855	<code>ObjectSetInteger(0,name,OBJPROP_CORNER,CORNER_LEFT_UPPER);</code>	Creates, updates, or removes an MT5 chart panel object.
2856	<code>ObjectSetInteger(0,name,OBJPROP_SELECTABLE,false);</code>	Creates, updates, or removes an MT5 chart panel object.
2857	<code>ObjectSetInteger(0,name,OBJPROP_HIDDEN,true);</code>	Creates, updates, or removes an MT5 chart panel object.
2858	<code>ObjectSetInteger(0,name,OBJPROP_BACK,false);</code>	Creates, updates, or removes an MT5 chart panel object.
2859	<code>ObjectSetInteger(0,name,OBJPROP_ZORDER,z);</code>	Creates, updates, or removes an MT5 chart panel object.
2860	<code>}</code>	Opens or closes a code block in Txt.
2861		Blank separator for readability.
2862		Blank separator for readability.
2863	<code>void DotLabel(string name,int x,int y,color clr,int size=22,int z=30)</code>	Declares or starts function DotLabel.
2864	<code>{</code>	Opens or closes a code block in DotLabel.
2865	<code>ObjectDelete(0,name);</code>	Creates, updates, or removes an MT5 chart panel object.
2866	<code>ObjectCreate(0,name,OBJ_LABEL,0,0,0);</code>	Creates, updates, or removes an MT5 chart panel object.
2867	<code>ObjectSetInteger(0,name,OBJPROP_XDISTANCE,x);</code>	Creates, updates, or removes an MT5 chart panel object.
2868	<code>ObjectSetInteger(0,name,OBJPROP_YDISTANCE,y);</code>	Creates, updates, or removes an MT5 chart panel object.
2869	<code>ObjectSetString(0,name,OBJPROP_TEXT,"?");</code>	Creates, updates, or removes an MT5 chart panel object.
2870	<code>ObjectSetInteger(0,name,OBJPROP_COLOR,clr);</code>	Creates, updates, or removes an MT5 chart panel object.
2871	<code>ObjectSetInteger(0,name,OBJPROP_FONTSIZE,size);</code>	Creates, updates, or removes an MT5 chart panel object.
2872	<code>ObjectSetString(0,name,OBJPROP_FONT,"Arial Black");</code>	Creates, updates, or removes an MT5 chart panel object.
2873	<code>ObjectSetInteger(0,name,OBJPROP_CORNER,CORNER_LEFT_UPPER);</code>	Creates, updates, or removes an MT5 chart panel object.
2874	<code>ObjectSetInteger(0,name,OBJPROP_SELECTABLE,false);</code>	Creates, updates, or removes an MT5 chart panel object.
2875	<code>ObjectSetInteger(0,name,OBJPROP_HIDDEN,true);</code>	Creates, updates, or removes an MT5 chart panel object.
2876	<code>ObjectSetInteger(0,name,OBJPROP_BACK,false);</code>	Creates, updates, or removes an MT5 chart panel object.
2877	<code>ObjectSetInteger(0,name,OBJPROP_ZORDER,z);</code>	Creates, updates, or removes an MT5 chart panel object.
2878	<code>}</code>	Opens or closes a code block in DotLabel.
2879		Blank separator for readability.
2880	<code>void SetDotColor(string name,bool enabled,color onColor)</code>	Declares or starts function SetDotColor.
2881	<code>{</code>	Opens or closes a code block in SetDotColor.
2882	<code>if(ObjectFind(0,name)<0)</code>	Checks a condition and runs the guarded branch in SetDotColor.
2883	<code>return;</code>	Returns a value or exits the current function in SetDotColor.
2884		Blank separator for readability.
2885	<code>ObjectSetString(0,name,OBJPROP_TEXT,"?");</code>	Creates, updates, or removes an MT5 chart panel object.
2886	<code>ObjectSetInteger(0,name,OBJPROP_COLOR,enabled ? onColor : C'38,45,62');</code>	Creates, updates, or removes an MT5 chart panel object.
2887	<code>}</code>	Opens or closes a code block in SetDotColor.
2888		Blank separator for readability.
2889	<code>void SetCandleDotColor(string name,bool valid,bool aboveOpen)</code>	Declares or starts function SetCandleDotColor.
2890	<code>{</code>	Opens or closes a code block in SetCandleDotColor.
2891	<code>if(ObjectFind(0,name)<0)</code>	Checks a condition and runs the guarded branch in SetCandleDotColor.
2892	<code>return;</code>	Returns a value or exits the current function in SetCandleDotColor.
2893		Blank separator for readability.
2894	<code>ObjectSetString(0,name,OBJPROP_TEXT,"?");</code>	Creates, updates, or removes an MT5 chart panel object.
2895	<code>if(!valid)</code>	Checks a condition and runs the guarded branch in SetCandleDotColor.
2896	<code>{</code>	Opens or closes a code block in SetCandleDotColor.
2897	<code>ObjectSetInteger(0,name,OBJPROP_COLOR,C'38,45,62');</code>	Creates, updates, or removes an MT5 chart panel object.
2898	<code>return;</code>	Returns a value or exits the current function in SetCandleDotColor.
2899	<code>}</code>	Opens or closes a code block in SetCandleDotColor.
2900		Blank separator for readability.
2901	<code>ObjectSetInteger(0,name,OBJPROP_COLOR,aboveOpen ? C'20,225,75' : C'245,38,38');</code>	Creates, updates, or removes an MT5 chart panel object.
2902	<code>}</code>	Opens or closes a code block in SetCandleDotColor.
2903		Blank separator for readability.
2904	<code>void Btn(string name,string text,int x,int y,int w,int h,color bg,color fg,int size=9)</code>	Declares or starts function Btn.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2905	{	Opens or closes a code block in Btn.
2906	ObjectDelete(0,name);	Creates, updates, or removes an MT5 chart panel object.
2907	ObjectCreate(0,name,OBJ_BUTTON,0,0,0);	Creates, updates, or removes an MT5 chart panel object.
2908	ObjectSetInteger(0,name,OBJPROP_XDISTANCE,x);	Creates, updates, or removes an MT5 chart panel object.
2909	ObjectSetInteger(0,name,OBJPROP_YDISTANCE,y);	Creates, updates, or removes an MT5 chart panel object.
2910	ObjectSetInteger(0,name,OBJPROP_XSIZE,w);	Creates, updates, or removes an MT5 chart panel object.
2911	ObjectSetInteger(0,name,OBJPROP_YSIZE,h);	Creates, updates, or removes an MT5 chart panel object.
2912	ObjectSetString(0,name,OBJPROP_TEXT,text);	Creates, updates, or removes an MT5 chart panel object.
2913	ObjectSetInteger(0,name,OBJPROP_COLOR,fg);	Creates, updates, or removes an MT5 chart panel object.
2914	ObjectSetInteger(0,name,OBJPROP_BGCOLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2915	ObjectSetInteger(0,name,OBJPROP_BORDER_COLOR,C'70,78,105');	Creates, updates, or removes an MT5 chart panel object.
2916	int uiSize=size;	Updates or initializes int uiSize in Btn.
2917	if(uiSize>=22) uiSize-=6;	Checks a condition and runs the guarded branch in Btn.
2918	else if(uiSize>=18) uiSize-=5;	Checks an alternate condition when the previous branch did not run in Btn.
2919	else if(uiSize>=14) uiSize-=3;	Checks an alternate condition when the previous branch did not run in Btn.
2920	else if(uiSize>=10) uiSize-=1;	Checks an alternate condition when the previous branch did not run in Btn.
2921	if(uiSize<6) uiSize=6;	Checks a condition and runs the guarded branch in Btn.
2922	ObjectSetInteger(0,name,OBJPROP_FONTSIZE,uiSize);	Creates, updates, or removes an MT5 chart panel object.
2923	ObjectSetString(0,name,OBJPROP_FONT,"Arial Bold");	Creates, updates, or removes an MT5 chart panel object.
2924	ObjectSetInteger(0,name,OBJPROP_CORNER,CORNER_LEFT_UPPER);	Creates, updates, or removes an MT5 chart panel object.
2925	ObjectSetInteger(0,name,OBJPROP_SELECTABLE,false);	Creates, updates, or removes an MT5 chart panel object.
2926	ObjectSetInteger(0,name,OBJPROP_HIDDEN,true);	Creates, updates, or removes an MT5 chart panel object.
2927	ObjectSetInteger(0,name,OBJPROP_BACK,false);	Creates, updates, or removes an MT5 chart panel object.
2928	ObjectSetInteger(0,name,OBJPROP_ZORDER,50);	Creates, updates, or removes an MT5 chart panel object.
2929	ObjectSetInteger(0,name,OBJPROP_STATE,false);	Creates, updates, or removes an MT5 chart panel object.
2930	}	Opens or closes a code block in Btn.
2931		Blank separator for readability.
2932	void EditBox(string name,string text,int x,int y,int w,int h,color bg,color fg,int size=10)	Declares or starts function EditBox.
2933	{	Opens or closes a code block in EditBox.
2934	ObjectDelete(0,name);	Creates, updates, or removes an MT5 chart panel object.
2935	ObjectCreate(0,name,OBJ_EDIT,0,0,0);	Creates, updates, or removes an MT5 chart panel object.
2936	ObjectSetInteger(0,name,OBJPROP_XDISTANCE,x);	Creates, updates, or removes an MT5 chart panel object.
2937	ObjectSetInteger(0,name,OBJPROP_YDISTANCE,y);	Creates, updates, or removes an MT5 chart panel object.
2938	ObjectSetInteger(0,name,OBJPROP_XSIZE,w);	Creates, updates, or removes an MT5 chart panel object.
2939	ObjectSetInteger(0,name,OBJPROP_YSIZE,h);	Creates, updates, or removes an MT5 chart panel object.
2940	ObjectSetString(0,name,OBJPROP_TEXT,text);	Creates, updates, or removes an MT5 chart panel object.
2941	ObjectSetInteger(0,name,OBJPROP_COLOR,fg);	Creates, updates, or removes an MT5 chart panel object.
2942	ObjectSetInteger(0,name,OBJPROP_BGCOLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2943	ObjectSetInteger(0,name,OBJPROP_BORDER_COLOR,C'85,95,125');	Creates, updates, or removes an MT5 chart panel object.
2944	int uiSize=size;	Updates or initializes int uiSize in EditBox.
2945	if(uiSize>=22) uiSize-=6;	Checks a condition and runs the guarded branch in EditBox.
2946	else if(uiSize>=18) uiSize-=5;	Checks an alternate condition when the previous branch did not run in EditBox.
2947	else if(uiSize>=14) uiSize-=3;	Checks an alternate condition when the previous branch did not run in EditBox.
2948	else if(uiSize>=10) uiSize-=1;	Checks an alternate condition when the previous branch did not run in EditBox.
2949	if(uiSize<6) uiSize=6;	Checks a condition and runs the guarded branch in EditBox.
2950	ObjectSetInteger(0,name,OBJPROP_FONTSIZE,uiSize);	Creates, updates, or removes an MT5 chart panel object.
2951	ObjectSetString(0,name,OBJPROP_FONT,"Arial Bold");	Creates, updates, or removes an MT5 chart panel object.
2952	ObjectSetInteger(0,name,OBJPROP_CORNER,CORNER_LEFT_UPPER);	Creates, updates, or removes an MT5 chart panel object.
2953	ObjectSetInteger(0,name,OBJPROP_SELECTABLE,false);	Creates, updates, or removes an MT5 chart panel object.
2954	ObjectSetInteger(0,name,OBJPROP_HIDDEN,true);	Creates, updates, or removes an MT5 chart panel object.
2955	ObjectSetInteger(0,name,OBJPROP_BACK,false);	Creates, updates, or removes an MT5 chart panel object.
2956	ObjectSetInteger(0,name,OBJPROP_ZORDER,70);	Creates, updates, or removes an MT5 chart panel object.
2957	ObjectSetInteger(0,name,OBJPROP_READONLY,false);	Creates, updates, or removes an MT5 chart panel object.
2958	ObjectSetInteger(0,name,OBJPROP_ALIGN,ALIGN_CENTER);	Creates, updates, or removes an MT5 chart panel object.
2959	}	Opens or closes a code block in EditBox.
2960		Blank separator for readability.
2961	void SetTxt(string name,string text,color clr)	Declares or starts function SetTxt.
2962	{	Opens or closes a code block in SetTxt.
2963	if(ObjectFind(0,name)<0)	Checks a condition and runs the guarded branch in SetTxt.
2964	return;	Returns a value or exits the current function in SetTxt.
2965	ObjectSetString(0,name,OBJPROP_TEXT,text);	Creates, updates, or removes an MT5 chart panel object.
2966	ObjectSetInteger(0,name,OBJPROP_COLOR,clr);	Creates, updates, or removes an MT5 chart panel object.
2967	}	Opens or closes a code block in SetTxt.
2968		Blank separator for readability.
2969	void SetRect(string name,color bg)	Declares or starts function SetRect.
2970	{	Opens or closes a code block in SetRect.
2971	if(ObjectFind(0,name)<0)	Checks a condition and runs the guarded branch in SetRect.
2972	return;	Returns a value or exits the current function in SetRect.
2973	ObjectSetInteger(0,name,OBJPROP_BGCOLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2974	ObjectSetInteger(0,name,OBJPROP_COLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2975	}	Opens or closes a code block in SetRect.
2976		Blank separator for readability.
2977	void SetBtn(string name,color bg,color fg,string text="")	Declares or starts function SetBtn.
2978	{	Opens or closes a code block in SetBtn.
2979	if(ObjectFind(0,name)<0)	Checks a condition and runs the guarded branch in SetBtn.
2980	return;	Returns a value or exits the current function in SetBtn.
2981	ObjectSetInteger(0,name,OBJPROP_BGCOLOR,bg);	Creates, updates, or removes an MT5 chart panel object.
2982	ObjectSetInteger(0,name,OBJPROP_COLOR,fg);	Creates, updates, or removes an MT5 chart panel object.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
2983	if(text!="")	Checks a condition and runs the guarded branch in SetBtn.
2984	ObjectSetString(0,name,OBJPROP_TEXT,text);	Creates, updates, or removes an MT5 chart panel object.
2985	ObjectSetInteger(0,name,OBJPROP_STATE,false);	Creates, updates, or removes an MT5 chart panel object.
2986	}	Opens or closes a code block in SetBtn.
2987		Blank separator for readability.
2988	void SetEditText(string name,double value,int digits)	Declares or starts function SetEditText.
2989	{	Opens or closes a code block in SetEditText.
2990	if(ObjectFind(0,name)<0)	Checks a condition and runs the guarded branch in SetEditText.
2991	return;	Returns a value or exits the current function in SetEditText.
2992	ObjectSetString(0,name,OBJPROP_TEXT,DoubleToString(value,digits));	Creates, updates, or removes an MT5 chart panel object.
2993	}	Opens or closes a code block in SetEditText.
2994		Blank separator for readability.
2995	void SectionTitle(string key,string title,int x,int y,int w,color bg,color fg)	Declares or starts function SectionTitle.
2996	{	Opens or closes a code block in SectionTitle.
2997	Rect(UI+key+"_BG",x,y,w,26,bg,5);	Draws or refreshes a dashboard UI element on the MT5 chart.
2998	Txt(UI+key+"_TXT",title,x+12,y+5,fg,8,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
2999	}	Opens or closes a code block in SectionTitle.
3000		Blank separator for readability.
3001	datetime DayStart(datetime t)	Declares or starts function DayStart.
3002	{	Opens or closes a code block in DayStart.
3003	MqlDateTime dt;	Executes one statement in DayStart.
3004	TimeToStruct(t,dt);	Calls helper function TimeToStruct in DayStart.
3005	dt.hour=0;	Updates or initializes dt.hour in DayStart.
3006	dt.min=0;	Updates or initializes dt.min in DayStart.
3007	dt.sec=0;	Updates or initializes dt.sec in DayStart.
3008	return StructToTime(dt);	Returns a value or exits the current function in DayStart.
3009	}	Opens or closes a code block in DayStart.
3010		Blank separator for readability.
3011	string MoneyText(double v)	Declares or starts function MoneyText.
3012	{	Opens or closes a code block in MoneyText.
3013	string cur=AccountInfoString(ACCOUNT_CURRENCY);	Updates or initializes string cur in MoneyText.
3014	if(StringLen(cur)>0)	Checks a condition and runs the guarded branch in MoneyText.
3015	return cur+" "+DoubleToString(v,2);	Returns a value or exits the current function in MoneyText.
3016	return DoubleToString(v,2);	Returns a value or exits the current function in MoneyText.
3017	}	Opens or closes a code block in MoneyText.
3018		Blank separator for readability.
3019	string SignedMoneyText(double v)	Declares or starts function SignedMoneyText.
3020	{	Opens or closes a code block in SignedMoneyText.
3021	string s=MoneyText(MathAbs(v));	Updates or initializes string s in SignedMoneyText.
3022	if(v>0.0)	Checks a condition and runs the guarded branch in SignedMoneyText.
3023	return "+"+s;	Returns a value or exits the current function in SignedMoneyText.
3024	if(v<0.0)	Checks a condition and runs the guarded branch in SignedMoneyText.
3025	return "-"+s;	Returns a value or exits the current function in SignedMoneyText.
3026	return s;	Returns a value or exits the current function in SignedMoneyText.
3027	}	Opens or closes a code block in SignedMoneyText.
3028		Blank separator for readability.
3029	void CalcDealStats(datetime fromTime,int &trades,int &wins,int &losses,double &net,double &grossProfit,double &grossLoss)	Declares or starts function CalcDealStats.
3030	{	Opens or closes a code block in CalcDealStats.
3031	trades=0;	Updates or initializes trades in CalcDealStats.
3032	wins=0;	Updates or initializes wins in CalcDealStats.
3033	losses=0;	Updates or initializes losses in CalcDealStats.
3034	net=0.0;	Updates or initializes net in CalcDealStats.
3035	grossProfit=0.0;	Updates or initializes grossProfit in CalcDealStats.
3036	grossLoss=0.0;	Updates or initializes grossLoss in CalcDealStats.
3037		Blank separator for readability.
3038	if(!HistorySelect(fromTime,TimeCurrent()))	Checks a condition and runs the guarded branch in CalcDealStats.
3039	return;	Returns a value or exits the current function in CalcDealStats.
3040		Blank separator for readability.
3041	int total=HistoryDealsTotal();	Updates or initializes int total in CalcDealStats.
3042	for(int i=0;i<total;i++)	Loops over a fixed range or collection in CalcDealStats.
3043	{	Opens or closes a code block in CalcDealStats.
3044	ulong deal=HistoryDealGetTicket(i);	Updates or initializes ulong deal in CalcDealStats.
3045	if(deal==0)	Checks a condition and runs the guarded branch in CalcDealStats.
3046	continue;	Executes one statement in CalcDealStats.
3047		Blank separator for readability.
3048	if(HistoryDealGetString(deal,DEAL_SYMBOL)!=_Symbol)	Checks a condition and runs the guarded branch in CalcDealStats.
3049	continue;	Executes one statement in CalcDealStats.
3050	if(!IsDashboardMagic((long)HistoryDealGetInteger(deal,DEAL_MAGIC)))	Checks a condition and runs the guarded branch in CalcDealStats.
3051	continue;	Executes one statement in CalcDealStats.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3052		Blank separator for readability.
3053	long entry=HistoryDealGetInteger(deal,DEAL_ENTRY);	Updates or initializes long entry in CalcDealStats.
3054	if(entry!=DEAL_ENTRY_OUT && entry!=DEAL_ENTRY_INOUT && entry!=DEAL_ENTRY_OUT_BY)	Checks a condition and runs the guarded branch in CalcDealStats.
3055	continue;	Executes one statement in CalcDealStats.
3056		Blank separator for readability.
3057	double p=HistoryDealGetDouble(deal,DEAL_PROFIT)+	Part of the EA logic in CalcDealStats.
3058	HistoryDealGetDouble(deal,DEAL_SWAP)+	Part of the EA logic in CalcDealStats.
3059	HistoryDealGetDouble(deal,DEAL_COMMISSION);	Calls helper function HistoryDealGetDouble in CalcDealStats.
3060		Blank separator for readability.
3061	trades++;	Executes one statement in CalcDealStats.
3062	net+=p;	Updates or initializes net+ in CalcDealStats.
3063		Blank separator for readability.
3064	if(p>0.0)	Checks a condition and runs the guarded branch in CalcDealStats.
3065	{	Opens or closes a code block in CalcDealStats.
3066	wins++;	Executes one statement in CalcDealStats.
3067	grossProfit+=p;	Updates or initializes grossProfit+ in CalcDealStats.
3068	}	Opens or closes a code block in CalcDealStats.
3069	else if(p<0.0)	Checks an alternate condition when the previous branch did not run in CalcDealStats.
3070	{	Opens or closes a code block in CalcDealStats.
3071	losses++;	Executes one statement in CalcDealStats.
3072	grossLoss+=MathAbs(p);	Updates or initializes grossLoss+ in CalcDealStats.
3073	}	Opens or closes a code block in CalcDealStats.
3074	}	Opens or closes a code block in CalcDealStats.
3075	}	Opens or closes a code block in CalcDealStats.
3076		Blank separator for readability.
3077	double OpenProfit()	Declares or starts function OpenProfit.
3078	{	Opens or closes a code block in OpenProfit.
3079	double p=0.0;	Updates or initializes double p in OpenProfit.
3080	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in OpenProfit.
3081	{	Opens or closes a code block in OpenProfit.
3082	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in OpenProfit.
3083	if(ticket==0)	Checks a condition and runs the guarded branch in OpenProfit.
3084	continue;	Executes one statement in OpenProfit.
3085		Blank separator for readability.
3086	if(PositionGetString(POSITION_SYMBOL)==_Symbol &&	Checks a condition and runs the guarded branch in OpenProfit.
3087	IsDashboardMagic((long)PositionGetInteger(POSITION_MAGIC)))	Part of the EA logic in OpenProfit.
3088	p+=PositionGetDouble(POSITION_PROFIT);	Updates or initializes p+ in OpenProfit.
3089	}	Opens or closes a code block in OpenProfit.
3090	return p;	Returns a value or exits the current function in OpenProfit.
3091	}	Opens or closes a code block in OpenProfit.
3092		Blank separator for readability.
3093	double CurrentPositionRR()	Declares or starts function CurrentPositionRR.
3094	{	Opens or closes a code block in CurrentPositionRR.
3095	if(!SelectOurPosition())	Checks a condition and runs the guarded branch in CurrentPositionRR.
3096	return 0.0;	Returns a value or exits the current function in CurrentPositionRR.
3097		Blank separator for readability.
3098	bool	Updates or initializes bool isBuy in CurrentPositionRR.
3099	isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE))==POSITION_TYPE_BUY);	
3099	double entry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double entry in CurrentPositionRR.
3100	double price=isBuy ? SymbolInfoDouble(_Symbol,SYMBOL_BID) :	Updates or initializes double price in CurrentPositionRR.
3100	SymbolInfoDouble(_Symbol,SYMBOL_ASK);	
3101	double risk=g_RiskDistance;	Updates or initializes double risk in CurrentPositionRR.
3102		Blank separator for readability.
3103	if(risk<=0.0)	Checks a condition and runs the guarded branch in CurrentPositionRR.
3104	risk=MathAbs(entry-PositionGetDouble(POSITION_SL));	Updates or initializes risk in CurrentPositionRR.
3105	if(risk<=0.0)	Checks a condition and runs the guarded branch in CurrentPositionRR.
3106	return 0.0;	Returns a value or exits the current function in CurrentPositionRR.
3107		Blank separator for readability.
3108	double move=isBuy ? price-entry : entry-price;	Updates or initializes double move in CurrentPositionRR.
3109	return move/risk;	Returns a value or exits the current function in CurrentPositionRR.
3110	}	Opens or closes a code block in CurrentPositionRR.
3111		Blank separator for readability.
3112	string SessionName()	Declares or starts function SessionName.
3113	{	Opens or closes a code block in SessionName.
3114	MqlDateTime dt;	Executes one statement in SessionName.
3115	TimeToStruct(TimeCurrent(),dt);	Calls helper function TimeToStruct in SessionName.
3116		Blank separator for readability.
3117	if(dt.hour>=13 && dt.hour<22)	Checks a condition and runs the guarded branch in SessionName.
3118	return "NEW YORK";	Returns a value or exits the current function in SessionName.
3119	if(dt.hour>=7 && dt.hour<16)	Checks a condition and runs the guarded branch in SessionName.
3120	return "LONDON";	Returns a value or exits the current function in SessionName.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3121	if(dt.hour>=0 && dt.hour<7)	Checks a condition and runs the guarded branch in SessionName.
3122	return "ASIA";	Returns a value or exits the current function in SessionName.
3123	return "OFF HOURS";	Returns a value or exits the current function in SessionName.
3124	}	Opens or closes a code block in SessionName.
3125		Blank separator for readability.
3126	string SessionTimeLeft()	Declares or starts function SessionTimeLeft.
3127	{	Opens or closes a code block in SessionTimeLeft.
3128	MqlDateTime dt;	Executes one statement in SessionTimeLeft.
3129	TimeToStruct(TimeCurrent(),dt);	Calls helper function TimeToStruct in SessionTimeLeft.
3130		Blank separator for readability.
3131	int endHour=24;	Updates or initializes int endHour in SessionTimeLeft.
3132	if(dt.hour>=13 && dt.hour<22)	Checks a condition and runs the guarded branch in SessionTimeLeft.
3133	endHour=22;	Updates or initializes endHour in SessionTimeLeft.
3134	else if(dt.hour>=7 && dt.hour<16)	Checks an alternate condition when the previous branch did not run in SessionTimeLeft.
3135	endHour=16;	Updates or initializes endHour in SessionTimeLeft.
3136	else if(dt.hour>=0 && dt.hour<7)	Checks an alternate condition when the previous branch did not run in SessionTimeLeft.
3137	endHour=7;	Updates or initializes endHour in SessionTimeLeft.
3138		Blank separator for readability.
3139	int mins=(endHour-dt.hour)*60-dt.min;	Updates or initializes int mins in SessionTimeLeft.
3140	if(mins<0)	Checks a condition and runs the guarded branch in SessionTimeLeft.
3141	mins=0;	Updates or initializes mins in SessionTimeLeft.
3142		Blank separator for readability.
3143	return IntegerToString(mins/60)+"H "+IntegerToString(mins%60)+"M";	Returns a value or exits the current function in SessionTimeLeft.
3144	}	Opens or closes a code block in SessionTimeLeft.
3145		Blank separator for readability.
3146	string CandleCloseTimerText()	Declares or starts function CandleCloseTimerText.
3147	{	Opens or closes a code block in CandleCloseTimerText.
3148	int periodSeconds=PeriodSeconds(_Period);	Updates or initializes int periodSeconds in CandleCloseTimerText.
3149	datetime barOpen=iTime(_Symbol,_Period,0);	Updates or initializes datetime barOpen in CandleCloseTimerText.
3150	if(periodSeconds<=0 barOpen<=0)	Checks a condition and runs the guarded branch in CandleCloseTimerText.
3151	return "---";	Returns a value or exits the current function in CandleCloseTimerText.
3152		Blank separator for readability.
3153	long elapsed=(long)(TimeCurrent()-barOpen);	Updates or initializes long elapsed in CandleCloseTimerText.
3154	long left=(long)periodSeconds-elapsed;	Updates or initializes long left in CandleCloseTimerText.
3155	if(left<0)	Checks a condition and runs the guarded branch in CandleCloseTimerText.
3156	left=0;	Updates or initializes left in CandleCloseTimerText.
3157		Blank separator for readability.
3158	if(left>=3600)	Checks a condition and runs the guarded branch in CandleCloseTimerText.
3159	return	Returns a value or exits the current function in CandleCloseTimerText.
3160	StringFormat("%02d:%02d:%02d", (int)(left/3600), (int)((left%3600)/60), (int)(left%60));	Blank separator for readability.
3161	return StringFormat("%02d:%02d", (int)(left/60), (int)(left%60));	Returns a value or exits the current function in CandleCloseTimerText.
3162	}	Opens or closes a code block in CandleCloseTimerText.
3163		Blank separator for readability.
3164	int QualityScore()	Declares or starts function QualityScore.
3165	{	Opens or closes a code block in QualityScore.
3166	int score=0;	Updates or initializes int score in QualityScore.
3167		Blank separator for readability.
3168	if(TerminalInfoInteger(TERMINAL_TRADE_ALLOWED) && MQLInfoInteger(MQL_TRADE_ALLOWED))	Checks a condition and runs the guarded branch in QualityScore.
3169	score++;	Executes one statement in QualityScore.
3170	if(AccountInfoInteger(ACCOUNT_TRADE_ALLOWED))	Checks a condition and runs the guarded branch in QualityScore.
3171	score++;	Executes one statement in QualityScore.
3172	if(!SpreadBlocksTrades() CurrentSpreadPoints()<=InpMaxSpreadPoints)	Checks a condition and runs the guarded branch in QualityScore.
3173	score++;	Executes one statement in QualityScore.
3174	string tradeReason="";	Updates or initializes string tradeReason in QualityScore.
3175	if(IsTradingAllowedForDirection(true,tradeReason) IsTradingAllowedForDirection(false,tradeReason))	Checks a condition and runs the guarded branch in QualityScore.
3176	score++;	Executes one statement in QualityScore.
3177	if(g_ArmedMode!=ARM_NONE g_AutoArm)	Checks a condition and runs the guarded branch in QualityScore.
3178	score++;	Executes one statement in QualityScore.
3179	if(HasEnoughBars())	Checks a condition and runs the guarded branch in QualityScore.
3180	score++;	Executes one statement in QualityScore.
3181		Blank separator for readability.
3182	PatternSignal sig;	Executes one statement in QualityScore.
3183	if(DetectPattern(1,true,sig) DetectPattern(1,false,sig))	Checks a condition and runs the guarded branch in QualityScore.
3184	score+=2;	Updates or initializes score+ in QualityScore.
3185		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3186	if(score>8)	Checks a condition and runs the guarded branch in QualityScore.
3187	score=8;	Updates or initializes score in QualityScore.
3188	return score;	Returns a value or exits the current function in QualityScore.
3189	}	Opens or closes a code block in QualityScore.
3190		Blank separator for readability.
3191	string QualityText(int score)	Declares or starts function QualityText.
3192	{	Opens or closes a code block in QualityText.
3193	if(score>=7)	Checks a condition and runs the guarded branch in QualityText.
3194	return "A+++";	Returns a value or exits the current function in QualityText.
3195	if(score>=5)	Checks a condition and runs the guarded branch in QualityText.
3196	return "B";	Returns a value or exits the current function in QualityText.
3197	return "C";	Returns a value or exits the current function in QualityText.
3198	}	Opens or closes a code block in QualityText.
3199		Blank separator for readability.
3200	color QualityColor(int score)	Declares or starts function QualityColor.
3201	{	Opens or closes a code block in QualityColor.
3202	if(score>=7)	Checks a condition and runs the guarded branch in QualityColor.
3203	return clrLime;	Returns a value or exits the current function in QualityColor.
3204	if(score>=5)	Checks a condition and runs the guarded branch in QualityColor.
3205	return clrGold;	Returns a value or exits the current function in QualityColor.
3206	return clrTomato;	Returns a value or exits the current function in QualityColor.
3207	}	Opens or closes a code block in QualityColor.
3208		Blank separator for readability.
3209	void CardBox(string key,string title,string v1,string l2,string v2,int x,int y,int w,int h,color accent)	Declares or starts function CardBox.
3210	{	Opens or closes a code block in CardBox.
3211	Rect(UI+key+"_BOX",x,y,w,h,C'10,16,30',4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3212	Border(UI+key+"_BRD",x,y,w,h,C'24,36,58');	Calls helper function Border in CardBox.
3213	Txt(UI+key+"_T",title,x+16,y+14,C'185,195,220',8,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3214	Txt(UI+key+"_V1",v1,x+16,y+42,accent,15,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3215	Txt(UI+key+"_L2",l2,x+16,y+78,C'150,160,185',8,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3216	Txt(UI+key+"_V2",v2,x+16,y+96,clrWhite,10,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3217	}	Opens or closes a code block in CardBox.
3218		Blank separator for readability.
3219	void StatBox(string key,string title,string value,int x,int y,int w,int h,color valueColor)	Declares or starts function StatBox.
3220	{	Opens or closes a code block in StatBox.
3221	Rect(UI+key+"_BOX",x,y,w,h,C'9,14,27',4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3222	Border(UI+key+"_BRD",x,y,w,h,C'24,34,52');	Calls helper function Border in StatBox.
3223	Txt(UI+key+"_T",title,x+12,y+14,C'150,160,185',8,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3224	Txt(UI+key+"_V",value,x+12,y+38,valueColor,13,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3225	}	Opens or closes a code block in StatBox.
3226		Blank separator for readability.
3227	void MiniCard(string key,string title,string v1,string l2,string v2,int x,int y,int w,int h,color accent)	Declares or starts function MiniCard.
3228	{	Opens or closes a code block in MiniCard.
3229	Rect(UI+key+"_BOX",x,y,w,h,C'8,15,29',4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3230	Border(UI+key+"_BRD",x,y,w,h,C'22,36,58');	Calls helper function Border in MiniCard.
3231		Blank separator for readability.
3232	if(h<=70)	Checks a condition and runs the guarded branch in MiniCard.
3233	{	Opens or closes a code block in MiniCard.
3234	Txt(UI+key+"_T",title,x+9,y+5,C'160,174,204',6,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3235	Txt(UI+key+"_V1",v1,x+9,y+20,accent,10,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3236	Txt(UI+key+"_L2",l2,x+9,y+38,C'120,136,166',6,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3237	Txt(UI+key+"_V2",v2,x+9,y+49,clrWhite,7,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3238	return;	Returns a value or exits the current function in MiniCard.
3239	}	Opens or closes a code block in MiniCard.
3240		Blank separator for readability.
3241	Txt(UI+key+"_T",title,x+10,y+9,C'160,174,204',6,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3242	Txt(UI+key+"_V1",v1,x+10,y+29,accent,11,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3243	Txt(UI+key+"_L2",l2,x+10,y+59,C'120,136,166',6,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3244	<code>Txt(UI+key+"_V2", v2, x+10, y+74, clrWhite, 8, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3245	<code>}</code>	Opens or closes a code block in MiniCard.
3246		Blank separator for readability.
3247		Blank separator for readability.
3248	<code>void MiniStat(string key, string title, string value, int x, int y, int w, int h, color valueColor)</code>	Declares or starts function MiniStat.
3249	<code>{</code>	Opens or closes a code block in MiniStat.
3250	<code>Rect(UI+key+"_BOX", x, y, w, h, C'8, 14, 26', 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3251	<code>Border(UI+key+"_BRD", x, y, w, h, C'24, 34, 52');</code>	Calls helper function Border in MiniStat.
3252		Blank separator for readability.
3253	<code>if(h<=32)</code>	Checks a condition and runs the guarded branch in MiniStat.
3254	<code>{</code>	Opens or closes a code block in MiniStat.
3255	<code>Txt(UI+key+"_T", title, x+7, y+4, C'140, 154, 184', 6, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3256	<code>Txt(UI+key+"_V", value, x+7, y+17, valueColor, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3257	<code>return;</code>	Returns a value or exits the current function in MiniStat.
3258	<code>}</code>	Opens or closes a code block in MiniStat.
3259		Blank separator for readability.
3260	<code>Txt(UI+key+"_T", title, x+8, y+6, C'140, 154, 184', 6, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3261	<code>Txt(UI+key+"_V", value, x+8, y+23, valueColor, 8, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3262	<code>}</code>	Opens or closes a code block in MiniStat.
3263		Blank separator for readability.
3264		Blank separator for readability.
3265	<code>void MetricRow(string key, string leftTitle, string leftValue, string rightTitle, string rightValue,</code>	Declares or starts function MetricRow.
3266	<code>int x, int y, int w, int h, color leftColor, color rightColor)</code>	Part of the EA logic in MetricRow.
3267	<code>{</code>	Opens or closes a code block in MetricRow.
3268	<code>int mid=x+w/2;</code>	Updates or initializes int mid in MetricRow.
3269	<code>Rect(UI+key+"_BOX", x, y, w, h, C'8, 13, 24', 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3270	<code>Border(UI+key+"_BRD", x, y, w, h, C'24, 36, 58');</code>	Calls helper function Border in MetricRow.
3271	<code>Txt(UI+key+"_L1", leftTitle, x+8, y+6, C'150, 165, 195', 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3272	<code>Txt(UI+key+"_V1", leftValue, x+8, y+20, leftColor, 10, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3273	<code>Txt(UI+key+"_L2", rightTitle, mid+6, y+6, C'150, 165, 195', 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3274	<code>Txt(UI+key+"_V2", rightValue, mid+6, y+20, rightColor, 10, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3275	<code>}</code>	Opens or closes a code block in MetricRow.
3276		Blank separator for readability.
3277		Blank separator for readability.
3278	<code>//=====</code>	Comment explaining the nearby EA code.
3279	<code>// LIVE CANDLE / SWEEP STRUCTURE PANEL</code>	Comment explaining the nearby EA code.
3280	<code>//=====</code>	Comment explaining the nearby EA code.
3281	<code>int H4BlockNumber(datetime t)</code>	Declares or starts function H4BlockNumber.
3282	<code>{</code>	Opens or closes a code block in H4BlockNumber.
3283	<code>MqlDateTime dt;</code>	Executes one statement in H4BlockNumber.
3284	<code>TimeToStruct(t, dt);</code>	Calls helper function TimeToStruct in H4BlockNumber.
3285	<code>return (dt.hour/4)+1; // broker-server day: 1,2,3,4,5,6</code>	Returns a value or exits the current function in H4BlockNumber.
3286	<code>}</code>	Opens or closes a code block in H4BlockNumber.
3287		Blank separator for readability.
3288	<code>bool LatestH4ShiftByParity(bool wantOdd, int &shiftOut)</code>	Declares or starts function LatestH4ShiftByParity.
3289	<code>{</code>	Opens or closes a code block in LatestH4ShiftByParity.
3290	<code>shiftOut=-1;</code>	Updates or initializes shiftOut in LatestH4ShiftByParity.
3291	<code>int bars=iBars(_Symbol, PERIOD_H4);</code>	Updates or initializes int bars in LatestH4ShiftByParity.
3292	<code>if(bars<2)</code>	Checks a condition and runs the guarded branch in LatestH4ShiftByParity.
3293	<code>return false;</code>	Returns a value or exits the current function in LatestH4ShiftByParity.
3294		Blank separator for readability.
3295	<code>int maxScan=MathMin(bars-1, 30);</code>	Updates or initializes int maxScan in LatestH4ShiftByParity.
3296	<code>for(int sh=0; sh<=maxScan; sh++)</code>	Loops over a fixed range or collection in LatestH4ShiftByParity.
3297	<code>{</code>	Opens or closes a code block in LatestH4ShiftByParity.
3298	<code>datetime t=iTime(_Symbol, PERIOD_H4, sh);</code>	Updates or initializes datetime t in LatestH4ShiftByParity.
3299	<code>if(t<=0)</code>	Checks a condition and runs the guarded branch in LatestH4ShiftByParity.
3300	<code>continue;</code>	Executes one statement in LatestH4ShiftByParity.
3301		Blank separator for readability.
3302	<code>int block=H4BlockNumber(t);</code>	Updates or initializes int block in LatestH4ShiftByParity.
3303	<code>bool isOdd=((block%2)==1);</code>	Updates or initializes bool isOdd in LatestH4ShiftByParity.
3304	<code>if(isOdd==wantOdd)</code>	Checks a condition and runs the guarded branch in LatestH4ShiftByParity.
3305	<code>{</code>	Opens or closes a code block in LatestH4ShiftByParity.
3306	<code>shiftOut=sh;</code>	Updates or initializes shiftOut in LatestH4ShiftByParity.
3307	<code>return true;</code>	Returns a value or exits the current function in LatestH4ShiftByParity.
3308	<code>}</code>	Opens or closes a code block in LatestH4ShiftByParity.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3309	}	Opens or closes a code block in LatestH4ShiftByParity.
3310		Blank separator for readability.
3311	return false;	Returns a value or exits the current function in LatestH4ShiftByParity.
3312	}	Opens or closes a code block in LatestH4ShiftByParity.
3313		Blank separator for readability.
3314	bool PriceAboveOpen(ENUM_TIMEFRAMES tf,int shift,bool &aboveOpen)	Declares or starts function PriceAboveOpen.
3315	{	Opens or closes a code block in PriceAboveOpen.
3316	aboveOpen=false;	Updates or initializes aboveOpen in PriceAboveOpen.
3317	if(iBars(_Symbol,tf)<=shift)	Checks a condition and runs the guarded branch in PriceAboveOpen.
3318	return false;	Returns a value or exits the current function in PriceAboveOpen.
3319		Blank separator for readability.
3320	double op=iOpen(_Symbol,tf,shift);	Updates or initializes double op in PriceAboveOpen.
3321	double bid=SymbolInfoDouble(_Symbol,SYMBOL_BID);	Updates or initializes double bid in PriceAboveOpen.
3322	double ask=SymbolInfoDouble(_Symbol,SYMBOL_ASK);	Updates or initializes double ask in PriceAboveOpen.
3323	double price=bid;	Updates or initializes double price in PriceAboveOpen.
3324	if(price<=0.0)	Checks a condition and runs the guarded branch in PriceAboveOpen.
3325	price=ask;	Updates or initializes price in PriceAboveOpen.
3326	if(op<=0.0 price<=0.0)	Checks a condition and runs the guarded branch in PriceAboveOpen.
3327	return false;	Returns a value or exits the current function in PriceAboveOpen.
3328		Blank separator for readability.
3329	aboveOpen=(price>=op);	Updates or initializes aboveOpen in PriceAboveOpen.
3330	return true;	Returns a value or exits the current function in PriceAboveOpen.
3331	}	Opens or closes a code block in PriceAboveOpen.
3332		Blank separator for readability.
3333	bool StandardSweep(ENUM_TIMEFRAMES tf,bool &highSweep,bool &lowSweep)	Declares or starts function StandardSweep.
3334	{	Opens or closes a code block in StandardSweep.
3335	highSweep=false;	Updates or initializes highSweep in StandardSweep.
3336	lowSweep=false;	Updates or initializes lowSweep in StandardSweep.
3337		Blank separator for readability.
3338	if(iBars(_Symbol,tf)<3)	Checks a condition and runs the guarded branch in StandardSweep.
3339	return false;	Returns a value or exits the current function in StandardSweep.
3340		Blank separator for readability.
3341	double h0=iHigh(_Symbol,tf,0);	Updates or initializes double h0 in StandardSweep.
3342	double l0=iLow(_Symbol,tf,0);	Updates or initializes double l0 in StandardSweep.
3343	double h1=iHigh(_Symbol,tf,1);	Updates or initializes double h1 in StandardSweep.
3344	double l1=iLow(_Symbol,tf,1);	Updates or initializes double l1 in StandardSweep.
3345		Blank separator for readability.
3346	if(h0<=0.0 l0<=0.0 h1<=0.0 l1<=0.0)	Checks a condition and runs the guarded branch in StandardSweep.
3347	return false;	Returns a value or exits the current function in StandardSweep.
3348		Blank separator for readability.
3349	highSweep=(h0>h1+_Point*0.1);	Updates or initializes highSweep in StandardSweep.
3350	lowSweep=(l0<l1-_Point*0.1);	Updates or initializes lowSweep in StandardSweep.
3351	return true;	Returns a value or exits the current function in StandardSweep.
3352	}	Opens or closes a code block in StandardSweep.
3353		Blank separator for readability.
3354	bool H4ParitySweep(bool wantOdd,bool &highSweep,bool &lowSweep)	Declares or starts function H4ParitySweep.
3355	{	Opens or closes a code block in H4ParitySweep.
3356	highSweep=false;	Updates or initializes highSweep in H4ParitySweep.
3357	lowSweep=false;	Updates or initializes lowSweep in H4ParitySweep.
3358		Blank separator for readability.
3359	int sh=-1;	Updates or initializes int sh in H4ParitySweep.
3360	if(!LatestH4ShiftByParity(wantOdd,sh))	Checks a condition and runs the guarded branch in H4ParitySweep.
3361	return false;	Returns a value or exits the current function in H4ParitySweep.
3362		Blank separator for readability.
3363	if(iBars(_Symbol,PERIOD_H4)<=sh+1)	Checks a condition and runs the guarded branch in H4ParitySweep.
3364	return false;	Returns a value or exits the current function in H4ParitySweep.
3365		Blank separator for readability.
3366	double h=iHigh(_Symbol,PERIOD_H4,sh);	Updates or initializes double h in H4ParitySweep.
3367	double l=iLow(_Symbol,PERIOD_H4,sh);	Updates or initializes double l in H4ParitySweep.
3368	double prevH=iHigh(_Symbol,PERIOD_H4,sh+1);	Updates or initializes double prevH in H4ParitySweep.
3369	double prevL=iLow(_Symbol,PERIOD_H4,sh+1);	Updates or initializes double prevL in H4ParitySweep.
3370		Blank separator for readability.
3371	if(h<=0.0 l<=0.0 prevH<=0.0 prevL<=0.0)	Checks a condition and runs the guarded branch in H4ParitySweep.
3372	return false;	Returns a value or exits the current function in H4ParitySweep.
3373		Blank separator for readability.
3374	highSweep=(h>prevH+_Point*0.1);	Updates or initializes highSweep in H4ParitySweep.
3375	lowSweep=(l<prevL-_Point*0.1);	Updates or initializes lowSweep in H4ParitySweep.
3376	return true;	Returns a value or exits the current function in H4ParitySweep.
3377	}	Opens or closes a code block in H4ParitySweep.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3378		Blank separator for readability.
3379	void BuildCandleStructureRow(string key,string label,int x,int y,int w,int rowH)	Declares or starts function BuildCandleStructureRow.
3380	{	Opens or closes a code block in BuildCandleStructureRow.
3381	Rect(UI+key+"_SEP",x+12,y+rowH-1,w-24,1,C'24,36,58',6);	Draws or refreshes a dashboard UI element on the MT5 chart.
3382	Txt(UI+key+"_L",label,x+20,y+6,clrWhite,8,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3383	// Smaller dot with more vertical padding = no overflow outside the row.	Comment explaining the nearby EA code.
3384	DotLabel(UI+key+"_DOT",x+w-56,y+5,C'38,45,62',13,35);	Calls helper function DotLabel in BuildCandleStructureRow.
3385	}	Opens or closes a code block in BuildCandleStructureRow.
3386		Blank separator for readability.
3387	void BuildSweepStructureRow(string key,string label,int x,int y,int w,int rowH)	Declares or starts function BuildSweepStructureRow.
3388	{	Opens or closes a code block in BuildSweepStructureRow.
3389	Rect(UI+key+"_SEP",x+12,y+rowH-1,w-24,1,C'24,36,58',6);	Draws or refreshes a dashboard UI element on the MT5 chart.
3390	Txt(UI+key+"_L",label,x+18,y+6,clrWhite,8,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3391		Blank separator for readability.
3392	int highX=x+w/2-62;	Updates or initializes int highX in BuildSweepStructureRow.
3393	int lowX=x+w-142;	Updates or initializes int lowX in BuildSweepStructureRow.
3394		Blank separator for readability.
3395	Txt(UI+key+"_HT","High",highX,y+7,clrWhite,7,false,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3396	DotLabel(UI+key+"_HDOT",highX+62,y+5,C'38,45,62',13,35);	Calls helper function DotLabel in BuildSweepStructureRow.
3397		Blank separator for readability.
3398	Txt(UI+key+"_LT","Low",lowX,y+7,clrWhite,7,false,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3399	DotLabel(UI+key+"_LDOT",lowX+66,y+5,C'38,45,62',13,35);	Calls helper function DotLabel in BuildSweepStructureRow.
3400	}	Opens or closes a code block in BuildSweepStructureRow.
3401		Blank separator for readability.
3402	int BuildStructureSections(int x,int y,int w)	Declares or starts function BuildStructureSections.
3403	{	Opens or closes a code block in BuildStructureSections.
3404	int gap=10;	Updates or initializes int gap in BuildStructureSections.
3405	int h=176;	Updates or initializes int h in BuildStructureSections.
3406	int leftW=(w-gap)/2;	Updates or initializes int leftW in BuildStructureSections.
3407	int rightW=w-leftW-gap;	Updates or initializes int rightW in BuildStructureSections.
3408	int leftX=x;	Updates or initializes int leftX in BuildStructureSections.
3409	int rightX=x+leftW+gap;	Updates or initializes int rightX in BuildStructureSections.
3410	int rowH=30;	Updates or initializes int rowH in BuildStructureSections.
3411	int rowY=y+48;	Updates or initializes int rowY in BuildStructureSections.
3412		Blank separator for readability.
3413	Rect(UI+"CS_BG",leftX,y,leftW,h,C'5,11,22',4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3414	Border(UI+"CS_BRD",leftX,y,leftW,h,C'28,46,74');	Calls helper function Border in BuildStructureSections.
3415	Rect(UI+"CS_ACC",leftX,y,4,h,C'0,140,255',6);	Draws or refreshes a dashboard UI element on the MT5 chart.
3416	Txt(UI+"CS_TITLE","CANDLE STRUCTURE",leftX+18,y+14,C'25,155,255',10,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3417	Txt(UI+"CS_NOTE","price vs open",leftX+leftW-132,y+18,C'110,130,160',6,false,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3418		Blank separator for readability.
3419	BuildCandleStructureRow("CS_D1","D1",leftX+8,rowY,leftW-16,rowH);	Calls helper function BuildCandleStructureRow in BuildStructureSections.
3420	rowY+=rowH;	Updates or initializes rowY+ in BuildStructureSections.
3421	BuildCandleStructureRow("CS_H40","H4 Odd",leftX+8,rowY,leftW-16,rowH);	Calls helper function BuildCandleStructureRow in BuildStructureSections.
3422	rowY+=rowH;	Updates or initializes rowY+ in BuildStructureSections.
3423	BuildCandleStructureRow("CS_H4E","H4 Even",leftX+8,rowY,leftW-16,rowH);	Calls helper function BuildCandleStructureRow in BuildStructureSections.
3424	rowY+=rowH;	Updates or initializes rowY+ in BuildStructureSections.
3425	BuildCandleStructureRow("CS_H1","H1",leftX+8,rowY,leftW-16,rowH);	Calls helper function BuildCandleStructureRow in BuildStructureSections.
3426		Blank separator for readability.
3427	rowY=y+48;	Updates or initializes rowY in BuildStructureSections.
3428	Rect(UI+"SW_BG",rightX,y,rightW,h,C'5,11,22',4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3429	Border(UI+"SW_BRD",rightX,y,rightW,h,C'28,46,74');	Calls helper function Border in BuildStructureSections.
3430	Rect(UI+"SW_ACC",rightX,y,4,h,C'0,140,255',6);	Draws or refreshes a dashboard UI element on the MT5 chart.
3431	Txt(UI+"SW_TITLE","SWEEP STRUCTURE",rightX+18,y+14,C'25,155,255',10,true,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3432	Txt(UI+"SW_NOTE","high red low green",rightX+rightW-172,y+18,C'110,130,160',6,false,25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3433		Blank separator for readability.
3434	BuildSweepStructureRow("SW_H1","H1",rightX+8,rowY,rightW-16,rowH);	Calls helper function BuildSweepStructureRow in BuildStructureSections.
3435	rowY+=rowH;	Updates or initializes rowY+ in BuildStructureSections.
3436	BuildSweepStructureRow("SW_H40","H4 Odd",rightX+8,rowY,rightW-16,rowH);	Calls helper function BuildSweepStructureRow in BuildStructureSections.
3437	rowY+=rowH;	Updates or initializes rowY+ in BuildStructureSections.
3438	BuildSweepStructureRow("SW_H4E","H4 Even",rightX+8,rowY,rightW-16,rowH);	Calls helper function BuildSweepStructureRow in BuildStructureSections.
3439	rowY+=rowH;	Updates or initializes rowY+ in BuildStructureSections.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3440	BuildSweepStructureRow("SW_D1", "D1", rightX+8, rowY, rightW-16, rowH);	Calls helper function BuildSweepStructureRow in BuildStructureSections.
3441		Blank separator for readability.
3442	return h;	Returns a value or exits the current function in BuildStructureSections.
3443	}	Opens or closes a code block in BuildStructureSections.
3444		Blank separator for readability.
3445	void UpdateStructureSections()	Declares or starts function UpdateStructureSections.
3446	{	Opens or closes a code block in UpdateStructureSections.
3447	bool above=false;	Updates or initializes bool above in UpdateStructureSections.
3448	bool valid=false;	Updates or initializes bool valid in UpdateStructureSections.
3449	int sh=-1;	Updates or initializes int sh in UpdateStructureSections.
3450		Blank separator for readability.
3451	// Candle Structure:	Comment explaining the nearby EA code.
3452	// Green = live price above opening.	Comment explaining the nearby EA code.
3453	// Red = live price below opening.	Comment explaining the nearby EA code.
3454	valid=PriceAboveOpen(PERIOD_D1,0, above);	Updates or initializes valid in UpdateStructureSections.
3455	SetCandleDotColor(UI+"CS_D1_D0T", valid, above);	Calls helper function SetCandleDotColor in UpdateStructureSections.
3456		Blank separator for readability.
3457	valid=PriceAboveOpen(PERIOD_H1,0, above);	Updates or initializes valid in UpdateStructureSections.
3458	SetCandleDotColor(UI+"CS_H1_D0T", valid, above);	Calls helper function SetCandleDotColor in UpdateStructureSections.
3459		Blank separator for readability.
3460	bool oddAbove=false;	Updates or initializes bool oddAbove in UpdateStructureSections.
3461	bool validOdd=LatestH4ShiftByParity(true, sh);	Updates or initializes bool validOdd in UpdateStructureSections.
3462	if(validOdd)	Checks a condition and runs the guarded branch in UpdateStructureSections.
3463	validOdd=PriceAboveOpen(PERIOD_H4, sh, oddAbove);	Updates or initializes validOdd in UpdateStructureSections.
3464	SetCandleDotColor(UI+"CS_H4_D0T", validOdd, oddAbove);	Calls helper function SetCandleDotColor in UpdateStructureSections.
3465		Blank separator for readability.
3466	bool evenAbove=false;	Updates or initializes bool evenAbove in UpdateStructureSections.
3467	bool validEven=LatestH4ShiftByParity(false, sh);	Updates or initializes bool validEven in UpdateStructureSections.
3468	if(validEven)	Checks a condition and runs the guarded branch in UpdateStructureSections.
3469	validEven=PriceAboveOpen(PERIOD_H4, sh, evenAbove);	Updates or initializes validEven in UpdateStructureSections.
3470	SetCandleDotColor(UI+"CS_H4E_D0T", validEven, evenAbove);	Calls helper function SetCandleDotColor in UpdateStructureSections.
3471		Blank separator for readability.
3472	// Sweep Structure:	Comment explaining the nearby EA code.
3473	// High sweep = red light.	Comment explaining the nearby EA code.
3474	// Low sweep = green light.	Comment explaining the nearby EA code.
3475	// If both swept, both lights stay ON.	Comment explaining the nearby EA code.
3476	bool hi=false;	Updates or initializes bool hi in UpdateStructureSections.
3477	bool lo=false;	Updates or initializes bool lo in UpdateStructureSections.
3478		Blank separator for readability.
3479	valid=StandardSweep(PERIOD_H1,hi, lo);	Updates or initializes valid in UpdateStructureSections.
3480	SetDotColor(UI+"SW_H1_HDOT", valid && hi, C'245,38,38');	Calls helper function SetDotColor in UpdateStructureSections.
3481	SetDotColor(UI+"SW_H1_LD0T", valid && lo, C'20,225,75');	Calls helper function SetDotColor in UpdateStructureSections.
3482		Blank separator for readability.
3483	valid=H4ParitySweep(true,hi, lo);	Updates or initializes valid in UpdateStructureSections.
3484	SetDotColor(UI+"SW_H40_HDOT", valid && hi, C'245,38,38');	Calls helper function SetDotColor in UpdateStructureSections.
3485	SetDotColor(UI+"SW_H40_LD0T", valid && lo, C'20,225,75');	Calls helper function SetDotColor in UpdateStructureSections.
3486		Blank separator for readability.
3487	valid=H4ParitySweep(false,hi, lo);	Updates or initializes valid in UpdateStructureSections.
3488	SetDotColor(UI+"SW_H4E_HDOT", valid && hi, C'245,38,38');	Calls helper function SetDotColor in UpdateStructureSections.
3489	SetDotColor(UI+"SW_H4E_LD0T", valid && lo, C'20,225,75');	Calls helper function SetDotColor in UpdateStructureSections.
3490		Blank separator for readability.
3491	valid=StandardSweep(PERIOD_D1,hi, lo);	Updates or initializes valid in UpdateStructureSections.
3492	SetDotColor(UI+"SW_D1_HDOT", valid && hi, C'245,38,38');	Calls helper function SetDotColor in UpdateStructureSections.
3493	SetDotColor(UI+"SW_D1_LD0T", valid && lo, C'20,225,75');	Calls helper function SetDotColor in UpdateStructureSections.
3494	}	Opens or closes a code block in UpdateStructureSections.
3495		Blank separator for readability.
3496	void BuildPanel()	Declares or starts function BuildPanel.
3497	{	Opens or closes a code block in BuildPanel.
3498	CalcPanelGeometry();	Calls helper function CalcPanelGeometry in BuildPanel.
3499	ObjectsDeleteAll(0,UI);	Calls helper function ObjectsDeleteAll in BuildPanel.
3500		Blank separator for readability.
3501	int x=g_X;	Updates or initializes int x in BuildPanel.
3502	int y=g_Y;	Updates or initializes int y in BuildPanel.
3503	int W=g_W;	Updates or initializes int W in BuildPanel.
3504	int H=g_H;	Updates or initializes int H in BuildPanel.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3505		Blank separator for readability.
3506	int pad=18;	Updates or initializes int pad in BuildPanel.
3507	int gap=10;	Updates or initializes int gap in BuildPanel.
3508	int inner=W-pad*2;	Updates or initializes int inner in BuildPanel.
3509		Blank separator for readability.
3510	color bg=C'3,8,16';	Updates or initializes color bg in BuildPanel.
3511	color card=C'7,13,24';	Updates or initializes color card in BuildPanel.
3512	color card2=C'15,18,38';	Updates or initializes color card2 in BuildPanel.
3513	color border=C'24,38,62';	Updates or initializes color border in BuildPanel.
3514	color soft=C'150,165,195';	Updates or initializes color soft in BuildPanel.
3515	color title=clrWhite;	Updates or initializes color title in BuildPanel.
3516		Blank separator for readability.
3517	int cy=y+10;	Updates or initializes int cy in BuildPanel.
3518		Blank separator for readability.
3519	Rect(UI+"BG", x, y, W, H, bg, 1);	Draws or refreshes a dashboard UI element on the MT5 chart.
3520	Border(UI+"BORDER", x, y, W, H, C'18,38,64');	Calls helper function Border in BuildPanel.
3521		Blank separator for readability.
3522	// Header only. Removed account balance, daily P&L, spread and time-server cards.	Comment explaining the nearby EA code.
3523	int headerH=50;	Updates or initializes int headerH in BuildPanel.
3524	Txt(UI+"TITLE", "3-CANDLE EA v6.9", x+pad, cy+2, clrWhite, 13, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3525	Txt(UI+"SUB", ClipText(_Symbol+" "+EnumToString(_Period)+" Clean Pro Controller", 68), x+pad, cy+28, C'145,170,215', 8, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3526	Txt(UI+"CONN", "EA ON", x+W-86, cy+5, clrLime, 8, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3527	Txt(UI+"VPS", "ONLINE", x+W-86, cy+25, clrLime, 7, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3528	cy+=headerH+gap;	Updates or initializes cy+ in BuildPanel.
3529		Blank separator for readability.
3530	// Status + daily lock row. This keeps useful lock info but removes the requested cards.	Comment explaining the nearby EA code.
3531	int statusH=72;	Updates or initializes int statusH in BuildPanel.
3532	int dlockW=260;	Updates or initializes int dlockW in BuildPanel.
3533	int statusW=inner-dlockW-gap;	Updates or initializes int statusW in BuildPanel.
3534		Blank separator for readability.
3535	Rect(UI+"STBG", x+pad, cy, statusW, statusH, card2, 4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3536	Border(UI+"STBRD", x+pad, cy, statusW, statusH, border);	Calls helper function Border in BuildPanel.
3537	Txt(UI+"STLBL", "STATUS", x+pad+16, cy+9, soft, 8, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3538	Txt(UI+"STVAL", "WAITING FOR SETUP", x+pad+16, cy+28, clrWhite, 15, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3539	Txt(UI+"STSUB", "Arm BUY or SELL and wait for next fresh model", x+pad+16, cy+54, C'180,190,215', 8, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3540		Blank separator for readability.
3541	int dlx=x+pad+statusW+gap;	Updates or initializes int dlx in BuildPanel.
3542	Rect(UI+"DLOCK_BOX", dlx, cy, dlockW, statusH, card, 4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3543	Border(UI+"DLOCK_BRD", dlx, cy, dlockW, statusH, border);	Calls helper function Border in BuildPanel.
3544	Txt(UI+"DLOCK_T", "DAILY LOSS LOCK", dlx+14, cy+9, soft, 7, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3545	Txt(UI+"DLOCK_V1", "0 / 2", dlx+14, cy+28, clrGold, 13, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3546	Txt(UI+"DLOCK_L2", "STATUS", dlx+126, cy+11, soft, 7, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3547	Txt(UI+"DLOCK_V2", "ACTIVE", dlx+126, cy+31, clrLime, 9, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3548	cy+=statusH+gap;	Updates or initializes cy+ in BuildPanel.
3549		Blank separator for readability.
3550	// Candle + Sweep structure. Real-time dots update from OnTimer().	Comment explaining the nearby EA code.
3551	cy+=BuildStructureSections(x+pad, cy, inner)+gap;	Updates or initializes cy+ in BuildPanel.
3552		Blank separator for readability.
3553	// Arming + quality row.	Comment explaining the nearby EA code.
3554	int leftW=(inner-gap)*65/100;	Updates or initializes int leftW in BuildPanel.
3555	int rightW=inner-leftW-gap;	Updates or initializes int rightW in BuildPanel.
3556	int mainH=150;	Updates or initializes int mainH in BuildPanel.
3557		Blank separator for readability.
3558	Rect(UI+"ARMBG", x+pad, cy, leftW, mainH, card, 4);	Draws or refreshes a dashboard UI element on the MT5 chart.
3559	Border(UI+"ARMBRD", x+pad, cy, leftW, mainH, border);	Calls helper function Border in BuildPanel.
3560	Txt(UI+"ARMT", "ARMING CONTROLS", x+pad+16, cy+12, title, 10, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.
3561		Blank separator for readability.
3562	int armY=cy+46;	Updates or initializes int armY in BuildPanel.
3563	int armBtnW=(leftW-52)/3;	Updates or initializes int armBtnW in BuildPanel.
3564	Btn(B_BUY, "ARM BUY", x+pad+16, armY, armBtnW, 38, C'0,120,50', clrWhite, 9);	Draws or refreshes a dashboard UI element on the MT5 chart.
3565	Btn(B_AUTO, "AUTO ARM", x+pad+26+armBtnW, armY, armBtnW, 38, C'25,36,52', clrWhite, 9);	Draws or refreshes a dashboard UI element on the MT5 chart.
3566	Btn(B_SELL, "ARM SELL", x+pad+36+armBtnW*2, armY, armBtnW, 38, C'145,28,28', clrWhite, 9);	Draws or refreshes a dashboard UI element on the MT5 chart.
3567		Blank separator for readability.
3568	int armInfoY=cy+102;	Updates or initializes int armInfoY in BuildPanel.
3569	int armCol=(leftW-32)/4;	Updates or initializes int armCol in BuildPanel.
3570	Txt(UI+"ASL", "ARM STATUS", x+pad+16, armInfoY, soft, 7, true, 25);	Draws or refreshes a dashboard UI element on the MT5 chart.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3571	<code>Txt(UI+"ASV", "OFF", x+pad+16, armInfoY+18, clrSilver, 8, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3572	<code>Txt(UI+"ATL", "ARM_TIMER", x+pad+16+armCol, armInfoY, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3573	<code>Txt(UI+"ATV", "00:00:00", x+pad+16+armCol, armInfoY+18, clrWhite, 9, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3574	<code>Txt(UI+"BIASL", "CC_TIMER", x+pad+16+armCol*2, armInfoY, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3575	<code>Txt(UI+"BIASV", "- - - -", x+pad+16+armCol*2, armInfoY+18, clrDeepSkyBlue, 8, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3576	<code>Txt(UI+"CONFL", "CONFIRM", x+pad+16+armCol*3, armInfoY, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3577	<code>Txt(UI+"CONFV", "NO", x+pad+16+armCol*3, armInfoY+18, clrSilver, 8, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3578		Blank separator for readability.
3579	<code>Rect(UI+"QUALBG", x+pad+leftW+gap, cy, rightW, mainH, card, 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3580	<code>Border(UI+"QUALBRD", x+pad+leftW+gap, cy, rightW, mainH, border);</code>	Calls helper function Border in BuildPanel.
3581	<code>Txt(UI+"QUALT", "TRADE_QUALITY_SCORE", x+pad+leftW+gap+16, cy+12, title, 10, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3582	<code>Txt(UI+"QUALV", "0 / 8", x+pad+leftW+gap+36, cy+48, clrWhite, 20, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3583	<code>Txt(UI+"QUALS", "WAITING", x+pad+leftW+gap+36, cy+86, C'180, 190, 215', 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3584	<code>Rect(UI+"QUALBARBG", x+pad+leftW+gap+22, cy+110, rightW-44, 6, C'30, 38, 52', 5);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3585	<code>Rect(UI+"QUALBARA", x+pad+leftW+gap+22, cy+110, (rightW-44)/3, 6, C'35, 175, 80', 6);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3586	<code>Rect(UI+"QUALBARB", x+pad+leftW+gap+22+(rightW-44)/3, cy+110, (rightW-44)/3, 6, C'225, 175, 80', 6);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3587	<code>Rect(UI+"QUALBARC", x+pad+leftW+gap+22+(rightW-44)*2/3, cy+110, (rightW-44)/3, 6, C'210, 175, 80', 6);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3588	<code>Txt(UI+"QUALR1", "7-8 A+++", x+pad+leftW+gap+24, cy+128, clrLime, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3589	<code>Txt(UI+"QUALR2", "5-6 B", x+pad+leftW+gap+108, cy+128, clrGold, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3590	<code>Txt(UI+"QUALR3", "0-4 C", x+pad+leftW+gap+174, cy+128, clrTomato, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3591	<code>cy+=mainH+gap;</code>	Updates or initializes cy+ in BuildPanel.
3592		Blank separator for readability.
3593	<code>// Risk + mode row.</code>	Comment explaining the nearby EA code.
3594	<code>int riskW=(inner-gap)*65/100;</code>	Updates or initializes int riskW in BuildPanel.
3595	<code>int modeW=inner-riskW-gap;</code>	Updates or initializes int modeW in BuildPanel.
3596	<code>int rowH=150;</code>	Updates or initializes int rowH in BuildPanel.
3597		Blank separator for readability.
3598	<code>Rect(UI+"RISKBG", x+pad, cy, riskW, rowH, card, 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3599	<code>Border(UI+"RISKBRD", x+pad, cy, riskW, rowH, border);</code>	Calls helper function Border in BuildPanel.
3600	<code>Txt(UI+"RISKT", "RISK_MANAGEMENT", x+pad+16, cy+12, title, 10, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3601		Blank separator for readability.
3602	<code>int groupY=cy+44;</code>	Updates or initializes int groupY in BuildPanel.
3603	<code>int groupH=72;</code>	Updates or initializes int groupH in BuildPanel.
3604	<code>int groupGap=10;</code>	Updates or initializes int groupGap in BuildPanel.
3605	<code>int groupW=(riskW-32-groupGap*2)/3;</code>	Updates or initializes int groupW in BuildPanel.
3606	<code>int gx=x+pad+16;</code>	Updates or initializes int gx in BuildPanel.
3607		Blank separator for readability.
3608	<code>Rect(UI+"LOTBOX", gx, groupY, groupW, groupH, C'9, 16, 30', 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3609	<code>Border(UI+"LOTBOXBRD", gx, groupY, groupW, groupH, C'22, 36, 58');</code>	Calls helper function Border in BuildPanel.
3610	<code>Txt(UI+"LOTL", "LOT_SIZE", gx+10, groupY+8, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3611	<code>Btn(B_LOT_DN, "-", gx+10, groupY+35, 28, 28, C'24, 34, 48', clrWhite, 9);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3612	<code>EditBox(E_LOT, DoubleToString(g_lot, VolumeDigits()), gx+43, groupY+35, 66, 28, C'12, 18, 30', clrWhite, 9);</code>	Calls helper function EditBox in BuildPanel.
3613	<code>Btn(B_LOT_UP, "+", gx+114, groupY+35, 28, 28, C'24, 34, 48', clrWhite, 9);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3614		Blank separator for readability.
3615	<code>gx+=groupW+groupGap;</code>	Updates or initializes gx+ in BuildPanel.
3616	<code>Rect(UI+"RISKBOX", gx, groupY, groupW, groupH, C'9, 16, 30', 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3617	<code>Border(UI+"RISKBOXBRD", gx, groupY, groupW, groupH, C'22, 36, 58');</code>	Calls helper function Border in BuildPanel.
3618	<code>Txt(UI+"RISKL", "RISK "+AccountInfoString(ACCOUNT_CURRENCY), gx+10, groupY+8, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3619	<code>Btn(B_RISK_DN, "-", gx+10, groupY+35, 28, 28, C'24, 34, 48', clrWhite, 9);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3620	<code>EditBox(E_RISK, DoubleToString(g_RiskMoney, 2), gx+43, groupY+35, 78, 28, C'12, 18, 30', clrWhite, 9);</code>	Calls helper function EditBox in BuildPanel.
3621	<code>Btn(B_RISK_UP, "+", gx+126, groupY+35, 28, 28, C'24, 34, 48', clrWhite, 9);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3622		Blank separator for readability.
3623	<code>gx+=groupW+groupGap;</code>	Updates or initializes gx+ in BuildPanel.
3624	<code>Rect(UI+"RRBOX", gx, groupY, groupW, groupH, C'9, 16, 30', 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3625	<code>Border(UI+"RRBOXBRD", gx, groupY, groupW, groupH, C'22, 36, 58');</code>	Calls helper function Border in BuildPanel.
3626	<code>Txt(UI+"RRL", "RR_TARGET", gx+10, groupY+8, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3627	<code>Btn(B_RR_DN, "-", gx+10, groupY+35, 28, 28, C'24, 34, 48', clrWhite, 9);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3628	<code>Txt(UI+"RRV", "1:3.0", gx+48, groupY+42, clrWhite, 9, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3629	<code>Btn(B_RR_UP, "+", gx+114, groupY+35, 28, 28, C'24, 34, 48', clrWhite, 9);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3630		Blank separator for readability.
3631	<code>Txt(UI+"LOTRANGE", "Min -- Step -- Max --", x+pad+16, cy+126, C'120, 135, 165', 7, false, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3632	<code>Txt(UI+"RISKAMT", "Risk --", x+pad+riskW/2-38, cy+126, C'120, 135, 165', 7, false, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3633	<code>Txt(UI+"RRTXT", "TP 3R SL model", x+pad+riskW-158, cy+126, C'120, 135, 165', 7, false, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3634		Blank separator for readability.
3635	<code>Rect(UI+"MODEBG", x+pad+riskW+gap, cy, modeW, rowH, card, 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3636	<code>Border(UI+"MODEBRD", x+pad+riskW+gap, cy, modeW, rowH, border);</code>	Calls helper function Border in BuildPanel.
3637	<code>Txt(UI+"MODET", "TRADE MODE", x+pad+riskW+gap+16, cy+12, title, 10, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3638	<code>Btn(B_MODE_SAFE, "SAFE MODE", x+pad+riskW+gap+16, cy+44, (modeW-44)/2, 32, C'0, 95, 170', clrWhite, 8);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3639	<code>Btn(B_MODE_ADV, "ADVANCED", x+pad+riskW+gap+28+(modeW-44)/2, cy+44, (modeW-44)/2, 32, C'2 4, 34, 48', clrWhite, 8);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3640	<code>Btn(B_PC, g_PC_On ? "PART ON" : "PART OFF", x+pad+riskW+gap+16, cy+95, 72, 24, g_PC_On ? C'20, 150, 65' : C'92, 34, 34', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3641	<code>Txt(UI+"PCSTAT", "Partial OFF", x+pad+riskW+gap+94, cy+99, C'180, 190, 210', 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3642	<code>EditBox(E_PC1, DoubleToString(g_PC1_Pct, 0), x+pad+riskW+gap+184, cy+93, 36, 24, C'12, 18, 3 0', clrYellow, 7);</code>	Calls helper function EditBox in BuildPanel.
3643	<code>EditBox(E_PC2, DoubleToString(g_PC2_Pct, 0), x+pad+riskW+gap+226, cy+93, 36, 24, C'12, 18, 3 0', clrYellow, 7);</code>	Calls helper function EditBox in BuildPanel.
3644	<code>EditBox(E_PC3, DoubleToString(g_PC3_Pct, 0), x+pad+riskW+gap+268, cy+93, 36, 24, C'12, 18, 3 0', clrYellow, 7);</code>	Calls helper function EditBox in BuildPanel.
3645	<code>Btn(B_SECOND, g_SecondEntryOn ? "2ND ON" : "2ND OFF", x+pad+riskW+gap+16, cy+122, 72, 24, g_SecondEntryOn ? C'20, 150, 65' : C'92, 34, 34', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3646	<code>Txt(UI+"SESTAT", "2nd entry OFF", x+pad+riskW+gap+94, cy+126, C'150, 165, 195', 7, false, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3647	<code>Btn(B_FIRST_TRAIL, g_FirstTrailOn ? "1TR ON" : "1TR OFF", x+pad+riskW+gap+228, cy+122, 72, 24, g_FirstTrailOn ? C'20, 150, 65' : C'92, 34, 34', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3648	<code>cy+=rowH+gap;</code>	Updates or initializes cy+ in BuildPanel.
3649		Blank separator for readability.
3650	<code>// Message footer. Account statistics and position info are removed.</code>	Comment explaining the nearby EA code.
3651	<code>int msgH=52;</code>	Updates or initializes int msgH in BuildPanel.
3652	<code>Rect(UI+"MSGBG", x+pad, cy, inner, msgH, card, 4);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3653	<code>Border(UI+"MSGBRD", x+pad, cy, inner, msgH, border);</code>	Calls helper function Border in BuildPanel.
3654	<code>Txt(UI+"MSGT", "MESSAGE", x+pad+16, cy+9, soft, 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3655	<code>Txt(UI+"MSG", "Ready", x+pad+16, cy+28, C'150, 165, 195', 7, true, 25);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3656		Blank separator for readability.
3657	<code>int manageBtnW=50;</code>	Updates or initializes int manageBtnW in BuildPanel.
3658	<code>int manageGap=6;</code>	Updates or initializes int manageGap in BuildPanel.
3659	<code>int manageX=x+pad+inner-(manageBtnW*6+manageGap*5)-12;</code>	Updates or initializes int manageX in BuildPanel.
3660	<code>int manageY=cy+14;</code>	Updates or initializes int manageY in BuildPanel.
3661	<code>Btn(B_CLOSE, "CLOSE", manageX, manageY, manageBtnW, 26, C'80, 12, 18', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3662	<code>Btn(B_BE, "BE", manageX+manageBtnW+manageGap, manageY, manageBtnW, 26, C'0, 70, 115', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3663	<code>Btn(B_FIRST_BE, "1BE", manageX+(manageBtnW+manageGap)*2, manageY, manageBtnW, 26, C'0, 92, 125', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3664	<code>Btn(B_FIRST_BE_EXIT, "1BEX", manageX+(manageBtnW+manageGap)*3, manageY, manageBtnW, 26, C' '0, 118, 96', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3665	<code>Btn(B_CLOSE50, "50%", manageX+(manageBtnW+manageGap)*4, manageY, manageBtnW, 26, C'90, 65, 0', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3666	<code>Btn(B_CLOSE_ALL, "ALL", manageX+(manageBtnW+manageGap)*5, manageY, manageBtnW, 26, C'55, 1 6, 95', clrWhite, 7);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3667		Blank separator for readability.
3668	<code>g_PanelBuilt=true;</code>	Updates or initializes g_PanelBuilt in BuildPanel.
3669	<code>}</code>	Opens or closes a code block in BuildPanel.
3670		Blank separator for readability.
3671	<code>void UpdatePanel()</code>	Declares or starts function UpdatePanel.
3672	<code>{</code>	Opens or closes a code block in UpdatePanel.
3673	<code>if(!g_PanelBuilt)</code>	Checks a condition and runs the guarded branch in UpdatePanel.
3674	<code>return;</code>	Returns a value or exits the current function in UpdatePanel.
3675		Blank separator for readability.
3676	<code>bool hasPos=HasOurPosition();</code>	Updates or initializes bool hasPos in UpdatePanel.
3677	<code>int openCount=CountOurPositions();</code>	Updates or initializes int openCount in UpdatePanel.
3678	<code>double balance=AccountInfoDouble(ACCOUNT_BALANCE);</code>	Updates or initializes double balance in UpdatePanel.
3679	<code>double equity=AccountInfoDouble(ACCOUNT_EQUITY);</code>	Updates or initializes double equity in UpdatePanel.
3680	<code>datetime now=TimeCurrent();</code>	Updates or initializes datetime now in UpdatePanel.
3681		Blank separator for readability.
3682	<code>int totalTrades=0, wins=0, losses=0;</code>	Updates or initializes int totalTrades in UpdatePanel.
3683	<code>double totalNet=0.0, grossProfit=0.0, grossLoss=0.0;</code>	Updates or initializes double totalNet in UpdatePanel.
3684	<code>CalcDealStats((datetime)0, totalTrades, wins, losses, totalNet, grossProfit, grossLoss);</code>	Calls helper function CalcDealStats in UpdatePanel.
3685		Blank separator for readability.
3686	<code>int dayTrades=0, dayWins=0, dayLosses=0;</code>	Updates or initializes int dayTrades in UpdatePanel.
3687	<code>double dayNet=0.0, dayGrossProfit=0.0, dayGrossLoss=0.0;</code>	Updates or initializes double dayNet in UpdatePanel.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3688	CalcDealStats(DayStart(now),dayTrades,dayWins,dayLosses,dayNet,dayGrossProfit,dayGr	Calls helper function CalcDealStats in UpdatePanel.
3689	ossLoss);	
3690	double riskMoney=NormalizeRiskMoney(g_RiskMoney);	Blank separator for readability.
3691	double dayR=(riskMoney>0.0 ? dayNet/riskMoney : 0.0);	Updates or initializes double riskMoney in UpdatePanel.
3692	double remainingR=MathMax(0.0,InpDailyLimitR-MathMax(0.0,-dayR));	Updates or initializes double dayR in UpdatePanel.
3693	int score=QualityScore();	Updates or initializes double remainingR in UpdatePanel.
3694	double openPL=OpenProfit();	Updates or initializes int score in UpdatePanel.
3695	double rr=CurrentPositionRR();	Updates or initializes double openPL in UpdatePanel.
3696	double rrPct=0.0;	Updates or initializes double rr in UpdatePanel.
3697	if(EffectiveRR())>0.0)	Updates or initializes double rrPct in UpdatePanel.
3698	rrPct=MathMax(0.0,MathMin(100.0,rr/EffectiveRR()*100.0));	Checks a condition and runs the guarded branch in UpdatePanel.
3699		Blank separator for readability.
3700	string timeText=TimeToString(now,TIME_SECONDS);	Updates or initializes rrPct in UpdatePanel.
3701	string dateText=TimeToString(now,TIME_DATE);	Blank separator for readability.
3702	string addEntryStatus="Open positions: "+IntegerToString(openCount)+" model	Updates or initializes string timeText in UpdatePanel.
3703	entries locked";	Updates or initializes string dateText in UpdatePanel.
3704	if(hasPos)	Updates or initializes string addEntryStatus in UpdatePanel.
3705	{	Checks a condition and runs the guarded branch in UpdatePanel.
3706	if(!g_SecondEntryOn)	Opens or closes a code block in UpdatePanel.
3707	addEntryStatus="Second entry OFF - one entry mode";	Checks a condition and runs the guarded branch in UpdatePanel.
3708	else if(openCount>=MaxEntriesPerCycle())	Updates or initializes addEntryStatus in UpdatePanel.
3709	addEntryStatus="Maximum "+IntegerToString(MaxEntriesPerCycle())+" entries	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3710	already open";	Updates or initializes addEntryStatus in UpdatePanel.
3711	else if(g_SecondEntryDone)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3712	addEntryStatus="Second entry already used for this trade";	Updates or initializes addEntryStatus in UpdatePanel.
3713	else if(g_SecondEntryBlocked)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3714	addEntryStatus="First trade reached	Updates or initializes addEntryStatus in UpdatePanel.
3715	"+DoubleToString(MathMax(0.1,InpSecondEntryCancelAtR,1)+"R, no second entry";	
3716	else	Fallback branch when prior conditions did not match in UpdatePanel.
3717	addEntryStatus="2nd waits for	Updates or initializes addEntryStatus in UpdatePanel.
3718	-"+DoubleToString(MathMax(0.1,InpSecondEntryTriggerLossR,1)+"R same SL/TP";	
3719	}	Opens or closes a code block in UpdatePanel.
3720	if(g_AutoArm)	Blank separator for readability.
3721	{	Checks a condition and runs the guarded branch in UpdatePanel.
3722	SetRect(UI+"STBG",C'28,35,12');	Opens or closes a code block in UpdatePanel.
3723	SetTxt(UI+"STVAL","AUTO WAIT",clrGold);	Draws or refreshes a dashboard UI element on the MT5 chart.
3724	SetTxt(UI+"STSUB",hasPos ? addEntryStatus : "Entry after 3rd candle	Draws or refreshes a dashboard UI element on the MT5 chart.
3725	closes",C'180,190,215');	Draws or refreshes a dashboard UI element on the MT5 chart.
3726	}	Opens or closes a code block in UpdatePanel.
3727	else if(g_ArmedMode==ARM_BUY)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3728	{	Opens or closes a code block in UpdatePanel.
3729	SetRect(UI+"STBG",C'5,50,18');	Draws or refreshes a dashboard UI element on the MT5 chart.
3730	SetTxt(UI+"STVAL","BUY WAIT",C'80,255,140');	Draws or refreshes a dashboard UI element on the MT5 chart.
3731	SetTxt(UI+"STSUB",hasPos ? addEntryStatus : "BUY enters after 3rd candle	Draws or refreshes a dashboard UI element on the MT5 chart.
3732	closes",C'180,190,215');	Draws or refreshes a dashboard UI element on the MT5 chart.
3733	}	Opens or closes a code block in UpdatePanel.
3734	else if(g_ArmedMode==ARM_SELL)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3735	{	Opens or closes a code block in UpdatePanel.
3736	SetRect(UI+"STBG",C'52,10,10');	Draws or refreshes a dashboard UI element on the MT5 chart.
3737	SetTxt(UI+"STVAL","SELL WAIT",C'255,105,105');	Draws or refreshes a dashboard UI element on the MT5 chart.
3738	SetTxt(UI+"STSUB",hasPos ? addEntryStatus : "SELL enters after 3rd candle	Draws or refreshes a dashboard UI element on the MT5 chart.
3739	closes",C'180,190,215');	Draws or refreshes a dashboard UI element on the MT5 chart.
3740	}	Opens or closes a code block in UpdatePanel.
3741	else if(hasPos)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3742	{	Opens or closes a code block in UpdatePanel.
3743	SetRect(UI+"STBG",C'16,44,24');	Draws or refreshes a dashboard UI element on the MT5 chart.
3744	SetTxt(UI+"STVAL",openCount>1 ? "POSITIONS OPEN" : "POSITION	Draws or refreshes a dashboard UI element on the MT5 chart.
3745	OPEN",C'110,255,165');	Draws or refreshes a dashboard UI element on the MT5 chart.
3746	SetTxt(UI+"STSUB","Managing "+IntegerToString(openCount)+" open position(s), SL,	Draws or refreshes a dashboard UI element on the MT5 chart.
3747	TP, BE and partials",C'180,190,215');	Draws or refreshes a dashboard UI element on the MT5 chart.
3748	}	Opens or closes a code block in UpdatePanel.
3749	else	Fallback branch when prior conditions did not match in UpdatePanel.
3750	{	Opens or closes a code block in UpdatePanel.
3751	SetRect(UI+"STBG",C'18,21,42');	Draws or refreshes a dashboard UI element on the MT5 chart.
3752	SetTxt(UI+"STVAL","WAITING",clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3753	SetTxt(UI+"STSUB","Arm BUY or SELL and wait for next model",C'180,190,215');	Draws or refreshes a dashboard UI element on the MT5 chart.
3754	}	Opens or closes a code block in UpdatePanel.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3747		Blank separator for readability.
3748	SetBtn(B_BUY,g_ArmedMode==ARM_BUY ? C'15,210,90' : C'8,120,55',clrWhite,	Draws or refreshes a dashboard UI element on the MT5 chart.
3749	g_ArmedMode==ARM_BUY ? "ARM BUY ON" : "ARM BUY");	Updates or initializes g_ArmedMode in UpdatePanel.
3750	SetBtn(B_AUTO,g_AutoArm ? C'0,95,170' : C'24,34,48',clrWhite,	Draws or refreshes a dashboard UI element on the MT5 chart.
3751	g_AutoArm ? "AUTO ON" : "AUTO ARM");	Executes one statement in UpdatePanel.
3752	SetBtn(B_SELL,g_ArmedMode==ARM_SELL ? C'230,48,48' : C'150,28,28',clrWhite,	Draws or refreshes a dashboard UI element on the MT5 chart.
3753	g_ArmedMode==ARM_SELL ? "ARM SELL ON" : "ARM SELL");	Updates or initializes g_ArmedMode in UpdatePanel.
3754	SetBtn(B_PC,g_PC_On ? C'20,150,65' : C'92,34,34',clrWhite,g_PC_On ? "PART ON" : "PART OFF");	Draws or refreshes a dashboard UI element on the MT5 chart.
3755	SetBtn(B_SECOND,g_SecondEntryOn ? C'20,150,65' : C'92,34,34',clrWhite,g_SecondEntryOn ? "2ND ON" : "2ND OFF");	Draws or refreshes a dashboard UI element on the MT5 chart.
3756	SetBtn(B_FIRST_TRAIL,g_FirstTrailOn ? C'20,150,65' : C'92,34,34',clrWhite,g_FirstTrailOn ? "1TR ON" : "1TR OFF");	Draws or refreshes a dashboard UI element on the MT5 chart.
3757	SetBtn(B_MODE_SAFE,g_AdvancedMode ? C'24,34,48' : C'0,95,170',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3758	SetBtn(B_MODE_ADV,g_AdvancedMode ? C'0,95,170' : C'24,34,48',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3759	SetBtn(B_CLOSE,hasPos ? C'110,18,24' : C'36,42,54',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3760	SetBtn(B_BE,hasPos ? C'0,88,145' : C'36,42,54',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3761	SetBtn(B_FIRST_BE,hasPos ? C'0,105,140' : C'36,42,54',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3762	SetBtn(B_FIRST_BE_EXIT,openCount>1 ? C'0,132,108' : C'36,42,54',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3763	SetBtn(B_CLOSE50,hasPos ? C'120,84,0' : C'36,42,54',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3764	SetBtn(B_CLOSE_ALL,hasPos ? C'74,26,126' : C'36,42,54',clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3765		Blank separator for readability.
3766	int sp=CurrentSpreadPoints();	Updates or initializes int sp in UpdatePanel.
3767	string lotRange="Min "+DoubleToString(BrokerMinLot(),VolumeDigits())+	Part of the EA logic in UpdatePanel.
3768	" Step "+DoubleToString(BrokerStepLot(),VolumeDigits())+	Part of the EA logic in UpdatePanel.
3769	" Max "+DoubleToString(BrokerMaxLot(),VolumeDigits());	Executes one statement in UpdatePanel.
3770	SetTxt(UI+"LOT RANGE",ClipText(lotRange,44),(SpreadBlocksTrades() && sp>InpMaxSpreadPoints) ? clrTomato : C'120,135,165');	Draws or refreshes a dashboard UI element on the MT5 chart.
3771		Blank separator for readability.
3772	SetTxt(UI+"BAL_V1",ClipText(MoneyText(balance),16),clrDeepSkyBlue);	Draws or refreshes a dashboard UI element on the MT5 chart.
3773	SetTxt(UI+"BAL_V2",ClipText(MoneyText(equity),16),clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3774	SetTxt(UI+"DPNL_V1",DoubleToString(dayR,2)+"R",dayNet>=0.0 ? clrLime : clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3775	SetTxt(UI+"DPNL_V2",DoubleToString(remainingR,2)+"R",remainingR>0.0 ? clrLime : clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3776	SetTxt(UI+"DLOCK_V1",IntegerToString(dayLosses)+" / "+IntegerToString(InpDailyLossLimitCount),	Draws or refreshes a dashboard UI element on the MT5 chart.
3777	dayLosses<InpDailyLossLimitCount ? clrGold : clrTomato);	Executes one statement in UpdatePanel.
3778	SetTxt(UI+"DLOCK_V2",dayLosses<InpDailyLossLimitCount ? "ACTIVE" : "LOCKED",	Draws or refreshes a dashboard UI element on the MT5 chart.
3779	dayLosses<InpDailyLossLimitCount ? clrLime : clrTomato);	Executes one statement in UpdatePanel.
3780	SetTxt(UI+"SPRD_V1",IntegerToString(sp)+" p", (SpreadBlocksTrades() && sp>InpMaxSpreadPoints) ? clrTomato : clrLime);	Draws or refreshes a dashboard UI element on the MT5 chart.
3781	SetTxt(UI+"SPRD_V2",SpreadBlocksTrades() ? "BLOCK" : "NO BLOCK",SpreadBlocksTrades() ? clrGold : clrLime);	Draws or refreshes a dashboard UI element on the MT5 chart.
3782	SetTxt(UI+"TIME_V1",timeText,clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3783	SetTxt(UI+"TIME_V2",ClipText(dateText,14),clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3784		Blank separator for readability.
3785	UpdateStructureSections();	Calls helper function UpdateStructureSections in UpdatePanel.
3786		Blank separator for readability.
3787	string armTxt="OFF";	Updates or initializes string armTxt in UpdatePanel.
3788	color armColor=clrSilver;	Updates or initializes color armColor in UpdatePanel.
3789	if(g_AutoArm)	Checks a condition and runs the guarded branch in UpdatePanel.
3790	{	Opens or closes a code block in UpdatePanel.
3791	armTxt="AUTO";	Updates or initializes armTxt in UpdatePanel.
3792	armColor=clrGold;	Updates or initializes armColor in UpdatePanel.
3793	}	Opens or closes a code block in UpdatePanel.
3794	else if(g_ArmedMode==ARM_BUY)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3795	{	Opens or closes a code block in UpdatePanel.
3796	armTxt="BUY";	Updates or initializes armTxt in UpdatePanel.
3797	armColor=clrLime;	Updates or initializes armColor in UpdatePanel.
3798	}	Opens or closes a code block in UpdatePanel.
3799	else if(g_ArmedMode==ARM_SELL)	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3800	{	Opens or closes a code block in UpdatePanel.
3801	armTxt="SELL";	Updates or initializes armTxt in UpdatePanel.
3802	armColor=clrTomato;	Updates or initializes armColor in UpdatePanel.
3803	}	Opens or closes a code block in UpdatePanel.
3804	SetTxt(UI+"ASV",armTxt,armColor);	Draws or refreshes a dashboard UI element on the MT5 chart.
3805		Blank separator for readability.
3806	int armedSecs=0;	Updates or initializes int armedSecs in UpdatePanel.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3807	<code>if((g_ArmedMode!=ARM_NONE g_AutoArm) && g_ArmMinSignalTime>0)</code>	Checks a condition and runs the guarded branch in UpdatePanel.
3808	<code>{</code>	Opens or closes a code block in UpdatePanel.
3809	<code> long elapsed=(long)(TimeCurrent()-g_ArmMinSignalTime);</code>	Updates or initializes long elapsed in UpdatePanel.
3810	<code> if(elapsed<0)</code>	Checks a condition and runs the guarded branch in UpdatePanel.
3811	<code> elapsed=0;</code>	Updates or initializes elapsed in UpdatePanel.
3812	<code> armedSecs=(int)elapsed;</code>	Updates or initializes armedSecs in UpdatePanel.
3813	<code> }</code>	Opens or closes a code block in UpdatePanel.
3814	<code> string timerTxt=StringFormat("%02d:%02d:%02d",armedSecs/3600,(armedSecs%3600)/60,armedSecs%60);</code>	Updates or initializes string timerTxt in UpdatePanel.
3815	<code> SetTxt(UI+"ATV",timerTxt,clrWhite);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3816		Blank separator for readability.
3817	<code> SetTxt(UI+"BIASV",CandleCloseTimerText(),clrDeepSkyBlue);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3818	<code> SetTxt(UI+"CONFV",score>=6 ? "YES" : "NO",score>=6 ? clrLime : clrSilver);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3819		Blank separator for readability.
3820	<code> SetTxt(UI+"QUALV",IntegerToString(score)+" / 8",QualityColor(score));</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3821	<code> SetTxt(UI+"QUALS",score>=6 ? QualityText(score)+" QUALITY" : "WAITING",QualityColor(score));</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3822		Blank separator for readability.
3823	<code> SetTxt(UI+"RISKAMT",ClipText(RiskPreviewText(),54),C'120,135,165');</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3824	<code> SetTxt(UI+"RRV", "1:"+DoubleToString(EffectiveRR(),1),clrWhite);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3825	<code> SetTxt(UI+"RRTXT", "TP "+DoubleToString(EffectiveRR(),1)+"R SL model",C'120,135,165');</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3826		Blank separator for readability.
3827	<code> if(g_PC_On && hasPos)</code>	Checks a condition and runs the guarded branch in UpdatePanel.
3828	<code> {</code>	Opens or closes a code block in UpdatePanel.
3829	<code> string pc="Partials watching";</code>	Updates or initializes string pc in UpdatePanel.
3830	<code> if(g_PC3_Done)</code>	Checks a condition and runs the guarded branch in UpdatePanel.
3831	<code> pc="PC1+PC2+PC3 done";</code>	Updates or initializes pc in UpdatePanel.
3832	<code> else if(g_PC2_Done)</code>	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3833	<code> pc="PC1+PC2 done";</code>	Updates or initializes pc in UpdatePanel.
3834	<code> else if(g_PC1_Done)</code>	Checks an alternate condition when the previous branch did not run in UpdatePanel.
3835	<code> pc="PC1 done";</code>	Updates or initializes pc in UpdatePanel.
3836	<code> SetTxt(UI+"PCSTAT",pc,clrGold);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3837	<code> }</code>	Opens or closes a code block in UpdatePanel.
3838	<code> else</code>	Fallback branch when prior conditions did not match in UpdatePanel.
3839	<code> {</code>	Opens or closes a code block in UpdatePanel.
3840	<code> SetTxt(UI+"PCSTAT",g_PC_On ? "Partials ready" : "Partials OFF",g_PC_On ? clrSkyBlue : clrGray);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3841	<code> }</code>	Opens or closes a code block in UpdatePanel.
3842	<code> SetTxt(UI+"SESTAT",SecondEntryStatusText(),g_SecondEntryOn ? clrSkyBlue : clrGray);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3843		Blank separator for readability.
3844	<code> if(hasPos)</code>	Checks a condition and runs the guarded branch in UpdatePanel.
3845	<code> {</code>	Opens or closes a code block in UpdatePanel.
3846	<code> SelectOurPosition();</code>	Calls helper function SelectOurPosition in UpdatePanel.
3847	<code> bool isBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);</code>	Updates or initializes bool isBuy in UpdatePanel.
3848	<code> double entry=PositionGetDouble(POSITION_PRICE_OPEN);</code>	Updates or initializes double entry in UpdatePanel.
3849	<code> double sl=PositionGetDouble(POSITION_SL);</code>	Updates or initializes double sl in UpdatePanel.
3850	<code> double tp=PositionGetDouble(POSITION_TP);</code>	Updates or initializes double tp in UpdatePanel.
3851	<code> double profit=PositionGetDouble(POSITION_PROFIT);</code>	Updates or initializes double profit in UpdatePanel.
3852	<code> double vol=PositionGetDouble(POSITION_VOLUME);</code>	Updates or initializes double vol in UpdatePanel.
3853		Blank separator for readability.
3854	<code> SetTxt(UI+"P1V", "OPEN",C'190,95,255');</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3855	<code> SetTxt(UI+"P2V",isBuy ? "BUY" : "SELL",isBuy ? clrLime : clrTomato);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3856	<code> SetTxt(UI+"P3V",DoubleToString(vol,VolumeDigits()),clrWhite);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3857	<code> SetTxt(UI+"P4V",DoubleToString(rr,2)+"R",profit>=0.0 ? clrLime : clrTomato);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3858	<code> SetTxt(UI+"P4M",SignedMoneyText(profit),profit>=0.0 ? clrLime : clrTomato);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3859	<code> SetTxt(UI+"P5V",PriceText(entry),clrWhite);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3860	<code> SetTxt(UI+"P6V",PriceText(sl),clrOrangeRed);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3861	<code> SetTxt(UI+"P7V",PriceText(tp),clrSpringGreen);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3862	<code> SetTxt(UI+"RRP2",DoubleToString(rr,2)+"R/"+DoubleToString(EffectiveRR(),2)+"R",clrWhite);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3863	<code> SetTxt(UI+"RRP3",DoubleToString(rrPct,0)+"%",rr>=0.0 ? clrLime : clrTomato);</code>	Draws or refreshes a dashboard UI element on the MT5 chart.
3864	<code> }</code>	Opens or closes a code block in UpdatePanel.
3865	<code> else</code>	Fallback branch when prior conditions did not match in UpdatePanel.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3866	{	Opens or closes a code block in UpdatePanel.
3867	SetTxt(UI+"P1V", "NONE", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3868	SetTxt(UI+"P2V", "---", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3869	SetTxt(UI+"P3V", "---", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3870	SetTxt(UI+"P4V", "0.00R", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3871	SetTxt(UI+"P4M", "\$0.00", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3872	SetTxt(UI+"P5V", "---", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3873	SetTxt(UI+"P6V", "---", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3874	SetTxt(UI+"P7V", "---", clrGray);	Draws or refreshes a dashboard UI element on the MT5 chart.
3875	SetTxt(UI+"RRP2", "0.00R/"+DoubleToString(EffectiveRR(),2)+"R", clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3876	SetTxt(UI+"RRP3", "0%", clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3877	}	Opens or closes a code block in UpdatePanel.
3878		Blank separator for readability.
3879	double winRate=(totalTrades>0 ? (double)wins*100.0/(double)totalTrades : 0.0);	Updates or initializes double winRate in UpdatePanel.
3880	double profitFactor=(grossLoss>0.0 ? grossProfit/grossLoss : (grossProfit>0.0 ? 99.0 : 0.0));	Updates or initializes double profitFactor in UpdatePanel.
3881	SetTxt(UI+"S1_V", IntegerToString(totalTrades), clrWhite);	Draws or refreshes a dashboard UI element on the MT5 chart.
3882	SetTxt(UI+"S2_V", DoubleToString(winRate,2)+"%", winRate>=50.0 ? clrLime : clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3883	SetTxt(UI+"S3_V", IntegerToString(wins), clrLime);	Draws or refreshes a dashboard UI element on the MT5 chart.
3884	SetTxt(UI+"S4_V", IntegerToString(losses), clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3885	SetTxt(UI+"S5_V", SignedMoneyText(openPL), openPL>=0.0 ? clrDeepSkyBlue : clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3886	SetTxt(UI+"S6_V", SignedMoneyText(totalNet), totalNet>=0.0 ? clrLime : clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3887	SetTxt(UI+"S7_V", DoubleToString(profitFactor,2), profitFactor>=1.0 ? clrDeepSkyBlue : clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3888	SetTxt(UI+"S8_V", "---", clrTomato);	Draws or refreshes a dashboard UI element on the MT5 chart.
3889		Blank separator for readability.
3890	SetTxt(UI+"MSG", ClipText(g_LastMessage,42), g_LastMsgColor);	Draws or refreshes a dashboard UI element on the MT5 chart.
3891		Blank separator for readability.
3892	ChartRedraw();	Calls helper function ChartRedraw in UpdatePanel.
3893	}	Opens or closes a code block in UpdatePanel.
3894		Blank separator for readability.
3895		Blank separator for readability.
3896	//=====	Comment explaining the nearby EA code.
3897	// SUPABASE WEB CONTROL BRIDGE	Comment explaining the nearby EA code.
3898	// React/Supabase -> Edge Function command queue -> This EA	Comment explaining the nearby EA code.
3899	// This block does not change the algo. It only calls the same internal	Comment explaining the nearby EA code.
3900	// functions used by your MT5 panel buttons.	Comment explaining the nearby EA code.
3901	//=====	Comment explaining the nearby EA code.
3902	void RefreshLotEdit();	Declares or starts function RefreshLotEdit.
3903	void RefreshRiskEdit();	Declares or starts function RefreshRiskEdit.
3904	void TCX_SupabasePostState(bool force);	Declares or starts function TCX_SupabasePostState.
3905		Blank separator for readability.
3906	string TCX_TrimRightSlash(string url)	Declares or starts function TCX_TrimRightSlash.
3907	{	Opens or closes a code block in TCX_TrimRightSlash.
3908	string out=url;	Updates or initializes string out in TCX_TrimRightSlash.
3909	StringTrimLeft(out);	Calls helper function StringTrimLeft in TCX_TrimRightSlash.
3910	StringTrimRight(out);	Calls helper function StringTrimRight in TCX_TrimRightSlash.
3911	while(StringLen(out)>0 && StringSubstr(out,StringLen(out)-1,1)!="/")	Repeats while the condition remains true in TCX_TrimRightSlash.
3912	out=StringSubstr(out,0,StringLen(out)-1);	Updates or initializes out in TCX_TrimRightSlash.
3913	return out;	Returns a value or exits the current function in TCX_TrimRightSlash.
3914	}	Opens or closes a code block in TCX_TrimRightSlash.
3915		Blank separator for readability.
3916	string TCX_JsonEscape(string s)	Declares or starts function TCX_JsonEscape.
3917	{	Opens or closes a code block in TCX_JsonEscape.
3918	StringReplace(s,"\\", "\\");	Calls helper function StringReplace in TCX_JsonEscape.
3919	StringReplace(s,"\"", "\"");	Calls helper function StringReplace in TCX_JsonEscape.
3920	StringReplace(s,"\\r", " ");	Calls helper function StringReplace in TCX_JsonEscape.
3921	StringReplace(s,"\\n", " ");	Calls helper function StringReplace in TCX_JsonEscape.
3922	StringReplace(s,"\\t", " ");	Calls helper function StringReplace in TCX_JsonEscape.
3923	return s;	Returns a value or exits the current function in TCX_JsonEscape.
3924	}	Opens or closes a code block in TCX_JsonEscape.
3925		Blank separator for readability.
3926	string TCX_BoolText(bool v)	Declares or starts function TCX_BoolText.
3927	{	Opens or closes a code block in TCX_BoolText.
3928	return v ? "true" : "false";	Returns a value or exits the current function in TCX_BoolText.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3929	}	Opens or closes a code block in TCX_BoolText.
3930		Blank separator for readability.
3931	string TCX_CommandStatusText()	Declares or starts function TCX_CommandStatusText.
3932	{	Opens or closes a code block in TCX_CommandStatusText.
3933	if(g_AutoArm) return "AUTO WAIT";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3934	if(g_ArmedMode==ARM_BUY) return "BUY WAIT";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3935	if(g_ArmedMode==ARM_SELL) return "SELL WAIT";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3936	int count=CountOurPositions();	Updates or initializes int count in TCX_CommandStatusText.
3937	if(count>1) return IntegerToString(count)+" POSITIONS OPEN";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3938	if(count==1) return "POSITION OPEN";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3939	int dashboardCount=CountDashboardPositions();	Updates or initializes int dashboardCount in TCX_CommandStatusText.
3940	if(dashboardCount>1) return IntegerToString(dashboardCount)+" DASHBOARD POSITIONS";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3941	if(dashboardCount==1) return "MANUAL POSITION OPEN";	Checks a condition and runs the guarded branch in TCX_CommandStatusText.
3942	return "IDLE";	Returns a value or exits the current function in TCX_CommandStatusText.
3943	}	Opens or closes a code block in TCX_CommandStatusText.
3944		Blank separator for readability.
3945	string TCX_ArmText()	Declares or starts function TCX_ArmText.
3946	{	Opens or closes a code block in TCX_ArmText.
3947	if(g_AutoArm) return "AUTO";	Checks a condition and runs the guarded branch in TCX_ArmText.
3948	if(g_ArmedMode==ARM_BUY) return "BUY";	Checks a condition and runs the guarded branch in TCX_ArmText.
3949	if(g_ArmedMode==ARM_SELL) return "SELL";	Checks a condition and runs the guarded branch in TCX_ArmText.
3950	return "OFF";	Returns a value or exits the current function in TCX_ArmText.
3951	}	Opens or closes a code block in TCX_ArmText.
3952		Blank separator for readability.
3953	string TCX_PositionJson()	Declares or starts function TCX_PositionJson.
3954	{	Opens or closes a code block in TCX_PositionJson.
3955	string positions="[";	Updates or initializes string positions in TCX_PositionJson.
3956	int count=0;	Updates or initializes int count in TCX_PositionJson.
3957	int eaCount=0;	Updates or initializes int eaCount in TCX_PositionJson.
3958	int manualCount=0;	Updates or initializes int manualCount in TCX_PositionJson.
3959	bool firstSet=false;	Updates or initializes bool firstSet in TCX_PositionJson.
3960	bool isBuy=true;	Updates or initializes bool isBuy in TCX_PositionJson.
3961	double entry=0.0;	Updates or initializes double entry in TCX_PositionJson.
3962	double sl=0.0;	Updates or initializes double sl in TCX_PositionJson.
3963	double tp=0.0;	Updates or initializes double tp in TCX_PositionJson.
3964	double profit=0.0;	Updates or initializes double profit in TCX_PositionJson.
3965	double vol=0.0;	Updates or initializes double vol in TCX_PositionJson.
3966	double rr=0.0;	Updates or initializes double rr in TCX_PositionJson.
3967	long firstMagic=0;	Updates or initializes long firstMagic in TCX_PositionJson.
3968	ulong firstTicket=0;	Updates or initializes ulong firstTicket in TCX_PositionJson.
3969		Blank separator for readability.
3970	for(int i=PositionsTotal()-1;i>=0;i--)	Loops over a fixed range or collection in TCX_PositionJson.
3971	{	Opens or closes a code block in TCX_PositionJson.
3972	ulong ticket=PositionGetTicket(i);	Updates or initializes ulong ticket in TCX_PositionJson.
3973	if(ticket==0)	Checks a condition and runs the guarded branch in TCX_PositionJson.
3974	continue;	Executes one statement in TCX_PositionJson.
3975		Blank separator for readability.
3976	string symbol=PositionGetString(POSITION_SYMBOL);	Updates or initializes string symbol in TCX_PositionJson.
3977	long magic=(long)PositionGetInteger(POSITION_MAGIC);	Updates or initializes long magic in TCX_PositionJson.
3978	if(symbol!=_Symbol !IsDashboardMagic(magic))	Checks a condition and runs the guarded branch in TCX_PositionJson.
3979	continue;	Executes one statement in TCX_PositionJson.
3980		Blank separator for readability.
3981	bool rowBuy=((ENUM_POSITION_TYPE)PositionGetInteger(POSITION_TYPE)==POSITION_TYPE_BUY);	Updates or initializes bool rowBuy in TCX_PositionJson.
3982	double rowEntry=PositionGetDouble(POSITION_PRICE_OPEN);	Updates or initializes double rowEntry in TCX_PositionJson.
3983	double rowSL=PositionGetDouble(POSITION_SL);	Updates or initializes double rowSL in TCX_PositionJson.
3984	double rowTP=PositionGetDouble(POSITION_TP);	Updates or initializes double rowTP in TCX_PositionJson.
3985	double rowProfit=PositionGetDouble(POSITION_PROFIT);	Updates or initializes double rowProfit in TCX_PositionJson.
3986	double rowVol=PositionGetDouble(POSITION_VOLUME);	Updates or initializes double rowVol in TCX_PositionJson.
3987	double rowRR=PositionRRFromValues(rowBuy,rowEntry,rowSL);	Updates or initializes double rowRR in TCX_PositionJson.
3988	string source=DashboardSourceText(magic);	Updates or initializes string source in TCX_PositionJson.
3989	bool managed=IsManagedMagic(magic);	Updates or initializes bool managed in TCX_PositionJson.
3990		Blank separator for readability.
3991	if(managed)	Checks a condition and runs the guarded branch in TCX_PositionJson.
3992	eaCount++;	Executes one statement in TCX_PositionJson.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
3993	else if(magic==0)	Checks an alternate condition when the previous branch did not run in TCX_PositionJson.
3994	manualCount++;	Executes one statement in TCX_PositionJson.
3995		Blank separator for readability.
3996	if(!firstSet)	Checks a condition and runs the guarded branch in TCX_PositionJson.
3997	{	Opens or closes a code block in TCX_PositionJson.
3998	firstSet=true;	Updates or initializes firstSet in TCX_PositionJson.
3999	isBuy=rowBuy;	Updates or initializes isBuy in TCX_PositionJson.
4000	entry=rowEntry;	Updates or initializes entry in TCX_PositionJson.
4001	sl=rowSL;	Updates or initializes sl in TCX_PositionJson.
4002	tp=rowTP;	Updates or initializes tp in TCX_PositionJson.
4003	profit=rowProfit;	Updates or initializes profit in TCX_PositionJson.
4004	vol=rowVol;	Updates or initializes vol in TCX_PositionJson.
4005	rr=rowRR;	Updates or initializes rr in TCX_PositionJson.
4006	firstMagic=magic;	Updates or initializes firstMagic in TCX_PositionJson.
4007	firstTicket=ticket;	Updates or initializes firstTicket in TCX_PositionJson.
4008	}	Opens or closes a code block in TCX_PositionJson.
4009		Blank separator for readability.
4010	if(count>0)	Checks a condition and runs the guarded branch in TCX_PositionJson.
4011	positions+=",";	Updates or initializes positions+ in TCX_PositionJson.
4012	positions+="{	Part of the EA logic in TCX_PositionJson.
4013	+"\"ticket\": \""+IntegerToString((long)ticket)+"\""	Part of the EA logic in TCX_PositionJson.
4014	+",\"type\": \""+(rowBuy ? "BUY" : "SELL")+"\""	Part of the EA logic in TCX_PositionJson.
4015	+",\"source\": \""+source+"\""	Part of the EA logic in TCX_PositionJson.
4016	+",\"magic\": \""+IntegerToString(magic)	Part of the EA logic in TCX_PositionJson.
4017	+",\"managed\": "+TCX_BoolText(managed)	Part of the EA logic in TCX_PositionJson.
4018	+",\"volume\": \""+DoubleToString(rowVol,VolumeDigits())	Part of the EA logic in TCX_PositionJson.
4019	+",\"entry\": \""+DoubleToString(rowEntry,_Digits)	Part of the EA logic in TCX_PositionJson.
4020	+",\"sl\": \""+DoubleToString(rowSL,_Digits)	Part of the EA logic in TCX_PositionJson.
4021	+",\"tp\": \""+DoubleToString(rowTP,_Digits)	Part of the EA logic in TCX_PositionJson.
4022	+",\"profit\": \""+DoubleToString(rowProfit,2)	Part of the EA logic in TCX_PositionJson.
4023	+",\"rr\": \""+DoubleToString(rowRR,2)	Part of the EA logic in TCX_PositionJson.
4024	+}";	Executes one statement in TCX_PositionJson.
4025	count++;	Executes one statement in TCX_PositionJson.
4026	}	Opens or closes a code block in TCX_PositionJson.
4027	positions+="]";	Updates or initializes positions+ in TCX_PositionJson.
4028		Blank separator for readability.
4029	if(count<=0 !firstSet)	Checks a condition and runs the guarded branch in TCX_PositionJson.
4030	return "{\"hasPosition\":false,\"count\":0,\"eaCount\":0,\"manualCount\":0,\"positions\":[]}";	Returns a value or exits the current function in TCX_PositionJson.
4031		Blank separator for readability.
4032	return "{\"hasPosition\":true"	Returns a value or exits the current function in TCX_PositionJson.
4033	+",\"count\": "+IntegerToString(count)	Part of the EA logic in TCX_PositionJson.
4034	+",\"eaCount\": "+IntegerToString(eaCount)	Part of the EA logic in TCX_PositionJson.
4035	+",\"manualCount\": "+IntegerToString(manualCount)	Part of the EA logic in TCX_PositionJson.
4036	+",\"ticket\": \""+IntegerToString((long)firstTicket)+"\""	Part of the EA logic in TCX_PositionJson.
4037	+",\"type\": \""+(isBuy ? "BUY" : "SELL")+"\""	Part of the EA logic in TCX_PositionJson.
4038	+",\"source\": \""+DashboardSourceText(firstMagic)+"\""	Part of the EA logic in TCX_PositionJson.
4039	+",\"magic\": "+IntegerToString(firstMagic)	Part of the EA logic in TCX_PositionJson.
4040	+",\"managed\": "+TCX_BoolText(IsManagedMagic(firstMagic))	Part of the EA logic in TCX_PositionJson.
4041	+",\"volume\": "+DoubleToString(vol,VolumeDigits())	Part of the EA logic in TCX_PositionJson.
4042	+",\"entry\": "+DoubleToString(entry,_Digits)	Part of the EA logic in TCX_PositionJson.
4043	+",\"sl\": "+DoubleToString(sl,_Digits)	Part of the EA logic in TCX_PositionJson.
4044	+",\"tp\": "+DoubleToString(tp,_Digits)	Part of the EA logic in TCX_PositionJson.
4045	+",\"profit\": "+DoubleToString(profit,2)	Part of the EA logic in TCX_PositionJson.
4046	+",\"rr\": "+DoubleToString(rr,2)	Part of the EA logic in TCX_PositionJson.
4047	+",\"positions\": "+positions	Part of the EA logic in TCX_PositionJson.
4048	+}";	Executes one statement in TCX_PositionJson.
4049	}	Opens or closes a code block in TCX_PositionJson.
4050		Blank separator for readability.
4051	string TCX_CandleRowJson(string label,ENUM_TIMEFRAMES tf,int shift)	Declares or starts function TCX_CandleRowJson.
4052	{	Opens or closes a code block in TCX_CandleRowJson.
4053	bool above=false;	Updates or initializes bool above in TCX_CandleRowJson.
4054	bool valid=PriceAboveOpen(tf,shift,above);	Updates or initializes bool valid in TCX_CandleRowJson.
4055	return "{\"label\": \""+TCX_JsonEscape(label)+"\", \"valid\": "+TCX_BoolText(valid)+"\", \"bull\": "+TCX_BoolText(valid && above)+"}";	Returns a value or exits the current function in TCX_CandleRowJson.
4056	}	Opens or closes a code block in TCX_CandleRowJson.
4057		Blank separator for readability.
4058	string TCX_SweepRowJson(string label,bool valid,bool high,bool low)	Declares or starts function TCX_SweepRowJson.
4059	{	Opens or closes a code block in TCX_SweepRowJson.
4060	return "{\"label\": \""+TCX_JsonEscape(label)+"\", \"valid\": "+TCX_BoolText(valid)+"\", \"high\": "+TCX_BoolText(valid && high)+"\", \"low\": "+TCX_BoolText(valid && low)+"}";	Returns a value or exits the current function in TCX_SweepRowJson.
4061	}	Opens or closes a code block in TCX_SweepRowJson.
4062		Blank separator for readability.
4063	string TCX_StructureJson()	Declares or starts function TCX_StructureJson.
4064	{	Opens or closes a code block in TCX_StructureJson.
4065	int shOdd=-1, shEven=-1;	Updates or initializes int shOdd in TCX_StructureJson.
4066	bool oddValid=LatestH4ShiftByParity(true,shOdd);	Updates or initializes bool oddValid in TCX_StructureJson.
4067	bool evenValid=LatestH4ShiftByParity(false,shEven);	Updates or initializes bool evenValid in TCX_StructureJson.
4068	bool hi=false, lo=false;	Updates or initializes bool hi in TCX_StructureJson.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4069		Blank separator for readability.
4070	string candles="[";	Updates or initializes string candles in TCX_StructureJson.
4071	candles+=TCX_CandleRowJson("D1",PERIOD_D1,0)+",";	Updates or initializes candles+ in TCX_StructureJson.
4072	candles+=TCX_CandleRowJson("H4 Odd",PERIOD_H4,oddValid ? shOdd : 0)+",";	Updates or initializes candles+ in TCX_StructureJson.
4073	candles+=TCX_CandleRowJson("H4 Even",PERIOD_H4,evenValid ? shEven : 0)+",";	Updates or initializes candles+ in TCX_StructureJson.
4074	candles+=TCX_CandleRowJson("H1",PERIOD_H1,0)+"]";	Updates or initializes candles+ in TCX_StructureJson.
4075		Blank separator for readability.
4076	string sweeps="[";	Updates or initializes string sweeps in TCX_StructureJson.
4077	bool valid=StandardSweep(PERIOD_H1,hi,lo);	Updates or initializes bool valid in TCX_StructureJson.
4078	sweeps+=TCX_SweepRowJson("H1",valid,hi,lo)+",";	Updates or initializes sweeps+ in TCX_StructureJson.
4079	valid=H4ParitySweep(true,hi,lo);	Updates or initializes valid in TCX_StructureJson.
4080	sweeps+=TCX_SweepRowJson("H4 Odd",valid,hi,lo)+",";	Updates or initializes sweeps+ in TCX_StructureJson.
4081	valid=H4ParitySweep(false,hi,lo);	Updates or initializes valid in TCX_StructureJson.
4082	sweeps+=TCX_SweepRowJson("H4 Even",valid,hi,lo)+",";	Updates or initializes sweeps+ in TCX_StructureJson.
4083	valid=StandardSweep(PERIOD_D1,hi,lo);	Updates or initializes valid in TCX_StructureJson.
4084	sweeps+=TCX_SweepRowJson("D1",valid,hi,lo)+"]";	Updates or initializes sweeps+ in TCX_StructureJson.
4085		Blank separator for readability.
4086	return "{\"candles\":\""+candles+"\",\"sweeps\":\""+sweeps+"}\";	Returns a value or exits the current function in TCX_StructureJson.
4087	}	Opens or closes a code block in TCX_StructureJson.
4088		Blank separator for readability.
4089	string TCX_DealReasonText(long reason)	Declares or starts function TCX_DealReasonText.
4090	{	Opens or closes a code block in TCX_DealReasonText.
4091	if(reason==DEAL_REASON_SL) return "SL";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4092	if(reason==DEAL_REASON_TP) return "TP";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4093	if(reason==DEAL_REASON_SO) return "Stop out";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4094	if(reason==DEAL_REASON_EXPERT) return "EA";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4095	if(reason==DEAL_REASON_CLIENT) return "Manual";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4096	if(reason==DEAL_REASON_MOBILE) return "Mobile";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4097	if(reason==DEAL_REASON_WEB) return "Web";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4098	if(reason==DEAL_REASON_ROLLOVER) return "Rollover";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4099	if(reason==DEAL_REASON_VMARGIN) return "Variation margin";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4100	if(reason==DEAL_REASON_SPLIT) return "Split";	Checks a condition and runs the guarded branch in TCX_DealReasonText.
4101	return "Close";	Returns a value or exits the current function in TCX_DealReasonText.
4102	}	Opens or closes a code block in TCX_DealReasonText.
4103		Blank separator for readability.
4104	string TCX_DealSideText(long entryType,long exitType)	Declares or starts function TCX_DealSideText.
4105	{	Opens or closes a code block in TCX_DealSideText.
4106	if(entryType==DEAL_TYPE_BUY)	Checks a condition and runs the guarded branch in TCX_DealSideText.
4107	return "BUY";	Returns a value or exits the current function in TCX_DealSideText.
4108	if(entryType==DEAL_TYPE_SELL)	Checks a condition and runs the guarded branch in TCX_DealSideText.
4109	return "SELL";	Returns a value or exits the current function in TCX_DealSideText.
4110	if(exitType==DEAL_TYPE_SELL)	Checks a condition and runs the guarded branch in TCX_DealSideText.
4111	return "BUY";	Returns a value or exits the current function in TCX_DealSideText.
4112	if(exitType==DEAL_TYPE_BUY)	Checks a condition and runs the guarded branch in TCX_DealSideText.
4113	return "SELL";	Returns a value or exits the current function in TCX_DealSideText.
4114	return "--";	Returns a value or exits the current function in TCX_DealSideText.
4115	}	Opens or closes a code block in TCX_DealSideText.
4116		Blank separator for readability.
4117	string TCX_JsonPrice(double price)	Declares or starts function TCX_JsonPrice.
4118	{	Opens or closes a code block in TCX_JsonPrice.
4119	if(price<=0.0)	Checks a condition and runs the guarded branch in TCX_JsonPrice.
4120	return "null";	Returns a value or exits the current function in TCX_JsonPrice.
4121	return DoubleToString(price,_Digits);	Returns a value or exits the current function in TCX_JsonPrice.
4122	}	Opens or closes a code block in TCX_JsonPrice.
4123		Blank separator for readability.
4124	ulong TCX_FindEntryDeal(long positionId,int exitIndex)	Part of the EA logic in TCX_JsonPrice.
4125	{	Opens or closes a code block in TCX_JsonPrice.
4126	for(int i=exitIndex-1;i>=0;i--)	Loops over a fixed range or collection in TCX_JsonPrice.
4127	{	Opens or closes a code block in TCX_JsonPrice.
4128	ulong deal=HistoryDealGetTicket(i);	Updates or initializes ulong deal in TCX_JsonPrice.
4129	if(deal==0)	Checks a condition and runs the guarded branch in TCX_JsonPrice.
4130	continue;	Executes one statement in TCX_JsonPrice.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4131	if(HistoryDealGetString(deal,DEAL_SYMBOL)!=_Symbol)	Checks a condition and runs the guarded branch in TCX_JsonPrice.
4132	continue;	Executes one statement in TCX_JsonPrice.
4133	if(!IsDashboardMagic((long)HistoryDealGetInteger(deal,DEAL_MAGIC)))	Checks a condition and runs the guarded branch in TCX_JsonPrice.
4134	continue;	Executes one statement in TCX_JsonPrice.
4135	if((long)HistoryDealGetInteger(deal,DEAL_POSITION_ID)!=positionId)	Checks a condition and runs the guarded branch in TCX_JsonPrice.
4136	continue;	Executes one statement in TCX_JsonPrice.
4137		Blank separator for readability.
4138	long entry=HistoryDealGetInteger(deal,DEAL_ENTRY);	Updates or initializes long entry in TCX_JsonPrice.
4139	if(entry==DEAL_ENTRY_IN entry==DEAL_ENTRY_INOUT)	Checks a condition and runs the guarded branch in TCX_JsonPrice.
4140	return deal;	Returns a value or exits the current function in TCX_JsonPrice.
4141	}	Opens or closes a code block in TCX_JsonPrice.
4142	return 0;	Returns a value or exits the current function in TCX_JsonPrice.
4143	}	Opens or closes a code block in TCX_JsonPrice.
4144		Blank separator for readability.
4145	string TCX_TradeHistoryJson(int maxRows)	Declares or starts function TCX_TradeHistoryJson.
4146	{	Opens or closes a code block in TCX_TradeHistoryJson.
4147	if(maxRows<=0)	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4148	maxRows=40;	Updates or initializes maxRows in TCX_TradeHistoryJson.
4149	if(maxRows>100)	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4150	maxRows=100;	Updates or initializes maxRows in TCX_TradeHistoryJson.
4151	if(!HistorySelect((datetime)0,TimeCurrent()))	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4152	return "[]";	Returns a value or exits the current function in TCX_TradeHistoryJson.
4153		Blank separator for readability.
4154	string json="[";	Updates or initializes string json in TCX_TradeHistoryJson.
4155	int rows=0;	Updates or initializes int rows in TCX_TradeHistoryJson.
4156	int total=HistoryDealsTotal();	Updates or initializes int total in TCX_TradeHistoryJson.
4157	for(int i=total-1;i>=0 && rows<maxRows;i--)	Loops over a fixed range or collection in TCX_TradeHistoryJson.
4158	{	Opens or closes a code block in TCX_TradeHistoryJson.
4159	ulong deal=HistoryDealGetTicket(i);	Updates or initializes ulong deal in TCX_TradeHistoryJson.
4160	if(deal==0)	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4161	continue;	Executes one statement in TCX_TradeHistoryJson.
4162	if(HistoryDealGetString(deal,DEAL_SYMBOL)!=_Symbol)	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4163	continue;	Executes one statement in TCX_TradeHistoryJson.
4164	long dealMagic=(long)HistoryDealGetInteger(deal,DEAL_MAGIC);	Updates or initializes long dealMagic in TCX_TradeHistoryJson.
4165	if(!IsDashboardMagic(dealMagic))	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4166	continue;	Executes one statement in TCX_TradeHistoryJson.
4167		Blank separator for readability.
4168	long dealEntry=HistoryDealGetInteger(deal,DEAL_ENTRY);	Updates or initializes long dealEntry in TCX_TradeHistoryJson.
4169	if(dealEntry!=DEAL_ENTRY_OUT && dealEntry!=DEAL_ENTRY_INOUT && dealEntry!=DEAL_ENTRY_OUT_BY)	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4170	continue;	Executes one statement in TCX_TradeHistoryJson.
4171		Blank separator for readability.
4172	long positionId=(long)HistoryDealGetInteger(deal,DEAL_POSITION_ID);	Updates or initializes long positionId in TCX_TradeHistoryJson.
4173	ulong entryDeal=TCX_FindEntryDeal(positionId,i);	Updates or initializes ulong entryDeal in TCX_TradeHistoryJson.
4174	long sourceMagic=(entryDeal>0 ?	Updates or initializes long sourceMagic in TCX_TradeHistoryJson.
4175	(long)HistoryDealGetInteger(entryDeal,DEAL_MAGIC) : dealMagic);	Updates or initializes long entryType in TCX_TradeHistoryJson.
4176	long exitType=HistoryDealGetInteger(deal,DEAL_TYPE);	Updates or initializes long exitType in TCX_TradeHistoryJson.
4177	datetime closeTime=(datetime)HistoryDealGetInteger(deal,DEAL_TIME);	Updates or initializes datetime closeTime in TCX_TradeHistoryJson.
4178	datetime openTime=(entryDeal>0 ?	Updates or initializes datetime openTime in TCX_TradeHistoryJson.
4179	(datetime)HistoryDealGetInteger(entryDeal,DEAL_TIME) : 0);	Updates or initializes double profit in TCX_TradeHistoryJson.
4180	double profit=HistoryDealGetDouble(deal,DEAL_PROFIT);	Updates or initializes double swap in TCX_TradeHistoryJson.
4181	double swap=HistoryDealGetDouble(deal,DEAL_SWAP);	Updates or initializes double commission in TCX_TradeHistoryJson.
4182	double commission=HistoryDealGetDouble(deal,DEAL_COMMISSION);	Updates or initializes double net in TCX_TradeHistoryJson.
4183	double net=profit+swap+commission;	Updates or initializes double sl in TCX_TradeHistoryJson.
4184	double sl=HistoryDealGetDouble(deal,DEAL_SL);	Updates or initializes double tp in TCX_TradeHistoryJson.
4185	double tp=HistoryDealGetDouble(deal,DEAL_TP);	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4186	if(sl<=0.0 && entryDeal>0)	Updates or initializes sl in TCX_TradeHistoryJson.
4187	sl=HistoryDealGetDouble(entryDeal,DEAL_SL);	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4188	if(tp<=0.0 && entryDeal>0)	Updates or initializes tp in TCX_TradeHistoryJson.
4189	tp=HistoryDealGetDouble(entryDeal,DEAL_TP);	Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4190	if(rows>0)	Checks a condition and runs the guarded branch in TCX_TradeHistoryJson.
4191	json+=",";	Updates or initializes json+ in TCX_TradeHistoryJson.
4192	json+="{ "	Part of the EA logic in TCX_TradeHistoryJson.
4193	+ "\"id\": \""+IntegerToString((long)deal)+"\""	Part of the EA logic in TCX_TradeHistoryJson.
4194	+ ", \"positionId\": \""+IntegerToString(positionId)+"\""	Part of the EA logic in TCX_TradeHistoryJson.
4195	+ ", \"source\": \""+DashboardSourceText(sourceMagic)+"\""	Part of the EA logic in TCX_TradeHistoryJson.
4196	+ ", \"magic\": \""+IntegerToString(sourceMagic)+"\""	Part of the EA logic in TCX_TradeHistoryJson.
4197	+ ", \"type\": \""+TCX_DealSideText(entryType,exitType)+"\""	Part of the EA logic in TCX_TradeHistoryJson.
4198	+ ", \"volume\": \""+DoubleToString(HistoryDealGetDouble(deal,DEAL_VOLUME),VolumeDigits())	Part of the EA logic in TCX_TradeHistoryJson.
4199	+ ", \"entryPrice\": \""+(entryDeal>0 ?	Builds or parses JSON used by the web control bridge.
	TCX_JsonPrice(HistoryDealGetDouble(entryDeal,DEAL_PRICE)) : "null")	
4200	+ ", \"exitPrice\": \""+TCX_JsonPrice(HistoryDealGetDouble(deal,DEAL_PRICE))	Builds or parses JSON used by the web control bridge.
4201	+ ", \"sl\": \""+TCX_JsonPrice(sl)	Builds or parses JSON used by the web control bridge.
4202	+ ", \"tp\": \""+TCX_JsonPrice(tp)	Builds or parses JSON used by the web control bridge.
4203	+ ", \"profit\": \""+DoubleToString(net,2)	Part of the EA logic in TCX_TradeHistoryJson.
4204	+ ", \"grossProfit\": \""+DoubleToString(profit,2)	Part of the EA logic in TCX_TradeHistoryJson.
4205	+ ", \"swap\": \""+DoubleToString(swap,2)	Part of the EA logic in TCX_TradeHistoryJson.
4206	+ ", \"commission\": \""+DoubleToString(commission,2)	Part of the EA logic in TCX_TradeHistoryJson.
4207	+ ", \"openTime\": \""+(openTime>0 ?	Builds or parses JSON used by the web control bridge.
	TCX_JsonEscape(TimeToString(openTime,TIME_DATE TIME_SECONDS)) : "")+"\""	
4208	+ ", \"closeTime\": \""+TCX_JsonEscape(TimeToString(closeTime,TIME_DATE TIME_SECONDS))+ "\""	Builds or parses JSON used by the web control bridge.
4209	+ ", \"openTimeSec\": \""+IntegerToString((long)openTime)	Part of the EA logic in TCX_TradeHistoryJson.
4210	+ ", \"closeTimeSec\": \""+IntegerToString((long)closeTime)	Part of the EA logic in TCX_TradeHistoryJson.
4211	+ ", \"reason\": \""+TCX_JsonEscape(TCX_DealReasonText(HistoryDealGetInteger(deal,DEAL_REASON)))+ "\""	Builds or parses JSON used by the web control bridge.
4212	+ ", \"comment\": \""+TCX_JsonEscape(HistoryDealGetString(deal,DEAL_COMMENT))+"\""	Builds or parses JSON used by the web control bridge.
4213	+ "}"	Executes one statement in TCX_TradeHistoryJson.
4214	rows++;	Executes one statement in TCX_TradeHistoryJson.
4215	}	Opens or closes a code block in TCX_TradeHistoryJson.
4216	json+="}";	Updates or initializes json+ in TCX_TradeHistoryJson.
4217	return json;	Returns a value or exits the current function in TCX_TradeHistoryJson.
4218	}	Opens or closes a code block in TCX_TradeHistoryJson.
4219		Blank separator for readability.
4220	string TCX_BuildStateJson()	Declares or starts function TCX_BuildStateJson.
4221	{	Opens or closes a code block in TCX_BuildStateJson.
4222	datetime now=TimeCurrent();	Updates or initializes datetime now in TCX_BuildStateJson.
4223	int totalTrades=0,wins=0,losses=0;	Updates or initializes int totalTrades in TCX_BuildStateJson.
4224	double totalNet=0.0,grossProfit=0.0,grossLoss=0.0;	Updates or initializes double totalNet in TCX_BuildStateJson.
4225	CalcDealStats((datetime)0,totalTrades,wins,losses,totalNet,grossProfit,grossLoss);	Calls helper function CalcDealStats in TCX_BuildStateJson.
4226		Blank separator for readability.
4227	int dayTrades=0,dayWins=0,dayLosses=0;	Updates or initializes int dayTrades in TCX_BuildStateJson.
4228	double dayNet=0.0,dayGrossProfit=0.0,dayGrossLoss=0.0;	Updates or initializes double dayNet in TCX_BuildStateJson.
4229	CalcDealStats(DayStart(now),dayTrades,dayWins,dayLosses,dayNet,dayGrossProfit,dayGrossLoss);	Calls helper function CalcDealStats in TCX_BuildStateJson.
4230		Blank separator for readability.
4231	double balance=AccountInfoDouble(ACCOUNT_BALANCE);	Updates or initializes double balance in TCX_BuildStateJson.
4232	double equity=AccountInfoDouble(ACCOUNT_EQUITY);	Updates or initializes double equity in TCX_BuildStateJson.
4233	double margin=AccountInfoDouble(ACCOUNT_MARGIN);	Updates or initializes double margin in TCX_BuildStateJson.
4234	double freeMargin=AccountInfoDouble(ACCOUNT_MARGIN_FREE);	Updates or initializes double freeMargin in TCX_BuildStateJson.
4235	double riskMoney=NormalizeRiskMoney(g_RiskMoney);	Updates or initializes double riskMoney in TCX_BuildStateJson.
4236	double dayR=(riskMoney>0.0 ? dayNet/riskMoney : 0.0);	Updates or initializes double dayR in TCX_BuildStateJson.
4237	double remainingR=MathMax(0.0,InpDailyLimitR-MathMax(0.0,-dayR));	Updates or initializes double remainingR in TCX_BuildStateJson.
4238	double openPL=OpenProfit();	Updates or initializes double openPL in TCX_BuildStateJson.
4239	int score=QualityScore();	Updates or initializes int score in TCX_BuildStateJson.
4240	int sp=CurrentSpreadPoints();	Updates or initializes int sp in TCX_BuildStateJson.
4241	double winRate=(totalTrades>0 ? (double)wins*100.0/(double)totalTrades : 0.0);	Updates or initializes double winRate in TCX_BuildStateJson.
4242	double profitFactor=(grossLoss>0.0 ? grossProfit/grossLoss : (grossProfit>0.0 ? 99.0 : 0.0));	Updates or initializes double profitFactor in TCX_BuildStateJson.
4243		Blank separator for readability.
4244	return "{"	Returns a value or exits the current function in TCX_BuildStateJson.
4245	+ "\"source\": \"mt5-ea\""	Part of the EA logic in TCX_BuildStateJson.
4246	+ ", \"serverTime\": \""+TCX_JsonEscape(TimeToString(now,TIME_DATE TIME_SECONDS))+"\""	Builds or parses JSON used by the web control bridge.
4247	+ ", \"accountLogin\": \""+IntegerToString((long)AccountInfoInteger(ACCOUNT_LOGIN))+"\""	Part of the EA logic in TCX_BuildStateJson.
4248	+ ", \"accountName\": \""+TCX_JsonEscape(AccountInfoString(ACCOUNT_NAME))+"\""	Builds or parses JSON used by the web control bridge.
4249	+ ", \"accountCompany\": \""+TCX_JsonEscape(AccountInfoString(ACCOUNT_COMPANY))+"\""	Builds or parses JSON used by the web control bridge.
4250	+ ", \"accountServer\": \""+TCX_JsonEscape(AccountInfoString(ACCOUNT_SERVER))+"\""	Builds or parses JSON used by the web control bridge.
4251	+ ", \"platform\": \"MetaTrader 5\""	Part of the EA logic in TCX_BuildStateJson.
4252	+ ", \"symbol\": \""+TCX_JsonEscape(_Symbol)+"\""	Builds or parses JSON used by the web control bridge.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4253	<code>+",\"period\\\":\\\""+TCX_JsonEscape(EnumToString(_Period))+\"\\\""</code>	Builds or parses JSON used by the web control bridge.
4254	<code>+",\"eaOnline\\\":true"</code>	Part of the EA logic in TCX_BuildStateJson.
4255	<code>+",\"webConnected\\\":"+TCX_BooleanText(g_WebConnected)</code>	Part of the EA logic in TCX_BuildStateJson.
4256	<code>+",\"webError\\\":\\\""+TCX_JsonEscape(g_WebLastError)+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4257	<code>+",\"status\\\":\\\""+TCX_CommandStatusText()+"\\\""</code>	Part of the EA logic in TCX_BuildStateJson.
4258	<code>+",\"arm\\\":\\\""+TCX_ArmText()+"\\\""</code>	Part of the EA logic in TCX_BuildStateJson.
4259	<code>+",\"autoArm\\\":"+TCX_BooleanText(g_AutoArm)</code>	Part of the EA logic in TCX_BuildStateJson.
4260	<code>+",\"advanced\\\":"+TCX_BooleanText(g_AdvancedMode)</code>	Part of the EA logic in TCX_BuildStateJson.
4261	<code>+",\"partialson\\\":"+TCX_BooleanText(g_PC_On)</code>	Part of the EA logic in TCX_BuildStateJson.
4262	<code>+",\"secondEntryOn\\\":"+TCX_BooleanText(g_SecondEntryOn)</code>	Part of the EA logic in TCX_BuildStateJson.
4263	<code>+",\"secondEntryDone\\\":"+TCX_BooleanText(g_SecondEntryDone)</code>	Part of the EA logic in TCX_BuildStateJson.
4264	<code>+",\"secondEntryBlocked\\\":"+TCX_BooleanText(g_SecondEntryBlocked)</code>	Part of the EA logic in TCX_BuildStateJson.
4265	<code>+",\"secondEntryStatus\\\":\\\""+TCX_JsonEscape(SecondEntryStatusText()+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4266	<code>+",\"secondEntryTriggerR\\\":"+DoubleToString(MathMax(0.1, InpSecondEntryTriggerLossR),1)</code>	Part of the EA logic in TCX_BuildStateJson.
4267	<code>+",\"secondEntryCancelAtR\\\":"+DoubleToString(MathMax(0.1, InpSecondEntryCancelAtR),1)</code>	Part of the EA logic in TCX_BuildStateJson.
4268	<code>+",\"firstTrailOn\\\":"+TCX_BooleanText(g_FirstTrailOn)</code>	Part of the EA logic in TCX_BuildStateJson.
4269	<code>+",\"firstTrailActive\\\":"+TCX_BooleanText(g_FirstTrailActive)</code>	Part of the EA logic in TCX_BuildStateJson.
4270	<code>+",\"firstTrailStatus\\\":\\\""+TCX_JsonEscape(FirstTrailStatusText()+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4271	<code>+",\"firstTrailTimeframe\\\":\\\"M5\\\""</code>	Part of the EA logic in TCX_BuildStateJson.
4272	<code>+",\"qualityScore\\\":"+IntegerToString(score)</code>	Part of the EA logic in TCX_BuildStateJson.
4273	<code>+",\"qualityText\\\":\\\""+QualityText(score)+"\\\""</code>	Part of the EA logic in TCX_BuildStateJson.
4274	<code>+",\"message\\\":\\\""+TCX_JsonEscape(g_LastMessage)+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4275	<code>+",\"lot\\\":"+DoubleToString(g_Lot, VolumeDigits())</code>	Part of the EA logic in TCX_BuildStateJson.
4276	<code>+",\"risk\\\":"+DoubleToString(g_RiskMoney, 2)</code>	Part of the EA logic in TCX_BuildStateJson.
4277	<code>+",\"riskPercent\\\":"+DoubleToString(RiskPercentOfBalance(g_RiskMoney), 2)</code>	Part of the EA logic in TCX_BuildStateJson.
4278	<code>+",\"riskMode\\\":\\\"money\\\""</code>	Part of the EA logic in TCX_BuildStateJson.
4279	<code>+",\"rr\\\":"+DoubleToString(EffectiveRR(),1)</code>	Part of the EA logic in TCX_BuildStateJson.
4280	<code>+",\"pc1\\\":"+DoubleToString(g_PC1_Pct,1)</code>	Part of the EA logic in TCX_BuildStateJson.
4281	<code>+",\"pc2\\\":"+DoubleToString(g_PC2_Pct,1)</code>	Part of the EA logic in TCX_BuildStateJson.
4282	<code>+",\"pc3\\\":"+DoubleToString(g_PC3_Pct,1)</code>	Part of the EA logic in TCX_BuildStateJson.
4283	<code>+",\"spread\\\":"+IntegerToString(sp)</code>	Part of the EA logic in TCX_BuildStateJson.
4284	<code>+",\"maxSpread\\\":"+IntegerToString(InpMaxSpreadPoints)</code>	Part of the EA logic in TCX_BuildStateJson.
4285	<code>+",\"spreadProtectionPoints\\\":"+IntegerToString(SpreadProtectionPoints())</code>	Part of the EA logic in TCX_BuildStateJson.
4286	<code>+",\"spreadBlock\\\":"+TCX_BooleanText(SpreadBlocksTrades() && sp>InpMaxSpreadPoints)</code>	Part of the EA logic in TCX_BuildStateJson.
4287	<code>+",\"balance\\\":"+DoubleToString(balance,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4288	<code>+",\"equity\\\":"+DoubleToString(equity,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4289	<code>+",\"margin\\\":"+DoubleToString(margin,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4290	<code>+",\"freeMargin\\\":"+DoubleToString(freeMargin,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4291	<code>+",\"currency\\\":\\\""+TCX_JsonEscape(AccountInfoString(ACCOUNT_CURRENCY))+\"\\\""</code>	Builds or parses JSON used by the web control bridge.
4292	<code>+",\"openPL\\\":"+DoubleToString(openPL,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4293	<code>+",\"totalNet\\\":"+DoubleToString(totalNet,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4294	<code>+",\"totalTrades\\\":"+IntegerToString(totalTrades)</code>	Part of the EA logic in TCX_BuildStateJson.
4295	<code>+",\"wins\\\":"+IntegerToString(wins)</code>	Part of the EA logic in TCX_BuildStateJson.
4296	<code>+",\"losses\\\":"+IntegerToString(losses)</code>	Part of the EA logic in TCX_BuildStateJson.
4297	<code>+",\"winRate\\\":"+DoubleToString(winRate,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4298	<code>+",\"profitFactor\\\":"+DoubleToString(profitFactor,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4299	<code>+",\"dayTrades\\\":"+IntegerToString(dayTrades)</code>	Part of the EA logic in TCX_BuildStateJson.
4300	<code>+",\"dayLosses\\\":"+IntegerToString(dayLosses)</code>	Part of the EA logic in TCX_BuildStateJson.
4301	<code>+",\"dayLossLimit\\\":"+IntegerToString(InpDailyLossLimitCount)</code>	Part of the EA logic in TCX_BuildStateJson.
4302	<code>+",\"dayNet\\\":"+DoubleToString(dayNet,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4303	<code>+",\"dayR\\\":"+DoubleToString(dayR,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4304	<code>+",\"remainingR\\\":"+DoubleToString(remainingR,2)</code>	Part of the EA logic in TCX_BuildStateJson.
4305	<code>+",\"session\\\":\\\""+TCX_JsonEscape(SessionName()+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4306	<code>+",\"sessionLeft\\\":\\\""+TCX_JsonEscape(SessionTimeLeft()+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4307	<code>+",\"candleTimer\\\":\\\""+TCX_JsonEscape(CandleCloseTimerText()+"\\\""</code>	Builds or parses JSON used by the web control bridge.
4308	<code>+",\"position\\\":"+TCX_PositionJson()</code>	Part of the EA logic in TCX_BuildStateJson.
4309	<code>+",\"structure\\\":"+TCX_StructureJson()</code>	Part of the EA logic in TCX_BuildStateJson.
4310	<code>+",\"tradeHistory\\\":"+TCX_TradeHistoryJson(50)</code>	Part of the EA logic in TCX_BuildStateJson.
4311	<code>+"}";</code>	Executes one statement in TCX_BuildStateJson.
4312	<code>}</code>	Opens or closes a code block in TCX_BuildStateJson.
4313		Blank separator for readability.
4314	<code>bool TCX_HttpPostFunction(string functionName,string body,string &response)</code>	Declares or starts function TCX_HttpPostFunction.
4315	<code>{</code>	Opens or closes a code block in TCX_HttpPostFunction.
4316	<code> response="";</code>	Updates or initializes response in TCX_HttpPostFunction.
4317	<code> if(!InpWebControlEnabled) return false;</code>	Checks a condition and runs the guarded branch in TCX_HttpPostFunction.
4318	<code> uint nowTick=GetTickCount();</code>	Updates or initializes uint nowTick in TCX_HttpPostFunction.
4319	<code> if(g_WebBackoffUntilTick>0 && nowTick<g_WebBackoffUntilTick)</code>	Checks a condition and runs the guarded branch in TCX_HttpPostFunction.
4320	<code> return false;</code>	Returns a value or exits the current function in TCX_HttpPostFunction.
4321		Blank separator for readability.
4322	<code> string base=TCX_TrimRightSlash(InpSupabaseFunctionsUrl);</code>	Updates or initializes string base in TCX_HttpPostFunction.
4323	<code> string url=base+"/"+functionName;</code>	Updates or initializes string url in TCX_HttpPostFunction.
4324	<code> string headers="Content-Type: application/json\r\n";</code>	Updates or initializes string headers in TCX_HttpPostFunction.
4325	<code> char data[];</code>	Executes one statement in TCX_HttpPostFunction.
4326	<code> char result[];</code>	Executes one statement in TCX_HttpPostFunction.
4327	<code> string resultHeaders="";</code>	Updates or initializes string resultHeaders in TCX_HttpPostFunction.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4328	StringToCharArray(body, data, 0, StringLen(body), CP_UTF8);	Calls helper function StringToCharArray in TCX_HttpPostFunction.
4329	ResetLastError();	Calls helper function ResetLastError in TCX_HttpPostFunction.
4330	int code=WebRequest("POST",url,headers,InpWebTimeoutMs,data,result,resultHeaders);	Sends an HTTP request from MT5 to a Supabase Edge Function.
4331	if(code<200 code>=300)	Checks a condition and runs the guarded branch in TCX_HttpPostFunction.
4332	{	Opens or closes a code block in TCX_HttpPostFunction.
4333	g_WebLastError=functionName+" HTTP "+IntegerToString(code)+" MT5err	Updates or initializes g_WebLastError in TCX_HttpPostFunction.
4334	g_WebConnected=false;	Updates or initializes g_WebConnected in TCX_HttpPostFunction.
4335	g_WebBackoffUntilTick=GetTickCount()+500;	Updates or initializes g_WebBackoffUntilTick in TCX_HttpPostFunction.
4336	return false;	Returns a value or exits the current function in TCX_HttpPostFunction.
4337	}	Opens or closes a code block in TCX_HttpPostFunction.
4338	response=CharArrayToString(result,0,-1,CP_UTF8);	Updates or initializes response in TCX_HttpPostFunction.
4339	g_WebConnected=true;	Updates or initializes g_WebConnected in TCX_HttpPostFunction.
4340	g_WebLastError="";	Updates or initializes g_WebLastError in TCX_HttpPostFunction.
4341	g_WebBackoffUntilTick=0;	Updates or initializes g_WebBackoffUntilTick in TCX_HttpPostFunction.
4342	return true;	Returns a value or exits the current function in TCX_HttpPostFunction.
4343	}	Opens or closes a code block in TCX_HttpPostFunction.
4344		Blank separator for readability.
4345	string TCX_JsonString(string src,string key,string def="")	Declares or starts function TCX_JsonString.
4346	{	Opens or closes a code block in TCX_JsonString.
4347	string needle="\""+key+"\"";	Updates or initializes string needle in TCX_JsonString.
4348	int p=StringFind(src,needle);	Updates or initializes int p in TCX_JsonString.
4349	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonString.
4350	p=StringFind(src,":",p);	Updates or initializes p in TCX_JsonString.
4351	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonString.
4352	p=StringFind(src,"\"",p+1);	Updates or initializes p in TCX_JsonString.
4353	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonString.
4354	int start=p+1;	Updates or initializes int start in TCX_JsonString.
4355	int end=start;	Updates or initializes int end in TCX_JsonString.
4356	while(end<StringLen(src))	Repeats while the condition remains true in TCX_JsonString.
4357	{	Opens or closes a code block in TCX_JsonString.
4358	string ch=StringSubstr(src,end,1);	Updates or initializes string ch in TCX_JsonString.
4359	if(ch=="\" " && (end==start StringSubstr(src,end-1,1)!="\\"))	Checks a condition and runs the guarded branch in TCX_JsonString.
4360	break;	Executes one statement in TCX_JsonString.
4361	end++;	Executes one statement in TCX_JsonString.
4362	}	Opens or closes a code block in TCX_JsonString.
4363	if(end<start) return def;	Checks a condition and runs the guarded branch in TCX_JsonString.
4364	string v=StringSubstr(src,start,end-start);	Updates or initializes string v in TCX_JsonString.
4365	StringReplace(v,"\\\"","\\");	Calls helper function StringReplace in TCX_JsonString.
4366	StringReplace(v,"\\\\","\\");	Calls helper function StringReplace in TCX_JsonString.
4367	return v;	Returns a value or exits the current function in TCX_JsonString.
4368	}	Opens or closes a code block in TCX_JsonString.
4369		Blank separator for readability.
4370	double TCX_JsonNumber(string src,string key,double def=0.0)	Declares or starts function TCX_JsonNumber.
4371	{	Opens or closes a code block in TCX_JsonNumber.
4372	string needle="\""+key+"\"";	Updates or initializes string needle in TCX_JsonNumber.
4373	int p=StringFind(src,needle);	Updates or initializes int p in TCX_JsonNumber.
4374	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonNumber.
4375	p=StringFind(src,":",p);	Updates or initializes p in TCX_JsonNumber.
4376	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonNumber.
4377	int start=p+1;	Updates or initializes int start in TCX_JsonNumber.
4378	while(start<StringLen(src))	Repeats while the condition remains true in TCX_JsonNumber.
4379	{	Opens or closes a code block in TCX_JsonNumber.
4380	string ch=StringSubstr(src,start,1);	Updates or initializes string ch in TCX_JsonNumber.
4381	if(ch!=" " && ch!="\t" && ch!="\r" && ch!="\n") break;	Checks a condition and runs the guarded branch in TCX_JsonNumber.
4382	start++;	Executes one statement in TCX_JsonNumber.
4383	}	Opens or closes a code block in TCX_JsonNumber.
4384	int end=start;	Updates or initializes int end in TCX_JsonNumber.
4385	while(end<StringLen(src))	Repeats while the condition remains true in TCX_JsonNumber.
4386	{	Opens or closes a code block in TCX_JsonNumber.
4387	string ch=StringSubstr(src,end,1);	Updates or initializes string ch in TCX_JsonNumber.
4388	if(StringFind("0123456789+-. ",ch)<0) break;	Checks a condition and runs the guarded branch in TCX_JsonNumber.
4389	end++;	Executes one statement in TCX_JsonNumber.
4390	}	Opens or closes a code block in TCX_JsonNumber.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4391	if(end<=start) return def;	Checks a condition and runs the guarded branch in TCX_JsonNumber.
4392	return StringToDouble(StringSubstr(src,start,end-start));	Returns a value or exits the current function in TCX_JsonNumber.
4393	}	Opens or closes a code block in TCX_JsonNumber.
4394		Blank separator for readability.
4395	bool TCX_JsonHasKey(string src,string key)	Declares or starts function TCX_JsonHasKey.
4396	{	Opens or closes a code block in TCX_JsonHasKey.
4397	string needle="\""+key+"\"";	Updates or initializes string needle in TCX_JsonHasKey.
4398	return (StringFind(src,needle)>=0);	Returns a value or exits the current function in TCX_JsonHasKey.
4399	}	Opens or closes a code block in TCX_JsonHasKey.
4400		Blank separator for readability.
4401	bool TCX_JsonBool(string src,string key,bool def=false)	Declares or starts function TCX_JsonBool.
4402	{	Opens or closes a code block in TCX_JsonBool.
4403	string needle="\""+key+"\"";	Updates or initializes string needle in TCX_JsonBool.
4404	int p=StringFind(src,needle);	Updates or initializes int p in TCX_JsonBool.
4405	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonBool.
4406	p=StringFind(src,":",p);	Updates or initializes p in TCX_JsonBool.
4407	if(p<0) return def;	Checks a condition and runs the guarded branch in TCX_JsonBool.
4408		Blank separator for readability.
4409	int start=p+1;	Updates or initializes int start in TCX_JsonBool.
4410	while(start<StringLen(src))	Repeats while the condition remains true in TCX_JsonBool.
4411	{	Opens or closes a code block in TCX_JsonBool.
4412	string ch=StringSubstr(src,start,1);	Updates or initializes string ch in TCX_JsonBool.
4413	if(ch!=" " && ch!="\t" && ch!="\r" && ch!="\n") break;	Checks a condition and runs the guarded branch in TCX_JsonBool.
4414	start++;	Executes one statement in TCX_JsonBool.
4415	}	Opens or closes a code block in TCX_JsonBool.
4416		Blank separator for readability.
4417	string v=StringSubstr(src,start,5);	Updates or initializes string v in TCX_JsonBool.
4418	StringToLower(v);	Calls helper function StringToLower in TCX_JsonBool.
4419	if(StringSubstr(v,0,4)=="true")	Checks a condition and runs the guarded branch in TCX_JsonBool.
4420	return true;	Returns a value or exits the current function in TCX_JsonBool.
4421	if(StringSubstr(v,0,5)=="false")	Checks a condition and runs the guarded branch in TCX_JsonBool.
4422	return false;	Returns a value or exits the current function in TCX_JsonBool.
4423	return def;	Returns a value or exits the current function in TCX_JsonBool.
4424	}	Opens or closes a code block in TCX_JsonBool.
4425		Blank separator for readability.
4426	void TCX_AckCommand(string id,string status,string message)	Declares or starts function TCX_AckCommand.
4427	{	Opens or closes a code block in TCX_AckCommand.
4428	if(id=="") return;	Checks a condition and runs the guarded branch in TCX_AckCommand.
4429	string res="";	Updates or initializes string res in TCX_AckCommand.
4430	string body="{\"ea_id\": \"\"+TCX_JsonEscape(InpSupabaseEaId)+\"\", \"ea_token\": \"\"+TCBuilds or parses JSON used by the web control bridge. X_JsonEscape(InpSupabaseEaToken)+\"\", \"command_id\": \"\"+TCX_JsonEscape(id)+\"\", \"status\": \"\"+TCX_JsonEscape(status)+\"\", \"message\": \"\"+TCX_JsonEscape(message)+\"\"}";	
4431	TCX_HttpPostFunction("ea-ack-command",body,res);	Calls helper function TCX_HttpPostFunction in TCX_AckCommand.
4432	}	Opens or closes a code block in TCX_AckCommand.
4433		Blank separator for readability.
4434	bool TCX_ApplyWebCommand(string id,string action,string raw)	Declares or starts function TCX_ApplyWebCommand.
4435	{	Opens or closes a code block in TCX_ApplyWebCommand.
4436	if(id!=" " && id==g_WebLastCommandId)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4437	return true;	Returns a value or exits the current function in TCX_ApplyWebCommand.
4438		Blank separator for readability.
4439	StringToUpper(action);	Calls helper function StringToUpper in TCX_ApplyWebCommand.
4440	string msg="";	Updates or initializes string msg in TCX_ApplyWebCommand.
4441		Blank separator for readability.
4442	if(action=="ARM_BUY")	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4443	{	Opens or closes a code block in TCX_ApplyWebCommand.
4444	string reason="";	Updates or initializes string reason in TCX_ApplyWebCommand.
4445	if(g_ArmedMode==ARM_BUY)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4446	{	Opens or closes a code block in TCX_ApplyWebCommand.
4447	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4448	msg="WEB BUY wait cancelled";	Updates or initializes msg in TCX_ApplyWebCommand.
4449	}	Opens or closes a code block in TCX_ApplyWebCommand.
4450	else if(CanStartModelEntry(reason))	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4451	{	Opens or closes a code block in TCX_ApplyWebCommand.
4452	SetArmMode(ARM_BUY);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4453	msg="WEB BUY armed - waiting for 3rd candle close";	Updates or initializes msg in TCX_ApplyWebCommand.
4454	}	Opens or closes a code block in TCX_ApplyWebCommand.
4455	else	Fallback branch when prior conditions did not match in TCX_ApplyWebCommand.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4456	{	Opens or closes a code block in TCX_ApplyWebCommand.
4457	msg=reason;	Updates or initializes msg in TCX_ApplyWebCommand.
4458	}	Opens or closes a code block in TCX_ApplyWebCommand.
4459	SetLastMessage(msg,g_ArmedMode==ARM_BUY ? clrLime : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4460	}	Opens or closes a code block in TCX_ApplyWebCommand.
4461	else if(action=="ARM_SELL")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4462	{	Opens or closes a code block in TCX_ApplyWebCommand.
4463	string reason="";	Updates or initializes string reason in TCX_ApplyWebCommand.
4464	if(g_ArmedMode==ARM_SELL)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4465	{	Opens or closes a code block in TCX_ApplyWebCommand.
4466	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4467	msg="WEB SELL wait cancelled";	Updates or initializes msg in TCX_ApplyWebCommand.
4468	}	Opens or closes a code block in TCX_ApplyWebCommand.
4469	else if(CanStartModelEntry(reason))	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4470	{	Opens or closes a code block in TCX_ApplyWebCommand.
4471	SetArmMode(ARM_SELL);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4472	msg="WEB SELL armed - waiting for 3rd candle close";	Updates or initializes msg in TCX_ApplyWebCommand.
4473	}	Opens or closes a code block in TCX_ApplyWebCommand.
4474	else	Fallback branch when prior conditions did not match in TCX_ApplyWebCommand.
4475	{	Opens or closes a code block in TCX_ApplyWebCommand.
4476	msg=reason;	Updates or initializes msg in TCX_ApplyWebCommand.
4477	}	Opens or closes a code block in TCX_ApplyWebCommand.
4478	SetLastMessage(msg,g_ArmedMode==ARM_SELL ? clrTomato : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4479	}	Opens or closes a code block in TCX_ApplyWebCommand.
4480	else if(action=="AUTO_ARM")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4481	{	Opens or closes a code block in TCX_ApplyWebCommand.
4482	string reason="";	Updates or initializes string reason in TCX_ApplyWebCommand.
4483	if(g_AutoArm)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4484	{	Opens or closes a code block in TCX_ApplyWebCommand.
4485	SetAutoArm(false);	Calls helper function SetAutoArm in TCX_ApplyWebCommand.
4486	msg="WEB auto arm disabled";	Updates or initializes msg in TCX_ApplyWebCommand.
4487	}	Opens or closes a code block in TCX_ApplyWebCommand.
4488	else if(CanStartModelEntry(reason))	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4489	{	Opens or closes a code block in TCX_ApplyWebCommand.
4490	SetAutoArm(true);	Calls helper function SetAutoArm in TCX_ApplyWebCommand.
4491	msg="WEB auto arm enabled - waits for 3rd candle close";	Updates or initializes msg in TCX_ApplyWebCommand.
4492	}	Opens or closes a code block in TCX_ApplyWebCommand.
4493	else	Fallback branch when prior conditions did not match in TCX_ApplyWebCommand.
4494	{	Opens or closes a code block in TCX_ApplyWebCommand.
4495	msg=reason;	Updates or initializes msg in TCX_ApplyWebCommand.
4496	}	Opens or closes a code block in TCX_ApplyWebCommand.
4497	SetLastMessage(msg,g_AutoArm ? clrGold : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4498	}	Opens or closes a code block in TCX_ApplyWebCommand.
4499	else if(action=="CANCEL")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4500	{	Opens or closes a code block in TCX_ApplyWebCommand.
4501	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4502	g_AutoArm=false;	Updates or initializes g_AutoArm in TCX_ApplyWebCommand.
4503	msg="WEB waiting mode cancelled";	Updates or initializes msg in TCX_ApplyWebCommand.
4504	SetLastMessage(msg,clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4505	}	Opens or closes a code block in TCX_ApplyWebCommand.
4506	else if(action=="CLOSE" action=="CLOSE_ALL")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4507	{	Opens or closes a code block in TCX_ApplyWebCommand.
4508	if(!InpWebAllowCloseCommands)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4509	{	Opens or closes a code block in TCX_ApplyWebCommand.
4510	msg="WEB close blocked by input setting";	Updates or initializes msg in TCX_ApplyWebCommand.
4511	SetLastMessage(msg,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
4512	return false;	Returns a value or exits the current function in TCX_ApplyWebCommand.
4513	}	Opens or closes a code block in TCX_ApplyWebCommand.
4514	CloseAllPositions();	Calls helper function CloseAllPositions in TCX_ApplyWebCommand.
4515	msg="WEB close command processed";	Updates or initializes msg in TCX_ApplyWebCommand.
4516	}	Opens or closes a code block in TCX_ApplyWebCommand.
4517	else if(action=="CLOSE_50")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4518	{	Opens or closes a code block in TCX_ApplyWebCommand.
4519	if(!InpWebAllowCloseCommands)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4520	{	Opens or closes a code block in TCX_ApplyWebCommand.
4521	msg="WEB close 50 blocked by input setting";	Updates or initializes msg in TCX_ApplyWebCommand.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4522	SetLastMessage(msg,clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
4523	return false;	Returns a value or exits the current function in TCX_ApplyWebCommand.
4524	}	Opens or closes a code block in TCX_ApplyWebCommand.
4525	CloseCurrentPercent(50.0);	Calls helper function CloseCurrentPercent in TCX_ApplyWebCommand.
4526	msg="WEB close 50 processed";	Updates or initializes msg in TCX_ApplyWebCommand.
4527	}	Opens or closes a code block in TCX_ApplyWebCommand.
4528	else if(action=="BREAK_EVEN" action=="BE")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4529	{	Opens or closes a code block in TCX_ApplyWebCommand.
4530	MoveToBreakEven();	Calls helper function MoveToBreakEven in TCX_ApplyWebCommand.
4531	msg="WEB break even processed";	Updates or initializes msg in TCX_ApplyWebCommand.
4532	}	Opens or closes a code block in TCX_ApplyWebCommand.
4533	else if(action=="FIRST_BREAK_EVEN" action=="FIRST_BE" action=="BE_FIRST")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4534	{	Opens or closes a code block in TCX_ApplyWebCommand.
4535	MoveFirstTradeToBreakEven();	Calls helper function MoveFirstTradeToBreakEven in TCX_ApplyWebCommand.
4536	msg=g_LastMessage;	Updates or initializes msg in TCX_ApplyWebCommand.
4537	}	Opens or closes a code block in TCX_ApplyWebCommand.
4538	else if(action=="FIRST_BE_EXIT" action=="FIRST_BREAK_EVEN_EXIT" action=="BE_EXIT_FIRST" action=="FIRST_PROFIT_EXIT")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4539	{	Opens or closes a code block in TCX_ApplyWebCommand.
4540	CloseFirstTradeAtBreakEvenExit();	Calls helper function CloseFirstTradeAtBreakEvenExit in TCX_ApplyWebCommand.
4541	msg=g_LastMessage;	Updates or initializes msg in TCX_ApplyWebCommand.
4542	}	Opens or closes a code block in TCX_ApplyWebCommand.
4543	else if(action=="TOGGLE_PARTIALS")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4544	{	Opens or closes a code block in TCX_ApplyWebCommand.
4545	g_PC_On=!g_PC_On;	Updates or initializes g_PC_On in TCX_ApplyWebCommand.
4546	msg=(g_PC_On ? "WEB partial close enabled" : "WEB partial close disabled");	Updates or initializes msg in TCX_ApplyWebCommand.
4547	SetLastMessage(msg,g_PC_On ? clrSkyBlue : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4548	}	Opens or closes a code block in TCX_ApplyWebCommand.
4549	else if(action=="TOGGLE_SECOND_ENTRY")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4550	{	Opens or closes a code block in TCX_ApplyWebCommand.
4551	g_SecondEntryOn=!g_SecondEntryOn;	Updates or initializes g_SecondEntryOn in TCX_ApplyWebCommand.
4552	if(!g_SecondEntryOn && HasOurPosition())	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4553	{	Opens or closes a code block in TCX_ApplyWebCommand.
4554	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4555	g_AutoArm=false;	Updates or initializes g_AutoArm in TCX_ApplyWebCommand.
4556	}	Opens or closes a code block in TCX_ApplyWebCommand.
4557	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in TCX_ApplyWebCommand.
4558	msg=(g_SecondEntryOn ? "WEB second entry enabled" : "WEB second entry disabled");	Updates or initializes msg in TCX_ApplyWebCommand.
4559	SetLastMessage(msg,g_SecondEntryOn ? clrSkyBlue : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4560	}	Opens or closes a code block in TCX_ApplyWebCommand.
4561	else if(action=="TOGGLE_FIRST_TRAIL")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4562	{	Opens or closes a code block in TCX_ApplyWebCommand.
4563	g_FirstTrailOn=!g_FirstTrailOn;	Updates or initializes g_FirstTrailOn in TCX_ApplyWebCommand.
4564	SyncFirstTrailState();	Calls helper function SyncFirstTrailState in TCX_ApplyWebCommand.
4565	msg=(g_FirstTrailOn ? "WEB first trade M5 trailing enabled" : "WEB first trade M5 trailing disabled");	Updates or initializes msg in TCX_ApplyWebCommand.
4566	SetLastMessage(msg,g_FirstTrailOn ? clrSkyBlue : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4567	}	Opens or closes a code block in TCX_ApplyWebCommand.
4568	else if(action=="SET_MODE")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4569	{	Opens or closes a code block in TCX_ApplyWebCommand.
4570	string mode=TCX_JsonString(raw,"mode","safe");	Builds or parses JSON used by the web control bridge.
4571	StringToLower(mode);	Calls helper function StringToLower in TCX_ApplyWebCommand.
4572	if(mode=="advanced" mode=="adv")	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4573	{	Opens or closes a code block in TCX_ApplyWebCommand.
4574	g_AdvancedMode=true;	Updates or initializes g_AdvancedMode in TCX_ApplyWebCommand.
4575	g_PC_On=true;	Updates or initializes g_PC_On in TCX_ApplyWebCommand.
4576	msg="WEB advanced mode enabled";	Updates or initializes msg in TCX_ApplyWebCommand.
4577	SetLastMessage(msg,clrSkyBlue);	Updates the EA status message, panel display, and web state dirty flag.
4578	}	Opens or closes a code block in TCX_ApplyWebCommand.
4579	else	Fallback branch when prior conditions did not match in TCX_ApplyWebCommand.
4580	{	Opens or closes a code block in TCX_ApplyWebCommand.
4581	g_AdvancedMode=false;	Updates or initializes g_AdvancedMode in TCX_ApplyWebCommand.
4582	g_PC_On=false;	Updates or initializes g_PC_On in TCX_ApplyWebCommand.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4583	msg="WEB safe mode enabled";	Updates or initializes msg in TCX_ApplyWebCommand.
4584	SetLastMessage(msg,clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4585	}	Opens or closes a code block in TCX_ApplyWebCommand.
4586	}	Opens or closes a code block in TCX_ApplyWebCommand.
4587	else if(action=="SET_RISK")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4588	{	Opens or closes a code block in TCX_ApplyWebCommand.
4589	double lot=TCX_JsonNumber(raw,"lot",g_Lot);	Builds or parses JSON used by the web control bridge.
4590	double risk=TCX_JsonNumber(raw,"risk",g_RiskMoney);	Builds or parses JSON used by the web control bridge.
4591	double rr=TCX_JsonNumber(raw,"rr",g_RRTarget);	Builds or parses JSON used by the web control bridge.
4592	g_Lot=NormalizeVolumeForPanel(lot);	Updates or initializes g_Lot in TCX_ApplyWebCommand.
4593	g_RiskMoney=NormalizeRiskMoney(risk);	Updates or initializes g_RiskMoney in TCX_ApplyWebCommand.
4594	g_RRTarget=NormalizeDouble(MathMax(0.1,MathMin(20.0,rr)),1);	Updates or initializes g_RRTarget in TCX_ApplyWebCommand.
4595	RefreshLotEdit();	Calls helper function RefreshLotEdit in TCX_ApplyWebCommand.
4596	RefreshRiskEdit();	Calls helper function RefreshRiskEdit in TCX_ApplyWebCommand.
4597	msg="WEB risk updated: lot "+DoubleToString(g_Lot,VolumeDigits())+", risk	Updates or initializes msg in TCX_ApplyWebCommand.
4598	"+MoneyText(g_RiskMoney)+"", RR 1:"+DoubleToString(g_RRTarget,1);	
4598	SetLastMessage(msg,clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4599	}	Opens or closes a code block in TCX_ApplyWebCommand.
4600	else if(action=="SET_PARTIALS")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4601	{	Opens or closes a code block in TCX_ApplyWebCommand.
4602	g_PC1_Pct=ClampPercent(TCX_JsonNumber(raw,"pc1",g_PC1_Pct));	Builds or parses JSON used by the web control bridge.
4603	g_PC2_Pct=ClampPercent(TCX_JsonNumber(raw,"pc2",g_PC2_Pct));	Builds or parses JSON used by the web control bridge.
4604	g_PC3_Pct=ClampPercent(TCX_JsonNumber(raw,"pc3",g_PC3_Pct));	Builds or parses JSON used by the web control bridge.
4605	bool explicitPartials=TCX_JsonHasKey(raw,"partialsOn");	Builds or parses JSON used by the web control bridge.
4606	bool explicitSecond=TCX_JsonHasKey(raw,"secondEntryOn");	Builds or parses JSON used by the web control bridge.
4607	bool explicitFirstTrail=TCX_JsonHasKey(raw,"firstTrailOn");	Builds or parses JSON used by the web control bridge.
4608	bool firstBeRequested=TCX_JsonBool(raw,"firstBreakEven",false);	Builds or parses JSON used by the web control bridge.
4609	bool firstBeExitRequested=TCX_JsonBool(raw,"firstBeExit",false);	Builds or parses JSON used by the web control bridge.
4610		Blank separator for readability.
4611	if(explicitPartials)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4612	g_PC_On=TCX_JsonBool(raw,"partialsOn",g_PC_On);	Builds or parses JSON used by the web control bridge.
4613		Blank separator for readability.
4614	if(TCX_JsonHasKey(raw,"secondEntryOn"))	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4615	{	Opens or closes a code block in TCX_ApplyWebCommand.
4616	g_SecondEntryOn=TCX_JsonBool(raw,"secondEntryOn",g_SecondEntryOn);	Builds or parses JSON used by the web control bridge.
4617	if(!g_SecondEntryOn && HasOurPosition())	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4618	{	Opens or closes a code block in TCX_ApplyWebCommand.
4619	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in TCX_ApplyWebCommand.
4620	g_AutoArm=false;	Updates or initializes g_AutoArm in TCX_ApplyWebCommand.
4621	}	Opens or closes a code block in TCX_ApplyWebCommand.
4622	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in TCX_ApplyWebCommand.
4623	}	Opens or closes a code block in TCX_ApplyWebCommand.
4624		Blank separator for readability.
4625	if(explicitFirstTrail)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4626	{	Opens or closes a code block in TCX_ApplyWebCommand.
4627	g_FirstTrailOn=TCX_JsonBool(raw,"firstTrailOn",g_FirstTrailOn);	Builds or parses JSON used by the web control bridge.
4628	SyncFirstTrailState();	Calls helper function SyncFirstTrailState in TCX_ApplyWebCommand.
4629	}	Opens or closes a code block in TCX_ApplyWebCommand.
4630		Blank separator for readability.
4631	SetEditText(E_PC1,g_PC1_Pct,0);	Calls helper function SetEditText in TCX_ApplyWebCommand.
4632	SetEditText(E_PC2,g_PC2_Pct,0);	Calls helper function SetEditText in TCX_ApplyWebCommand.
4633	SetEditText(E_PC3,g_PC3_Pct,0);	Calls helper function SetEditText in TCX_ApplyWebCommand.
4634		Blank separator for readability.
4635	if(firstBeExitRequested)	Checks a condition and runs the guarded branch in TCX_ApplyWebCommand.
4636	{	Opens or closes a code block in TCX_ApplyWebCommand.
4637	CloseFirstTradeAtBreakEvenExit();	Calls helper function CloseFirstTradeAtBreakEvenExit in TCX_ApplyWebCommand.
4638	msg=g_LastMessage;	Updates or initializes msg in TCX_ApplyWebCommand.
4639	}	Opens or closes a code block in TCX_ApplyWebCommand.
4640	else if(firstBeRequested)	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4641	{	Opens or closes a code block in TCX_ApplyWebCommand.
4642	MoveFirstTradeToBreakEven();	Calls helper function MoveFirstTradeToBreakEven in TCX_ApplyWebCommand.
4643	msg=g_LastMessage;	Updates or initializes msg in TCX_ApplyWebCommand.
4644	}	Opens or closes a code block in TCX_ApplyWebCommand.
4645	else	Fallback branch when prior conditions did not match in TCX_ApplyWebCommand.
4646	{	Opens or closes a code block in TCX_ApplyWebCommand.
4647	msg=(explicitPartials explicitSecond explicitFirstTrail) ? "WEB trade	Updates or initializes msg in TCX_ApplyWebCommand.
4648	controls updated" : "WEB partial values updated";	
4648	SetLastMessage(msg,clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4649	}	Opens or closes a code block in TCX_ApplyWebCommand.
4650	}	Opens or closes a code block in TCX_ApplyWebCommand.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4651	else if(action=="PING" action=="NOOP")	Checks an alternate condition when the previous branch did not run in TCX_ApplyWebCommand.
4652	{	Opens or closes a code block in TCX_ApplyWebCommand.
4653	msg="WEB ping ok";	Updates or initializes msg in TCX_ApplyWebCommand.
4654	}	Opens or closes a code block in TCX_ApplyWebCommand.
4655	else	Fallback branch when prior conditions did not match in TCX_ApplyWebCommand.
4656	{	Opens or closes a code block in TCX_ApplyWebCommand.
4657	msg="Unknown web command: "+action;	Updates or initializes msg in TCX_ApplyWebCommand.
4658	SetLastMessage(msg, clrTomato);	Updates the EA status message, panel display, and web state dirty flag.
4659	return false;	Returns a value or exits the current function in TCX_ApplyWebCommand.
4660	}	Opens or closes a code block in TCX_ApplyWebCommand.
4661		Blank separator for readability.
4662	g_WebLastCommandId=id;	Updates or initializes g_WebLastCommandId in TCX_ApplyWebCommand.
4663	UpdatePanel();	Calls helper function UpdatePanel in TCX_ApplyWebCommand.
4664	g_WebStateDirty=true;	Updates or initializes g_WebStateDirty in TCX_ApplyWebCommand.
4665	return true;	Returns a value or exits the current function in TCX_ApplyWebCommand.
4666	}	Opens or closes a code block in TCX_ApplyWebCommand.
4667		Blank separator for readability.
4668	bool TCX_WebReady()	Declares or starts function TCX_WebReady.
4669	{	Opens or closes a code block in TCX_WebReady.
4670	if(!InpWebControlEnabled)	Checks a condition and runs the guarded branch in TCX_WebReady.
4671	return false;	Returns a value or exits the current function in TCX_WebReady.
4672	if(StringLen(InpSupabaseFunctionsUrl)<20 StringFind(InpSupabaseFunctionsUrl, "YOUR_PROJECT_REF")>=0)	Checks a condition and runs the guarded branch in TCX_WebReady.
4673	return false;	Returns a value or exits the current function in TCX_WebReady.
4674	if(StringLen(InpSupabaseEaId)<8 StringFind(InpSupabaseEaId, "PASTE_EA_ID")>=0)	Checks a condition and runs the guarded branch in TCX_WebReady.
4675	return false;	Returns a value or exits the current function in TCX_WebReady.
4676	if(StringLen(InpSupabaseEaToken)<12 StringFind(InpSupabaseEaToken, "PASTE_EA_SECRET")>=0)	Checks a condition and runs the guarded branch in TCX_WebReady.
4677	return false;	Returns a value or exits the current function in TCX_WebReady.
4678	return true;	Returns a value or exits the current function in TCX_WebReady.
4679	}	Opens or closes a code block in TCX_WebReady.
4680		Blank separator for readability.
4681	void TCX_SupabasePollCommands()	Declares or starts function TCX_SupabasePollCommands.
4682	{	Opens or closes a code block in TCX_SupabasePollCommands.
4683	if(!TCX_WebReady())	Checks a condition and runs the guarded branch in TCX_SupabasePollCommands.
4684	return;	Returns a value or exits the current function in TCX_SupabasePollCommands.
4685		Blank separator for readability.
4686	uint nowTick=GetTickCount();	Updates or initializes uint nowTick in TCX_SupabasePollCommands.
4687	int ms=MathMax(200, InpWebPollMilliseconds);	Updates or initializes int ms in TCX_SupabasePollCommands.
4688	if(g_WebLastPollTick>0 && (uint)(nowTick-g_WebLastPollTick)<(uint)ms)	Checks a condition and runs the guarded branch in TCX_SupabasePollCommands.
4689	return;	Returns a value or exits the current function in TCX_SupabasePollCommands.
4690	g_WebLastPollTick=nowTick;	Updates or initializes g_WebLastPollTick in TCX_SupabasePollCommands.
4691		Blank separator for readability.
4692	string res="";	Updates or initializes string res in TCX_SupabasePollCommands.
4693	string body="{\"ea_id\": \"\"+TCX_JsonEscape(InpSupabaseEaId)+\"\", \"ea_token\": \"\"+TCBuilds or parses JSON used by the web control bridge. X_JsonEscape(InpSupabaseEaToken)+\"\", \"symbol\": \"\"+TCX_JsonEscape(_Symbol)+\"\", \"magi c\": \"\"+IntegerToString(InpMagic)+\"\"}";	
4694	if(!TCX_HttpPostFunction("ea-next-command", body, res))	Checks a condition and runs the guarded branch in TCX_SupabasePollCommands.
4695	return;	Returns a value or exits the current function in TCX_SupabasePollCommands.
4696		Blank separator for readability.
4697	bool hasCommand=(StringFind(res, "\"action\"")>=0 && StringFind(res, "\"id\"")>=0 && StringFind(res, "\"command\": null")<0);	Updates or initializes bool hasCommand in TCX_SupabasePollCommands.
4698	if(!hasCommand)	Checks a condition and runs the guarded branch in TCX_SupabasePollCommands.
4699	return;	Returns a value or exits the current function in TCX_SupabasePollCommands.
4700		Blank separator for readability.
4701	string id=TCX_JsonString(res, "id", "");	Builds or parses JSON used by the web control bridge.
4702	string action=TCX_JsonString(res, "action", "");	Builds or parses JSON used by the web control bridge.
4703	if(action=="")	Checks a condition and runs the guarded branch in TCX_SupabasePollCommands.
4704	return;	Returns a value or exits the current function in TCX_SupabasePollCommands.
4705		Blank separator for readability.
4706	bool ok=TCX_ApplyWebCommand(id, action, res);	Updates or initializes bool ok in TCX_SupabasePollCommands.
4707	TCX_AckCommand(id, ok ? "done" : "failed", g_LastMessage);	Acknowledges a command after the EA handles it.
4708	}	Opens or closes a code block in TCX_SupabasePollCommands.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4709		Blank separator for readability.
4710	void TCX_SupabasePostState(bool force)	Declares or starts function TCX_SupabasePostState.
4711	{	Opens or closes a code block in TCX_SupabasePostState.
4712	if(!TCX_WebReady() !InpWebPostLiveData)	Checks a condition and runs the guarded branch in TCX_SupabasePostState.
4713	return;	Returns a value or exits the current function in TCX_SupabasePostState.
4714		Blank separator for readability.
4715	uint nowTick=GetTickCount();	Updates or initializes uint nowTick in TCX_SupabasePostState.
4716	int ms=1000;	Updates or initializes int ms in TCX_SupabasePostState.
4717	if(!force && !g_WebStateDirty && g_WebLastPostTick>0 && (uint)(nowTick-g_WebLastPostTick)<(uint)ms)	Checks a condition and runs the guarded branch in TCX_SupabasePostState.
4718	return;	Returns a value or exits the current function in TCX_SupabasePostState.
4719	g_WebLastPostTick=nowTick;	Updates or initializes g_WebLastPostTick in TCX_SupabasePostState.
4720		Blank separator for readability.
4721	string res="";	Updates or initializes string res in TCX_SupabasePostState.
4722	string body="{\"ea_id\": \"\"+TCX_JsonEscape(InpSupabaseEaId)+\"\", \"ea_token\": \"\"+TCX_JsonEscape(InpSupabaseEaToken)+\"\", \"state\": \"\"+TCX_BuildStateJson()+\"\"}";	Builds or parses JSON used by the web control bridge.
4723	if(TCX_HttpPostFunction("ea-post-state",body,res))	Checks a condition and runs the guarded branch in TCX_SupabasePostState.
4724	g_WebStateDirty=false;	Updates or initializes g_WebStateDirty in TCX_SupabasePostState.
4725	}	Opens or closes a code block in TCX_SupabasePostState.
4726		Blank separator for readability.
4727	void EnsurePanel()	Declares or starts function EnsurePanel.
4728	{	Opens or closes a code block in EnsurePanel.
4729	if(!g_PanelBuilt ObjectFind(0,UI+"B6")<0 ObjectFind(0,B_BUY)<0 ObjectFind(0,B_SELL)<0)	Checks a condition and runs the guarded branch in EnsurePanel.
4730	{	Opens or closes a code block in EnsurePanel.
4731	BuildPanel();	Calls helper function BuildPanel in EnsurePanel.
4732	UpdatePanel();	Calls helper function UpdatePanel in EnsurePanel.
4733	}	Opens or closes a code block in EnsurePanel.
4734	}	Opens or closes a code block in EnsurePanel.
4735		Blank separator for readability.
4736	void RuntimeLoop(bool allowTradeScan)	Declares or starts function RuntimeLoop.
4737	{	Opens or closes a code block in RuntimeLoop.
4738	EnsurePanel();	Calls helper function EnsurePanel in RuntimeLoop.
4739	SyncPositionState();	Calls helper function SyncPositionState in RuntimeLoop.
4740	if(allowTradeScan)	Checks a condition and runs the guarded branch in RuntimeLoop.
4741	{	Opens or closes a code block in RuntimeLoop.
4742	MonitorSecondEntry();	Calls second-entry monitoring logic.
4743	MonitorFirstTradeTrailing();	Calls first-trade trailing logic.
4744	}	Opens or closes a code block in RuntimeLoop.
4745	else	Fallback branch when prior conditions did not match in RuntimeLoop.
4746	{	Opens or closes a code block in RuntimeLoop.
4747	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in RuntimeLoop.
4748	SyncFirstTrailState();	Calls helper function SyncFirstTrailState in RuntimeLoop.
4749	}	Opens or closes a code block in RuntimeLoop.
4750	TCX_SupabasePollCommands();	Posts Supabase for pending website commands.
4751		Blank separator for readability.
4752	if(g_PC_On)	Checks a condition and runs the guarded branch in RuntimeLoop.
4753	MonitorPartialClose();	Calls partial close monitoring logic.
4754		Blank separator for readability.
4755	datetime barTime=iTime(_Symbol,_Period,0);	Updates or initializes datetime barTime in RuntimeLoop.
4756	if(barTime!=g_LastBarTime)	Checks a condition and runs the guarded branch in RuntimeLoop.
4757	{	Opens or closes a code block in RuntimeLoop.
4758	g_LastBarTime=barTime;	Updates or initializes g_LastBarTime in RuntimeLoop.
4759	DrawLatestClosedSignal();	Calls helper function DrawLatestClosedSignal in RuntimeLoop.
4760	}	Opens or closes a code block in RuntimeLoop.
4761		Blank separator for readability.
4762	// Trade scan is internally gated to the first tick after a candle closes.	Comment explaining the nearby EA code.
4763	if(allowTradeScan && (g_ArmedMode!=ARM_NONE g_AutoArm))	Checks a condition and runs the guarded branch in RuntimeLoop.
4764	TryArmedExecution();	Calls helper function TryArmedExecution in RuntimeLoop.
4765		Blank separator for readability.
4766	UpdatePanel();	Calls helper function UpdatePanel in RuntimeLoop.
4767	TCX_SupabasePostState(false);	Posts the latest EA state to Supabase for the website.
4768	ChartHeartbeat();	Calls helper function ChartHeartbeat in RuntimeLoop.
4769	}	Opens or closes a code block in RuntimeLoop.
4770		Blank separator for readability.
4771	//=====	Comment explaining the nearby EA code.
4772	// EDIT HANDLERS	Comment explaining the nearby EA code.
4773	//=====	Comment explaining the nearby EA code.
4774	void RefreshLotEdit()	Declares or starts function RefreshLotEdit.
4775	{	Opens or closes a code block in RefreshLotEdit.
4776	if(ObjectFind(0,E_LOT)>=0)	Checks a condition and runs the guarded branch in RefreshLotEdit.
4777	ObjectSetString(0,E_LOT,OBJPROP_TEXT,DoubleToString(g_Lot,VolumeDigits()));	Creates, updates, or removes an MT5 chart panel object.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4778	}	Opens or closes a code block in RefreshLotEdit.
4779		Blank separator for readability.
4780	void RefreshRiskEdit()	Declares or starts function RefreshRiskEdit.
4781	{	Opens or closes a code block in RefreshRiskEdit.
4782	if(ObjectFind(0,E_RISK)>=0)	Checks a condition and runs the guarded branch in RefreshRiskEdit.
4783	ObjectSetString(0,E_RISK,OBJPROP_TEXT,DoubleToString(g_RiskMoney,2));	Creates, updates, or removes an MT5 chart panel object.
4784	}	Opens or closes a code block in RefreshRiskEdit.
4785		Blank separator for readability.
4786	void HandleLotEdit()	Declares or starts function HandleLotEdit.
4787	{	Opens or closes a code block in HandleLotEdit.
4788	string raw=ObjectGetString(0,E_LOT,OBJPROP_TEXT);	Updates or initializes string raw in HandleLotEdit.
4789		Blank separator for readability.
4790	double maxLot=InpPanelLotMax;	Updates or initializes double maxLot in HandleLotEdit.
4791	if(maxLot<=0.0)	Checks a condition and runs the guarded branch in HandleLotEdit.
4792	maxLot=100.0;	Updates or initializes maxLot in HandleLotEdit.
4793	if(InpClampLotToBrokerMax)	Checks a condition and runs the guarded branch in HandleLotEdit.
4794	maxLot=MathMin(maxLot,BrokerMaxLot());	Updates or initializes maxLot in HandleLotEdit.
4795		Blank separator for readability.
4796	g_Lot=NormalizeVolumeForPanel(CleanNumber(raw,g_Lot,BrokerMinLot(),maxLot));	Updates or initializes g_Lot in HandleLotEdit.
4797	RefreshLotEdit();	Calls helper function RefreshLotEdit in HandleLotEdit.
4798	SetLastMessage("Lot set to "+DoubleToString(g_Lot,VolumeDigits()),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4799	UpdatePanel();	Calls helper function UpdatePanel in HandleLotEdit.
4800	}	Opens or closes a code block in HandleLotEdit.
4801		Blank separator for readability.
4802	void HandleRiskEdit()	Declares or starts function HandleRiskEdit.
4803	{	Opens or closes a code block in HandleRiskEdit.
4804	string raw=ObjectGetString(0,E_RISK,OBJPROP_TEXT);	Updates or initializes string raw in HandleRiskEdit.
4805	g_RiskMoney=CleanMoney(raw,g_RiskMoney);	Updates or initializes g_RiskMoney in HandleRiskEdit.
4806	RefreshRiskEdit();	Calls helper function RefreshRiskEdit in HandleRiskEdit.
4807	SetLastMessage("Risk set to "+MoneyText(g_RiskMoney),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4808	UpdatePanel();	Calls helper function UpdatePanel in HandleRiskEdit.
4809	}	Opens or closes a code block in HandleRiskEdit.
4810		Blank separator for readability.
4811	void HandlePartialEdit(string obj)	Declares or starts function HandlePartialEdit.
4812	{	Opens or closes a code block in HandlePartialEdit.
4813	string raw=ObjectGetString(0,obj,OBJPROP_TEXT);	Updates or initializes string raw in HandlePartialEdit.
4814		Blank separator for readability.
4815	if(obj==E_PC1)	Checks a condition and runs the guarded branch in HandlePartialEdit.
4816	{	Opens or closes a code block in HandlePartialEdit.
4817	g_PC1_Pct=CleanPercent(raw,g_PC1_Pct);	Updates or initializes g_PC1_Pct in HandlePartialEdit.
4818	SetEditText(E_PC1,g_PC1_Pct,0);	Calls helper function SetEditText in HandlePartialEdit.
4819	}	Opens or closes a code block in HandlePartialEdit.
4820	else if(obj==E_PC2)	Checks an alternate condition when the previous branch did not run in HandlePartialEdit.
4821	{	Opens or closes a code block in HandlePartialEdit.
4822	g_PC2_Pct=CleanPercent(raw,g_PC2_Pct);	Updates or initializes g_PC2_Pct in HandlePartialEdit.
4823	SetEditText(E_PC2,g_PC2_Pct,0);	Calls helper function SetEditText in HandlePartialEdit.
4824	}	Opens or closes a code block in HandlePartialEdit.
4825	else if(obj==E_PC3)	Checks an alternate condition when the previous branch did not run in HandlePartialEdit.
4826	{	Opens or closes a code block in HandlePartialEdit.
4827	g_PC3_Pct=CleanPercent(raw,g_PC3_Pct);	Updates or initializes g_PC3_Pct in HandlePartialEdit.
4828	SetEditText(E_PC3,g_PC3_Pct,0);	Calls helper function SetEditText in HandlePartialEdit.
4829	}	Opens or closes a code block in HandlePartialEdit.
4830		Blank separator for readability.
4831	SetLastMessage("Partial book values updated",clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4832	UpdatePanel();	Calls helper function UpdatePanel in HandlePartialEdit.
4833	}	Opens or closes a code block in HandlePartialEdit.
4834		Blank separator for readability.
4835	//=====	Comment explaining the nearby EA code.
4836	// EVENTS	Comment explaining the nearby EA code.
4837	//=====	Comment explaining the nearby EA code.
4838	int OnInit()	Declares or starts function OnInit.
4839	{	Opens or closes a code block in OnInit.
4840	g_InitTime=TimeCurrent();	Updates or initializes g_InitTime in OnInit.
4841	SymbolSelect(_Symbol,true);	Calls helper function SymbolSelect in OnInit.
4842		Blank separator for readability.
4843	g_Lot=NormalizeVolumeForPanel(InpLot);	Updates or initializes g_Lot in OnInit.
4844	g_RRTarget=(InpRiskReward>0.0 ? InpRiskReward : 3.0);	Updates or initializes g_RRTarget in OnInit.
4845	g_RiskMoney=NormalizeRiskMoney(InpRiskMoney);	Updates or initializes g_RiskMoney in OnInit.
4846	g_PC_On=InpPartialCloseEnabled;	Updates or initializes g_PC_On in OnInit.
4847	g_AdvancedMode=InpPartialCloseEnabled;	Updates or initializes g_AdvancedMode in OnInit.
4848	g_SecondEntryOn=InpSecondEntryEnabled;	Updates or initializes g_SecondEntryOn in OnInit.
4849	g_FirstTrailOn=InpFirstTrailEnabled;	Updates or initializes g_FirstTrailOn in OnInit.
4850	g_PC1_Pct=ClampPercent(InpPC1_Percent);	Updates or initializes g_PC1_Pct in OnInit.
4851	g_PC2_Pct=ClampPercent(InpPC2_Percent);	Updates or initializes g_PC2_Pct in OnInit.
4852	g_PC3_Pct=ClampPercent(InpPC3_Percent);	Updates or initializes g_PC3_Pct in OnInit.
4853		Blank separator for readability.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4854	Trade.SetExpertMagicNumber(InpMagic);	Executes or modifies a trade through the MetaTrader trade API.
4855	Trade.SetDeviationInPoints(InpDeviationPoints);	Executes or modifies a trade through the MetaTrader trade API.
4856	Trade.SetMarginMode();	Executes or modifies a trade through the MetaTrader trade API.
4857	Trade.SetTypeFillingBySymbol(_Symbol);	Executes or modifies a trade through the MetaTrader trade API.
4858		Blank separator for readability.
4859	g_LastBarTime=0; // Force pattern check on first tick	Part of the EA logic in OnInit.
4860		Blank separator for readability.
4861	DeleteTradeLevelObjects();	Calls helper function DeleteTradeLevelObjects in OnInit.
4862	BuildPanel();	Calls helper function BuildPanel in OnInit.
4863	ChartHeartbeat();	Calls helper function ChartHeartbeat in OnInit.
4864	SyncPositionState();	Calls helper function SyncPositionState in OnInit.
4865	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in OnInit.
4866	SyncFirstTrailState();	Calls helper function SyncFirstTrailState in OnInit.
4867	DrawHistoricalSignals();	Calls helper function DrawHistoricalSignals in OnInit.
4868	UpdatePanel();	Calls helper function UpdatePanel in OnInit.
4869		Blank separator for readability.
4870	SetLastMessage("Controller ready - buttons wait for next model, spread no block",clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4871	UpdatePanel();	Calls helper function UpdatePanel in OnInit.
4872	if(InpWebControlEnabled)	Checks a condition and runs the guarded branch in OnInit.
4873	EventSetMillisecondTimer(MathMax(200,InpWebPollMilliseconds));	Calls helper function EventSetMillisecondTimer in OnInit.
4874	else	Fallback branch when prior conditions did not match in OnInit.
4875	EventSetTimer(1);	Calls helper function EventSetTimer in OnInit.
4876	ChartRedraw();	Calls helper function ChartRedraw in OnInit.
4877		Blank separator for readability.
4878	return INIT_SUCCEEDED;	Returns a value or exits the current function in OnInit.
4879	}	Opens or closes a code block in OnInit.
4880		Blank separator for readability.
4881	void OnDeinit(const int reason)	Declares or starts function OnDeinit.
4882	{	Opens or closes a code block in OnDeinit.
4883	EventKillTimer();	Calls helper function EventKillTimer in OnDeinit.
4884	Comment("");	Calls helper function Comment in OnDeinit.
4885	ObjectsDeleteAll(0,FX);	Calls helper function ObjectsDeleteAll in OnDeinit.
4886	ChartRedraw();	Calls helper function ChartRedraw in OnDeinit.
4887	}	Opens or closes a code block in OnDeinit.
4888		Blank separator for readability.
4889	void OnTick()	Declares or starts function OnTick.
4890	{	Opens or closes a code block in OnTick.
4891	RuntimeLoop(true);	Calls helper function RuntimeLoop in OnTick.
4892	}	Opens or closes a code block in OnTick.
4893		Blank separator for readability.
4894	void OnTimer()	Declares or starts function OnTimer.
4895	{	Opens or closes a code block in OnTimer.
4896	RuntimeLoop(false);	Calls helper function RuntimeLoop in OnTimer.
4897	}	Opens or closes a code block in OnTimer.
4898		Blank separator for readability.
4899	void OnChartEvent(const int id,const long &lparam,const double &dparam,const string &sparam)	Declares or starts function OnChartEvent.
4900	{	Opens or closes a code block in OnChartEvent.
4901	if(id==CHARTEVENT_CHART_CHANGE)	Checks a condition and runs the guarded branch in OnChartEvent.
4902	{	Opens or closes a code block in OnChartEvent.
4903	BuildPanel();	Calls helper function BuildPanel in OnChartEvent.
4904	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
4905	return;	Returns a value or exits the current function in OnChartEvent.
4906	}	Opens or closes a code block in OnChartEvent.
4907		Blank separator for readability.
4908	if(id==CHARTEVENT_OBJECT_ENDEDIT)	Checks a condition and runs the guarded branch in OnChartEvent.
4909	{	Opens or closes a code block in OnChartEvent.
4910	if(sparam==E_LOT)	Checks a condition and runs the guarded branch in OnChartEvent.
4911	{	Opens or closes a code block in OnChartEvent.
4912	HandleLotEdit();	Calls helper function HandleLotEdit in OnChartEvent.
4913	return;	Returns a value or exits the current function in OnChartEvent.
4914	}	Opens or closes a code block in OnChartEvent.
4915	if(sparam==E_RISK)	Checks a condition and runs the guarded branch in OnChartEvent.
4916	{	Opens or closes a code block in OnChartEvent.
4917	HandleRiskEdit();	Calls helper function HandleRiskEdit in OnChartEvent.
4918	return;	Returns a value or exits the current function in OnChartEvent.
4919	}	Opens or closes a code block in OnChartEvent.
4920	if(sparam==E_PC1 sparam==E_PC2 sparam==E_PC3)	Checks a condition and runs the guarded branch in OnChartEvent.
4921	{	Opens or closes a code block in OnChartEvent.
4922	HandlePartialEdit(sparam);	Calls helper function HandlePartialEdit in OnChartEvent.
4923	return;	Returns a value or exits the current function in OnChartEvent.
4924	}	Opens or closes a code block in OnChartEvent.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4925	}	Opens or closes a code block in OnChartEvent.
4926		Blank separator for readability.
4927	if(id!=CHARTEVENT_OBJECT_CLICK)	Checks a condition and runs the guarded branch in OnChartEvent.
4928	return;	Returns a value or exits the current function in OnChartEvent.
4929		Blank separator for readability.
4930	if(ObjectFind(0,sparam)>=0)	Checks a condition and runs the guarded branch in OnChartEvent.
4931	ObjectSetInteger(0,sparam,OBJPROP_STATE,false);	Creates, updates, or removes an MT5 chart panel object.
4932		Blank separator for readability.
4933	if(sparam==B_BUY)	Checks a condition and runs the guarded branch in OnChartEvent.
4934	{	Opens or closes a code block in OnChartEvent.
4935	string reason="";	Updates or initializes string reason in OnChartEvent.
4936	if(g_ArmedMode==ARM_BUY)	Checks a condition and runs the guarded branch in OnChartEvent.
4937	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in OnChartEvent.
4938	else if(CanStartModelEntry(reason))	Checks an alternate condition when the previous branch did not run in OnChartEvent.
4939	SetArmMode(ARM_BUY);	Calls helper function SetArmMode in OnChartEvent.
4940	else	Fallback branch when prior conditions did not match in OnChartEvent.
4941	{	Opens or closes a code block in OnChartEvent.
4942	SetLastMessage(reason,clrGold);	Updates the EA status message, panel display, and web state dirty flag.
4943	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
4944	return;	Returns a value or exits the current function in OnChartEvent.
4945	}	Opens or closes a code block in OnChartEvent.
4946		Blank separator for readability.
4947	SetLastMessage(g_ArmedMode==ARM_BUY ? "BUY armed - waiting for 3rd candle close" : "BUY wait cancelled", g_ArmedMode==ARM_BUY ? clrLime : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4948	UpdatePanel();	Updates or initializes g_ArmedMode in OnChartEvent.
4949	return;	Calls helper function UpdatePanel in OnChartEvent.
4950		Returns a value or exits the current function in OnChartEvent.
4951	}	Opens or closes a code block in OnChartEvent.
4952		Blank separator for readability.
4953	if(sparam==B_SELL)	Checks a condition and runs the guarded branch in OnChartEvent.
4954	{	Opens or closes a code block in OnChartEvent.
4955	string reason="";	Updates or initializes string reason in OnChartEvent.
4956	if(g_ArmedMode==ARM_SELL)	Checks a condition and runs the guarded branch in OnChartEvent.
4957	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in OnChartEvent.
4958	else if(CanStartModelEntry(reason))	Checks an alternate condition when the previous branch did not run in OnChartEvent.
4959	SetArmMode(ARM_SELL);	Calls helper function SetArmMode in OnChartEvent.
4960	else	Fallback branch when prior conditions did not match in OnChartEvent.
4961	{	Opens or closes a code block in OnChartEvent.
4962	SetLastMessage(reason,clrGold);	Updates the EA status message, panel display, and web state dirty flag.
4963	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
4964	return;	Returns a value or exits the current function in OnChartEvent.
4965	}	Opens or closes a code block in OnChartEvent.
4966		Blank separator for readability.
4967	SetLastMessage(g_ArmedMode==ARM_SELL ? "SELL armed - waiting for 3rd candle close" : "SELL wait cancelled", g_ArmedMode==ARM_SELL ? clrTomato : clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4968	UpdatePanel();	Updates or initializes g_ArmedMode in OnChartEvent.
4969	return;	Calls helper function UpdatePanel in OnChartEvent.
4970		Returns a value or exits the current function in OnChartEvent.
4971	}	Opens or closes a code block in OnChartEvent.
4972		Blank separator for readability.
4973	if(sparam==B_AUTO)	Checks a condition and runs the guarded branch in OnChartEvent.
4974	{	Opens or closes a code block in OnChartEvent.
4975	string reason="";	Updates or initializes string reason in OnChartEvent.
4976	if(g_AutoArm)	Checks a condition and runs the guarded branch in OnChartEvent.
4977	SetAutoArm(false);	Calls helper function SetAutoArm in OnChartEvent.
4978	else if(CanStartModelEntry(reason))	Checks an alternate condition when the previous branch did not run in OnChartEvent.
4979	SetAutoArm(true);	Calls helper function SetAutoArm in OnChartEvent.
4980	else	Fallback branch when prior conditions did not match in OnChartEvent.
4981	{	Opens or closes a code block in OnChartEvent.
4982	SetLastMessage(reason,clrGold);	Updates the EA status message, panel display, and web state dirty flag.
4983	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
4984	return;	Returns a value or exits the current function in OnChartEvent.
4985	}	Opens or closes a code block in OnChartEvent.
4986		Blank separator for readability.
4987	SetLastMessage(g_AutoArm ? "Auto arm enabled - waiting for 3rd candle close" : "Auto arm disabled",	Updates the EA status message, panel display, and web state dirty flag.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
4988	g_AutoArm ? clrGold : clrSilver);	Executes one statement in OnChartEvent.
4989	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
4990	return;	Returns a value or exits the current function in OnChartEvent.
4991	}	Opens or closes a code block in OnChartEvent.
4992		Blank separator for readability.
4993	if(sparam==B_CANCEL)	Checks a condition and runs the guarded branch in OnChartEvent.
4994	{	Opens or closes a code block in OnChartEvent.
4995	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in OnChartEvent.
4996	g_AutoArm=false;	Updates or initializes g_AutoArm in OnChartEvent.
4997	SetLastMessage("Waiting mode cancelled",clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
4998	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
4999	return;	Returns a value or exits the current function in OnChartEvent.
5000	}	Opens or closes a code block in OnChartEvent.
5001		Blank separator for readability.
5002	if(sparam==B_CLOSE sparam==B_CLOSE_ALL)	Checks a condition and runs the guarded branch in OnChartEvent.
5003	{	Opens or closes a code block in OnChartEvent.
5004	CloseAllPositions();	Calls helper function CloseAllPositions in OnChartEvent.
5005	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5006	return;	Returns a value or exits the current function in OnChartEvent.
5007	}	Opens or closes a code block in OnChartEvent.
5008		Blank separator for readability.
5009	if(sparam==B_CLOSE50)	Checks a condition and runs the guarded branch in OnChartEvent.
5010	{	Opens or closes a code block in OnChartEvent.
5011	CloseCurrentPercent(50.0);	Calls helper function CloseCurrentPercent in OnChartEvent.
5012	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5013	return;	Returns a value or exits the current function in OnChartEvent.
5014	}	Opens or closes a code block in OnChartEvent.
5015		Blank separator for readability.
5016	if(sparam==B_BE)	Checks a condition and runs the guarded branch in OnChartEvent.
5017	{	Opens or closes a code block in OnChartEvent.
5018	MoveToBreakEven();	Calls helper function MoveToBreakEven in OnChartEvent.
5019	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5020	return;	Returns a value or exits the current function in OnChartEvent.
5021	}	Opens or closes a code block in OnChartEvent.
5022		Blank separator for readability.
5023	if(sparam==B_FIRST_BE)	Checks a condition and runs the guarded branch in OnChartEvent.
5024	{	Opens or closes a code block in OnChartEvent.
5025	MoveFirstTradeToBreakEven();	Calls helper function MoveFirstTradeToBreakEven in OnChartEvent.
5026	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5027	return;	Returns a value or exits the current function in OnChartEvent.
5028	}	Opens or closes a code block in OnChartEvent.
5029		Blank separator for readability.
5030	if(sparam==B_FIRST_BE_EXIT)	Checks a condition and runs the guarded branch in OnChartEvent.
5031	{	Opens or closes a code block in OnChartEvent.
5032	CloseFirstTradeAtBreakEvenExit();	Calls helper function CloseFirstTradeAtBreakEvenExit in OnChartEvent.
5033	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5034	return;	Returns a value or exits the current function in OnChartEvent.
5035	}	Opens or closes a code block in OnChartEvent.
5036		Blank separator for readability.
5037	if(sparam==B_PC)	Checks a condition and runs the guarded branch in OnChartEvent.
5038	{	Opens or closes a code block in OnChartEvent.
5039	g_PC_On=!g_PC_On;	Updates or initializes g_PC_On in OnChartEvent.
5040	SetLastMessage(g_PC_On ? "Partial close enabled" : "Partial close disabled",	Updates the EA status message, panel display, and web state dirty flag.
5041	g_PC_On ? clrSkyBlue : clrSilver);	Executes one statement in OnChartEvent.
5042	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5043	return;	Returns a value or exits the current function in OnChartEvent.
5044	}	Opens or closes a code block in OnChartEvent.
5045		Blank separator for readability.
5046	if(sparam==B_SECOND)	Checks a condition and runs the guarded branch in OnChartEvent.
5047	{	Opens or closes a code block in OnChartEvent.
5048	g_SecondEntryOn=!g_SecondEntryOn;	Updates or initializes g_SecondEntryOn in OnChartEvent.
5049	if(!g_SecondEntryOn && HasOurPosition())	Checks a condition and runs the guarded branch in OnChartEvent.
5050	{	Opens or closes a code block in OnChartEvent.
5051	SetArmMode(ARM_NONE);	Calls helper function SetArmMode in OnChartEvent.
5052	g_AutoArm=false;	Updates or initializes g_AutoArm in OnChartEvent.
5053	}	Opens or closes a code block in OnChartEvent.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
5054	SyncSecondEntryState();	Calls helper function SyncSecondEntryState in OnChartEvent.
5055	2R" : SetLastMessage(g_SecondEntryOn ? "Second entry enabled - waits for -0.5R before "Second entry disabled",	Updates the EA status message, panel display, and web state dirty flag.
5056	g_SecondEntryOn ? clrSkyBlue : clrSilver);	Executes one statement in OnChartEvent.
5057	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5058	return;	Returns a value or exits the current function in OnChartEvent.
5059	}	Opens or closes a code block in OnChartEvent.
5060		Blank separator for readability.
5061	if(sparam==B_FIRST_TRAIL)	Checks a condition and runs the guarded branch in OnChartEvent.
5062	{	Opens or closes a code block in OnChartEvent.
5063	g_FirstTrailOn=!g_FirstTrailOn;	Updates or initializes g_FirstTrailOn in OnChartEvent.
5064	SyncFirstTrailState();	Calls helper function SyncFirstTrailState in OnChartEvent.
5065	M5 SetLastMessage(g_FirstTrailOn ? "First trade M5 trailing enabled" : "First trade M5 trailing disabled",	Updates the EA status message, panel display, and web state dirty flag.
5066	g_FirstTrailOn ? clrSkyBlue : clrSilver);	Executes one statement in OnChartEvent.
5067	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5068	return;	Returns a value or exits the current function in OnChartEvent.
5069	}	Opens or closes a code block in OnChartEvent.
5070		Blank separator for readability.
5071	if(sparam==B_MODE_SAFE)	Checks a condition and runs the guarded branch in OnChartEvent.
5072	{	Opens or closes a code block in OnChartEvent.
5073	g_AdvancedMode=false;	Updates or initializes g_AdvancedMode in OnChartEvent.
5074	g_PC_On=false;	Updates or initializes g_PC_On in OnChartEvent.
5075	SetLastMessage("Safe mode: fixed TP, no partials",clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5076	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5077	return;	Returns a value or exits the current function in OnChartEvent.
5078	}	Opens or closes a code block in OnChartEvent.
5079		Blank separator for readability.
5080	if(sparam==B_MODE_ADV)	Checks a condition and runs the guarded branch in OnChartEvent.
5081	{	Opens or closes a code block in OnChartEvent.
5082	g_AdvancedMode=true;	Updates or initializes g_AdvancedMode in OnChartEvent.
5083	g_PC_On=true;	Updates or initializes g_PC_On in OnChartEvent.
5084	SetLastMessage("Advanced mode: partials and BE controls ready",clrSkyBlue);	Updates the EA status message, panel display, and web state dirty flag.
5085	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5086	return;	Returns a value or exits the current function in OnChartEvent.
5087	}	Opens or closes a code block in OnChartEvent.
5088		Blank separator for readability.
5089	if(sparam==B_LOT_UP)	Checks a condition and runs the guarded branch in OnChartEvent.
5090	{	Opens or closes a code block in OnChartEvent.
5091	double step=InpLotButtonStep;	Updates or initializes double step in OnChartEvent.
5092	if(step<=0.0)	Checks a condition and runs the guarded branch in OnChartEvent.
5093	step=BrokerStepLot();	Updates or initializes step in OnChartEvent.
5094	g_Lot=NormalizeVolumeForPanel(g_Lot+step);	Updates or initializes g_Lot in OnChartEvent.
5095	RefreshLotEdit();	Calls helper function RefreshLotEdit in OnChartEvent.
5096	SetLastMessage("Lot set to "+DoubleToString(g_Lot,VolumeDigits()),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5097	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5098	return;	Returns a value or exits the current function in OnChartEvent.
5099	}	Opens or closes a code block in OnChartEvent.
5100		Blank separator for readability.
5101	if(sparam==B_LOT_DN)	Checks a condition and runs the guarded branch in OnChartEvent.
5102	{	Opens or closes a code block in OnChartEvent.
5103	double step=InpLotButtonStep;	Updates or initializes double step in OnChartEvent.
5104	if(step<=0.0)	Checks a condition and runs the guarded branch in OnChartEvent.
5105	step=BrokerStepLot();	Updates or initializes step in OnChartEvent.
5106	g_Lot=NormalizeVolumeForPanel(g_Lot-step);	Updates or initializes g_Lot in OnChartEvent.
5107	RefreshLotEdit();	Calls helper function RefreshLotEdit in OnChartEvent.
5108	SetLastMessage("Lot set to "+DoubleToString(g_Lot,VolumeDigits()),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5109	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5110	return;	Returns a value or exits the current function in OnChartEvent.
5111	}	Opens or closes a code block in OnChartEvent.
5112		Blank separator for readability.
5113	if(sparam==B_RISK_UP)	Checks a condition and runs the guarded branch in OnChartEvent.
5114	{	Opens or closes a code block in OnChartEvent.
5115	double step=InpRiskMoneyButtonStep;	Updates or initializes double step in OnChartEvent.
5116	if(step<=0.0)	Checks a condition and runs the guarded branch in OnChartEvent.
5117	step=10.0;	Updates or initializes step in OnChartEvent.
5118	g_RiskMoney=NormalizeRiskMoney(g_RiskMoney+step);	Updates or initializes g_RiskMoney in OnChartEvent.
5119	RefreshRiskEdit();	Calls helper function RefreshRiskEdit in OnChartEvent.

Appendix A - EA Line-by-Line Source Explanation

Line	EA source	Explanation
5120	SetLastMessage("Risk set to "+MoneyText(g_RiskMoney),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5121	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5122	return;	Returns a value or exits the current function in OnChartEvent.
5123	}	Opens or closes a code block in OnChartEvent.
5124		Blank separator for readability.
5125	if(sparam==B_RISK_DN)	Checks a condition and runs the guarded branch in OnChartEvent.
5126	{	Opens or closes a code block in OnChartEvent.
5127	double step=InpRiskMoneyButtonStep;	Updates or initializes double step in OnChartEvent.
5128	if(step<=0.0)	Checks a condition and runs the guarded branch in OnChartEvent.
5129	step=10.0;	Updates or initializes step in OnChartEvent.
5130	g_RiskMoney=NormalizeRiskMoney(g_RiskMoney-step);	Updates or initializes g_RiskMoney in OnChartEvent.
5131	RefreshRiskEdit();	Calls helper function RefreshRiskEdit in OnChartEvent.
5132	SetLastMessage("Risk set to "+MoneyText(g_RiskMoney),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5133	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5134	return;	Returns a value or exits the current function in OnChartEvent.
5135	}	Opens or closes a code block in OnChartEvent.
5136		Blank separator for readability.
5137	if(sparam==B_RR_UP)	Checks a condition and runs the guarded branch in OnChartEvent.
5138	{	Opens or closes a code block in OnChartEvent.
5139	g_RRTarget=MathMin(20.0,g_RRTarget+0.1);	Updates or initializes g_RRTarget in OnChartEvent.
5140	g_RRTarget=NormalizeDouble(g_RRTarget,1);	Updates or initializes g_RRTarget in OnChartEvent.
5141	SetLastMessage("RR target set to 1:"+DoubleToString(g_RRTarget,1),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5142	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5143	return;	Returns a value or exits the current function in OnChartEvent.
5144	}	Opens or closes a code block in OnChartEvent.
5145		Blank separator for readability.
5146	if(sparam==B_RR_DN)	Checks a condition and runs the guarded branch in OnChartEvent.
5147	{	Opens or closes a code block in OnChartEvent.
5148	g_RRTarget=MathMax(0.1,g_RRTarget-0.1);	Updates or initializes g_RRTarget in OnChartEvent.
5149	g_RRTarget=NormalizeDouble(g_RRTarget,1);	Updates or initializes g_RRTarget in OnChartEvent.
5150	SetLastMessage("RR target set to 1:"+DoubleToString(g_RRTarget,1),clrSilver);	Updates the EA status message, panel display, and web state dirty flag.
5151	UpdatePanel();	Calls helper function UpdatePanel in OnChartEvent.
5152	return;	Returns a value or exits the current function in OnChartEvent.
5153	}	Opens or closes a code block in OnChartEvent.
5154	}	Opens or closes a code block in OnChartEvent.
5155	//+-----+	Banner comment marking a major EA section.